



BUILD WEB APPS WITH ASP.NET



Sr No.	Index
1	Article
2	Article
1	Hello World
1.1	What is C#?
1.2	Run Some C#
1.3	Getting Input
1.4	Comments
1.5	C# in the Wild
1.6	Review
1	Concept Review
1	Quiz
1	Project : Console Creatures
3	Article
2	Data Types and Variables
2.1	Introduction to Data Types and Variables in C#
2.2	C# Data Types
2.3	Creating Variables with Types
2.4	Handling Errors
2.5	Converting Data Types
2.6	Review
2	Concept Review
3	WORKING WITH NUMBERS
3.1	Introduction to Working with Numbers
3.2	Numerical Data Types
3.3	Arithmetic Operators
3.4	Operator Shortcuts
3.5	Modulo
3.6	Built-In Methods
3.7	Using Documentation
3.8	Review

4	WORKING WITH TEXT
4.1	Introduction to Working with Text
4.2	Building Strings
4.3	String Concatenation
4.4	String Interpolation
4.5	Get Info About Strings
4.6	Get Parts of Strings
4.7	Manipulate Strings
4.8	Review
4	Quiz
2	Project : Mad Libs
3	Project : Money Maker
5	UNDERSTANDING LOGIC IN C#
5.1	Introduction to Logic in C#
5.2	Boolean Data Types
5.3	Comparison Operators
5.4	Truth Table
5.5	Logical Operators
5.6	Review
6	CONDITIONAL STATEMENTS
6.1	Introduction to Conditional Statements
6.2	If Statements
6.3	If...Else... Statements
6.4	Else If Statements
6.5	Switch Statements
6.6	Ternary Operators
6.7	Review
6	Concept Review
6	Quiz
4	Project : Password Checker
5	Project : Choose Your Own Adventure
7	METHOD CALLS AND INPUT
7.1	Introduction to Methods

7.2	Call a Method
7.3	Capture Output
7.4	Define a Method
7.5	Define Parameters
7.6	A Note on Parameters
7.7	Optional Parameters
7.8	Named Arguments
7.9	Method Overloading
7.10	Review
7	Concept Review
8	METHOD OUTPUT
8.1	Introduction to Method Output
8.2	Return
8.3	Return Errors
8.4	Out
8.5	Using Out
8.6	Out Errors
8.7	Review
8	Quiz
6	Project : Architect Arithmetic
9	ALTERNATE EXPRESSIONS
9.1	Introduction to Alternate Expressions
9.2	Expression-bodied Definitions
9.3	Methods as Arguments
9.4	Lambda Expressions
9.5	Shorter Lambda Expressions
9.6	Review
10	ARRAYS
10.1	Introduction to Arrays
10.2	Building Arrays
10.3	Array Length
10.4	Accessing Array Items
10.5	Editing Arrays

10.6	Built-In Methods
10.7	Built-In Methods
10.8	Review
10	Concept Review
11	LOOPS
11.1	Introduction to Loops
11.2	While Loop
11.3	Do...While Loop
11.4	For Loop
11.5	For Each Loop
11.6	Comparing Loops
11.7	Jump Statements
11.8	Review
11	Concept Review
11	Quiz
7	Project : Caesar Cipher
12	BASIC CLASSES AND OBJECTS
12.1	Introduction to Classes
12.2	Making Classes
12.3	Fields
12.4	Properties
12.5	Automatic Properties
12.6	Public vs. Private
12.7	Get-Only Properties
12.8	Methods
12.9	Constructors
12.10	this
12.11	Overloading Constructors
12.12	Review
12	Concept Review
12	Quiz
13	STATIC MEMBERS
13.1	Introduction to Static

13.2	Static Fields and Properties
13.3	Static Methods
13.4	Static Constructors
13.5	Static Classes
13.6	Common Static Errors
13.7	Main()
13.8	Review
13	Quiz
8	Project : The Object of Your Affection
14	INTERFACES
14.1	Introduction to Interfaces
14.2	Build an Interface
14.3	Implementing an Interface
14.4	What Interfaces Cannot Do
14.5	Implementing an Interface Again
14.6	Finish Truck Class
14.7	Testing Interfaces
14.8	Review
14	Concept Review
15	INHERITANCE
15.1	Introduction to Inheritance
15.2	Superclass and Subclass
15.3	Create a Superclass
15.4	Access Inherited Members with Protected
15.5	Remove Duplicate Code
15.6	Access Inherited Members with Base
15.7	Override Inherited Members
15.8	Make Inherited Members Abstract
15.9	Review
15	Quiz
16	WHAT IS THE BACK-END?
16.1	Front and Back
16.2	The Web Server

16.3	So What is the Back-end?
16.4	Storing Data
16.5	What is an API?
16.6	Authorization and Authentication
16.7	Different Back-end Stacks
16.8	Review
4	Article
5	Article
6	Article
17	RAZOR PAGES SYNTAX I
17.1	Introduction to Razor Pages
17.2	Structure of a Razor View Page
17.3	Writing C# in a Razor Page
17.4	Conditionals in Razor Pages: If Statements
17.5	Conditionals in Razor Pages: Switch Statements
17.6	Working with Loops
17.7	Review
17	Concept Review
18	RAZOR PAGES SYNTAX II
18.1	Introduction to the Page Model
18.2	Working with the Page Model
18.3	ViewData
18.4	Sharing Pages
18.5	Working with ViewImports
18.6	Using Partialcs
18.7	Tag Helpers
18.8	Review
18	Quiz
7	Article
19	PAGE MODEL BASICS
19.1	Re-Introduction To Page Models
19.2	OnGet()
19.3	OnPost()

19.4	Model Binding
19.5	asp-for
19.6	asp-route-{value}
19.7	OnGetAsync()
19.8	OnPostAsync()
19.9	Review
19	Concept Review
20	ROUTING
20.1	Introduction to Routing
20.2	Customize the URL
20.3	Route Templates
20.4	Add Optional Route Templates
20.5	Add Constrained Route Templates
20.6	asp-route-{value} revisited
20.7	Review
21	REDIRECTION
21.1	Introduction to Redirection
21.2	RedirectToPage()
21.3	Page()
21.4	NotFound()
21.5	Async Redirects
21.6	Review
21	Quiz
9	Project : Grocer.ly
	ASP.NET: Databases

1.Article

Welcome to ASP.NET

Learn about the ASP.NET Skill Path

Welcome to the Build Web Apps with ASP.NET Skill Path! We're excited to start this journey with you.

Build Web Apps with ASP.NET is a Skill Path that will teach you the fundamentals of programming in C#, along with how to build web sites with a working front-end and back-end. As you work through this path, you'll take Codecademy lessons, complete practice projects, and build apps on your own computer. This material will prepare you to:

- Become an intermediate programmer in C#
- Build web pages using Razor syntax
- Build page-focused web apps using ASP.NET Core
- Perform CRUD database operations
- Use Visual Studio to build web apps on your own computer

We'll assume that you know some basic HTML and CSS. If you don't, run through our [Learn HTML](#) and [Learn CSS](#) courses before starting.

This skill path thoroughly covers C# before getting into any web development. Stick with it and your hard work will pay off.

Let's get started!

2.Article

What is .NET?

Learn about the .NET platform that powers C# applications

If you're learning C#, you're going to encounter a lot of "dot net": .NET, .NET Framework, .NET Core, dotnet, ASP.NET, the list goes on. This article will demystify these concepts and give you some context before diving into C# programming.

What is .NET?

Formally, .NET is "an open source developer platform, created by Microsoft, for building many different types of applications. You can write .NET apps in C#, F#, Visual C++, or Visual Basic."

Informally, .NET is the tool that lets you build and run C# programs (we'll avoid F#, Visual C++, Visual Basic for now).

When you download .NET, you're really downloading a bunch of programs that:

- Translate your C# code into instructions that a computer can understand
- Provide utilities for building software, like tools for printing text to the screen and finding the current time
- Define a set of data types that make it easier for you to store information in your programs, like text, numbers, and dates

There are a few versions of .NET. They do the same job but they are meant for different operating systems:

- .NET Framework is the original version of .NET that only runs on Windows computers.
- .NET Core is the new, cross-platform version of .NET that runs on Windows, MacOS, and Linux computers. Since it's more flexible and [Microsoft is actively enhancing this version](#), we'll be using this one throughout the path.

Whenever we refer to “.NET” from now on, we really mean .NET Core.

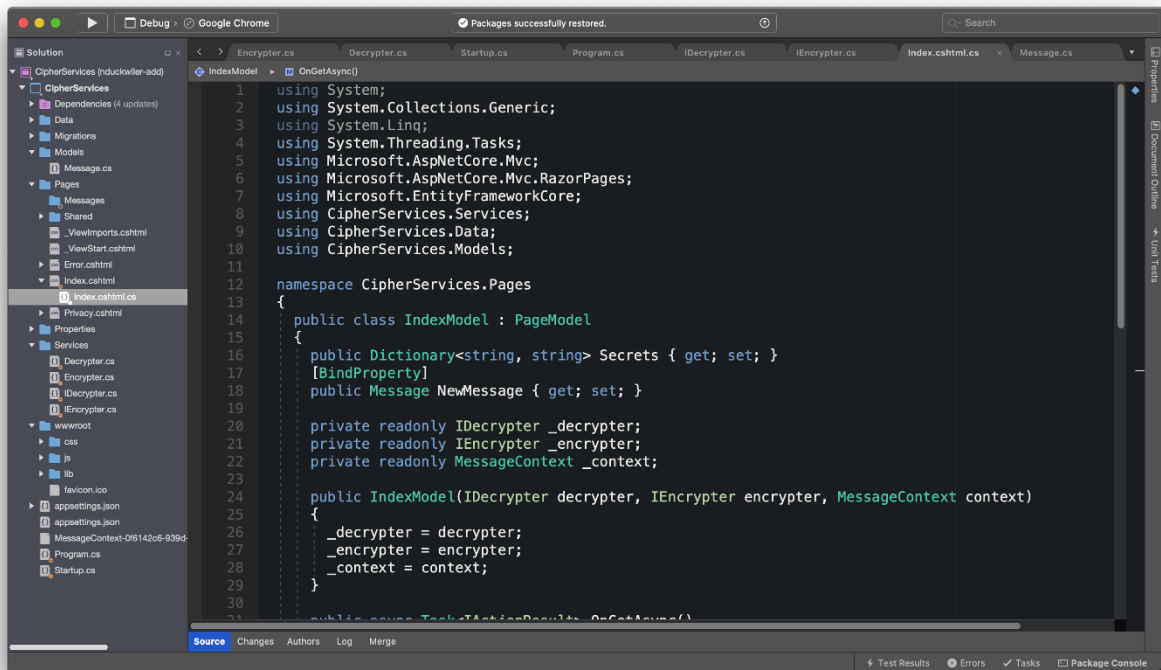
Conveniently, [Microsoft itself plans to rename .NET Core to .NET](#) towards the end of 2020.

How do I get .NET?

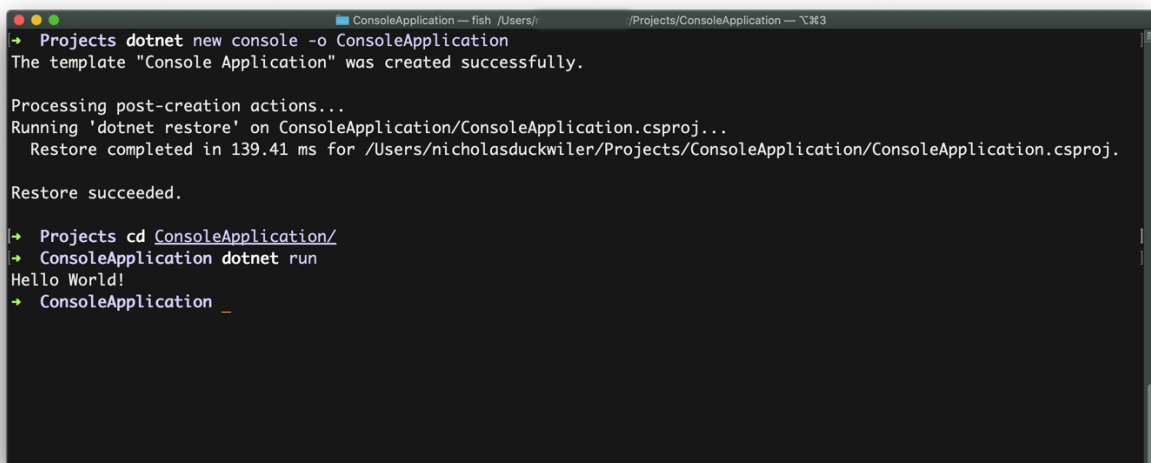
So we know we want to work with C#, and we know we need .NET. How do we get the .NET platform on our computers?

The two main ways are:

Download [Visual Studio](#), an [integrated development environment \(IDE\)](#) for .NET applications. It comes as an app, like the web browser you're using to view this article. It comes with the .NET platform, a code editor, and additional tools to help you write code.



Download the [.NET SDK](#) (software development kit). It also comes with the .NET platform, but it has no code editing tools. Instead of an app on your computer, the SDK is accessed via a command-line interface (CLI) — to use an SDK, you'll open up a terminal on your computer and type commands instead of clicking buttons. In this example, you can see a terminal in which the user ran commands like **dotnet new** and **dotnet run** to build a new application, then ran it (the app just prints out "Hello World!").



You'll see both of these throughout the skill path.

What about ASP.NET?

If we want to build programs specifically for the web, like websites, you'll need to add more tools on top of .NET. One of the most popular is ASP.NET. To keep them separate, developers call .NET a *platform* and ASP.NET a *framework*. This is all you need to know about ASP.NET to get started with C#: we'll cover the details in a separate article.

Summary

Hopefully, this helps you see the big picture before diving into the details of C#:

- C# is a [programming language](#)
- To build programs with C#, you need the .NET platform
- .NET comes in two major flavors, .NET Framework for Windows and .NET Core for Windows/MacOS/Linux
- To get .NET Core on your computer, download Visual Studio or the .NET SDK
- To build programs like web apps and web services, you need the ASP.NET framework on top of .NET

Now let's get coding!

1. HELLO WORLD

1.1 What is C#?

What would you like to build? If you can name it, you can probably build it with C#.

This programming language can be used to make interactive websites, mobile apps, video games, augmented reality (AR), virtual reality (VR), desktop applications, and back-end services – just to name a few. For example, the mobile game [Pokemon Go](#) and the [Stack Overflow website](#) were built with frameworks that can be run with C# (Unity and ASP.NET, respectively).

Even if you already know a programming language, learning C# (pronounced “cee sharp”) can make you a better programmer:

- Unlike languages like Ruby and JavaScript, C# has you define the *type* of each data in a program. Assigning a type essentially tells a computer what operations can and cannot be performed on a piece of data. This style of coding helps programmers avoid a large class of errors that are common to Ruby and JavaScript .
- If you’re familiar with Java, you’ll recognize how C# programs are built—by defining objects that interact with each other, which makes code reusable and easy to manage.

1.1 What is C#? Instructions

What are your goals with learning C#? (There are no wrong answers!)

Keep those in mind when taking this course.



1.2 Run Some C#

Time to run some C# yourself!

There are two panels here: a text editor containing some C# code and a console, or terminal, that shows output. When you run the code, you'll see some text printed to the console.

The code is contained in a file called **Program.cs**. For now, just focus on the line:

```
Console.WriteLine("Hello World!");
```

`Console.WriteLine()` is a command that prints text to a console. Whatever is in between the parentheses will be printed to the console! In this case it's Hello World!.

1.2 Run Some C# Instructions

- 1.

Change the code in the editor so that your own name is printed to the console!
Make sure to run the code after you edit the text.

Program.cs

```
using System;

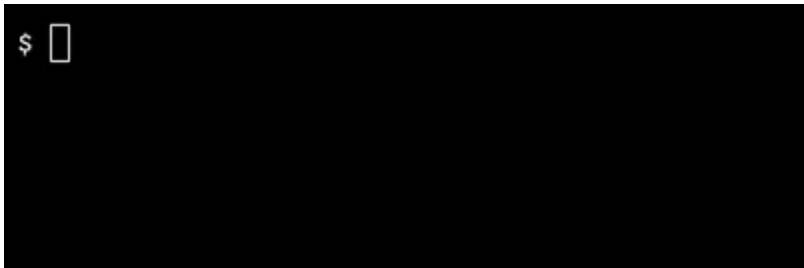
namespace HelloWorld
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```

OUTPUT

Hello World!

1.3 Getting Input

We can also read input from a user. The command `Console.ReadLine()` captures text that a user types into the console.



In this example, the program writes a question to the console and waits for user input. Once the user types something and presses Enter (or Return), the input is printed back out to the user.

```
Console.WriteLine("How old are you?");  
string input = Console.ReadLine();  
Console.WriteLine($"You are {input} years old!");
```

The word `input` represents a variable (variables are explored more in other lessons). For now, know that the word `input` represents the text the user typed in the console. It's labeled string because, in C#, a piece of text is called a string.

`input` is used in the following line so that the printed message will change based on what the user types. For example, if you ran the program and responded to the question with 101, then the value of `input` would be "101" and `You are 101 years old!` would be printed to the console.

1.3 Getting Input Instructions

1.

Run the code and answer the question.

For interactive code like this, run it by typing the command
`dotnet run`

in the console and pressing Enter.

Program.cs

```
using System;
```

```
namespace GettingInput
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main()
```

```
        {
```

```
            Console.WriteLine("How old are you?");
```

```
            string input = Console.ReadLine();
```

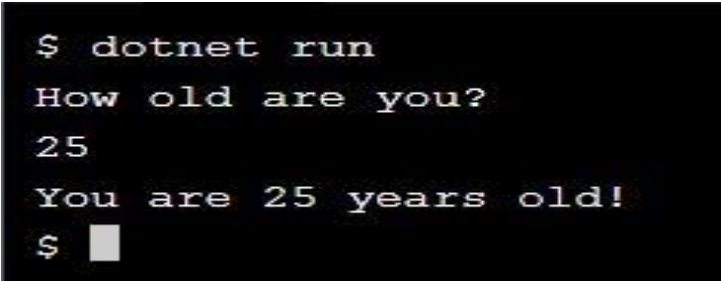
```
            Console.WriteLine($"You are {input} years old!");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

A terminal window with a black background and white text. The text shows the command '\$ dotnet run' being executed, followed by the program's output: 'How old are you?', the user input '25', and the program response 'You are 25 years old!'. The prompt '\$' is followed by a small grey square cursor.

```
$ dotnet run
How old are you?
25
You are 25 years old!
$ █
```

1.4 Comments

Ironically, an essential tool in programming is the ability to tell a computer to ignore a part of a program. Text written in a program but not run by the computer is called a *comment*. In C#, anything after a `//` or between `/*` and `*/` is a comment. In spoken word we call these symbols “forward slashes” and “asterisks”.

Comments can:

- Provide context for why something is written the way it is:

```
/* This variable will be used to count the number of times anyone tweets the  
word persnickety */  
int persnicketyCount = 0;
```

- Help other people reading the code understand it faster:

```
/* Calculates tomorrow's rain likelihood as a number between 0 and 100 */  
ComplicatedRainCalculationForTomorrow();
```

- Ignore a line of code and see how a program will run without it:

```
// string usefulValue = OldSloppyCode();  
string usefulValue = NewCleanCode();
```

Developers tend to use `//` for short, one-line comments and `/* */` for anything longer, but the choice is up to you!

1.4 Comments Instructions

1.

Add a comment to the code right above the first `Console.WriteLine()`.

The comment should explain what this program does.

Program.cs

```
using System;
```

```
namespace GettingInput
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main()
```

```
        {
```

```
            Console.WriteLine("How old are you?");
```

```
            string input = Console.ReadLine();
```

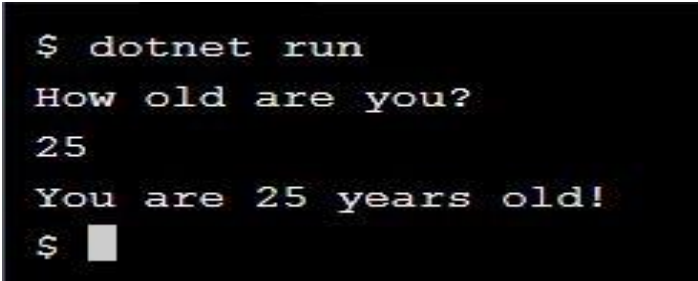
```
            Console.WriteLine($"You are {input} years old!");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT



```
$ dotnet run
How old are you?
25
You are 25 years old!
$ █
```

1.5 C# in the Wild

You've seen some examples of C# code in action, but learning a programming language is more than just memorizing commands: it's understanding how it differs from other technologies, joining its community, and applying it to real-life problems. Here's what you need to know:

1. C# technologies are fast: A software company called [Raygun](#) built their application using Node.js, a framework for JavaScript. When their app started to slow down, they switched to .NET, a framework for C#. Their performance skyrocketed. As CEO John Daniel puts it, "using the same size server, we were able to go from 1,000 requests per second...with Node.js, to 20,000 requests per second with .NET Core." In other words, they increased throughput by 2,000%.
2. The C# community is big: In 2019, [Github ranked C# as the fifth most popular programming language](#) and [StackOverflow ranked it seventh](#).
3. C# is employable: Thanks to good design and the popularity of frameworks supporting the language, C# can get you access to a lot of great jobs. Search on any job site and you'll find companies looking for C# and .NET developers to build chat applications, financial trading programs, medical record systems, and beyond. In the 2019 HackerRank Developer Skills Report, expertise in [two C#-compatible frameworks, ASP.NET and .NET Core, were among the top ten most sought-after by managers](#).

1.5 C# in the Wild Instructions

Hover over the image to see it come to life!

Use this time to think about how you want to use C# in your career.

1.6 Review

Congrats! You finished your first lesson in C#. In this lesson you learned:

- C# is used to make interactive websites, mobile apps, video games, augmented and virtual reality (AR and VR), back-end services, and desktop applications
- .NET generally refers to the family of programs and commands that let you make applications with C#
- C# and .NET jobs are out there! Build video games with [Unity](#), build websites with [ASP.NET](#)...The skills you learn on Codecademy can open new doors
- The command `Console.WriteLine()` prints text to the console
- The command `Console.ReadLine()` captures user input in the console
- *Comments* are lines of code that are ignored by your computer; they're intended to be read by developers instead. Make them with `//` or `/*` and `*/`

1.6 Review Instructions

Here's the first C# application we used earlier.

Use `Console.WriteLine()` commands to make it your own!

Program.cs

```
using System;
```

```
namespace HelloWorld
```

```
{
```

```
    class Program
```

```
{  
    static void Main()  
    {  
        Console.WriteLine("Hello Codey!");  
    }  
}
```

Concept Review

Console.ReadLine()

The `Console.ReadLine()` method is used to get user input. The user input can be stored in a variable. This method can also be used to prompt the user to press enter on the keyboard.

```
Console.WriteLine("Enter your name: ");
```

```
name = Console.ReadLine();
```

C#

C# is a general-purpose, type-safe, object-oriented programming language. It was developed around 2000 by Microsoft as part of its .NET initiative.

Comments

Comments are bits of text that are not executed. These lines can be used to leave notes and increase the readability of the program.

- Single line comments are created with two forward slashes `//`.
- Multi-line comments start with `/*` and end with `*/`. They are useful for commenting out large blocks of code.

```
// This is a single line comment
```

```
/* This is a multi-line comment  
and continues until the end  
of comment symbol is reached */
```

Console.WriteLine()

The `Console.WriteLine()` method is used to print text to the console. It can also be used to print other data types and values stored in variables.

```
Console.WriteLine("Hello, world!");
```

```
// Prints: Hello, world!
```

.NET Platform

.NET is a free, cross-platform, open source developer platform for building many different types of applications.

With .NET, you can use multiple languages, editors, and libraries to build for web, mobile, desktop, gaming, and IoT. Whether you're working in C#, F#, or Visual Basic, your code will run natively on any compatible OS. Different .NET implementations handle the heavy lifting for you.

1. Quiz

1. What is the outcome of these lines of code?

```
Console.WriteLine("Press Enter to begin...");  
Console.ReadLine();  
Console.WriteLine("Let's get started!");
```

- a) The text Let's get started! is printed to the console.
- b) No text is printed until the user types into the console.
- c) The text Press Enter to begin... is printed to the console and the computer waits for the user to hit Enter on the keyboard.
- d) The texts Press Enter to begin... and Let's get started! are printed to the console.

2. How is the .NET platform related to C#?

- a) .NET is a part of C#.
- b) .NET apps can be written in C#

- c) .NET is not related to C#.
- d) .NET is the same as C#.

3. What is NOT a good reason to use comments in your code?

- a) To instruct a computer to execute a block of code.
- b) Help other people reading the code understand it faster.
- c) Ignore a line of code and see how a program will run without it.
- d) Provide context for why something is written the way it is.

4. What is the outcome of this line of code?

`Console.ReadLine();`

- a) The computer outputs whatever the user last entered in the console.
- b) The computer copies all of the text in the console.
- c) The computer outputs `Console.ReadLine()` to the console.
- d) The computer waits for the user to type something in the console.

5. What is the outcome of this line of code?

`Console.WriteLine("Open Sesame!");`

- a) The text `Open Sesame!` is printed to the console.
- b) The text `WriteLine` is printed to the console.
- c) The text `Console.WriteLine("Open Sesame!");` is printed to the console.
- d) No text is printed to the console.

6. What is one professional benefit of learning C#?

- a) Web services written with C# are always slower than those written with Node.js.
- b) There are no platforms that support C#.
- c) It is popular among recruiters looking to hire engineers.
- d) There is no community around C#.

7. What can you make with C#?

- a) C# can only be used to build websites.
- b) C# can only be used to build games.
- c) C# can only be used to build mobile apps.
- d) C# can be used to make websites, mobile apps, games, and more.

8. In C#, what are these lines of code called?

```
// Idling standing by
```

```
/*
```

```
Nothing to see here!
```

```
*/
```

- a) .NET platforms
- b) Commands
- c) Comments
- d) Output

Project-1

Console Creatures

C# can be used to make nearly anything – even creatures! Using the `Console.WriteLine()` command, you can print text to the console and make something like this:

```
.-.  
(o o)  
| O |  
| |  
'~~~'
```

Each part of the ghost is a line of text, using symbols like parentheses () and tildes ~. Spooky, right? These type of drawings are sometimes called [ASCII art](#), and you can find them all across the internet.

The C# program is already set-up for you in the text editor. For now, you can ignore everything except the line with `Console.WriteLine();`. You'll be copying and editing this command throughout the project.

Make sure to use double quotes (") and a semicolon (;) like so:

```
Console.WriteLine("This will be printed to the console!");
```

TASKS

Build a Creature

1.

Run the code once to see what we have to start.

You may need to use the “Save” button.

2.

Before the first `Console.WriteLine()` command, write a comment explaining the purpose of this program.

3.

Since the ghostly head has already been created, draw the eyes: write a new `Console.WriteLine()` command with this text inside the parentheses:

```
"(o o)"
```

4.

Draw the mouth: write another `Console.WriteLine()` command, this time using the text:

```
"| O |"
```

5.

Make the body of the ghost by using two more `Console.WriteLine()` commands and this text:

```
"|  |"
```

```
"~~~~"
```

6.

Name your ghost by adding another line of text below it.

7.

You've built your very own apparition! You can build all kinds of ghouls with this technique. Find even more inspiration in the hint.

```

c_  _
  \|_|_|_
    \ ( o_o)
      > ~ >
        /   /\
       /    /\
      \   )  c/
       /   /
      /  /|
     (  (\
    |  | \
    | / \ )
    | |  ) |
   / ) ( _/
  ( _ /

```

–Dancing Man

Remember:

- The backslash \ is an escape character in C#, so \" will show up as one quote in the output (") and \\ will show up as one backslash in the output (\).
- Spaces matter! " (" is not the same as "(".

Program.cs

```
using System;
```

```
namespace ConsoleCreatures
```

```
{
```

```
    class Program
```

```

{
    static void Main()
    {
        Console.WriteLine(" .-.");
    }
}

```

OUTPUT

```

 .-.
(o o)
| O |
|   |
'~~~'

I'm a friendly ghost named Casper!

```

3.Article

Go Off-Platform with C#

Learn how to set up a local development environment and run C# code on your own computer! This article covers downloading, installing, and using Microsoft Visual Studio for MacOS, Windows, and Linux.

By the end of this article you will be able to run a C# console application on your own computer!

C#, like any programming language, is a way to communicate with your computer: giving it commands to print things to a console, lay out a website, run a video game, etc. At this point we assume you've seen a C# console application in the Codecademy learning environment. If you haven't, take [the first lesson](#).

To use C#, we need to teach your computer how to understand the language. We do this by downloading some free software.

There are multiple options, but we recommend downloading [Microsoft Visual Studio](#). This app helps you write, debug, and run C# code. It is maintained by the same company that first developed the C# language and it's very popular amongst C# developers. In a 2017 survey of over 5,000 people, the software company JetBrains found that [92% of C# developers use Visual Studio to write and run C# code on their own computers](#).

Everyone's computer system, sometimes called a local development environment, is different. That means that everyone's setup process will be different as well. We've included videos and documentation that describe the process for MacOS, Windows, and Linux. Take your pick and happy coding!

- [Setting up C# on MacOS](#)
- [Setting up C# on Windows](#)
- [Setting up C# on Linux](#): This links to MonoDevelop, a good replacement for Visual Studio (since VS isn't available on Linux at the time of writing this article)

2. Data Types and Variables

2.1 Introduction to Data Types and Variables in C#

When we write programs, we're telling the computer how to process pieces of information, like the numbers in a calculation, or printing text to the screen. So how does a computer distinguish between a number and a character? Why is it important to be able to tell the difference?

Languages like C# tell a computer about the type of data in its program using *data types*. Data types represent the different types of information that we can use in our programs and how they should be used.

Without data types, computers would try and perform processes that are impossible, like squaring a piece of text or capitalizing a number. That's how we get bugs!

C# is *strongly-typed*, so it requires us to specify the data types that we're using. It is also *statically-typed*, which means it will check that we used the correct types before the program even runs. Both language features are important because they help write scalable code with fewer bugs.

Using types will come up in several different places when learning C#. To start, we'll examine how types impact variable declaration and the usage of different data types in a program.

In this lesson, we'll look at:

- Common C# data types
- How to create variables that are statically typed
- How to convert variables from one data type to another

2.1 Introduction to Data Types and Variables in C# Instructions

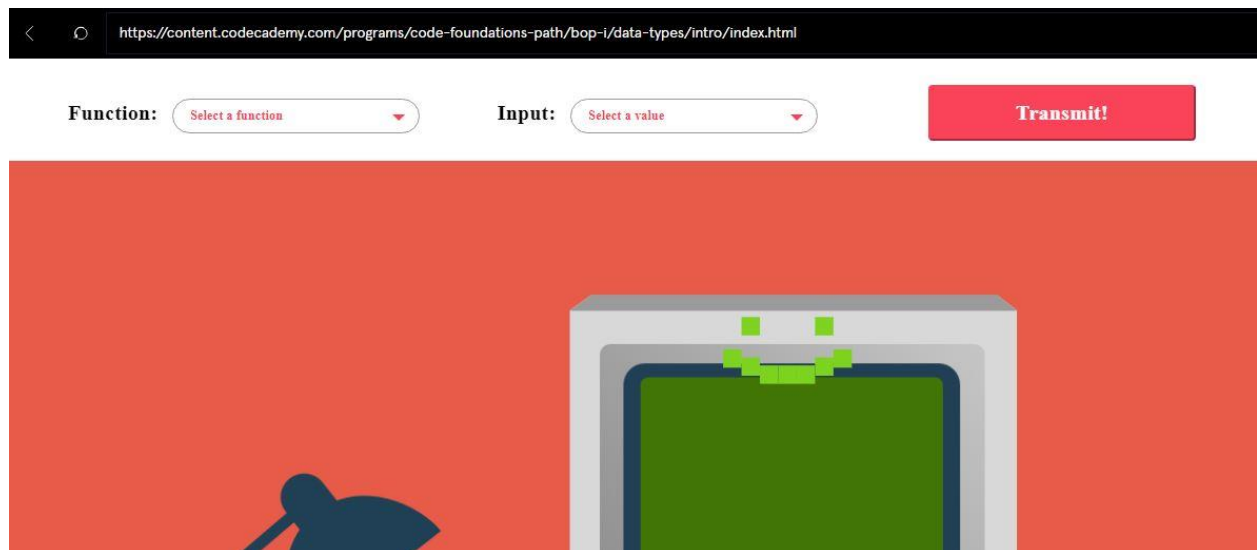
Computers can do different things with different kinds of data. This computer will process data according to the function that you give it. If the function matches the data type, you will get an answer! If it doesn't, you'll see an error.

The functions do the following:

- **capitalize**: will turn lowercase characters into uppercase characters
- **square**: will square a number
- **evaluate**: will determine if an input is true or false

The data types include:

- **int** (4637): whole integer number
- **string** (kangaroo): a piece of text
- **bool** (true): represents the logical idea of true or false



2.2 C# Data Types

Data types tell us a few things about a piece of data, like:

- How it can be stored
- What operations we can perform with it
- Different methods it can be used with

Data types are present in all programming languages, but are particularly important in C#. That's because C# is known as a strongly-typed language—it requires that the programmer *specify the data type of every value and expression*. While it means writing more code, using types has long term benefits like built-in documentation and increased readability.

As we can see to the diagram to the right, C# has several built-in data types. You don't need to memorize all of them, but pay specific attention to these common ones that we'll use throughout our lessons:

- `int` - whole numbers, like: 1, -56, 948
- `double` - decimal numbers, like: 239.43909, -660.01
- `char` - single characters, like: "a", "&", "£"
- `string` - string of characters, like: "dog", "hello world"
- `bool` - boolean values, like: true or false

2.2 C# Data Types Instructions

Review the diagram to the right to see the difference between different C# data types and their usage. Continue when you're ready.

Type	Description	Size (bytes)	.NET type	Range
int	Whole numbers	4	System.Int32	-2, 147, 483, 648 to 2, 147, 483, 647
long	Whole numbers (bigger range)	8	System.Int64	-9, 223, 372, 036, 854, 775, 808 to 9, 223, 372, 036, 854, 775, 807
float	floating-point numbers	4	System.Single	$\pm 3.4 \times 10^{38}$
double	Double precision (more accurate) FP numbers	8	System.Double	$\pm 1.7 \times 10^{308}$
decimal	Monetary values	16	System.Decimal	28 significant figures
char	Single character	2	System.Char	N/A
bool	Boolean	1	System.Boolean	True or False
DateTime	Moments in time	8	System.DateTime	0:0:00 on 01/01/0001 to 23:59:59 on 12/31/9999
string	Sequence of characters	2 per character	System.String	N/A

2.3 Creating Variables with Types

When we use data in our programs, it's good practice to save them in a *variable*. A variable is basically like a box in our computer memory where we can store values used in our code.

In C#, data types and variables are closely intertwined. Remember how C# is strongly-typed? Every time we declare a variable, we have to specify what kind of data type that variable is going to hold.

There are two ways we can assign variables. We can do it on two lines:

```
// Declare an integer  
int myAge;  
myAge = 32;
```

Or, we can be more concise and just do it on one:

```
// Declare a string  
string countryName = "Netherlands";
```

In each case, we first write the data type, then the variable name, then use the equals sign = to assign the variable a value.

Once we've defined a variable, we can use them throughout our program. For example, here's a short program that prints a few math equations to the console:

```
int evenNumber = 22;  
int oddNumber = 45;  
Console.WriteLine(evenNumber + oddNumber); // Prints 67  
Console.WriteLine(oddNumber - evenNumber); // Prints 23
```

If we want to change the values, it's only necessary to change it in one place instead of everywhere it is used.

2.3 Creating Variables with Types Instructions

1.

To practice creating variables, we're going to write a program that prints information about a dog to the console. We'll be working with the types string, int, double, and bool.

First, create two string variables. The first one is called name and has the value "Shadow". The second one is called breed and has the value "Golden Retriever".

2.

Next, create a variable to hold the age. Name the variable age and store the value 5.

3.

Next, create a variable to hold the weight. Name the variable weight and store the value 65.22.

4.

Next, create a variable that can be either true or false. Name the variable spayed and set it to true.

5.

Use Console.WriteLine() to print each variable to the console.

Program.cs

```
using System;
```

```
namespace Form
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
{  
    // Create Variables  
  
    // Print variables to the console  
  
}  
}  
}
```

OUTPUT

Shadow

Golden Retriever

5

65.22

True

2.4 Handling Errors

When you program, you'll come across a lot of errors. And that's ok! And when you're first starting to learn a strongly-typed language, errors can be pretty common.

So, what happens if you forget to specify a data type for your variable?:

```
randomData = "asdf jskdf";  
Console.WriteLine(randomData);
```

C# will give an error because it doesn't want you to have random data being used in your application. The error will look something like this:

The name 'randomData' does not exist in the current context [CS0103:]

If you use the wrong type definition, like an int when it's supposed to be a double:

```
int score = 45.39;
```

You might see an error like this:

Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)

We also want to watch out for how we name our variables. It's good practice to use camelCase styling, and they should only contain underscores, letters, and digits.

```
string iAmAVariable;  
string i'mnot; // this will cause errors
```

There are also a few words that you can't use. These are known as *reserved keywords*. [Reserved keywords](#) are words that the language uses, so they already have specific definitions that shouldn't be re-written. If we use one of them as a variable name, we risk overwriting it and causing significant errors in our program. For example, we can't name a variable string because that word is reserved for defining data types.

Lastly, it's important to double-check spelling and punctuation! Don't forget to end each statement with a semicolon ;.

Luckily, many IDEs will point out these potential errors before we even run our code! But it's still good to be prepared to handle these errors if you should see them, including when you're coding on Codecademy.

2.4 Handling Errors Instructions

1.

The following code has a couple of bugs in it. Run the code and see what errors appear. Once you know the errors, try fixing it line by line.

Program.cs

```
using System;
```

```
namespace BugSquasher
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            int number = 38498.3222;
```

```
            dinosaur = "Barney";
```

```
            double lock = 293.000;
```

```
            bool is.yes = true;
```

```
            string band = "The Low Anthem"
```

```
        }
```

```
}  
}
```

OUTPUT

```
Program.cs(13,14): error CS1001: Identifier  
expected [/home/ccuser/workspace/csharp-data-  
types-variables-handling-errors-csharp/e7-  
workspace.csproj]  
Program.cs(13,14): error CS1002: ; expected  
[/home/ccuser/workspace/csharp-data-types-  
variables-handling-errors-csharp/e7-  
workspace.csproj]  
Program.cs(13,19): error CS1003: Syntax error,  
'(' expected [/home/ccuser/workspace/csharp-  
data-types-variables-handling-errors-csharp/e7-  
workspace.csproj]  
Program.cs(13,19): error CS1525: Invalid  
expression term '='  
[/home/ccuser/workspace/csharp-data-types-  
variables-handling-errors-csharp/e7-  
workspace.csproj]  
Program.cs(13,28): error CS1026: ) expected  
[/home/ccuser/workspace/csharp-data-types-  
variables-handling-errors-csharp/e7-  
workspace.csproj]
```

```
Program.cs(15,12): error CS1002: ; expected
[/home/ccuser/workspace/csharp-data-types-
variables-handling-errors-csharp/e7-
workspace.csproj]
Program.cs(15,12): error CS1525: Invalid
expression term 'is'
[/home/ccuser/workspace/csharp-data-types-
variables-handling-errors-csharp/e7-
workspace.csproj]
Program.cs(15,14): error CS1525: Invalid
expression term '.'
[/home/ccuser/workspace/csharp-data-types-
variables-handling-errors-csharp/e7-
workspace.csproj]
Program.cs(17,37): error CS1002: ; expected
[/home/ccuser/workspace/csharp-data-types-
variables-handling-errors-csharp/e7-
workspace.csproj]

The build failed. Fix the build errors and run
again.
```

2.5 Converting Data Types

Because variables have to be strictly typed, there may be some circumstances where we want to change the type of data a variable is storing. This strategy is known as *data type conversion*.

For example, let's try converting a double to an integer:

```
double myDouble = 3.2;
```

```
// Round myDouble to the nearest whole number  
int myInt = myDouble;
```

But if you tried to run this code, it wouldn't work. Sorry.

However, if you did the reverse and turned an int into a double:

```
int myInt = 3;  
// Turn it into a double  
double myDouble = myInt;
```

It's fine! Why is that?

C# checks to make sure that when we convert data types from one to another that we're not losing any data, because that could cause problems in our code.

Because of that, there are a couple different ways to do data type conversion:

- *implicit* conversion: happens automatically if no data will be lost in the conversion. That's why it's possible to convert an int (which can hold less data) to a double (which can hold more), but not the other way around.
- *explicit* conversion: requires a cast operator to convert a data type into another one. So if we do want to convert a double to an int, we could use the operator (int).

So, if we're to fix the error in our previous code, we'd rewrite it as follows:

```
double myDouble = 3.2;
```

```
// Round myDouble to the nearest whole number  
int myInt = (int)myDouble;
```

It's also possible to convert data types using built-in methods. For most data types, there is a `Convert.ToX()` method, like `Convert.ToString()` and `Convert.ToDouble()`. For a full list of `Convert` class built-in methods, check out the [Microsoft Documentation](#).

2.5 Converting Data Types Instructions

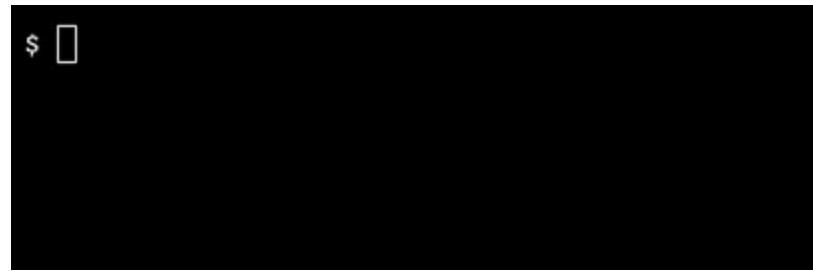
1.

One example of when we have to use conversions is when we ask a user to input a numerical value. Even if that value is an integer or a decimal, `Console.ReadLine()` will always return a string.

Let's write a program that asks a learner for their favorite number and see if we can *implicitly* convert their response to an int.

To start, below the `Console.Write()` statement, create an int variable named `faveNumber` and set it equal to `Console.ReadLine()`.

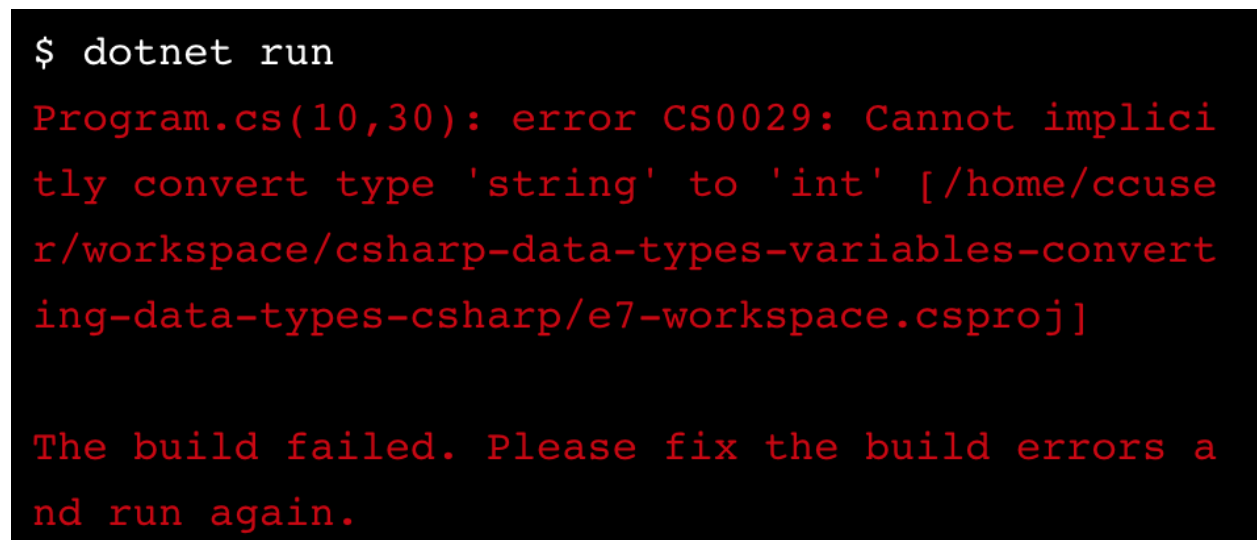
To run the program, press the **Run** button to save your work, then type `dotnet run` into the console.



```
$ 
```

2.

Hmm. That didn't work. Instead, we got the error message:



```
$ dotnet run
Program.cs(10,30): error CS0029: Cannot implicitly
convert type 'string' to 'int' [/home/ccuse
r/workspace/csharp-data-types-variables-converting-data-types-csharp/e7-workspace.csproj]

The build failed. Please fix the build errors a
nd run again.
```

Looks like we're going to have to cast their response as an int some other way!

Try *explicitly* casting the value of faveNumber as an int and rerun the program. What happens this time?

3.

If you tried dotnet run again, you'll see that (int) didn't work either. That's because it is not possible to explicitly convert a string into an int (or vice versa) in C#. This time, let's try using a built-in method to do the conversion.

Look at [this article on converting strings to int](#). It lists a few of the methods in the Convert class, including: Convert.ToInt32(). This method takes a string and outputs an integer. Let's try it!

Delete the explicit casting (int) from the code editor. Add the Convert.ToInt32() method so that it takes the user input as a string.

Run the code again. Did you run into any errors?

Program.cs

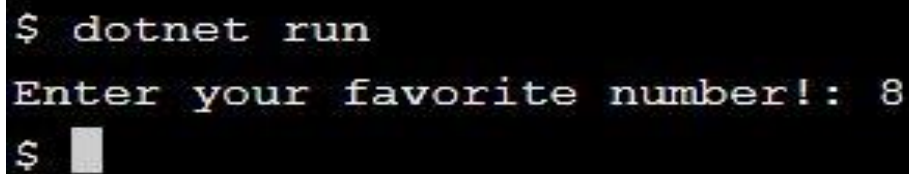
```
using System;

namespace FavoriteNumber
{
    class Program
    {
        static void Main(string[] args)
        {
            // Ask user for fave number
            Console.Write("Enter your favorite number! ");
```

```
// Turn that answer into an int
```

```
}  
}  
}
```

OUTPUT

A terminal window with a black background and white text. The first line shows the command '\$ dotnet run'. The second line shows the program's output: 'Enter your favorite number!: 8'. The third line shows the prompt '\$' followed by a small grey square cursor.

```
$ dotnet run  
Enter your favorite number!: 8  
$ █
```

2.6 Review

Congratulations! In this lesson, you learned about how data types and variables work in C#. Topics covered include:

- C# built-in data types, including int, double, char, string, and bool
- How to create, name, and use variables
- Common errors you might encounter
- Converting data types from one to another

For more information on using types and declaring variables, check out [Microsoft's C# documentation](#).

Want to keep track of the different data types? Download [this handy cheatsheet](#).

Now that you know a few things about variables and data types, try writing a program that:

- Converts a boolean to a string
- Changes a string to a list of chars
- Converts a data type we didn't cover to another data type!

2.6 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above!

Program.cs

```
using System;
```

```
namespace Review
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            /* use this space to write your own short program!
```

```
            Here's what you learned:
```


DATA TYPES: int, double, char, string, bool

VARIABLES: datatype variableName = value;

COMMON ERRORS: wrong type, wrong value, no semicolon

DATA TYPE CONVERSION: implicit, explicit, methods

Good luck! */

```
}  
}  
}
```

Concept Review

Variables and Types

A variable is a way to store data in the computer's memory to be used later in the program. C# is a type-safe language, meaning that when variables are declared it is necessary to define their data type.

Declaring the types of variables allows the compiler to stop the program from being run when variables are used incorrectly, i.e, an int being used when a string is needed or vice versa.

```
string foo = "Hello";
```

```
string bar = "How are you?";  
int x = 5;
```

```
Console.WriteLine(foo);  
// Prints: Hello
```

Math.Sqrt()

Math.Sqrt() is a Math class method which is used to calculate the square root of the specified value.

```
double x = 81;
```

```
Console.Write(Math.Sqrt(x));
```

```
// Prints: 9
```

Arithmetic Operators

Arithmetic operators are used to modify numerical values:

- + addition operator
- - subtraction operator
- * multiplication operator
- / division operator
- % modulo operator (returns the remainder)

```
int result;
```

```
result = 10 + 5; // 15
```

```
result = 10 - 5; // 5
```

```
result = 10 * 5; // 50
```

```
result = 10 / 5; // 2
```

```
result = 10 % 5; // 0
```

Unary Operator

Operators can be combined to create shorter statements and quickly modify existing variables. Two common examples:

- ++ operator increments a value.
- -- operator decrements a value.

```
int a = 10;
```

```
a++;
```

```
Console.WriteLine(a);
```

```
// Prints: 11
```

Math.Pow()

Math.Pow() is a Math class method that is used to raise a number to a specified power. It returns a number of double type.

```
double pow_ab = Math.Pow(6, 2);
```

```
Console.WriteLine(pow_ab);
```

```
// Prints: 36
```

.ToUpper() in C#

In C#, .ToUpper() is a string method that converts every character in a string to uppercase. If a character does not have an uppercase equivalent, it remains unchanged. For example, special symbols remain unchanged.

```
string str2 = "This is C# Program xsdd_ $#%";
```

```
// string converted to Upper case
```

```
string upperstr2 = str2.ToUpper();
```

```
//upperstr2 contains "THIS IS C# PROGRAM XSDD_ $#%"
```

IndexOf() in C#

In C#, the IndexOf() method is a string method used to find the index position of a specified character in a string. The method returns -1 if the character isn't found.

```
string str = "Divyesh";
```

```
// Finding the index of character
```

```
// which is present in string and
```

```
// this will show the value 5
```

```
int index1 = str.IndexOf('s');
```

```
Console.WriteLine("The Index Value of character 's' is " + index1);
```

```
//The Index Value of character 's' is 5
```

Bracket Notation

Strings contain characters. One way these char values can be accessed is with bracket notation. We can even store these chars in separate variables.

We access a specific character by using the square brackets on the string, putting the index position of the desired character between the brackets. For example, to get the first character, you can specify variable[0]. To get the last character, you can subtract one from the length of the string.

```
// Get values from this string.
```

```
string value = "Dot Net Perls";
```

```
//variable first contains letter D
```

```
char first = value[0];
```

```
//Second contains letter o
```

```
char second = value[1];
```

```
//last contains letter s
```

```
char last = value[value.Length - 1];
```

Escape Character Sequences in C#

In C#, an escape sequence refers to a combination of characters beginning with a back slash \ followed by letters or digits. It's used to make sure that the program reads certain characters as part of a string. For example, it can be used to include quotation marks within a string that you would like to print to console. Escape sequences can do other things using specific characters. \n is used to create a new line.

Substring() in C#

In C#, Substring() is a string method used to retrieve part of a string while keeping the original data intact. The substring that you retrieve can be stored in a variable for use elsewhere in your program.

```
string myString = "Divyesh";
```

```
string test1 = myString.Substring(2);
```

String Concatenation in C#

Concatenation is the process of appending one string to the end of another string. The simplest method of adding two strings in C# is using the + operator.

```
// Declare strings
string firstName = "Divyesh";
string lastName = "Goardnan";

// Concatenate two string variables
string name = firstName + " " + lastName;
Console.WriteLine(name);
//This code will output Divyesh Goardnan
```

.ToLower() in C#

In C#, .ToLower() is a string method that converts every character to lowercase. If a character does not have a lowercase equivalent, it remains unchanged. For example, special symbols remain unchanged.

```
string mixedCase = "This is a MIXED case string.";

// Call ToLower instance method, which returns a new copy.
string lower = mixedCase.ToLower();

//variable lower contains "this is a mixed case string."
```

String Length in C#

The string class has a Length property, which returns the number of characters in the string.

```
string a = "One example";  
Console.WriteLine("LENGTH: " + a.Length);  
// This code outputs 11
```

String Interpolation in C#

String interpolation provides a more readable and convenient syntax to create formatted strings. It allows us to insert variable values and expressions in the middle of a string so that we don't have to worry about punctuation or spaces.

```
int id = 100  
  
// We can use an expression with a string interpolation.  
string multipliedNumber = $"The multiplied ID is {id * 10}.";   
  
Console.WriteLine(multipliedNumber);  
// This code would output "The multiplied ID is 1000."
```

String New-Line

The character combination `\n` represents a newline character when inside a C# string.

For example passing "Hello\nWorld" to Console.WriteLine() would print Hello and World on separate lines in the console.


```
Console.WriteLine("Hello\nWorld");
```

```
// The console output will look like:
```

```
// Hello
```

```
// World
```

Console.ReadLine()

The `Console.ReadLine()` method is used to get user input. The user input can be stored in a variable. This method can also be used to prompt the user to press enter on the keyboard.

```
Console.WriteLine("Enter your name: ");
```

```
name = Console.ReadLine();
```

Comments

Comments are bits of text that are not executed. These lines can be used to leave notes and increase the readability of the program.

- Single line comments are created with two forward slashes `//`.
- Multi-line comments start with `/*` and end with `*/`. They are useful for commenting out large blocks of code.

```
// This is a single line comment
```

```
/* This is a multi-line comment
```

and continues until the end
of comment symbol is reached */

Console.WriteLine()

The Console.WriteLine() method is used to print text to the console. It can also be used to print other data types and values stored in variables.

```
Console.WriteLine("Hello, world!");
```

```
// Prints: Hello, world!
```

3. WORKING WITH NUMBERS

3.1 Introduction to Working with Numbers

No matter what you want to make in C#, you'll need numbers!

Art: what are the dimensions of your canvas?

Games: how fast can your player move?

Business: how much does your product cost?

In this lesson, we'll look at a few of the most commonly used numerical data types in C#. By the end of this lesson, you will be able to manipulate numerical data and write programs that perform calculations using arithmetic operators and built-in methods.

3.1 Introduction to Working with Numbers Instructions

Take a look at the C# program in the code editor window. What do you think it will do? Run the code and see if you're correct.

Program.cs

```
using System;
```

```
namespace BusinessSolution
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {
        // Calculating Net Income
        double revenue = 853023.45;
        double expenses = 438374.11;
        double netIncome = revenue - expenses;

        Console.WriteLine(netIncome);

        // Calculating a Break-Even Point
        double fixedCosts = 912849.30;
        double salesPrice = 29.99;
        double variableCostPerUnit = 17.65;

        double breakEvenVolume = fixedCosts / (salesPrice - variableCostPerUnit);
        Console.WriteLine(breakEvenVolume);
    }
}

```

OUTPUT

414649.34

73974.8217179903

3.2 Numerical Data Types

In C#, there are several ways of representing numerical data. Your usage of each will depend on your application. When choosing a data type, think about the following questions:

Do I need a whole number or do I need something that will represent a fraction, or a decimal? If I want to use a decimal, how precise do I need to be? Depending on your application, whether it's a hobby project or building a B2B financial services software, you'll need a different data type. Is performance a factor? Most times, choosing a data type that takes up less memory will result in faster applications.

Let's look at two data types that we can use to represent different numerical values:

Int

An *int* is a whole integer value, like 4, 100, or 2349. They're a good way to count units of things. For example, if we wanted to track the number of coin flips a user makes, we'd use an *int*. It doesn't make sense to have 0.5 coin flips!

To define a variable with the type *int*, you would write it as follows:

```
int variableName = 7;
```

Double and Decimal

If we need to use a decimal value, we have a few options: *float*, *double*, and *decimal*. These values are useful for anything that requires more precision than a whole number, like measuring the precise location of an object in 3D space.

A *double* is usually the best choice of the three because it is more precise than a *float*, but faster to process than a *decimal*. However, make sure to use a *decimal* for financial applications, since it is the most precise.

To define a variable with the type double, you would write it as follows:

```
double variableName = 39.76876;
```

To define a variable with the type decimal, you would write it as follows:

```
decimal variableName = 489872.76m;
```

Don't forget the m character after the number! This character tells C# that we're defining a decimal and not a double.

3.1 Numerical Data Types Instructions

1.

Several large pizza chains employ C# developers. Let's imagine that you work for the chain, Giant Brutus. Your boss gives you some data and wants you to enter it into a C# program.

The first value they give you is the number of pizza shops they own, which is 4332. Save this number to a variable named `pizzaShops`. Which data type should you use for the variable?

2.

The next value is the number of employees, which is 86,928. Save this number to the variable `totalEmployees`.

3.

The total revenue for a single Big Brutus franchise store is 390,819.28. Save this number to the variable `revenue`.

4.

Print the three variables to the console.

Program.cs

```
using System;
```

```
namespace Numbers
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Number of pizza shops
```

```
            // Number of employees
```

```
            // Revenue
```

```
            // Log the values to the console:
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

4332

86928

390819.28

3.3 Arithmetic Operators

So what can we do with numerical data? A first step is to write expressions using *arithmetic operators*.

Arithmetic operators include:

- addition +
- subtraction -
- multiplication *
- division /

We can use these symbols to perform operations on numbers and create new values.

```
int answer = 4 + 19;  
Console.Write(answer);
```

```
// prints 23
```

When using operators, it's important to pay attention to data types. If we use two integers, it will return an integer *every time*. However, if we combine an integer with a double, the answer will be a double. Let's look at the following example:


```
Console.WriteLine(5 / 3);  
Console.WriteLine(5 / 3.0);
```

```
// prints 1  
// prints 1.66667
```

The first operation that we log uses two ints. While 3 doesn't go into 5 evenly, we are still left with an int whole number. In the second operation, we use an int and a double, so the final result is a double.

C# follows [the order of operations](#). If we do $1 + 2 * 3$, should the answer be 9 or 7? C# follows a set of rules to determine which operations to perform first (the answer is 7). It's good practice to use parentheses to explicitly tell C# how to calculate these expressions.

Notice in the following example, even if the addition symbol appears like it should come first, the multiplication operation will happen first.

```
int answer = 8 + (9 * 3);  
Console.Write(answer);
```

```
// prints 35
```

3.3 Arithmetic Operators Instructions

1.

Did you know that your age would be different on another planet? Different planets orbit the sun at different rates, so 1 year on earth can be much shorter or much longer on another planet, depending on their position in the solar system.

Let's start by saving your age on Earth in a variable named `userAge`.

2.

We're going to write an equation that calculates your age on Jupiter.

To calculate a person's age on another planet, we need to know how long it takes each planet to orbit the Sun (relative to a year on Earth):

Jupiter takes **11.86** years

Create a variable named `jupiterYears` and save the amount of years it takes to orbit as their value.

3.

Next we'll write the calculations. The formula for calculating an age on another planet is: divide the user age in Earth years by the length of the other planet's year (in Earth years) to get your age in that planet's years.

Calculate your age in Jupiter years and save it to the variable `jupiterAge`.

4.

If we were to fly to Jupiter, [it could take as long as 2,242 \(Earth\) days or about 6.142466 years!](#)

Save the value 6.142466 to a variable named `journeyToJupiter`.

5.

When you reach Jupiter:

- What would be your age in Earth years? Save your answer to `newEarthAge`.
- What would be your age in Jupiter years? Save your answer to `newJupiterAge`.

6.

Log the ages on the different planets to the console so you can see your age on each planet currently, and after your journey to Jupiter.

Program.cs

using System;

namespace PlanetCalculations

{

class Program

{

static void Main(string[] args)

{

// Your Age

// Length of years on Jupiter (in Earth years)

// Age on Jupiter

// Time to Jupiter

// New Age on Earth

```
// New Age on Jupiter
```

```
// Log calculations to console
```

```
}  
}  
}
```

OUTPUT

```
2.52951096121417
```

```
36.142466
```

```
3.04742546374368
```

3.4 Operator Shortcuts

Often we need to update a variable in our program. We can do so by modifying that variable using an arithmetic expression, then re-saving it to the same variable name:

```
int apple = 0;  
apple = apple + 1;  
Console.Write(apple); // prints 1
```

Programmers are always looking for shortcuts. For instance, we can condense the same program above using the shorthand `++`. The combined addition signs represent the idea of *incrementing by one*. We can do the same with the subtraction symbol `--`.

```
// a shorter way to do the same thing
int apple = 0;
apple++;
Console.Write(apple); // prints 1
```

If we want the amount to increment by another value, say 3, we would do the following:

```
int apple = 0;
apple += 3; // is the same as apple = apple + 3
Console.Write(apple); // prints 3
```

Again, if we want to decrement, you would do `--3`.

Ultimately, programmers disagree on this, so everyone does it differently! On Codecademy, we prefer clarity over conciseness, so we'll use the long form (`apple = apple + 3`) except for incrementing by one (`apple++`).

3.4 Operator Shortcuts Instructions

1.

Ever heard of the phrase, “two steps forward, one step back?” It means that you can make progress in some task, but also might suffer some setbacks. It's also a great way to illustrate the concept of incrementing and decrementing!

Start by creating a variable named `steps`. Set it to 0.

2.

Two steps forward

Next, increment the value of `steps` by two and resave this new value to `steps`.

3.

One step back.

Now, decrement the variable steps by 1.

4.

Print the final value of steps to the console.

Program.cs

```
using System;
```

```
namespace MakingProgress
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // declare steps variable
```

```
            // Two steps forward
```

```
// One step back
```

```
// Print result to the console
```

```
    }  
  }  
}
```

OUTPUT

1

3.5 Modulo

One arithmetic operator that we haven't covered yet and may be less familiar is a *modulo*. A modulo returns a *remainder*, what is left over when we divide a number by another number.

$4 \% 3 = 1$

$4 \% 2 = 0$

The modulo is the same as the percentage symbol, but it's important to remember it's different meaning in this context.

Modulos are useful because they let us know if a number "fits" into a larger number, or if there will be a remainder. For example, how many eggs will be left over if I try and fit 56 eggs into crates of a dozen eggs?

```
int eggs = 56;
```

```
int crateAmount = 12;
```

```
int eggsLeftOver = eggs % crateAmount;  
Console.WriteLine(eggsLeftOver); // prints 8
```

It can also be used to check if a number is odd or even. If a number is even, taking its modulo with 2 it will return a 0 and if it is odd it will return a 1:

```
int myNum = 85939824;  
Console.WriteLine(myNum % 2); // prints 0, so number is even
```

3.5 Modulo Instructions

1.

You're teaching computer science in a classroom and need to break up your students into teams.

Start by creating a variable named `students` that has the value 18.

2.

You need to find a number that will go evenly into 18 (without a remainder) so that there are an even number of students. The groups should have more than 2 students in each group, but no more than 5.

Create a variable named `groupSize`. Enter a number between 3 and 5.

3.

Inside of a `Console.WriteLine()` statement, use the modulo operator to see if students will divide evenly into `groupSize`. If it does not, change the value of `groupSize` until it does.

```
using System;
```



```
namespace ClassTeams
{
    class Program
    {
        static void Main(string[] args)
        {
            // Number of students

            // Number of students in a group

            // Does groupSize go evenly into students?

        }
    }
}
```

OUTPUT

0

3.6 Built-In Methods

So how do we do more advanced mathematical operations? For example, how would we perform a square root on a number if the program doesn't recognize a square root symbol?

There are several built-in methods that we can use to manipulate numerical data and perform more complex mathematical calculations. Here are a few:

- `Math.Abs()`—will find the absolute value of a number. Example: `Math.Abs(-5)` returns 5.
- `Math.Sqrt()`—will find the square root of a number. Example: `Math.Sqrt(16)` returns 4.
- `Math.Floor()`—will round the given double or decimal down to the nearest whole number. Example: `Math.Floor(8.65)` returns 8.
- `Math.Min()`—returns the smaller of two numbers. Example: `Math.Min(39, 12)` returns 12.

3.6 Built-In Methods Instructions

1.

In this exercise, we're going to use built-in methods to determine which number is smaller between the square roots of two different numbers.

First, find the square root of `numberOne` and round the answer down so it doesn't have a decimal. Save this value to a new double variable `numberOneSqrt`.

2.

Do the same process to variable `numberTwo` and save this value to a new double variable `numberTwoSqrt`.

3.

Inside of a `Console.WriteLine()` statement, use a built-in method that returns the smallest of two numbers, using the values `numberOneSqrt` and `numberTwoSqrt`.

Which value gets printed to the console?

4.

Did NaN get printed to the console? NaN stands for “Not a Number” in C#. So what happened?

The built-in method `Math.Sqrt()` can only take a positive number as a value, but the value of `numberTwo` is negative. But we can fix it. The method `Math.Abs()` will find the [absolute value](#) of a number. A good reminder that when we use built-in methods, to check the documentation so we know how to use them!

Inside of the `Math.Sqrt()` method, add the `Math.Abs()` method, so it takes the variable `numberTwo` as a value. Re-run the code and see what gets printed to the console this time!

Program.cs

```
using System;
```

```
namespace LowestNumber
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Starting variables
```

```
            int numberOne = 12932;
```

```
int numberTwo = -2828472;

// Use built-in methods and save to variable

// Use built-in methods and save to variable

// Print the lowest number

}

}

}
```

OUTPUT

113

3.7 Using Documentation

We introduced you to a few common built-in methods, but there are many more out there! Now's a good time to practice your documentation search skills. Take a moment to look up the following built-in methods to understand their usage:

- `Math.Pow()`
- `Math.Max()`
- `Math.Ceiling()`

(If you're having trouble locating what you need, we recommend the [Microsoft .NET API documentation](#)).

3.7 Using Documentation Instructions

1.

In this exercise, you'll need to use the documentation linked above.

Inside of the first `Console.WriteLine()` command, raise `numberOne` to the `numberTwo` power.

2.

Inside of the second `Console.WriteLine()` command, round `numberOne` up.

3.

Inside of the third `Console.WriteLine()` command, find the larger number between `numberOne` and `numberTwo`.

Program.cs

```
using System;
```

```
namespace DocumentationHunt
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
{  
  
    double numberOne = 6.5;  
    double numberTwo = 4;  
  
    // Raise numberOne to the numberTwo power  
    Console.WriteLine();  
  
    // Round numberOne up  
    Console.WriteLine();  
  
    // Find the larger number between numberOne and numberTwo  
    Console.WriteLine();  
  
}  
}  
}
```

OUTPUT

1785.0625

7

6.5

3.8 Review

Great job! You just learned about numerical data types and how to work with numerical data in a few different ways:

- Use arithmetic operators to write expressions.
- Combine operators together to write more concise programs.
- Use the modulo operator (%) to find remainders.
- Use built-in methods to do more complex math.
- Use documentation to look new things up.

Now that you know a few ways of working with numbers, try writing some small programs that use what you learned in this lesson. Some ideas:

- Write a program that calculates compound interest
- Write a program that finds your age in dog years

What other program ideas can you come up with?

3.8 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above!

```
using System;
```

```
namespace Review
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
{
```

```
/* use this space to write your own short program!
```

```
Here's what you learned:
```

```
DATA TYPES: int, double
```

```
ARITHMETIC OPERATORS: +, -, *, /
```

```
INCREMENT/DECREMENT: ++, --
```

```
MODULO: %
```

```
BUILT-IN METHODS: Abs, Pow, Sqrt, Floor, Ceiling, Min, Max
```

```
Good luck! */
```

```
}
```

```
}
```

```
}
```


4. WORKING WITH TEXT

4.1 Introduction to Working with Text

Working with text is a fundamental part of writing programs. Whether it's to provide instructions to a user, gathering data like a name or address, or writing a new form of poetry, text enables us to bring human language into computational form.

In this lesson, we'll look at the two commonly used text data types in C#: `char` and `string`. By the end of this lesson, you will be able to manipulate textual data and write programs that take in user inputs and output customizable messages using variables, operators, and built-in methods.

4.1 Introduction to Working with Text Instructions

Take a look at the program on the right. What do you think the outcome will be? This program uses many of the techniques that you'll learn in this lesson!

Program.cs

```
using System;
```

```
namespace MadTeaParty
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {
        string drink = "wine";

        string madTeaParty = $"\"Have some {drink},\" the March Hare said in an
        encouraging tone. \nAlice looked all round the table, but there was nothing on it
        but tea.\n\"I don't see any {drink},\" she remarked.\n\"There isn't any,\" said the
        March Hare.";

        int storyLength = madTeaParty.Length;

        string toFind = "March Hare";

        string findLowerCase = toFind.ToLower();
        int findMarchHare = madTeaParty.IndexOf(toFind);

        Console.WriteLine(madTeaParty.Substring(findMarchHare));

        Console.WriteLine($"This scene is {storyLength} long.\n");

        Console.WriteLine($"The term we're looking for is {toFind} and is located at
        index {findMarchHare}.");

    }
}

```

OUTPUT

March Hare said in an encouraging tone.

Alice looked all round the table, but there was nothing on it but tea.

"I don't see any wine," she remarked.

"There isn't any," said the March Hare.

This scene is 211 long.

The term we're looking for is March Hare and is located at index 22.

4.2 Building Strings

A *string* is a group of characters surrounded by quotation marks, like "https://codecademy.com" or "To be or not to be." A string is just a collection of a smaller data type, *char*, which is a single character like "a" or "?".

To define a variable as a string, you write the data type, then the variable name. Then set it equal to the value, which is inside of quotation marks :

```
string variableName = "puppy";
```

The variable is named variableName, it's of type string, and its value is "puppy".

Escape Character Sequences

What happens when you need to include quotes in a string? You can use an escape sequence. An escape sequence places a backslash (\) before the inner quotation marks, so the program doesn't read them accidentally as the end of sequence.

```
string withoutSlash = "Ifemelu said, \"Hello!\"";
```

```
string withSlash = "Ifemelu said, \\\"Hello!\\\"";
```

We can use escape character sequences to create a newline. That means that when we print the string to the console, it will print that line below the rest. If printed on its own, it will create an empty line. To create a newline, use the character combination `\n`.

```
string newLine = "Ifemelu walked \n to the park.";
```

For more on escape sequences, check out [Microsoft's Documentation](#).

4.2 Building Strings Instructions

1.

Create a string variable named `firstSentence` and save the following string to it:

It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife.

2.

Create a second string variable named `firstSpeech` and save the following string to it:

"My dear Mr. Bennet," said his lady to him one day, "have you heard that Netherfield Park is let at last?"

Make sure to use escape characters when necessary.

3.

Using `Console.WriteLine()`, print each variable to the console. In between them, print a newline to the console.

Program.cs

```
using System;
```

```
namespace PrideAndPrejudice
{
    class Program
    {
        static void Main(string[] args)
        {
            // First string variable

            // Second string variable

            // Print variable and newline

        }
    }
}
```

OUTPUT

It is a truth universally acknowledged, that a single man in possession of a good fortune, must be in want of a wife.

"My dear Mr. Bennet," said his lady to him one day, "have you heard that Netherfield Park is let at last?"

4.3 String Concatenation

Often, we want to combine strings together, or combine strings with a value that we've saved to a variable.

A common way to do is by using *string concatenation*. String concatenation is when we combine strings using the addition symbol (+), literally adding one string to another.

```
string yourFaveMusician = "David Bowie";  
string myFaveMusician = "Solange";
```

```
Console.WriteLine("Your favorite musician is " + yourFaveMusician + " and mine is "  
+ myFaveMusician + ".");
```

We write the first part of our string, end it with a double quote ("), include an addition symbol and then put our variable. However, there are a couple of things to pay attention to:

- If we want to concatenate a string with something that is another data type, C# will implicitly convert that value to a string.
- Make sure that you include spaces and proper punctuation so that when it prints out, your variable strings aren't squished between the rest of the statement. Notice how we have to create a one character string at the end to include a period.

4.3 String Concatenation Instructions

1.

Let's use string concatenation to tell a story. Come up with a beginning, middle, and end to a story and save them to the corresponding variables beginning, middle, and end.

2.

Concatenate your three strings and store the result in variable named story.

3.

Print story to the console.

Are the spacing and punctuation correct? If not, go back and change the story strings so that it prints correctly.

Program.cs

```
using System;
```

```
namespace StoryTime
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Declare the variables
```

```
            // Concatenate the string and the variables
```

```
            // Print the story to the console
```

```
}  
  
}  
  
}
```

OUTPUT

Once upon a time, the kid climbed a tree. Everyone lived happily ever after.

4.4 String Interpolation

While string concatenation is easy to read, it can get annoying to write. Another option is *string interpolation*. String interpolation was introduced in C# 6 and it enables us to insert our variable values and expressions in the middle of a string, without having to worry about spaces and punctuation.

```
string yourFaveMusician = "David Bowie";  
string myFaveMusician = "Solange";
```

```
Console.WriteLine($"Your favorite musician is {yourFaveMusician} and mine is  
{myFaveMusician}.");
```

Notice how we just have one string, which we append with the dollar sign symbol \$. Make sure there isn't a space in between the \$ and the starting quotation mark ". Whenever we need to insert a variable, we surround it with braces {}. For more on string interpolation, [check out Microsoft's documentation](#).

In older documentation, you may come across a slightly similar syntax. This style is known as string formatting or composite formatting. Since the introduction of string interpolation, it is rarely used. But in case you come across it, [check out Microsoft's documentation](#).

4.4 String Interpolation Instructions

1.

Let's revisit our story, but use string interpolation. This time we've provided you with the variables.

Using the provided beginning, middle, and end variables, interpolate the story variable.

2.

Print the story to the console. Reflect on using string concatenation and string interpolation to achieve the same outcome. Which one did you prefer?

Program.cs

```
using System;
```

```
namespace StoryTime
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Declare the variables
```

```
            string beginning = "Once upon a time,";
```

```
            string middle = "The kid climbed a tree";
```

```
            string end = "Everyone lived happily ever after.";
```

```
            // Interpolate the string and the variables
```

```
// Print the story to the console

    }

}

}
```

OUTPUT

Once upon a time, it was a beautiful day. The kid climbed a tree and the birds flew through the sky. Everyone lived happily ever after.

4.5 Get Info About Strings

In addition to containing the value of a piece of text, strings also contain information about themselves. It can be useful to know these properties when working with strings. There are several built-in .NET methods that we can use to get more information about strings.

Length

Since strings are composed of a set of characters, we can find out how many characters exist in a string with the `.Length` method. A common example is if we're building a form and need to make sure a user submission doesn't exceed a certain character length.

```
string userTweet = Console.ReadLine();
```

```
userTweet.Length; // returns the length of the tweet
```

We append the `.Length` property to a string that we have, such as a user's tweet.

IndexOf

We can also find the position of a specific character or substring using `.IndexOf()`. This method is useful for searching to see if something exists in a string.

If it does exist within a string, the method will return the *position* of the search term in the larger string. Each character in a string has a unique position, like an address. Positions starts at 0 and increment by 1.

```
string word = "radio";
```

```
word.IndexOf("a"); // returns 1
```

Since positioning starts at 0, the second thing in the string will return a 1. If it doesn't exist in the string the method will return a -1. If we pass it an empty string, it will return 0. If it occurs more than once, it will return the first instance.

4.5 Get Info About Strings Instructions

1.

You've been asked to build a program that verifies some information about a piece of data.

First, check the length of password and save the result to the variable `passwordLength`.

2.

Next, let's see if this password contains any special characters, like an exclamation point (!). Save the result to the variable passwordCheck. Run the program to see the results printed to the console.

Program.cs

```
using System;
```

```
namespace PasswordCheck
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Create password
```

```
            string password = "a92301j2add";
```

```
            // Get password length
```

```
            // Check if password uses symbol
```

```
            // Print results
```

```
            Console.WriteLine($"The user password is {password}. Its length is {password.Length} and it receives a {passwordCheck} check.");
```

```
}  
  
}  
  
}
```

OUTPUT

The user password is a92301j2add. Its length is 11 and it receives a -1 check.

4.6 Get Parts of Strings

We can also use built-in .NET methods to grab parts of strings or specific characters in a string.

Substring

.Substring() grabs part of a string using the specified character position, continues until the end of the string, and returns a new string. .IndexOf() is usually used first to get the specific character position.

```
string plantName = "Cactaceae, Cactus";  
int charPosition = plantName.IndexOf("Cactus"); // returns 11  
string commonName = plantName.Substring(charPosition); // returns Cactus
```

.Substring() is useful if we only want to use part of a string but keep the original data intact. So in this instance, we want to keep the string plantName, but just grab the "Cactus" portion of the string. We use .IndexOf() to find where "Cactus" starts, then use .Substring() with the position information to save "Cactus" to the new variable commonName.

We can also pass `.Substring()` a second argument, which will determine the number of characters in the resulting substring. For example, the following code shows how we can use `.Substring()` with two arguments to specify the length of our substring:

```
string name = "Codecademy";  
int start = 2;  
int length = 6;  
string substringName = name.Substring(start, length); // returns 'decade'
```

Bracket Notation

Bracket notation is a style of syntax that uses brackets `[]` and an integer value to identify a particular value in a collection. In this case, we can use it to find a specific character in a string.

```
string plantName = "Cactaceae, Cactus";  
int charPosition = plantName.IndexOf("u"); // returns 15  
char u = plantName[charPosition]; // returns u
```

Similar to the example above, we first use `.IndexOf()` to grab the character position, which in this case is 15. We then take the string value and append it with a set of brackets `[]` and place the `charPosition` value inside the brackets.

4.6 Get Parts of Strings Instructions

1.

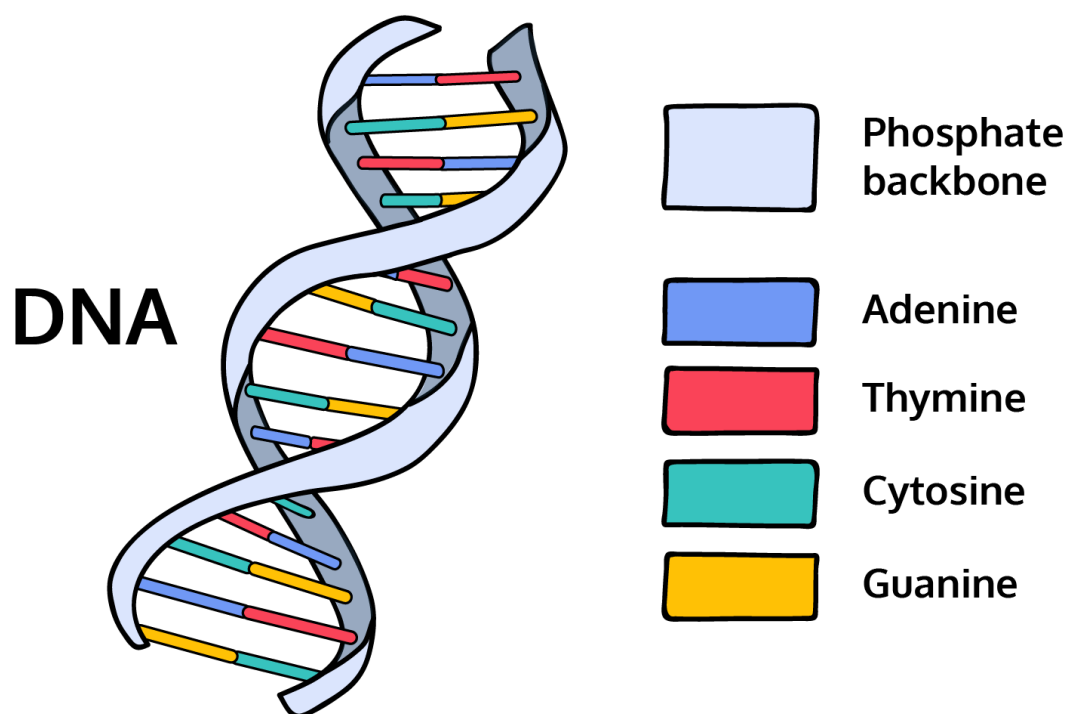
In **program.cs**, you are given a string defined as `startStrand`. Your goal is to find the ending condition for a DNA strand, "TGA", and then print out the substring of `startStrand` that starts from the first character and goes through "TGA". This will output a valid DNA strand.

You don't need much understanding of biology to follow these directions, but if you'd like a more in-depth explanation/background, click the dropdown below:

More info on DNA

DNA is composed of a series of nucleotides abbreviated as:

- A: Adenine
- C: Cytosine
- G: Guanine
- T: Thymine



So a strand of DNA could look something like ACGAATTCCG.

A protein has the following qualities (along with a few others):

1. It begins with a “start codon”: ATG.
2. It ends with a “stop codon”: TGA.

First, use `IndexOf()` to find the location of "TGA" in `startStrand`. Save this as a variable called `tga`.

2.

Create two variables called `startPoint` and `length`. `startPoint` should be set to 0, and `length` should be set to three more than the value of `tga`.

`length` should be set to three more than the value of `tga` for two reasons:

- `IndexOf()` returns the starting point of `tga` in `startStrand` (where the character "T" is). To capture "GA" as well, add two.
- When using `.Substring()` with *two* arguments, `.Substring()` captures the amount of characters indicated by the second argument. Because strings in C# are zero-indexed, you will need to *add one more* in order to properly capture the last character. In this example, the "A" in "TGA" falls at index 8 but is the 9th character — if we only set the `length` to 8, the final "A" would not be included. Therefore, instead of defining `length` as `tga + 2`, you should define it as `tga + 3`.

3.

Now use `Substring()` to retrieve the substring of `startStrand` that starts from `startPoint` and has a `length` of `length`. Save this to a variable called `dna`.

Print out this string using `Console.WriteLine(dna)`.

4.

You are worried there might have been a mutation within this strand of DNA! Take a look at the fourth character to determine whether or not the strand has mutated:

- If it is equal to G, there has been no mutation.
- If it is equal to C or A, there has been a mutation.

Use bracket notation to grab the fourth character in `dna` and show the result in the console. Has there been a mutation?

Program.cs

using System;

namespace DNA

{

class Program

{

static void Main(string[] args)

{

// dna strand

string startStrand = "ATGCGATGAGCTTAC";

// find location of "tga"

// start point and stop point variables

// define final strand

// DNA mutation search

}

```
}  
}
```

OUTPUT

ATGCGATGA

C

4.7 Manipulate Strings

There are also built-in .NET methods that we can use to manipulate text data. Using these methods on a string doesn't change the string itself, but creates an entirely new one.

ToUpper, ToLower

We can quickly change the case of our strings using the methods `.ToUpper()` and `.ToLower()`. These methods are useful if we want to make our text look like an e.e. cumming's poem or make it appear like we forgot to turn off our caps lock key.

```
string shouting = "I'm not shouting, you're shouting".ToUpper();  
Console.WriteLine(shouting);  
// prints I'M NOT SHOUTING, YOU'RE SHOUTING.
```

4.7 Manipulate Strings Instructions

1.

You're writing a movie script but are having trouble following the style guide. You need to transform this sentence so that the first two words (Close on) are in all caps and the rest is lower case.

Close on a portrait of the HANDSOME PRINCE -- as the BEAST'S giant paw slashes it.

Your assistant already started on this program and separated out the necessary strings and saved them in the variables `cameraDirections` and `sceneDescription`. You need to finish the program.

First, make all of the letters in `cameraDirections` uppercase and re-save them to the variable `cameraDirections`.

2.

Now make all of the letters in `sceneDescription` lowercase and resave them to the variable `sceneDescription`. Print your results to the console.

Program.cs

```
using System;
```

```
namespace MovieScript
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Script line
```

```
            string script = "Close on a portrait of the HANDSOME PRINCE -- as the BEAST'S  
giant paw slashes it.";
```

```

// Get camera directions
int charPosition = script.IndexOf("Close");
int length = "Close on".Length;
string cameraDirections = script.Substring(charPosition, length);

// Get scene description
charPosition = script.IndexOf("a portrait");
string sceneDescription = script.Substring(charPosition);

// Make camera directions uppercase

// Make scene description lowercase

// Print results

}
}
}

```

OUTPUT

CLOSE ON a portrait of the handsome prince -- as the beast's giant paw slashes it.

4.8 Review

Great job! You just learned about how to work with textual data in a few different ways:

- How to save char and string values to a variable.
- Use the addition symbol (+) to concatenate strings.
- Interpolate strings for easier string construction.
- Find information about a string using .Length and .IndexOf().
- Grab characters and parts of strings using bracket notation and .Substring().
- Use built-in methods such as .ToUpper() and .ToLower() to manipulate strings.

Now use what you've learned to write a short program! Some ideas:

- Write a program that randomly converts part of a text to uppercase and lowercase to look like [randomcase](#).
- Write a program that takes in a series of random words to construct an automated poem, in the style of e.e. cummings.

4.8 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above!

```
using System;
```

```
namespace Review
```

```
{
```

```
class Program
{
    static void Main(string[] args)
    {
        /* use this space to write your own short program!
        Here's what you learned:

        DATA TYPES: char, string
        STRING INTERPOLATION: $"blah blah"
        STRING INFO: .Length, .IndexOf()
        PARTS OF STRINGS: bracketNotation[], .Substring()
        STRING MANIPULATION: .ToUpper(), .ToLower()

        Good luck! */

    }
}
}
```

4. Quiz

1. Which of these will run without errors:
 - a) ToLower(username)

- b) `String.ToLower(username)`
- c) `username.ToLower()`

2. Access the first letter of the string initials

Initials _____;

- `[i]`
- `[0]`
- `.0`
- `{0}`

Click or drag and drop to fill in the blank

3. Given this variable definition, which statement would throw an error?

`double num = 3;`

- a) `num + 2;`
- b) `num % 2;`
- c) `Math.Sqrt(num);`
- d) `num.ToUpper();`

4. Write a program that uses string interpolation and the variables `treeName` and `treefamily` to print: "The maple tree is part of the deciduous family."

```
string treeName = "maple";
```

```
string treeFamily = "deciduous";
```

```
Console.WriteLine(_____ "The _____ tree is part of the _____ family");
```

- `[treeName]`

- \$
- +
- {treeFamily}
- &
- %
- {treeName}
- [treeFamily]

Click or drag and drop to fill in the blank

5. Set the variable toy equal to 1. Increase the value of toy by 4 so that toy now equals 5.

toy _____ 1;

toy _____ 4;

- --
- =
- ++
- +=
- -=

Click or drag and drop to fill in the blank

6. Which line of code can we use to convert the int variable speed to a double variable named specificSpeed?

int speed = 65;

- a) double specificSpeed = (int)speed
- b) double specificSpeed = speed

c) `double specificSpeed = Convert.ToString(speed)`

7. In C#, a computer checks for type errors:

- a) After a program runs.
- b) While a program runs.
- c) Before a program runs.

8. If the output of this code is 2, which operator would appear between the operands 20 and 6?

`Console.WriteLine(20 ____ 6);`

- `%`
- `*`
- `+`
- `-`
- `/`

Click or drag and drop to fill in the blank

9. Set the variable `gallery` equal to "Gagosian" and the variable `price` to 12.3

____ `gallery = "Gagosian";`

____ `price = 12.3;`

- `char`
- `string`
- `int`
- `double`

- bool

Click or drag and drop to fill in the blank

10. Set the variable hours equal to 10. Decrease the value of hours by 2 so that hours now equals 8.

hours ____ 10;

hours ____ 2;

- --
- =
- ++
- -=
- +=

Click or drag and drop to fill in the blank

11. What will the output of this program be?

```
int num = 1;  
string population = "all";
```

```
Console.WriteLine(num + " ring to rule them " + population);
```

- a) It will throw an error.
- b) 1 ring to rule them all
- c) num + " ring to rule them " + population

12. In C#, what type of conversion would you use to change a double into an int?

- a) Implicit
- b) Explicit

Project-2

Mad Libs

In C#, variables and string interpolation allow us to transform a piece of text by swapping out different pieces of information.

In this project, we'll use C# to write a Mad Libs word game! Mad Libs are short stories with blanks for the player to fill in that represent different parts of speech. The end result is a really hilarious and strange story.

Here's an example of the "Roses are Red" poem changed into a Mad Lib:

Roses are _____
Adjective

_____ **are blue**
Noun

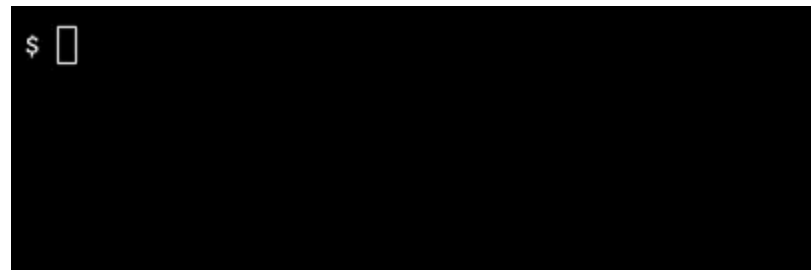
_____ **is** _____
Noun Adjective

And so are you!

Mad Libs require: A short story with blank spaces (asking for different types of words). Words from the player to fill in those blanks.

For this project, we have provided the story, but it will be up to you to complete the following: Prompt the user for inputs. Print the story with the inputs in the right places.

It's important to note that for this project, you should test your app periodically — when you hit save, your app will not run! To run your app, enter `dotnet run` into the terminal.



Let's begin!

TASKS

MadLibs Set Up

1.

Begin by completing the multi-line comment that describes this program.

2.

Inform the user that the program is running. You need to be constantly thinking from the users' point of view – they are the ones who run your program. Make sure that your program is easy for others to understand!

Before the string story, print a message to let the user know that Mad Libs has started.

3.

Give your story a title. Change the value of the variable title to a title that you like.

4.

Take a look at the string variable named story. It is set equal to a string that contains our story.

Creating the Variables

5.

Now we're going to start creating the variables that we will use in our story. Make sure to declare *all* of your variables *above* the variable story, otherwise you won't be able to use them in the story!

The story that we have provided is going to need a main character. Ask the user to input a name, and store the user's input in a variable with a string type:

```
Console.Write("Enter a name: ");  
string name = Console.ReadLine();
```

Note: It's good practice to use short, but descriptive variable names.

6.

You will need to ask the user for three [adjectives](#). An adjective is a word that describes a noun, like a color ('blue'), or feeling ('silly'), texture ('soft').

Ask the user for input three separate times. Store each adjective that the user inputs into variables with descriptive names, like you did when you asked the user for a name.

7.

It's a good practice to save and run your code every few steps to make sure there are no bugs. In the terminal, type the following command and press Enter on your keyboard: `dotnet run`

8.

Moving on! You'll also need to ask the user for one [verb](#). A verb is a word that represents an action, like 'sit', 'eat', 'sleep'

Ask the user to input a verb and store their response in a variable.

9.

The story also needs two [nouns](#). A noun is a person ('girl'), place ('cabin'), or thing ('toaster').

Ask the user to input two nouns. Store each noun in its own variable.

10.

This is where the story can get really fun and weird. Ask the user to input one of each of the following:

- An animal
- A food
- A fruit
- A superhero
- A country
- A dessert
- A year

Make sure to save the inputs into different variables. Run your code again to make sure you have everything in place before the next task!

Using String Interpolation

11.

At this point, we have all the words needed for the story. Great job!

The next step is to insert all of the user's inputs into the blank spaces of the story using string interpolation.

Put a \$ in front of the first quotation mark " in story and replace each underscore _ with a set of empty braces {}.

12.

Write the names of each variable inside of the brackets. Here's the order that they should appear in:

1. Name
2. First adjective
3. Second adjective
4. Animal
5. Food
6. Verb
7. First noun
8. Fruit
9. Third adjective
10. Name
11. Superhero
12. Name
13. Country
14. Name
15. Dessert
16. Name

17.Year

18.Second noun

Complete and Run the Program

13.

Write a line of code that prints out the completed story to the terminal.

14.

Let's run the program!

Save the program. Then, in the terminal, type the following command and press Enter:

dotnet run

Program.cs

using System;

namespace MadLibs

{

class Program

{

static void Main(string[] args)

{

/*

This program ...

Author: ...


```
*/
```

```
// Let the user know that the program is starting:
```

```
// Give the Mad Lib a title:
```

```
string title = "TITLE";
```

```
Console.WriteLine(title);
```

```
// Define user input and variables:
```

```
// The template for the story:
```

```
string story = "This morning _ woke up feeling _. 'It is going to be a _ day!'
Outside, a bunch of _s were protesting to keep _ in stores. They began to _ to the
rhythm of the _, which made all the _s very _. Concerned, _ texted _, who flew _
to _ and dropped _ in a puddle of frozen _. _ woke up in the year _, in a world
where _s ruled the world.";
```

```
// Print the story:
```

```
}
```

```
}
```

}

OUTPUT

```
$ dotnet run
Mad Libs has started!
A Day in the Life
Enter a name: vishva
Enter an adjective: beautiful
Enter another adjective: rested
Enter an animal: chickens
Enter a food: eggs
Enter a verb: cluck
Enter a noun: drums
Enter a fruit: excited
Enter another adjective: john
Enter a superhero: superman
Enter a country: france
Enter a dessert: icecream
Enter a year: 2080
Enter another noun: aliens
```

```
This morning vishva woke up feeling beautiful. 'It is going to be a rested day!' Outside, a bunch of chickens were protesting to keep eggs in stores. They began to cluck to the rhythm of the drums, which made all the exciteds very john. Concerned, vishva texted superman, who flew vishva to france and dropped vishva in a puddle of frozen icecream. vishva woke up in the year 2080, in a world where aliens ruled the world.
```

Project-3

Money Maker

You have three types of coins:

- A bronze coin is worth 1 cent
- A silver coin is worth 5 cents
- A gold coin is worth 10 cents

What is the minimum number of coins that equals 98 cents?

It's a hard question to answer in your head, but it's a fun problem to solve with programming. In this project you'll use C# to build a Money Maker: a program in which a user enters an amount and gets the minimum number of coins that equal that value.

For example, 16 cents could be:

- 16 bronze coins,
- 3 silver coins and 1 bronze coin, or
- 1 gold coin, 1 silver coin, 1 bronze coin

We'd like the last option because it uses the fewest coins.

This project will get you comfortable with division (/), rounding down (`Math.Floor()`), and modulo (%): You can find how many coins "fit" into an amount by dividing the amount by the coin value, rounding down, and finding the remainder. Let's walk through an example:

1. The user enters 15 cents. A gold coin is 10 cents, so 1.5 gold coins fit into the total.

```
goldCoinsIn15Cents = 15 / 10; // equals 1.5
```

2. But we can't divide coins in half, so instead, we round down to the nearest whole number:

```
actualGoldCoins = Math.Floor( 15 / 10 ); // equals 1
```

3. You can find the remainder using modulo:

```
double remainder = 15 % 10; // equals 5
```

4. Using the remainder, repeat the process with the smaller coins.

TASKS

Capture Input

1.

Run the code once to understand the starting code. Use `dotnet run` in the terminal.

2.

Use `Console.WriteLine()` and `Console.ReadLine()` to ask the user for the amount to convert and capture the value in a variable.

3.

Convert the captured value (a string) to a number.

You can convert a value using `Convert.ToDouble()`.

4.

Before we get to calculating, let the user know what you are about to do. For example, if the user entered 16, the program should write to the console:

16 cents is equal to...

5.

Run the code to check your work so far. At this point you should see something like:

Welcome to Money Maker!

Enter an amount to convert to coins:

Once you enter a number — say 16 — and press Enter you should see something like:

16 cents is equal to...

Convert To Coins

6.

To convert to coins, we need to know the value of each type of coin! Define two variables that hold those values.

- A gold coin is worth 10 cents
- A silver coin is worth 5 cents

Don't worry about bronze coins for now.

7.

To find the maximum number of gold coins that “fit” into the amount (e.g. one gold coin fits into 11 cents):

1. Divide the amount by the value of a gold coin
2. Round down to the nearest integer
3. Store the result in a double variable called goldCoins

8.

Use the modulo (%) operator to find the remaining amount and store it in a double variable.

9.

Find the maximum number of silver coins that “fit” into the remainder:

1. Divide the remainder by the value of a silver coin

2. Round down to the nearest integer
3. Store the result in a double variable called silverCoins

10.

Use the modulo (%) operator to find the remaining amount and store it in the existing remainder variable.

```
remainder = remainder % silverValue;
```

11.

Print out all of the coins! If your input was 16, your output should look something like:

16 cents is equal to...

Gold coins: 1

Silver coins: 1

Bronze coins: 1

We didn't need to do extra work to find the number of bronze coins because it's just the remainder!

12.

Test your application to make sure it works! Some test cases are provided in the hint.

Here are two optional challenges:

- The program doesn't work if the user enters a decimal, like 16.2 cents. Using type conversion or a Math method, round down their input to the nearest whole cent.
- Use another currency that has different coin amounts. For example, the US uses 1, 5, 10, and 25 cent coins called pennies, nickels, dimes, and quarters, respectively.

Program.cs

```
using System;
```

```
namespace MoneyMaker
```

```
{
```

```
    class MainClass
```

```
    {
```

```
        public static void Main(string[] args)
```

```
        {
```

```
            Console.WriteLine("Welcome to Money Maker!");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

```
$ dotnet run
Welcome to Money Maker!
Enter an amount to convert to coins:
16
16 cents is equal to...
Gold coins: 1
Silver coins: 1
Bronze coins: 1
$ █
```

5. UNDERSTANDING LOGIC IN C#

5.1 Introduction to Logic in C#

Computers are constantly checking the state of something. Is this program running or not? Does this variable exist or not? Is this value equal to that value?

These yes or no questions demonstrate that a number of *binaries* —a relationship between two entities—exist. Computer programs essentially function off of binaries: true and false, yes and no, ones and zeroes, on and off.

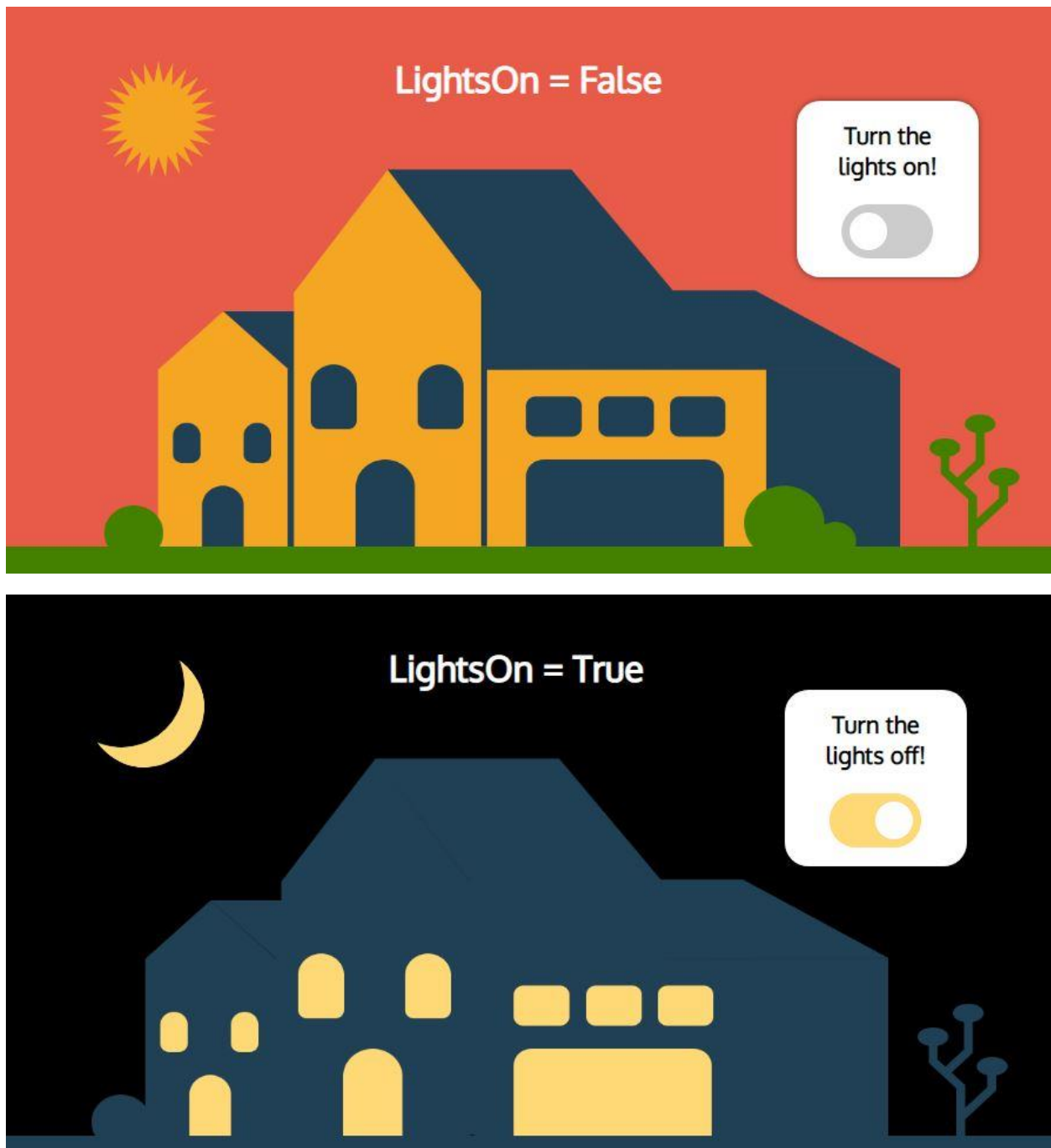
Distinguishing between binaries is the foundation of *Boolean logic*. Boolean logic is based on the idea that all values are either *true* or *false*. Logic is important to computer science because it is an early attempt at translating the human capacity for reason to computers. If a computer can use logic to reason about certain situations, it can use that rationale to make decisions.

In this lesson, we'll explore how Boolean logic works in C# and show you how you can begin to implement boolean data types and expressions in your own programs.

5.1 Introduction to Logic in C# Instructions

Programmers first were able to illustrate the idea of binary logic through turning on and off electric circuits, where on equaled true and off equaled false. In fact, the relationship between electricity and logic is the basis for digitization - how we store information with electrical circuits!

Flip the switch to see the scene change from day to night and watch the lights turn on and off. Notice how the variable `LightsOn` stays the same, but its value changes from true to false.



5.2 Boolean Data Types

In C#, we can represent Boolean values using the `bool` data type. Booleans, unlike numbers or strings, only have two values: `true` and `false`.

To define a variable as a boolean, you define the data type as bool. Then write the variable name and set it equal to the value, either true or false:

```
bool variableName = true;
```

The variable is named variableName, its of type bool, and its value is true.

While we use the words true and false to represent boolean values, it's important to remember that they are different from the strings "true" and "false".

5.2 Boolean Data Types Instructions

1.

Time to practice writing variables that use a bool data type!

Answer these True/False questions by saving your answer to the specified variables.

True or False: The number 500 is greater than 24.

Save your answer to a variable named answerOne.

2.

True or False: "coffee" contains the letter "a".

Save your answer to a variable named answerTwo.

Program.cs

```
using System;
```

```
namespace BooleanDataTypes
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {

    }
}
}

```

5.3 Comparison Operators

When writing a program, we often need to check if a value is correct or compare two values. Comparison operators allow us to compare values and evaluate their relationship. Rather than evaluating to an integer, they evaluate to boolean values. Expressions that evaluate to boolean values are known as boolean expressions.

Comparison operators include:

- *Equals* ==: returns true if the value to the left is equal to the value to the right.
- *Inequality operator* !=: returns true if the two values are not equal.
- *Less than* <: returns true if the value to the left is less than the value to the right.
- *Greater than* >: returns true if the value to the left is more than the value to the right.
- *Less than or equal to* <=: returns true if the value to the left is less than or equal to the value on the right.

- *Greater than or equal to >=*: returns true if the value to the left is more than or equal to the value to the right.

Here's what a boolean expression using comparison operators can look like:

```
bool answer = 3 < 75;  
Console.WriteLine(answer); // prints True
```

In this example, we use the less than < comparison operator to compare the values 3 and 75, then save the expression to a variable named answer with a bool data type. If we were to print the value of answer to the console, it would print out True, since the number 3 is less than the number 75.

In addition to comparing integers, we can also compare variables, strings, and even boolean values:

```
bool answer = (true == false);  
Console.WriteLine(answer); //prints False
```

Here, we use the equals comparison operator to see if the Boolean value true is equal to false. This time, the expression evaluates to false. We'll look more into comparing boolean values in the next exercise.

5.3 Comparison Operators Instructions

1.

You are driving to your family's house for a holiday and want to see if you'll get there before dinner. Dinner will begin at 6:00 PM and the house is 95 miles away. If you leave at 2:00PM and drive an average of 30 miles per hour, will you get there early (before 6:00pm)?

Create a double variable named timeToDinner and save the difference in hours between 2:00pm and 6:00pm.

2.

Save the value 95 to a double variable named distance.

Save the value 30 to a double variable named rate.

3.

We can calculate how long it will take us by dividing the distance variable by the rate variable.

Write out the expression and save it to the variable tripDuration.

4.

Create a bool variable named answer. Save the appropriate comparison that checks if tripDuration is *less than or equal to* timeToDinner.

5.

Print answer to the console. Will you arrive before dinner begins?

Program.cs

```
using System;
```

```
namespace ComparisonOperators
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
}  
}  
}
```

OUTPUT

True

5.4 Truth Table

We can also use operators that use Boolean values as inputs and output. Logical operators, also known as Boolean operators, can be used to create Boolean expressions.

Logical operators include:

- AND &&: Both expressions are evaluated and will return True *only* if both expressions evaluate to True. Otherwise, it will return False.
- OR ||: Both expressions are evaluated and will return True if *at least one* of the expressions evaluates to True. Otherwise, it will return False.
- NOT !: An expression, no matter its logical value, evaluates to its opposite. What is True becomes False and what is False becomes True.

Let's look at some examples:

```
bool andExample = ((4 > 1) && (2 < 7));  
// (True AND True) evaluates to True
```

In this example, both comparisons evaluate to True, so the overall expression evaluates to True.

```
bool orExample = ((8 > 6) || (3 > 6));  
// (True OR False) evaluates to True
```

Here, only one comparison evaluates to True and the other evaluates to False, so the expression evaluates to True.

```
bool notExample = !(1 < 3);  
// NOT (True) evaluates to False
```

In this last example, the comparison evaluates to True, so the expression evaluates to False.

A common way to visualize these relationships is using a diagram known as a *truth table*. Truth tables allow us to quickly see what the outcome is for different relationships between Boolean values. Handling two Boolean values is simple, but longer expressions can be very complex. It's crucial that you are familiar with these fundamentals before going ahead.

5.4 Truth Table Instructions

We can visualize the relationship of Boolean values and logical operators using a diagram known as a *truth table*. Truth tables allow us to quickly see what the outcome is for different relationships between boolean values.

For example, what is the result of False OR True?

Boolean Operators								
AND			OR			NOT		
A	B	A AND B	A	B	A OR B	A	NOT A	
True	True	True	True	True	True	True	False	
True	False	False	True	False	True	False	True	
False	True	False	False	True	True			
False	False	False	False	False	False			

5.5 Logical Operators

As we saw in the truth table, a Boolean expression that uses logical operators can be as simple as evaluating two boolean values:

```
bool answer = true && false; // evaluates to False
```

In this case, we're saying that answer is equal to the evaluation of true AND false. According to the truth table, answer will return False.

But more often, Boolean expressions are extremely complex. Rather than determining if one relationship is true or false, we can evaluate several expressions by connecting them with logical operators and then determining the validity of the overall expression.

```
bool answer = (9 < 3) || (100 < 45); // evaluates to False
```

```
bool another = ((3439 > 40) && (1 < 3)) || answer; // evaluates to True
```

We can use logical values to start chaining logical statements. Let's start by finding the value of answer.

First, the computer will evaluate each comparison statement:

- `bool answer = (9 < 3) || (100 < 45)`

Both of these statements evaluate to false:

- `bool answer = false || false`

Since both statements evaluate to false and we're using an OR operator, the overall expression will return false:

- `bool answer = false`

Now we can start evaluating the value of another. Again, we'll start by evaluating the comparison statements:

- `bool another = ((3439 > 40) && (1 < 3)) || answer`

Both statements evaluate to true:

- `bool another = (true && true) || answer`

Since both statements evaluate to true and we're using an AND operator, the overall expression returns true:

- `bool another = true || answer`

Since we already know that `answer` is false and we're evaluating it with a true value using an OR operator `another`, it will return true:

- `bool another = true.`

5.5 Logical Operators Instructions

1.

You and your friend are planning a trip together and are trying to decide where to go. You each have specific criteria that it needs to fulfill. You want it to have a beach and a city.

Create a bool variable named `yourNeeds`. Write a logical comparison that captures your criteria.

2.

Your friend wants a beach or hiking.

Create a bool variable named `friendNeeds`. Write a logical comparison that captures their criteria.

3.

Your current pick is Barcelona, which is a city that has a beach. Will both you and your friend be happy?

Write a logical comparison that compares `yourNeeds` and `friendNeeds` and save it to `tripDecision`.

Print `tripDecision` to the console. Will you agree on Barcelona? You should only go if it satisfies both your needs and your friend's needs.

Program.cs

```
using System;
```

```
namespace LogicalOperators
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            bool beach = true;
```

```
            bool hiking = false;
```

```
            bool city = true;
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

True

5.6 Review

Great job! You just learned about logic and boolean values, including:

- How to define variables with a bool data type
- How to use comparison operators with different data types to return boolean values
- What a truth table is and how to read one
- How to use logical operators to compare boolean values and expressions

Now that you know a few things about boolean data types, comparison operators and logical operators, try writing a program that:

- Checks a password if it has uppercase letters and doesn't include symbols
- Tells you if you should watch a TV show if it has your favorite actor or is your favorite drama

5.6 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above!

Program.cs

```
using System;
```

```
namespace Review
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            /* use this space to write your own short program!
```

Here's what you learned:

BOOL TYPE: `bool variableName`

COMPARISON OPERATORS: `==, <, >, <=, >=`

LOGICAL OPERATORS: `&&, ||, !`

Good luck! `*/`

```
}  
  
}  
  
}
```

6. CONDITIONAL STATEMENTS

6.1 Introduction to Conditional Statements

We make decisions all the time in our life based on different conditions. Are you going to drink tea or coffee? Study history or biology? Buy a new shirt or save your money?

We can program computers to make decisions based on different conditions. We can specify to the computer the *order* in which it should execute certain instructions, or that it should only execute *certain* instructions in specific cases. That means that depending on the conditions, the instructions that our program executes can change.

The order that computer programs execute a set of instructions is known as *control flow*. We can use different *control structures* to alter the flow of our program. Control structures let us handle different situations that might arise and make our programs more flexible. In C#, these type of statements are known as conditional or [selection statements](#).

Boolean logic and conditional statements go hand in hand. A computer will determine if a condition is true or false and execute a set of instructions accordingly. Make sure that you're comfortable with Boolean values, comparison operators, and logical operators!

We'll look at a couple of different structures that use Boolean logic to control the flow of our programs, including:

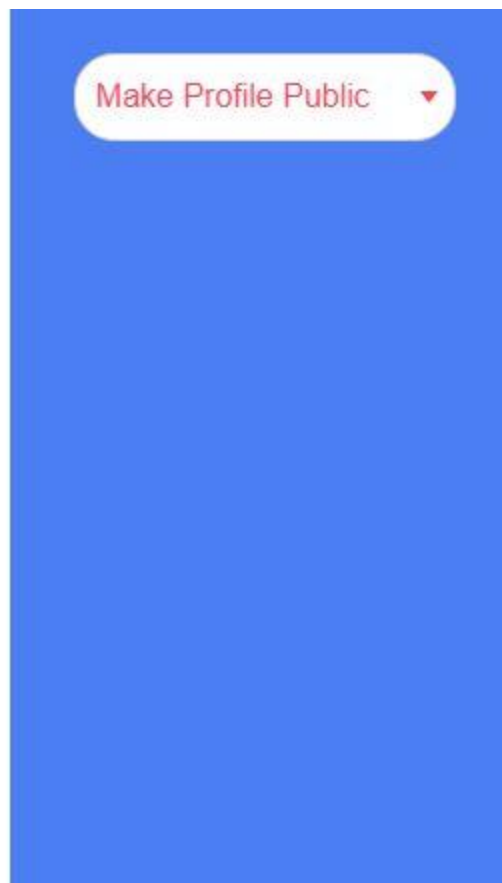
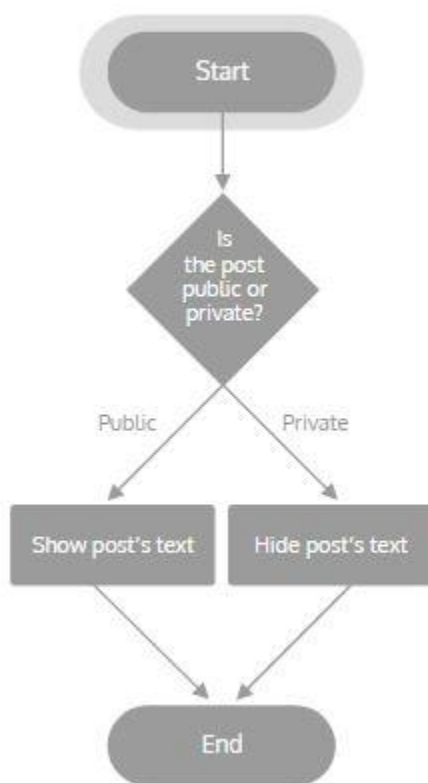
- If statements
- If...Else... statements
- Else if statements
- Switch statements

- Ternary Operators

6.1 Introduction to Conditional Statements Instructions

Conditional control structures, or just conditionals, allow programs to do different things in different scenarios. As you can see, they follow a logic similar to how humans think, making it easy to write clear code while still handling complex processes.

Select “Make Profile Public”, then click through each step of the conditional structure. Select “Make Profile Private”, then click through each step of the conditional structure. What’s the control flow for each?



6.2 If Statements

Conditional statements are the most commonly used control structures in programming. They rely on the computer being able to reason whether conditions are true or false.

The most basic conditional statement is an *if statement*. An *if statement* executes a block of code if specified condition is true.

In C#, we write an if statement using the following syntax:

```
string color = "blue";

if (color == "blue")
{
    // this code block will execute only if the value of color is
    // equivalent to "blue"
    Console.WriteLine("color is blue");
}
```

In this example, our if conditional statement checks to see if the value of the variable color is equivalent to the string "blue". Since it is, it will execute the code in the code block and print the string "color is blue" to the console. If color equals another value or was null, the program skips over the block and moves on to the next instruction.

When writing an if statement, make sure to pay attention to:

- Parentheses: we place the boolean expression that the if statement is evaluating in parentheses ().
- Braces: after the boolean expression, we write a set of braces {}. Write the code that will execute if the boolean expression evaluates to true inside these braces.
- Indentation: while whitespace won't impact our program, it is convention to indent the code inside the braces by two spaces.

6.2 If Statements Instructions

1.

You take your laundry to the laundromat once you have exactly three pairs of socks or less left. Write an if statement to check whether or not it's time to take your laundry in. Leave the code block empty for now.

2.

If the condition is true, have it print Time to do laundry! to the console.

3.

Now try changing the value of socks to 6. What happens this time?

Program.cs

```
using System;
```

```
namespace IfStatement
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            int socks = 3;
```



```
}  
  
}  
  
}
```

6.3 If...Else... Statements

What if we want another set of instructions to execute if the condition is false? An else clause can be added to an if statement to provide code that will only be executed if the if condition is false.

In C#, we write an if..else... statement using the following syntax:

```
string color = "red";  
  
if (color == "blue")  
{  
    // this code block will execute only if the value of color is  
    // equivalent to "blue"  
    Console.WriteLine("color is blue");  
}  
else  
{  
    // this code block will execute if the value of color is  
    // NOT equivalent to "blue"  
    Console.WriteLine("color is NOT blue");  
}
```

In this example, we're still checking to see if color equals "blue". However, this time we also added a second code block that will execute if the result of (color == "blue") is false. Since color equals "red" this time, it will skip the first code block and execute the second code block, before moving on with the rest of the program.

When writing an if...else... statement, make sure to pay attention to:

- else follows if: In an if...else... statement, the else statement and its corresponding code block still need to follow the if statement and code block.
- Number of code blocks: Make sure that if you include an else statement, that you include a code block with it.

6.3 If...Else... Statements Instructions

1.

You're deciding where to go for lunch. If the line isn't long at SaladMart (10 people or less) AND the weather is nice, you'll go there.

Write an if statement where if the condition is true, it prints out SaladMart.

2.

However, if the line is long and the weather is bad, you'll go to Soup N Sandwich. Next, add an else statement that prints out Soup N Sandwich.

3.

Now try changing the value of people to 12 and weather to bad and see what gets printed to the console.

Program.cs

```
using System;
```

```
namespace IfElseStatement
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {
        int people = 10;
        string weather = "nice";

    }
}

```

OUTPUT

Soup N Sandwich

6.4 Else If Statements

What if we want to handle multiple conditions and have a different thing happen each time? Conditional statements can be chained by combining if and else statements into else if. After an initial if statement, one or more else if blocks can check additional conditions. An optional else block can be added at the end to catch cases that do not match any of the conditions.

In C#, we write an if..else if... statement using the following syntax:

```

string color = "red";

if (color == "blue")
{
    // this code block will execute only if the value of color is
    // equivalent to "blue"
}

```

```

    Console.WriteLine("color is blue");
}
else if (color == "red")
{
    // this code block will execute if the value of color is
    // equivalent to "red"
    Console.WriteLine("color is NOT blue");
}
else // this is optional
{
    // this code block will execute if the value of color is
    // NOT equivalent to "blue" OR "red"
    Console.WriteLine("color is NOT blue OR red");
}

```

In this example, the program checks to see if color equals "blue" OR "red". Depending on what color is equivalent to, it will execute the corresponding code block. If it isn't equal to either of those colors, it will execute the else block before moving on with the rest of the program.

When using else if statement, make sure to pay attention to:

- Each else if statement gets its own condition: make sure to specify the condition an else if is evaluating. Just like an if statement, this condition goes in parentheses and if true, will execute what is in the code block.
- else follows else if: If you choose to include an else statement (it's optional), make sure it comes after any else if statements you might have.

6.4 Else If Statements Instructions

1.

You're having board game night with your friends, but you're not sure how many people will show up. You want to write a program that prints out what game to play depending on the number of guests:

If 4 or more people show up, you'll play Catan. If 1 to 3 people show up, you'll play Innovation. If no one shows up, you'll play Solitaire.

First, write the conditional statement for if you have at least 4 friends show up. If the condition is true, have it print out Catan.

2.

Next, modify the statement with another condition that outputs the game Innovation if at least 1 person shows up.

3.

Lastly, if no one shows up, have the program print Solitaire to the console.

4.

Now try changing the value of guests to 0 and see what gets printed to the console.

Program.cs

```
using System;
```

```
namespace ElselfStatement
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            int guests = 3;
```

```
}  
  
}  
  
}
```

OUTPUT

Solitaire

6.5 Switch Statements

Using multiple else if statements can get unwieldy pretty quickly. Imagine writing an else if statement for every possible number of guests. And you invited 20 people. You have to write a lot of repetitive code, which is hard to read and prone to errors.

If it's necessary to evaluate several conditions with their own unique output, a *switch statement* is the way to go. Switch statements allow for compact control flow structures by evaluating a single expression and executing code blocks based on a matched case.

In C#, we write a switch statement using the following syntax:

```
string color;  
  
switch (color)  
{  
    case "blue":  
        // execute if the value of color is "blue"  
        Console.WriteLine("color is blue");  
        break;
```

```

case "red":
    // execute if the value of color is "red"
    Console.WriteLine("color is red");
    break;
case "green":
    // execute if the value of color is "green"
    Console.WriteLine("color is green");
    break;
default:
    // execute if none of the above conditions are met
    break;
}

```

In this example, the program checks to see what the value of color equals. If it matches any of the specified cases, it will execute the code associated with that case. In C#, the *break* keyword allows programs to exit a block when a specific condition is met. If none of the conditions are met, the code inside the default case will run.

When using a switch statement, make sure to pay attention to:

- Cases: rather than writing out each condition, if we're evaluating one value we use *cases* to specify different potential values.
- Braces: rather than each case having its own code block, the entire statement lives within one set of braces {}.
- Colons: to distinguish between different cases, we state the case value, followed by a colon :. The code that should execute if that case is met follows.
- Break: Each case code needs to end with a break keyword.
- Default: Every switch statement needs a default case.

6.5 Switch Statements Instructions

1.

You want to build a simple movie recommender that gives the top movie in a particular genre.

First, create a string variable named `genre` and save the value "Horror" to it.

2.

Create a switch statement using `genre`. Don't add any cases to the code block yet.

3.

Next, add the following movie genres as cases to the switch statement. Make sure to also add a default case. Add a break statement to each case.

Genres:

- Drama
- Comedy
- Adventure
- Horror
- Science Fiction

4.

Next, add `Console.WriteLine()` statements to each case in the switch statement so that the program prints out different movie titles based on the selected genre. For the default case, print "No movie found".

Take a look at the following table to see the [top movie for five different genres](#):

Genre	Movie
Drama	Citizen Kane
Comedy	Duck Soup

Genre	Movie
Adventure	King Kong
Horror	Psycho
Science Fiction	2001: A Space Odyssey

5.

Let's turn this into something a user can make use of. Swap out "Horror" for `Console.ReadLine()` to get the user's response and save it to genre. Before that, add a `Console.WriteLine()` that prompts the user to pick a genre.

Type `dotnet run` into the terminal to see the program in action.

Program.cs

```
using System;
```

```
namespace SwitchStatement
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

```
$ dotnet run
Choose a movie genre:
Comedy
Duck Soup
```

6.6 Ternary Operators

The *ternary operator* allows for a compact syntax in the case of binary decisions. Like an if...else statement, it evaluates a single condition and executes one expression if the condition is true and the second expression otherwise.

Here's an example of a ternary operator in action:

```
string color = "blue";
string result = (color == "blue") ? "blue" : "NOT blue";
```

```
Console.WriteLine(result);
```

In this example, we create a variable result and save the outcome of the ternary operator expression. The expression starts with the Boolean expressions (color == "blue"), followed by the ternary operator ?, then the two possible outcomes, separated by a colon :. The first outcome, "blue" will be saved to result if the Boolean expression evaluates to true, otherwise it will store the second outcome.

Ternary operators can also be chained, like else if statements. But careful! Since the entire expression exists on one line, it can quickly become unreadable.

When using ternary operators, make sure to pay attention to:

- Parentheses: we place the boolean expression that the statement is evaluating in parentheses ().

- The ? operator: make sure this comes after the statement and before the outcomes.
- Colon: This separates the two possible outcomes.

6.6 Ternary Operators Instructions

1.

You're growing peppers and wrote a program that lets you know if it's time to pick them. If a pepper is at least 3.5 inches, it's time to be picked. If it's not ready, the program should tell you to "wait a little longer".

Start by creating a string variable named message. Save the comparison statement that checks to see if the pepperLength is 3.5 inches or more.

Note: This will throw an error, since we have not completed our statement.

2.

Next, write out your ternary operation. If a pepper is ready to be picked (3.5 inches or longer) your program should set message to "ready!" If it's not ready it should set message to "wait a little longer".

3.

Print the value of message to the console.

```
using System;
```

```
namespace TernaryOperator
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {
        int pepperLength = 4;

    }
}

```

OUTPUT

ready!

6.7 Review

Great job! You just learned about how to create programs that use control flow. Here's a few of the things we covered:

- Using if, else if, and else keywords to write conditional statements
- Writing switch statements for situations where there are many conditions
- Using ternary operators for shorter conditional statements

Now that you know a few things about control flow, try writing a program that:

- Has a user guess a random number between 1-10 and lets them know if they got it correct, are too low, or are too high
- Asks users to select their favorite fast food and tells them what type of cat they are (or basically, any kind of BuzzFeed style quiz)

- Checks if it's your birthday. If it is, it will print out a celebratory ASCII banner and if not, it will tell you how many days until your birthday.

6.7 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above! This is a regular console, so make sure to press **Run** to save your code, then type `dotnet run` in the console to see it in action.

Program.cs

```
using System;
```

```
namespace Review
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            /* use this space to write your own short program!
```

```
            Here's what you learned:
```

```
            CONDITIONALS: if, if...else, else if
```

```
            SWITCH STATEMENT: switch (condition)
```

```
            TERNARY OPERATOR: (condition) ? true : false
```

```
Good luck! */
```

```
}  
}  
}
```

Concept Review

Boolean Expressions

A *boolean expression* is any expression that evaluates to, or returns, a boolean value.

```
// These expressions all evaluate to a boolean value.
```

```
// Therefore their values can be stored in boolean variables.
```

```
bool a = (2 > 1);
```

```
bool b = a && true;
```

```
bool c = !false || (7 < 8);
```

Boolean Type

The `bool` data type can be either `true` or `false` and is based on the concept that the validity of all logical statements must be either `true` or `false`.

Booleans encode the science of logic into computers, allowing for logical reasoning in programs. In a broad sense, the computer can encode the truthfulness or falseness of certain statements, and based on that information, completely alter the behavior of the program.

```
bool skyIsBlue = true;
```

```
bool penguinsCanFly = false;
```

```
Console.WriteLine($"True or false, is the sky blue? {skyIsBlue}.");
```

```
// This simple program illustrates how booleans are declared. However, the real  
power of booleans requires additional programming constructs such as  
conditionals.
```

Logical Operators

Logical operators receive boolean expressions as input and return a boolean value.

The `&&` operator takes two boolean expressions and returns true only if they both evaluate to true.

The `||` operator takes two boolean expressions and returns true if either one evaluates to true.

The `!` operator takes one boolean expression and returns the opposite value.

```
// These variables equal true.
```

```
bool a = true && true;
```

```
bool b = false || true;
```

```
bool c = !false;
```

```
// These variables equal false.
```

```
bool d = true && false;
```

```
bool e = false || false;
```

```
bool f = !true;
```

Comparison Operators

A *comparison operator*, as the name implies, compares two expressions and returns either true or false depending on the result of the comparison. For example, if we compared two int values, we could test to see if one number is greater than the other, or if both numbers are equal. Similarly, we can test one string for equality against another string.

```
int x = 5;
```

```
Console.WriteLine(x < 6); // Prints "True" because 5 is less than 6.
```

```
Console.WriteLine(x > 8); // Prints "False" because 5 is not greater than 8.
```

```
string foo = "foo";
```

```
Console.WriteLine(foo == "bar"); // Prints "False" because "foo" does not equal "bar".
```

Truth Tables

A *truth table* is a way to visualize boolean logic. Since booleans only have two possible values, that means that we can compactly list out in a table all the possible input and output pairs for unary and binary boolean operators.

The image below gives the *truth tables* for the *AND*, *OR*, and *NOT* operators. For each row, the last column represents the output given that the other columns were fed as input to the corresponding operator.

Boolean Operators

AND			OR			NOT	
A	B	A AND B	A	B	A OR B	A	NOT A
True	True	True	True	True	True	True	False
True	False	False	True	False	True	False	True
False	True	False	False	True	True		
False	False	False	False	False	False		

Conditional Control

Conditional statements or *conditional* control structures allow a program to have different behaviors depending on certain *conditions* being met.

Intuitively, this mimics the way humans make simple decisions and act upon them. For example, reasoning about whether to go outside might look like:

- Condition: *Is it raining outside?*
 - If it is raining outside, then *bring an umbrella*.
 - Otherwise, *do not bring an umbrella*.

We could keep adding clauses to make our reasoning more sophisticated, such as “If it is sunny, then wear sunscreen”.

Control Flow

In programming, *control flow* is the order in which statements and instructions are executed. Programmers are able to change a program's *control flow* using *control structures* such as conditionals.

Being able to alter a program's *control flow* is powerful, as it lets us adapt a running program's behavior depending on the state of the program. For example, suppose a user is using a banking application and wants to withdraw \$500. We certainly want the application to behave differently depending on whether the user has \$20 or \$1000 in their bank account!

If Statements

In C#, an *if statement* executes a block of code based on whether or not the *boolean expression* provided in the parentheses is true or false.

If the expression is true then the block of code inside the braces, {}, is executed. Otherwise, the block is skipped over.

```
if (true) {  
    // This code is executed.  
    Console.WriteLine("Hello User!");  
}  
  
if (false) {  
    // This code is skipped.  
    Console.WriteLine("This won't be seen :(");  
}
```

Break Keyword

One of the uses of the break keyword in C# is to exit out of switch/case blocks and resume program execution after the switch code block. In C#, each case code

block inside a switch statement needs to be exited with the break keyword (or some other jump statement), otherwise the program will not compile. It should be called once all of the instructions specific to that particular case have been executed.

```
string color = "blue";
```

```
switch (color) {  
    case "red":  
        Console.WriteLine("I don't like that color.");  
        break;  
    case "blue":  
        Console.WriteLine("I like that color.");  
        break;  
    default:  
        Console.WriteLine("I feel ambivalent about that color.");  
        break;  
}  
  
// Regardless of where the break statement is in the above switch statement,  
// breaking will resume the program execution here.  
Console.WriteLine("- Steve");
```

Ternary Operator

In C#, the *ternary operator* is a special syntax of the form: condition ? expression1 : expression2.

It takes one boolean condition and two expressions as inputs. Unlike an if statement, the *ternary operator* is an expression itself. It evaluates to either

its first input expression or its second input expression depending on whether the condition is true or false, respectively.

```
bool isRaining = true;

// This sets umbrellaOrNot to "Umbrella" if isRaining is true,
// and "No Umbrella" if isRaining is false.

string umbrellaOrNot = isRaining ? "Umbrella" : "No Umbrella";

// "Umbrella"

Console.WriteLine(umbrellaOrNot);
```

Else Clause

An else followed by braces, {}, containing a code block, is called an else clause. else clauses must always be preceded by an if statement.

The block inside the braces will only run if the expression in the accompanying if condition is false. It is useful for writing code that runs *only if* the code inside the if statement is not executed.

```
if (true) {

    // This block will run.

    Console.WriteLine("Seen!");

} else {

    // This will not run.

    Console.WriteLine("Not seen!");

}

if (false) {
```

```
// Conversely, this will not run.  
Console.WriteLine("Not seen!");  
} else {  
    // Instead, this will run.  
    Console.WriteLine("Seen!");  
}
```

If and Else If

A common pattern when writing multiple if and else statements is to have an else block that contains another nested if statement, which can contain another else, etc. A better way to express this pattern in C# is with else if statements. The first condition that evaluates to true will run its associated code block. If none are true, then the optional else block will run if it exists.

```
int x = 100, y = 80;
```

```
if (x > y)  
{  
    Console.WriteLine("x is greater than y");  
}  
else if (x < y)  
{  
    Console.WriteLine("x is less than y");  
}  
else  
{
```

```
Console.WriteLine("x is equal to y");  
}
```

Switch Statements

A switch statement is a control flow structure that evaluates one expression and decides which code block to run by trying to match the result of the expression to each case. In general, a code block is executed when the value given for a case equals the evaluated expression, i.e, when == between the two values returns true. switch statements are often used to replace if else structures when all conditions test for equality on one value.

// The expression to match goes in parentheses.

```
switch (fruit) {  
    case "Banana":  
        // If fruit == "Banana", this block will run.  
        Console.WriteLine("Peel first.");  
        break;  
    case "Durian":  
        Console.WriteLine("Strong smell.");  
        break;  
    default:  
        // The default block will catch expressions that did not match any above.  
        Console.WriteLine("Nothing to say.");  
        break;  
}
```

// The switch statement above is equivalent to this:

```
if (fruit == "Banana") {  
    Console.WriteLine("Peel first.");  
} else if (fruit == "Durian") {  
    Console.WriteLine("Strong smell.");  
} else {  
    Console.WriteLine("Nothing to say.");  
}
```

6. Quiz

1. Use the ternary conditional operator to complete this statement:

num > 6 _____ "big!" _____ "small!";

- ;
- !
- ?
- :
- ==

Click or drag and drop to fill in the blank

2. What will be printed to the console when this code is executed?

```
bool hasAnger = true;  
string expr;
```

```
if (hasAnger)
```

```

{
    expr = "<( ` ^')>";
}
else
{
    expr = "~\_(`ツ)\_/~";
}

```

Console.WriteLine(expr);

- a) true
- b) <(` ^')>
- c) ~_(`ツ)_/~
- d) expr

3. Complete this statement such that it evaluates to true.

int bottles = 3;

bottles _____ 3

- ==
- >
- !=
- <

Click or drag and drop to fill in the blank

4. What will be printed to the console when this code is executed?

int x = -100;


```

if (x == 0)
{
    Console.WriteLine("x is equal to zero.");
}
else if (x > 0)
{
    Console.WriteLine("x is greater than zero.");
}
else
{
    Console.WriteLine("x is less than zero.");
}

```

- a) x is less than zero.
- b) x is greater than zero.
- c) x is equal to zero.
- d) Nothing will be printed.

5. What's wrong with this code?

```

int num = 2;

switch (num)
{
    case 1:
        Console.WriteLine("One for the money");
    case 2:
        Console.WriteLine("Two for the show");
    default:
        Console.WriteLine("Let's go!");
}

```

- a) num should be type string.
- b) There are no switch statements in C#.

- c) There are not enough cases.
- d) There are no break statements.

6. What possible values can a bool variable take?

- a) on and off
- b) true and false
- c) "true" and "false"
- d) right and wrong

7. What will happen when this code is executed?

```
int x = 50;
```

```
if (x == 0)
{
    Console.WriteLine("x is equal to zero.");
}
else if x > 0
{
    Console.WriteLine("x is greater than zero.");
}
else
{
    Console.WriteLine("x is less than zero.");
}
```

- a) "x is equal to zero." is printed to the console.
- b) There will be an error.
- c) "x is less than zero." is printed to the console.
- d) "x is greater than zero." is printed to the console.

8. Complete this statement such that it evaluates to true

```
bool isItRaining = true;  
bool isItSunny = false;
```

```
(____) && (30 > 28);
```

- false
- isItRaining
- isItSunny
- 28 > 30

Click or drag and drop to fill in the blank

9. Complete this statement such that it evaluates to false.

True _____ false

- ||
- >=
- !
- &&

Click or drag and drop to fill in the blank

Project-4

Password Checker

A computer can run thousands of operations per minute. If someone wanted to steal your information, they use each operation to make a guess at your password. With one high-performance computer and the right software, [a 6 letter single-case password can be broken in one day](#).

Apart from quitting the internet entirely, what can we do to protect ourselves? We can increase our password's strength: make it longer and more complex.

In this project you'll make a program that measures the strength of any given password based on the following criteria. A strong password:

- is at least 8 characters long
- has lowercase letters
- has uppercase letters
- has numerical digits
- has symbols, like #, ?, !

The program will ask the user to input a password, and using conditional logic and control flow, it will rate the password.

If you plan to do this project on your own computer, outside of Codecademy, make sure to copy the **Tools.cs** file in addition to **Program.cs**.

TASKS

Setup

1.

Let's start by defining all of the standards for a password.

- `minLength` — a number with the minimum length
- `uppercase` — a string with all uppercase letters
- `lowercase` — a string with all lowercase letters
- `digits` — a string with all 10 digits

- `specialChars` — a string made of special characters

2.

Now we'll capture input from the user. Ask the user to enter a password and capture their input in a variable.

Test

3.

We'll score the user's password: one point for each standard satisfied. Define a variable `score` to hold their score and set it to 0.

4.

If the password is greater than or equal to the minimum length, add a point to the score. Use an if statement.

5.

If the password contains uppercase letters, add a point.

We've provided a custom tool that checks for certain characters in a string, called `Tools.Contains()`.

Call it like this:

```
Tools.Contains(target, list);
```

`target` would be the password and `list` would be a string of characters you'd like to check for. If it finds a match, it returns `true`. If it does not, it returns `false`.

For example:

```
Tools.Contains("password", "abc"); // true
```

```
Tools.Contains("password", "123"); // false
```

6.

If the password contains lowercase letters, add a point.

7.

If the password contains digits, add a point.

8.

If the password contains special characters, add a point.

9.

Before we move on, make sure everything is working so far.

- After the if statements, print the score to the console
- Save your work
- In the console, enter `dotnet run` and press Enter
- Try the examples in the hint!

Tell the User

10.

Now that we've determined the password's score, we need to let the user know how they did.

Use a switch statement to log different messages for different scores:

- For scores of 4 and 5, the password is extremely strong
- For scores of 3, the password is strong
- For scores of 2, the password is medium

- For scores of 1, the password is weak
- If none of those match, the password doesn't meet any of the standards

11.

Test your program with a few passwords! Here are some examples to use:

- word scores a 1. Weak.
- woRD scores a 2. Medium.
- 1woRD scores a 3. Strong.
- 2woRDsss scores a 4. Extremely strong!
- 2woRDsss! scores a 5. Extremely strong again!
- " " scores a 0. It's a bunch of spaces...It doesn't meet any of the standards.

If you want an extra challenge, try adding these requirements:

- If the password is password or 1234, give it a score of 0.
- If you're familiar with [regular expressions](#), use them instead of Tools.Contains().

Program.cs

```
using System;
```

```
namespace PasswordChecker
```

```
{
```

```
    class Program
```

```
    {
```

```
        public static void Main(string[] args)
```

```
        {
```

```
}  
}  
}
```

OUTPUT

```
$ dotnet run  
Please enter a password:  
123  
Password score: 1  
The password is weak  
□
```

Project-5

Choose Your Own Adventure

In this project, you'll use logic and conditional statements to write a classic text-based Choose Your Own Adventure Game. Depending on what choices your user makes, the program will have a different result. If you're interested in game development, this is a great project to get your started! While we're only working with text, the principles of user input and control flow are used to build even highly advanced games.

We've provided you with a template to write a game, but we *highly encourage* you to take what you've learned here and use your creativity to write an original game, either here on Codecademy or off-platform!

TASKS

Set Up the Program

1.

Take a look at the program in the text editor. We use the command `Console.Write()` to give instructions to the user, then `Console.ReadLine()` to get input from the user. We save that input to a variable named `name`. We can then interpolate the value of that variable in a message we print to the console using `Console.WriteLine()`.

We'll be using these commands frequently throughout the project, so make sure you're comfortable with them before moving forward!

2.

Let's start our story. Print the following sentences to the console:

It begins on a cold rainy night. You're sitting in your room and hear a noise coming from down the hall. Do you go investigate?

3.

So far, we've explained the story and asked our user to make a decision. Let's give our user some instructions for *how* to make a decision in this game.

Use `Console.Write()` to inform the user that they should:

Type YES or NO:

4.

Now we can use `Console.ReadLine()` to capture the user's decision. Save the user's input to a string variable named `noiseChoice`.

5.

What if a user doesn't type YES, but instead types yes? Modify the noiseChoice variable so that it's value is uppercase.

6.

It's a good practice to save and run your code every few steps to make sure there are no bugs. In the terminal, type the following command and press Enter on your keyboard:

```
dotnet run
```

Adding Conditional Statements

7.

We want our program to do something different based on a user's decision. If they type YES, then we want them to continue on in the game and if they choose NO, we want the game to end.

Write an if statement that checks to see if a user writes NO. If the user picks NO, then have the program print out:

```
Not much of an adventure if we don't leave our room!  
THE END.
```

Use two Console.WriteLine() statements to break up the text.

8.

Write an else if statement that checks to see if a user writes YES.

9.

If the condition in the else if statement is true, have it print the following text to the console:

You walk into the hallway and see a light coming from under a door down the hall.

You walk towards it. Do you open it or knock?

10.

Now it's time for our user to make another decision. Let's tell them what they can type using `Console.Write()`:

Type OPEN or KNOCK:

Save their response to a variable named `doorChoice`. Make sure to change the value of `doorChoice` to uppercase.

11.

Let's write another conditional statement based on the user's choice. Write an if statement for if a user chooses KNOCK and an else if statement for if they choose OPEN.

12.

In the if statement, have it print the following text to the console:

A voice behind the door speaks. It says, "Answer this riddle: "
"Poor people have it. Rich people need it. If you eat it you die. What is it?"

13.

Using `Console.Write()`, tell the user to answer the riddle:

Type your answer:

Save their response to a variable named `riddleAnswer`.

14.

The correct answer to the riddle is NOTHING. So if a user types NOTHING, the program will print the following to the screen:

The door opens and NOTHING is there.

You turn off the light and run back to your room and lock the door.

THE END.

If a user types anything else, the program will print the following to the screen:

You answered incorrectly. The door doesn't open.

THE END.

Adding a Switch Statement

15.

Return to the else if statement that checks to see if doorChoice is equal to OPEN.

If the condition is true, have it print the following text to the console:

The door is locked! See if one of your three keys will open it.

16.

Using Console.Write(), get the user to give a number for the key they want to use:

Enter a number (1-3):

Save their response to a string variable named keyChoice. Make sure that the value of keyChoice is always uppercase!

17.

We're going to use the value of the variable keyChoice for our switch statement case. Write a switch statement that checks to see if a value is equal to either "1", "2", or "3".

18.

For each case, print out something to the console to finish the story.

If a user chooses "1", print:

You choose the first key. Lucky choice!
The door opens and NOTHING is there. Strange...
THE END.

If a user chooses "2", print:

You choose the second key. The door doesn't open.
THE END.

If a user chooses "3", print:

You choose the third key. The door doesn't open.
THE END.

Run the Program

19.

Let's run the program! Save the program. Then, in the terminal, type the following command and press Enter:

```
dotnet run
```

Try different options. When you hit the end of the program, re-run it and make a different decision than you did before. Keep doing that until you've gone through all the possible endings to make sure your program looks correct, no matter what options your user chooses.

If you are feeling ambitious, here are some extensions:

- Modify the story and the code to create your very own Choose Your Adventure
- Include more cases to check for other user input (what if a user picks a choice you didn't include?)
- Use ternary operators

Program.cs

```

using System;

namespace ChooseYourOwnAdventure
{
    class Program
    {
        static void Main(string[] args)
        {
            /* THE MYSTERIOUS NOISE */

            // Start by asking for the user's name:
            Console.Write("What is your name?: ");
            string name = Console.ReadLine();
            Console.WriteLine($"Hello, {name}! Welcome to our story.");

        }
    }
}

```

OUTPUT

```
$ dotnet run
What is your name?: vishva
Hello, vishva! Welcome to our story.
It begins on a cold rainy night. You're sitting in your room and hear a noise coming from down the hall. Do you go investigate?
Type YES or NO: yes
You walk into the hallway and see a light coming from under a door down the hall.
You walk towards it. Do you open it or knock?
Type OPEN or KNOCK: open
The door is locked! See if one of your three keys will open it.
Enter a number (1-3): 2
You choose the second key. The door doesn't open.
THE END.
$ ☐
```

7. METHOD CALLS AND INPUT

7.1 Introduction to Methods

Imagine making a hamburger:

1. Place the bread down
2. Add the burger patty
3. Add the pickles
4. Place the bread on top

What if you had to say each step every time you ordered a hamburger? It's tedious. It takes a long time. (How do you fit that on a menu?) And it risks making mistakes.

In this lesson you will learn a solution to that problem: *methods*. A method is a reusable set of instructions that perform a specific task. Developers use methods to make their code organized, maintainable, and repeatable.

You might hear methods also called functions — for now, the difference isn't important to us.

By the end of this lesson you will be able to recognize methods, call them with multiple arguments, capture their returned values, and define a basic method.

7.1 Introduction to Methods Instructions

1. First make two burgers without methods: add bread, patty, pickles, and bread.
2. Then make two burgers with the method `makeHamburger()`

3. Which way is faster? Which is more prone to mistakes?



7.2 Call a Method

You've been using methods since you started learning C#! Commands like `Console.WriteLine()` and `Math.Min()` are methods.

Each method has a different behavior: The first method prints something to the console, and the second finds the smallest of two given numbers. We activate a method's behavior by *calling* it. In C# we do this by adding parentheses to the end of a method name.

```
Console.WriteLine();
```

Some methods accept inputs called *arguments*. `Console.WriteLine()` accepts one string argument. That argument will be printed to the console.

```
// This prints "I'm hungry!"  
Console.WriteLine("I'm hungry!");
```

Other methods accept multiple arguments, like `Math.Min()`. It expects two number inputs.

```
Math.Min(3, 5);
```

You've probably seen built-in methods for each data type too. Every string has access to methods like `IndexOf()` and `Substring()`.

```
string name = "beatrice";  
name.Substring(0, 3); // returns "bea"
```

7.2 Call a Method Instructions

1.

Call `Math.Min()` with two arguments (any two integers will do). You can [view the documentation](#) if you're not sure how to use this method.

2.

Call `Console.WriteLine()` using the variable `msg` as an argument.

3.

Get the first letter of the `msg` string using `Substring()`. You can [view the documentation](#) if you're not sure how to use this method.

Program.cs

```
using System;
```

```
namespace CallAMethod
```

```
{
```

```
    class Program
```

```

{
    static void Main(string[] args)
    {
        string msg = "Yabba dabba doo!";
    }
}

```

OUTPUT

Yabba dabba doo!

Y

7.3 Capture Output

Like a math function or a factory machine, a method takes input and *returns* output. We've just seen how input works (arguments). Let's see how output works.

When a method *returns* a value, it essentially passes a piece of data to wherever it was called. One way to capture the returned value of a method is with a variable:

```
int smallerNumber = Math.Min(3, 4);
```

`Math.Min()` returns the value 3, so you can imagine that value taking its place.

```
int smallerNumber = 3;
```

We can then use that variable as input to other methods, like `Console.WriteLine()`:

```
Console.WriteLine(smallerNumber);
```

In this case, we can take a shortcut by nesting the method calls:

```
Console.WriteLine(Math.Min(3, 4));
```

Now the value returned by `Math.Min()` is used as input to `Console.WriteLine()`.

Not every method returns a value. `Console.WriteLine()`, for example, prints 3 to the console but it doesn't pass the value 3 to its caller. If you're not sure what a method returns you can always [check the Microsoft documentation](#).

7.3 Capture Output Instructions

1.

The designer of C# is "Anders Hejlsberg". His first name is nice, but we want to print the second name alone.

First, find the index of the space (" ") in the string `designer` and store it in a variable `indexOfSpace`.

You'll need to use [the `IndexOf\(\)` method](#).

2.

Use `Substring()` and the variable `indexOfSpace` to get the second name. Store the returned value in a variable `secondName`.

3.

Print `secondName` to the console.

Program.cs

```
using System;
```

```
namespace CaptureOutput
```

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            string designer = "Anders Hejlsberg";  
        }  
    }  
}
```

OUTPUT

Hejlsberg

7.4 Define a Method

Up until now, you've been calling built-in methods: methods that are available whenever you use C#. Sometimes you need a custom method for your specific program. In that case, you'll need to define your own!

The basic structure of a method definition looks like this:

```
static void YourMethodName()  
{  
}  
}
```

We'll skip over static and void for the moment.

In C#, it's convention to use PascalCase to name your method. The name starts with an uppercase letter and each word following begins with an uppercase as

well. It's not required in C#, but it makes your code easier to read for other developers.

The body of your method goes between the curly braces: { }. Whenever the method is called, the code in the body is executed.

```
static void YourMethodName()  
{  
    Console.WriteLine("Hi there!");  
}
```

Just like any other method, we call it with parentheses:

```
YourMethodName();
```

Look closely at the code in the editor and you'll see that you've been defining methods all along! Main() is a method. Every time you run the code, the Main() method is executed.

Since Main() is already a method, we'll define our own methods outside of Main().

7.4 Define a Method Instructions

1.

Define a method named VisitPlanets() outside of the Main() method and run the code.

VisitPlanets() can print anything you'd like to the console, but something like "You visited many new planets..." would be appropriate.

2.

Why isn't your method executed? It's not called within Main(). Call it in Main() and run the code again.

Program.cs

```
using System;
```

```
namespace DefineAMethod
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

You visited many new planets...

7.5 Define Parameters

Remember calling methods with arguments, like `Math.Min(3, 4)`? Methods that you define can use arguments as well, making them more versatile and useful.

While we are defining our method, we don't know the actual argument values that will be used when calling the method. But we do know the expected data type and how it will be used. We can use this information to define a *parameter*, which sort of works like a variable within a method. Imagine it as a placeholder for the actual argument value.

```
static void YourMethodName(string identity)
```

```
{
```

```
    Console.WriteLine(identity);  
}
```

The `YourMethodName()` method now has one parameter named `identity` of type `string`.

Separate multiple parameters with commas:

```
static void YourMethodName(string identity, int age)  
{  
    Console.WriteLine($"{identity} is {age} years old.");  
}
```

When you call your method, the values to be used for each parameter are called *arguments*. In this case "Yoda" and 900 are arguments for the `identity` and `age` parameters.

```
YourMethodName("Yoda", 900);
```

7.5 Define Parameters Instructions

1.

The `VisitPlanets()` method has been re-written for you here.

Add an `int` parameter named `numberOfPlanets` to the method.

2.

Change the method body so that it uses the parameter. If someone were to call your method, it should print how many planets they visited. For example, calling `VisitPlanets(3)` would cause this to be printed:

You visited 3 new planets...

3.

Call `VisitPlanets()` three times in `Main()` with different arguments.

Program.cs

```
using System;
```

```
namespace DefineParameters
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
        static void VisitPlanets()
```

```
        {
```

```
            Console.WriteLine("You visited many new planets...");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

You visited 3 new planets...

You visited 4 new planets...

You visited 5 new planets...

7.6 A Note on Parameters

One thing to watch for with parameters: they can only be used inside their method!

```
static void YourMethodName(string message)
{
    Console.WriteLine(message);
}
Console.WriteLine(message); // causes an error!
```

You'll see an error like...

error CS0103: The name 'message' does not exist in the current context

When talking to other developers about this type of issue, you might hear the term *scope*. While the entire concept of scope won't be covered in this lesson, you should know how it applies here: a parameter's scope is within its method, which means that the name (message in this case) is only valid within its method. If the parameter name is used outside the method, it has no meaning, so it throws an error.

7.6 A Note on Parameters Instructions

1.

Try causing an error by using the parameter in VisitPlanets() outside of the method body.

For example, if your parameter is named numberOfPlanets, you could try to print numberOfPlanets in Main().

Program.cs

```
using System;
```

```
namespace ANoteOnParameters
```

```
{
```

```

class Program
{
    static void Main(string[] args)
    {
        VisitPlanets(3);
        VisitPlanets(4);
        VisitPlanets(5);
    }

    static void VisitPlanets(int numberOfPlanets)
    {
        Console.WriteLine($"You visited {numberOfPlanets} new planets...");
    }
}

```

OUTPUT

Program.cs(12,25): error CS0103: The name 'numberOfPlanets' does not exist in the current context [/home/ccuser/workspace/csharp-a-note-on-parameters/c-sharp-hello-world-run-some-c-sharp.csproj]

The build failed. Fix the build errors and run again.

7.7 Optional Parameters

To make our functions even more flexible, we can make certain parameters *optional*. If someone calls your method without all the parameters, the method will assign a default value to those missing parameters.

All you have to do is use the equals sign (=) when defining the method. In this example, punctuation is an optional parameter, and its default value is ".".

```
static void Main(string[] args)
{
    YourMethodName("I'm hungry", "!"); // prints "I'm hungry!"
    YourMethodName("I'm hungry"); // prints "I'm hungry."
}

static void YourMethodName(string message, string punctuation = ".")
{
    Console.WriteLine(message + punctuation);
}
```

7.7 Optional Parameters Instructions

1.

Make the existing parameter in `VisitPlanets()` optional.

The default value should be 0.

2.

Call the method without the optional parameter in `Main()`.

Program.cs

```
using System;
```

```
namespace OptionalParameters
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            VisitPlanets(3);
            VisitPlanets(4);
            VisitPlanets(5);
        }

        static void VisitPlanets(int numberOfPlanets)
        {
            Console.WriteLine($"You visited {numberOfPlanets} new planets...");
        }
    }
}

```

OUTPUT

You visited 3 new planets...

You visited 4 new planets...

You visited 5 new planets...

You visited 0 new planets...

7.8 Named Arguments

Say your method has lots of optional parameters, but you only want to specify one when you call it.

In this example, your method has five optional parameters:

```
static void YourMethodName(int a = 0, int b = 0, int c = 0, int d = 0, int e = 0) {...}
```

When you call the method, you only want to specify d. But calling the method this way would set a to 4, not d!

```
YourMethodName(4);
```

Refer to the parameter by its name instead:

```
YourMethodName(d: 4);
```

With named arguments, you can list them in any order:

```
YourMethodName(d: 4, b: 1, a: 2);
```

You can also mix named arguments with positional arguments, but positional arguments **MUST** come before named arguments:

```
YourMethodName(2, 1, d: 4) // a is 2, b is 1, d is 4
```

```
YourMethodName(d: 4, 2, 1) // Error!
```

Named arguments aren't always necessary, but they can be useful when:

- a method has many optional parameters
- you want to differentiate between similar arguments

7.8 Named Arguments Instructions

1.

The VisitPlanets() method has some new optional parameters.

First, call the method in Main() with no arguments.

2.

Call the method again, but define only the numberOfPlanets parameter as 2.

3.

Call the method one more time, now defining the numberOfPlanets as 2 and name with your name.

Program.cs

```
using System;
```

```
namespace NamedArguments
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
        static void VisitPlanets(
```

```
            string adjective = "brave",
```

```
            string name = "Cosmonaut",
```

```
            int numberOfPlanets = 0,
```

```
            double gForce = 4.2)
```

```
        {
```

```
            Console.WriteLine($"Welcome back, {adjective} {name}.");
```

```
            Console.WriteLine($"You visited {numberOfPlanets} new planets...");
```

```
        Console.WriteLine($"...while experiencing a g-force of {gForce} g!");  
    }  
}  
}
```

OUTPUT

Welcome back, brave Cosmonaut.

You visited 0 new planets...

...while experiencing a g-force of 4.2 g!

Welcome back, brave Cosmonaut.

You visited 2 new planets...

...while experiencing a g-force of 4.2 g!

Welcome back, brave Farfel.

You visited 2 new planets...

...while experiencing a g-force of 4.2 g!

7.9 Method Overloading

Say you want to use `Math.Round()`, a built-in method. You go to the [Microsoft documentation](#) to learn how to use it, and find at least 8 different versions! They all have the same name: `Math.Round()`.

What's happening here is called *method overloading*, and each “version” is called an *overload*. Though they have the same name, the *overloads* are different because they have either (i) different parameter types or (ii) different number of parameters. This is useful if you want the same method to have different behavior based on its inputs.

Let's examine this concept with these two overloads: `Math.Round(Double, Int32)` and `Math.Round(Double)`.

The first overload, `Math.Round(Double, Int32)`, rounds the double to the int's number of decimal points

```
Math.Round(3.14159, 2); // returns 3.14
```

The second, `Math.Round(Double)`, rounds the double to the nearest integer.

```
Math.Round(3.14159); // returns 3
```

In C#, when we say that the methods are “different”, we are really talking about their method *signatures*, which is the method's name and parameter types in order.

For example, both methods above are named `Round()`, but one has `Double` and `Int32` parameters, and the other has a `Double` parameter.

7.9 Method Overloading Instructions

1.

Let's practice implementing our own overloads. Let's build a method `NamePets` with two overloads.

First write a method `NamePets()` that takes two string arguments.

If you call it, like:

```
NamePets("Laika", "Albert");
```

it should announce the newly named pets in the console, like:

Your pets Laika and Albert will be joining your voyage across space!

2.

Then write another method `NamePets` that takes three string arguments. When you call it:

```
NamePets("Mango", "Puddy", "Bucket");
```

it should announce the newly named pets in the console, like:

Your pets Mango, Puddy, and Bucket will be joining your voyage across space!

3.

Add a third NamePets method with zero arguments. When you call it:

```
NamePets();
```

it should empathize with you, like:

Aw, you have no spacefaring pets :(

4.

Call each method overload once in Main().

Program.cs

```
using System;
```

```
namespace MethodOverloading
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
    }
```

```
}  
}
```

OUTPUT

Your pets Laika and Albert will be joining your voyage across space!

Your pets Mango, Puddy, and Bucket will be joining your voyage across space!

Aw, you have no spacefaring pets :(

7.10 Review

You learned a lot this lesson: congrats on finishing! Here's what you've covered:

- Call a method with its name and parentheses:

```
VisitPlanets();
```

- Store a method's returned value in a variable:

```
double result = Math.Round(3.14159, 2);
```

- Define a basic method with the following syntax:

```
static void VisitPlanets()  
{  
}
```

- Every time an application is started, the `Main()` method is called.
- Values passed to a method are called *arguments*. When defined in the method, they are *parameters*.
- Method parameters can only be used within the method body.
- Method parameters can be *optional* if given a default value using `equals =` syntax:

```
static void VisitPlanets(int numberOfPlanets = 0)
```

- When calling a method, pass arguments by position or by name. If using names, use the colon (:) syntax:

```
VisitPlanets(numberOfPlanets: 9);
```

- In *method overloading*, multiple methods can have the same name, as long as they have different method *signatures*.
- A method *signature* is a method's name and parameter types in order.

7.10 Review Instructions

1.

Make sure you know how to apply all of these concepts before moving on!

To pass this last exercise:

- Call NamePets() with two arguments
- Call VisitPlanets(), and specify only the numberOfPlanets

Program.cs

```
using System;
```

```
namespace ReviewMethodCallsAndInput
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
}
```

```
static void NamePets()
```

```
{
```

```
    Console.WriteLine("Aw, you have no spacefaring pets :(");
```

```
}
```

```
static void NamePets(string pet1, string pet2)
```

```
{
```

```
    Console.WriteLine($"Your pets {pet1} and {pet2} will be joining your voyage  
across space!");
```

```
}
```

```
static void NamePets(string pet1, string pet2, string pet3)
```

```
{
```

```
    Console.WriteLine($"Your pets {pet1}, {pet2}, and {pet3} will be joining your  
voyage across space!");
```

```
}
```

```
static void VisitPlanets(
```

```
    string adjective = "brave",
```

```
    string name = "Cosmonaut",
```

```
    int numberOfPlanets = 0,
```

```
    double gForce = 4.2)
```

```
{
```

```
Console.WriteLine($"Welcome back, {adjective} {name}.");  
Console.WriteLine($"You visited {numberOfPlanets} new planets...");  
Console.WriteLine($"...while experiencing a g-force of {gForce} g!");  
}  
  
}  
}
```

OUTPUT

Your pets Laika and Albert will be joining your voyage across space!

Welcome back, brave Cosmonaut.

You visited 8 new planets...

...while experiencing a g-force of 4.2 g!

Concept Review

Optional Parameters

In C#, methods can be given *optional parameters*. A parameter is optional if its declaration specifies a *default argument*. Methods with an *optional parameter* can be called with or without passing in an argument for that parameter. If a method is called without passing in an argument for the *optional parameter*, then the parameter is initialized with its default value.

To define an *optional parameter*, use an equals sign after the parameter declaration followed by its default value.

```
// y and z are optional parameters.  
static int AddSomeNumbers(int x, int y = 3, int z = 2)  
{  
    return x + y + z;  
}
```

// Any of the following are valid method calls.

AddSomeNumbers(1); // Returns 6.

AddSomeNumbers(1, 1); // Returns 4.

AddSomeNumbers(3, 3, 3); // Returns 9.

Variables Inside Methods

Parameters and variables declared inside of a method cannot be used outside of the method's body. Attempting to do so will cause an error when compiling the program!

```
static void DeclareAndPrintVars(int x)  
{  
    int y = 3;  
    // Using x and y inside the method is fine.  
    Console.WriteLine(x + y);  
}  
  
static void Main()  
{
```

```
DeclareAndPrintVars(5);
```

// x and y only exist inside the body of DeclareAndPrintVars, so we cannot use them here.

```
    Console.WriteLine(x * y);  
}
```

Void Return Type

In C#, methods that do not return a value have a void return type.

void is not an actual data type like int or string, as it represents the lack of an output or value.

// This method has no return value

```
static void DoesNotReturn()  
{  
    Console.WriteLine("Hi, I don't return like a bad library borrower.");  
}
```

// This method returns an int

```
static int ReturnsAnInt()  
{  
    return 2 + 3;  
}
```

Method Declaration

In C#, a *method declaration*, also known as a *method header*, includes everything about the method other than the method's body. The method declaration includes:

- the method name
- parameter types
- parameter order
- parameter names
- return type
- optional modifiers

A method declaration you've seen often is the declaration for the Main method (note there is more than one valid Main declaration):

```
static void Main(string[] args)
```

```
// This is an example of a method header.
```

```
static int MyMethodName(int parameter1, string parameter2) {
```

```
    // Method body goes here...
```

```
}
```

Return Keyword

In C#, the return statement can be used to return a value from a method back to the method's caller.

When return is invoked, the current method terminates and control is returned to where the method was originally called. The value that is returned by the method must match the method's *return type*, which is specified in the *method declaration*.

```
static int ReturnAValue(int x)
{
    // We return the result of computing x * 10 back to the caller.
    // Notice how we are returning an int, which matches the method's return type.
    return x * 10;
}
```

```
static void Main()
{
    // We can use the returned value any way we want, such as storing it in a
    variable.
    int num = ReturnAValue(5);
    // Prints 50 to the console.
    Console.WriteLine(num);
}
```

Out Parameters

return can only return one value. When multiple values are needed, out parameters can be used.

out parameters are prefixed with out in the method header. When called, the argument for each out parameter must be a *variable* prefixed with out.

The out parameters become aliases for the variables that were passed in. So, we can assign values to the parameters, and they will persist on the variables we passed in after the method terminates.

```

// f1, f2, and f3 are out parameters, so they must be prefixed with `out`.
static void GetFavoriteFoods(out string f1, out string f2, out string f3)
{
    // Notice how we are assigning values to the parameters instead of using
    `return`.
    f1 = "Sushi";
    f2 = "Pizza";
    f3 = "Hamburgers";
}

static void Main()
{
    string food1;
    string food2;
    string food3;

    // Variables passed to out parameters must also be prefixed with `out`.
    GetFavoriteFoods(out food1, out food2, out food3);

    // After the method call, food1 = "Sushi", food2 = "Pizza", and food3 =
    "Hamburgers".

    Console.WriteLine($"My top 3 favorite foods are {food1}, {food2}, and {food3}");
}

```

Expression-Bodied Methods

In C#, *expression-bodied methods* are short methods written using a special concise syntax. A method can only be written in *expression body* form when the method body consists of a single statement or expression. If the body is a single expression, then that expression is used as the method's return value.

The general syntax is `returnType funcName(args...) => expression;`. Notice how “fat arrow” notation, `=>`, is used instead of curly braces. Also note that the `return` keyword is not needed, since the expression is implicitly returned.

```
static int Add(int x, int y)
{
    return x + y;
}
```

```
static void PrintUpper(string str)
{
    Console.WriteLine(str.ToUpper());
}
```

// The same methods written in expression-body form.

```
static int Add(int x, int y) => x + y;
```

```
static void PrintUpper(string str) => Console.WriteLine(str.ToUpper());
```

Lambda Expressions

A *lambda expression* is a block of code that is treated like any other value or expression. It can be passed into methods, stored in variables, and created inside methods.

In particular, *lambda expressions* are useful for creating *anonymous methods*, methods with no name, to be passed into methods that require method arguments. Their concise syntax is more elegant than declaring a regular method when they are being used as one off method arguments.

```
int[] numbers = { 3, 10, 4, 6, 8 };
```

```
static bool isTen(int n) {  
    return n == 10;  
}
```

```
// `Array.Exists` calls the method passed in for every value in `numbers` and  
returns true if any call returns true.
```

```
Array.Exists(numbers, isTen);
```

```
Array.Exists(numbers, (int n) => {  
    return n == 10;  
});
```

```
// Typical syntax
```

```
// (input-parameters) => { <statements> }
```

Shorter Lambda Expressions

There are multiple ways to shorten the concise lambda expression syntax.

- The parameter type can be removed if it can be inferred.
- The parentheses can be removed if there is only one parameter.

As a side note, the usual rules for expression-bodied methods also apply to lambdas.

```
int[] numbers = { 7, 7, 7, 4, 7 };
```

```
Array.Find(numbers, (int n) => { return n != 7; });
```

// The type specifier on `n` can be inferred based on the array being passed in and the method body.

```
Array.Find(numbers, (n) => { return n != 7; });
```

// The parentheses can be removed since there is only one parameter.

```
Array.Find(numbers, n => { return n != 7; });
```

// Finally, we can apply the rules for expression-bodied methods.

```
Array.Find(numbers, n => n != 7);
```

8. METHOD OUTPUT

8.1 Introduction to Method Output

What's the outcome of calling a method?

Sometimes a message is printed to the console:

```
Console.WriteLine("Hello World!");
```

Sometimes a value is returned:

```
Math.Floor(15.6); // Returns 15
```

Sometimes a variable is altered:

```
int number;  
bool success = Int32.TryParse("10602", out number);  
// number is 10602 and success is True
```

If you don't understand all of the code examples, that's okay! By the end of this lesson you will be able to:

- Define methods with output using the return and out keywords
- Recognize and resolve errors related to method output

Let's get started!

8.1 Introduction to Method Output Instructions

To remind yourself of some earlier concepts about method input, review the code on the right. We assume you are comfortable with these concepts before taking this lesson.

Program.cs

```
using System;

namespace IntroMethodOutput
{
    class Program
    {
        static void Main(string[] args)
        {
            // Call a method with multiple arguments
            NamePets("Laika", "Albert");

            // Call a method with named arguments
            VisitPlanets(numberOfPlanets: 8);
        }

        // Define a method with multiple parameters
        static void NamePets(string pet1, string pet2)
        {
            Console.WriteLine($"Your pets {pet1} and {pet2} will be joining your voyage across space!");
        }
    }
}
```



```
}
```

```
// Define a method with optional parameters
```

```
static void VisitPlanets(
```

```
    string adjective = "brave",
```

```
    string name = "Cosmonaut",
```

```
    int numberOfPlanets = 0,
```

```
    double gForce = 4.2)
```

```
{
```

```
    Console.WriteLine($"Welcome back, {adjective} {name}.");
```

```
    Console.WriteLine($"You visited {numberOfPlanets} new planets...");
```

```
    Console.WriteLine($"...while experiencing a g-force of {gForce} g!");
```

```
}
```

```
}
```

```
}
```

OUTPUT

Your pets Laika and Albert will be joining your voyage across space!

Welcome back, brave Cosmonaut.

You visited 8 new planets...

...while experiencing a g-force of 4.2 g!

8.2 Return

The basic way to *return* values from a method is to use a return statement! (A well-constructed programming language shouldn't have a lot of surprises.)

Let's start with an example in the below code. It contains a definition of the Yell() method, which returns a string, and it calls that method in Main().

When we execute this program, the code in Main() runs first:

1. Yell() is called.
2. In Yell(), the uppercase version of "who's there?" is created and returned.
3. Back in Main(), that returned value is stored in the variable output and then printed to the console:

```
static string Yell(string phrase)
{
    return phrase.ToUpper();
}

public static void Main()
{
    string output = Yell("who's there?");
    Console.WriteLine(output); // Prints WHO'S THERE?
}
```

Here's a more generic definition: the keyword return tells the computer to exit the method and return a value to wherever the method was called.

When a method is declared, it must announce the type of value it will return. In this case, Yell() returns a string, so it has the string modifier (right before the name Yell).

That first line of the method is called a *method declaration*, so we can say that the method declaration must contain the type of the return value.

Generally, the method declaration is a combination of details including: the access modifiers, return type, method name, and parameter types. This lesson will not cover access modifiers, like static, so that we can focus on the return type, like string.

8.2 Return Instructions

1.

Let's define a static method `DecoratePlanet()` that takes a planet name as input and returns a fancy welcome to the planet.

First, write the method declaration. It should have a string parameter and return a string.

2.

Write the method body so that it returns a fancy welcome to the planet. For example, calling

```
DecoratePlanet("Mars");
```

returns:

```
"*.*.* Welcome to Mars *.*.*"
```

3.

Call the method with the argument "Jupiter" and print its output to the console.

Program.cs

```
using System;
```

```
namespace Return
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
{  
  
}  
  
}  
}
```

OUTPUT

```
*..*..* Welcome to Jupiter *..*..*
```

8.3 Return Errors

As we mentioned before, we don't like surprises — they lead to mistakes. So, when we call a method, we'd like to know what type of value will be returned. This is done in the method definition.

The method definition must contain the type of the return value: if a method returns an integer, its return type must be `int`; if it returns text, it must be `string`, and so on. If the method returns nothing, use `void`.

If a method returns a type different from its stated return type, it will throw an error. Here are some common errors you may see —

This error means you must state a return type before the method name:

error CS1520: Method must have a return type

This error means that your method doesn't return a value, when it should:

error CS0161: [MethodName]: not all code paths return a value

In some cases, this error means that your method returns a `string` when it should be an `int` (this one can be caused by a lot of things outside of methods):

error CS0029: Cannot implicitly convert type 'string' to 'int'

It's important to remember that running into errors is a natural part of coding. As a teacher once put it ["Great programmers understand that errors are part of the process, and they know how to find the solution to each while learning something new from them."](#)

8.3 Return Errors Instructions

1.

This code has a bunch of errors! Run the code to find them.

2.

Fix the first error by adding a return type to one of the methods.

3.

Fix the second error by adding a return to one of the methods.

4.

Fix the last error by changing the return type of one of the methods.

Program.cs

```
using System;
```

```
namespace ReturnErrors
```

```
{
```

```
    class Program
```

```
    {
```

```

static void Main(string[] args)
{
    Console.WriteLine(DecoratePlanet("Mars"));
    Console.WriteLine("Is Pluto really a dwarf...?");
    Console.WriteLine(IsPlutoADwarf());
    Console.WriteLine("Then how many planets are there in the galaxy...?");
    Console.WriteLine(CountThePlanets());
}

```

```

static DecoratePlanet(string planet)
{
    return $"*..*..* Welcome to {planet} *..*..*";
}

```

```

static bool IsPlutoADwarf()
{
    bool answer = true;
}

```

```

static int CountThePlanets()
{
    return "8 planets, usually";
}

}

```

```
}
```

OUTPUT

```
*..*..* Welcome to Mars *..*..*
```

```
Is Pluto really a dwarf...?
```

```
True
```

```
Then how many planets are there in the galaxy...?
```

```
8 planets, usually
```

8.4 Out

A method can only *return* one value, but sometimes you need to output two pieces of information. Calling a method that uses an out parameter is one way to return multiple values.

For example, the `Int32.TryParse()` method tries to parse its input as an integer. If it can properly parse the input, the method returns true and sets its out variable to the new value. If it cannot properly parse the input, the method returns false and sets the out variable to 0.

This is what the method's signature looks like:

```
public static bool TryParse (string s, out int result);
```

The method returns a boolean and accepts a string and a variable that has been declared of type `int` as input.

Here's how `Int32.TryParse()` and the out parameter are used:

```
int number;  
bool success = Int32.TryParse("10602", out number);  
// number is 10602 and success is true  
int number2;
```

```
bool success2 = Int32.TryParse(" !!! ", out number2);  
// number2 is 0 and success2 is false
```

The second parameter is labeled out, which means that it must be assigned a value within the method.

For a shortcut, you can declare the int variable within the method call:

```
bool success = Int32.TryParse("10602", out int number);
```

8.4 Out Instructions

1.

Let's parse another string ageAsString to an integer.

First, define:

- an int named ageAsInt
- a bool named outcome

2.

Use Int32.TryParse() to convert ageAsString:

- ageAsInt should be used as the out argument
- outcome should capture the returned value

3.

Print outcome and ageAsInt to the console.

4.

Repeat the process with nameAsString:

Define:

- an int named nameAsInt

- a bool named outcome2

Use `Int32.TryParse()` to convert `nameAsString`:

- `nameAsInt` should be used as the out argument
- `outcome2` should capture the returned value

Print the returned value and the out variable to the console.

Program.cs

```
using System;
```

```
namespace OutParameters
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string ageAsString = "102";
```

```
            string nameAsString = "Granny";
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

True

102

False

0

8.5 Using Out

We can use out parameters in our own methods as well. In this example, `Yell()` converts phrase to uppercase and sets a boolean variable to true:

```
static string Yell(string phrase, out bool wasYellCalled)
{
    wasYellCalled = true;
    return phrase.ToUpper();
}
```

- The out parameter must have the out keyword and its expected type
- The out parameter must be set to a value before the method ends

When calling the method, don't forget to use the out keyword as well:

```
string message = "garrrr";
Yell(message, out bool flag);
// returns "GARRRR" and flag is true
```

8.5 Using Out Instructions

1.

Declare a method `Whisper()` with a string parameter and out bool parameter. It should return a string.

2.

Define the method body. Whisper() should work like Yell(), but instead of returning an uppercase string, it returns a lowercase string.

Once defined, you should be able to call it like:

```
string statement = "GARRRR";  
Whisper(statement, out bool marker);  
// should return "garrrr" and set marker to true;
```

3.

Call Whisper() in the Main() method and print the returned value to the console.

Make sure to use an out modifier when calling the method!

Program.cs

```
using System;
```

```
namespace UsingOut  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
  
        }  
    }  
}
```

OUTPUT

garrrr

8.6 Out Errors

As with return, out is a very useful keyword, but it can lead to errors if used incorrectly. Here are two common ones:

This error means that the out parameter needs to be assigned a value within the method:

error CS0177: The out parameter 'success' must be assigned to before control leaves the current method

This error means you called a method that expects an 'out' parameter but you didn't use the out keyword when calling it:

error CS1620: Argument 2 must be passed with the 'out' keyword

8.6 Out Errors Instructions

1.

This code has a bunch of errors! Run the code to find them.

2.

Fix the first error by using out when calling Whisper().

3.

Fix the second error by assigning a value to the out parameter wasWhisperCalled in the method body of Whisper().

Program.cs

```
using System;

namespace OutErrors
{
    class Program
    {
        static void Main(string[] args)
        {
            string statement = "GARRRR";
            bool marker;

            string murmur = Whisper(statement, marker);

            Console.WriteLine(murmur);
        }

        static string Whisper(string phrase, out bool wasWhisperCalled)
        {
            return phrase.ToLower();
        }
    }
}
```

OUTPUT

garrrr

8.7 Review

Congrats on finishing! You can now use and define methods with output.

Here's what else you've learned in this lesson:

- Methods return values with the return keyword.
- Every method has a return type, designated in its method signature. That type must match the type of the value actually returned.
- If a method returns no type, its return type is void.
- out parameters can be used to return multiple values from a method.

You can always review this material on the [Microsoft documentation](#).

8.7 Review Instructions

1.

Make sure you know how to apply all of these concepts before moving on!

Call `DecoratePlanet()` with the argument `destination`.

Store the returned value in the variable `welcomeMessage`.

2.

Call `Int32.TryParse()`, using `galaxyString` and `galaxyInt` as arguments.

Store the returned value in the variable `outcome`.

Program.cs

```
using System;
```

```
namespace ReviewMethodOutput
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            // Define variables
```

```
            string destination = "Neptune";
```

```
            string galaxyString = "8";
```

```
            int galaxyInt;
```

```
            string welcomeMessage;
```

```
            bool outcome;
```

```
            // Call DecoratePlanet() and TryParse() here
```

```
            // Print results
```

```
            Console.WriteLine(welcomeMessage);
```

```
            Console.WriteLine($"Parsed to int? {outcome}: {galaxyInt}");
```

```
        }
```

```
    // Define a method that returns a string
```

```

        static string DecoratePlanet(string planet)
        {
            return $"*..*..* Welcome to {planet} *..*..*";
        }

// Define a method with out
static string Whisper(string phrase, out bool wasWhisperCalled)
{
    wasWhisperCalled = true;
    return phrase.ToLower();
}
    }
}

```

OUTPUT

```

*..*..* Welcome to Neptune *..*..*

Parsed to int? True: 8

```

8. Quiz

1. The Main() method is executed when...
 - a) When you save your code.
 - b) The application is started.
 - c) Any time a method is called.

d) When there is an error.

2. What will be printed to the console?

```
public static void Main(string[] args)
{
    Console.WriteLine(SpaceSuit());
}

static string SpaceSuit(string animal)
{
    return $"This is a spacesuit for a(n) {animal}.";
}

static string SpaceSuit()
{
    return "Here's a spacesuit.";
}
```

- a) This is a spacesuit for a(n) .
- b) Here's a spacesuit.
- c) SpaceSuit()
- d) This is a spacesuit for a(n) animal.

3. What will be printed to the console?

```
public static void Main(string[] args)
{
    Console.WriteLine(Launch(launchWord: "blast off"));
}

static string Launch(
    string countDown = "3, 2, 1",
    string launchWord = "lift off")
{
```

```
return $"{countDown}...{launchWord}!";  
}
```

- a) 3, 2, 1...lift off!
- b) 3, 2, 1...blast off!
- c) {countDown}...{launchWord}!
- d) ...blast off!

4. Why will this code throw an error?

```
public static string CombineWords(string a, string b)  
{  
    return a + b;  
}
```

```
Console.WriteLine(a + b);
```

- a) string is not a valid return type.
- b) CombineWords is not a valid method name.
- c) You cannot use + with strings.
- d) a and b are not defined outside of the method body.

5. Why will this code throw an error?

```
public static void PercentDifference(double actual, double expected)  
{  
    return (actual - expected) / expected;  
}
```

- a) return is not a valid keyword.
- b) The curly braces are in the wrong place.
- c) The return type is wrong.
- d) Methods cannot take a double as input.

6. Return an all-caps string, i.e. "HELLO".

"hello" _____ ;

- ;
- {}
- ()
- ToUpper
- ToLower
- .

Click or drag and drop to fill in the blank

7. In this code, what is the argument?

Convert.ToInt32(x);

- a) Convert
- b) ToInt32
- c) ()
- d) x

8. Call the method Score() within the Main() method.

```
public static void Main(string[] args)
{
    _____ ;
}
```

```
public static void Score()
{
    Console.WriteLine("GOOOOAL!");
}
```

- {}
- ()
- Console.WriteLine
- Score

Click or drag and drop to fill in the blank

9. Find the minimum of int variables a and b. Assume that they are already defined in the code.

____(____);

- a
- a, b
- Math.Min
- b
- String.ToLower

Click or drag and drop to fill in the blank

10. Define *method*.

- The code inside curly braces { }.
- A reusable set of instructions that perform a specific task.
- A datatype.
- Any operator in C#.

11. Which line is the method declaration on?

```
static int AddTwo(int num)
{
    int result = num + 2;
    return result;
```

}

- a) 3: int result...
- b) 1: static int...
- c) 2: {
- d) 4: return...

12. What will be printed to the console?

```
public static void Main(string[] args)
{
    Console.WriteLine(Sight("Dauna"));
}
```

```
static string Sight(string viewer, string spaceThing = "meteor")
{
    return $"{viewer} saw a {spaceThing}!";
}
```

- a) Dauna saw a meteor!
- b) Dauna saw a spaceThing!
- c) Dauna saw a !
- d) viewer saw a spaceThing!

13. Write a method declaration that accepts one out parameter of type string. It should not return any value.

```
public static _____ HideMessage( _____ message)
```

- void
- double
- int
- string

- bool
- out

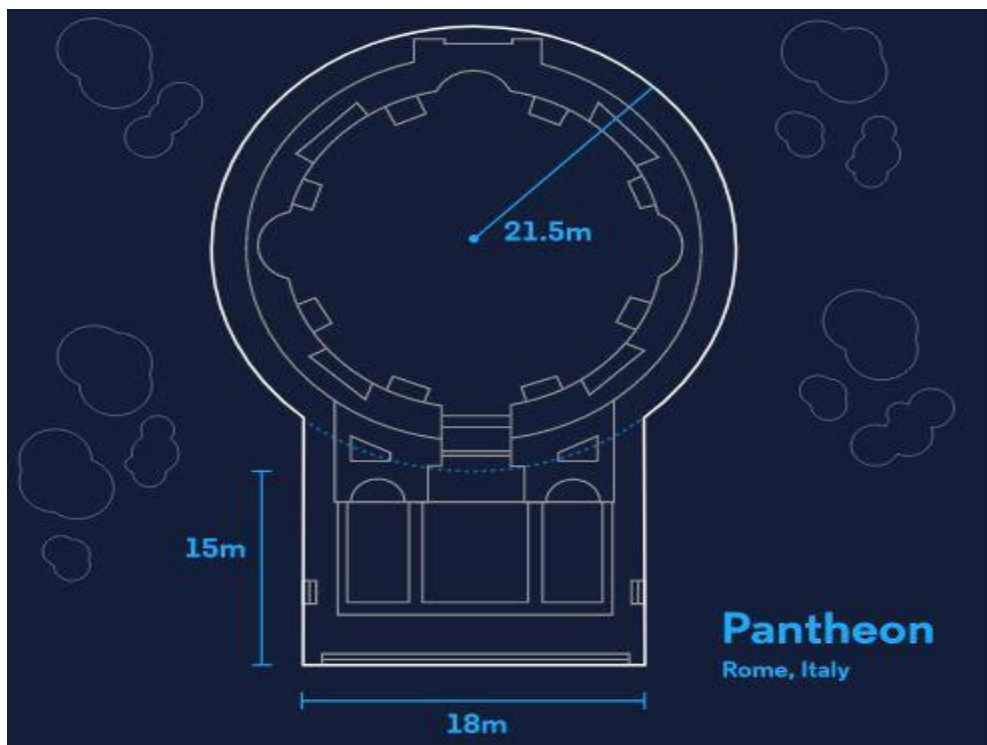
Click or drag and drop to fill in the blank

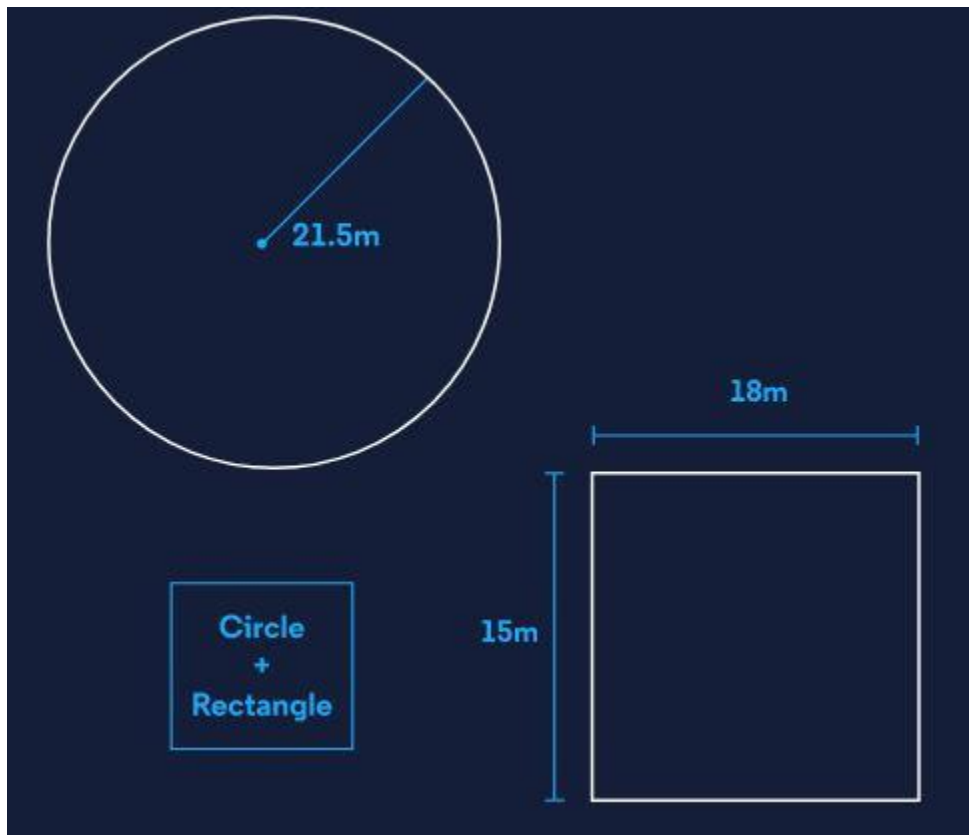
Project-6

Architect Arithmetic

Architects have big ideas – but big ideas can be expensive. How expensive? Depends on the size.

In this project, you'll use methods to build a program that calculates the material cost for any architect's floor plan. For example, the floor plan of the Pantheon in Rome, Italy:





...can be (approximately) broken into circles and rectangles:

Using basic formulas, we can calculate the area of each shape and find the total:

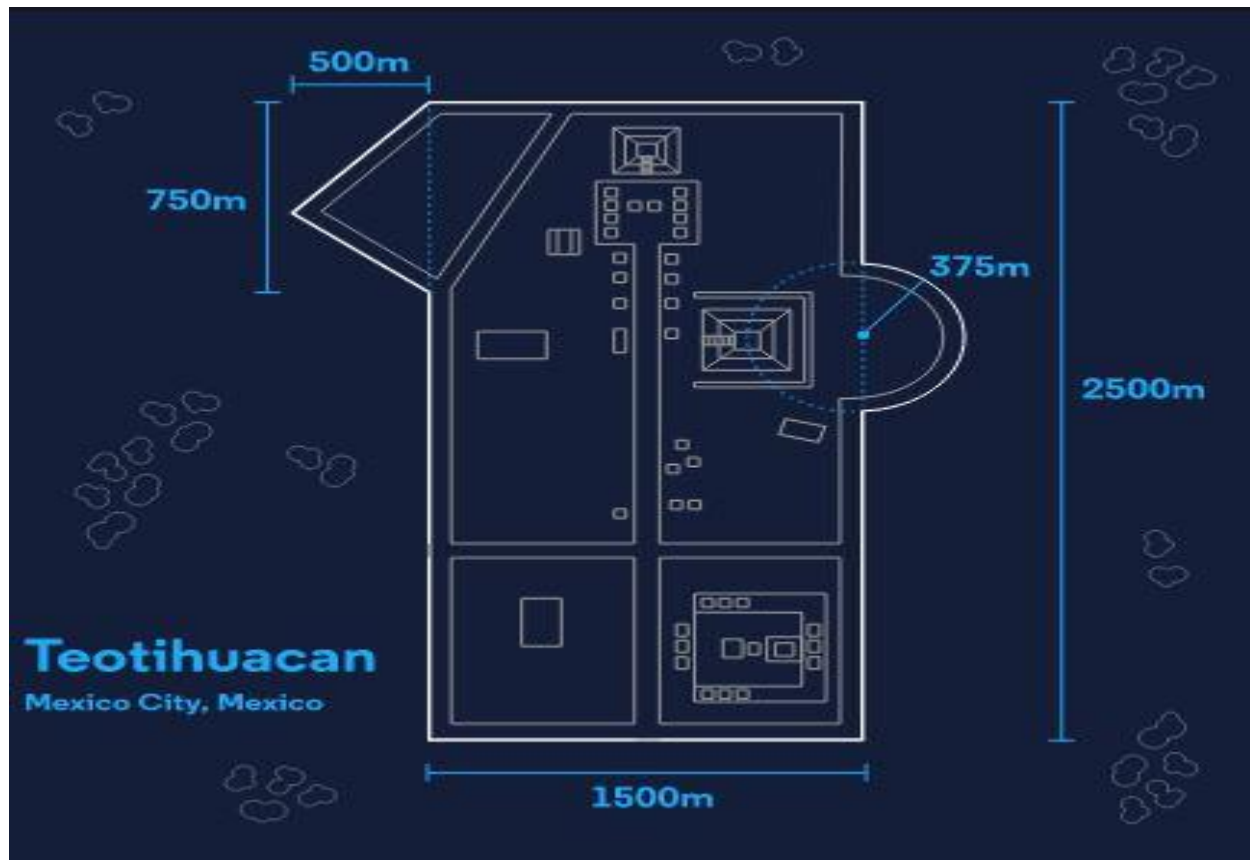
```
double totalArea = Circle(21.5) + Rect(15, 18);
```

You'll be defining area methods like `Circle()` and `Rect()`, but we'll provide the formulas for you to use.

Finally, we multiply the total area by the cost of each unit of flooring material. Let's assume that tiles cost £100 each:

```
double tileCost = 100;  
double totalCost = totalArea * tileCost;
```

Although the floor plan isn't exactly one circle and rectangle, this approximation is good enough for estimating costs. Let's get started!



TASKS

Define Methods

1.

We'll need to define a method for each basic shape. Let's start with rectangles. Define a method with two parameters (length and width) that returns the area of the rectangle.

Here's the formula for rectangular area:

$$area=length \times width$$

For all numbers in this project, use the double data type.

2.

Now circles. Define a method with one parameter (radius) that returns the area of the circle.

Use Math.PI to represent the number pi, and Math.Pow() to square the radius.

$$area = \pi \times radius^2$$

3.

We'll also need triangles. Define a method with two parameters (bottom and height) that returns the area of the triangle.

$$area = 0.5 \times bottom \times height$$

4.

Test that your methods work with these test cases:

- A rectangle with length 4 and width 5 has an area of 20.
- A circle with radius 4 has an area of about 50.
- A triangle with base 10 and height 9 has an area of 45.

Don't forget that you need to use dotnet run to run the program!

Calculate Cost

5.

Find the area of the floor plan. We'll use an adapted version of the ancient city in Mexico: Teotihuacan, which you can see as image to the right.

On paper or in your head, break down the floor plan using circles, rectangles, and triangles.

6.

Calculate the area of each part, add them together, and store the result in a variable.

7.

Multiply the total area by the cost of flooring material – 180 Mexican pesos – and store the result in a variable.

8.

Write the result to the console, explaining what the number means. Use string interpolation.

9.

Pesos should only have two decimal places, so use a built-in method to round the value.

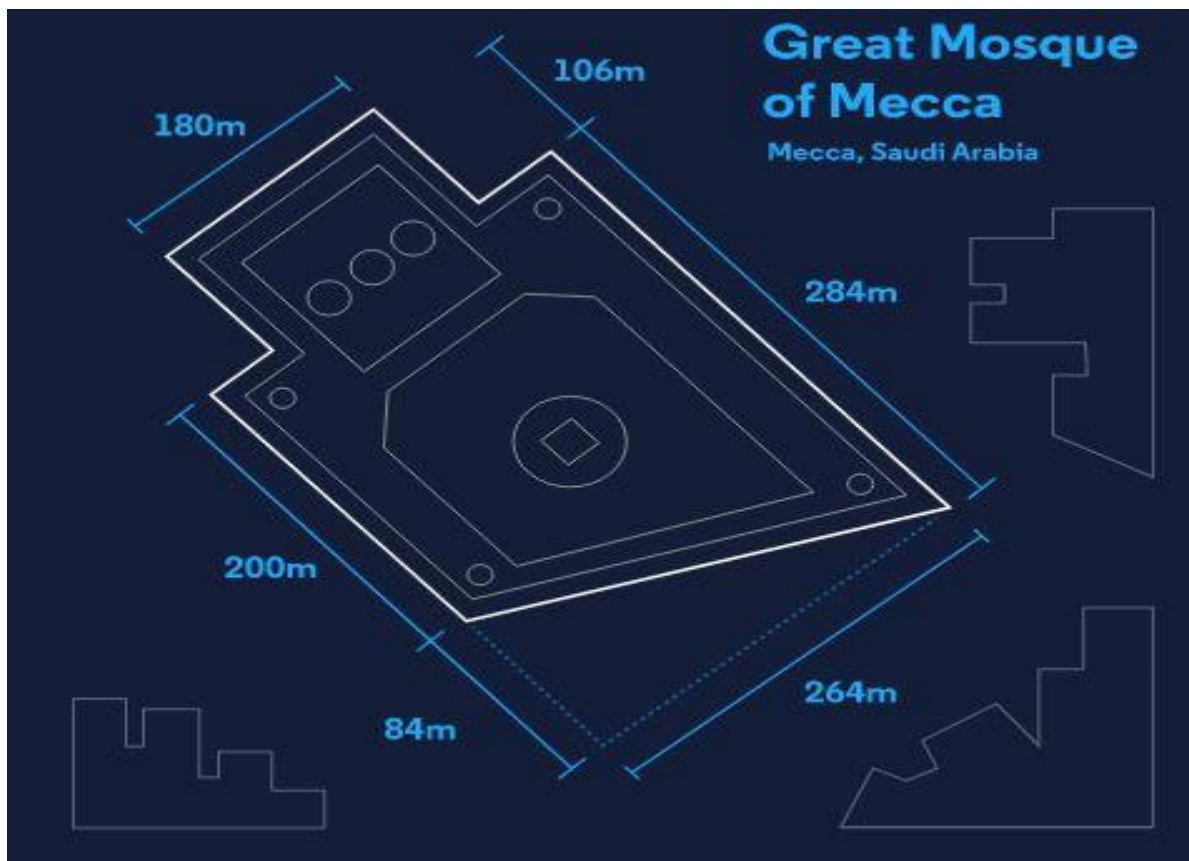
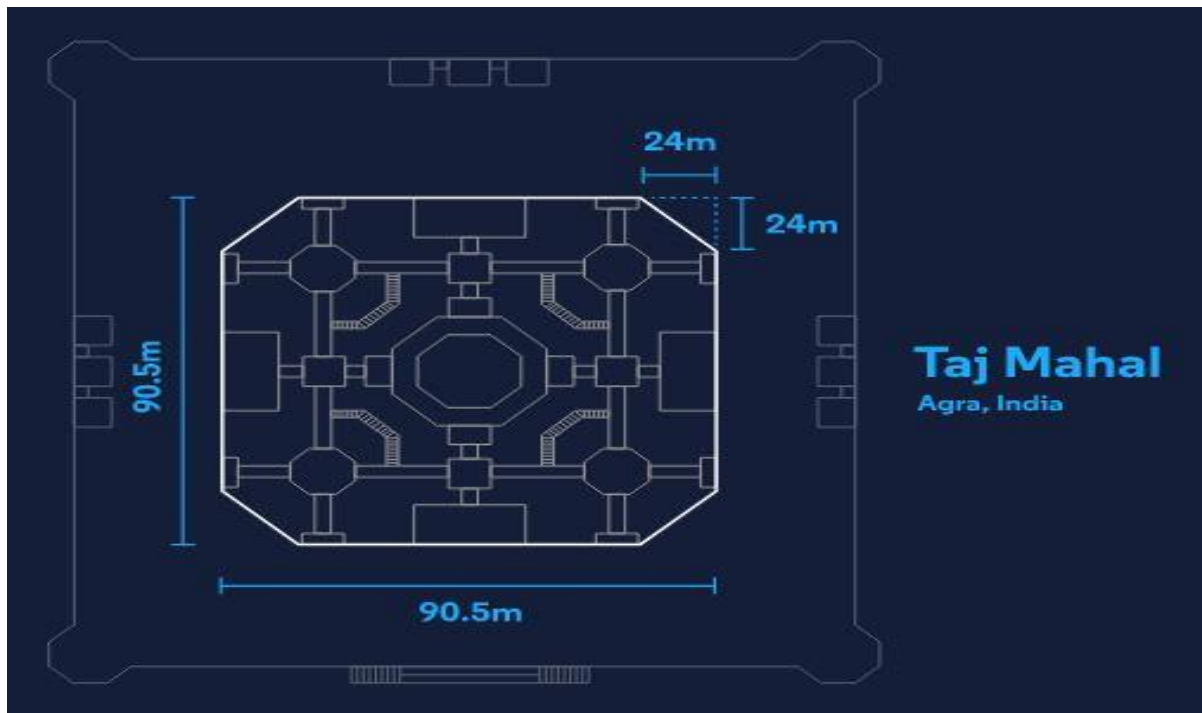
10.

For an extra challenge, try these additions:

- Make the entire program a method so that you can execute it in your Main() method with one line:

CalculateTotalCost();

- Determine the total cost for the Taj Mahal in Agra, India and the Al-Masjid al-haram (Great Mosque) in Mecca, Saudi Arabia.



- Use conditional statements and Console.ReadLine() to allow users to pick which monument for which they'd like to calculate a cost.

What monument would you like to work with? Teotihuacan
The plan for that monument costs 748510782.02 pesos!

Program.cs

```
using System;

namespace ArchitectArithmetic
{
    class Program
    {
        public static void Main(string[] args)
        {

        }

    }
}
```

OUTPUT

\$ dotnet run

The cost of flooring the floor plan is 20747.79 Mexican pesos.

9. ALTERNATE EXPRESSIONS

9.1 Introduction to Alternate Expressions

Good developers don't work too hard. We all push ourselves to get better at coding, but when something seems difficult or repetitive, there's usually a better way to do it!

Methods are a good example:

```
bool IsEven(int num)
{
    return num % 2 == 0;
}
```

Why bother with these curly braces ({ }) and return keywords for such a short method?

In C# there are other ways to define a method, which can save us effort in typing and make our code easier to read. They're called *expression-bodied definitions* and *lambda expressions*.

From now on, when you see these phrases just think: "These are shortcuts for defining methods!" This lesson will show you how to use them.

9.1 Introduction to Alternate Expressions Instructions

Check out these basic method definitions in the text editor. The *method signature* tells us the types of input to and output from the method. The *method body* lives inside the curly braces { } and sometimes contains the return keyword.

Take Welcome() for example:

- The signature tells us that the method takes in a string and outputs nothing.
- The body prints some text to the console.

Program.cs

```
using System;
```

```
namespace AlternateExpressions
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Welcome("Earth");
```

```
            double days = 500;
```

```
            double rotations = DaysToRotations(days);
```

```
        }
```

```
        static double DaysToRotations(double days)
```

```
        {
```

```
            return days / 365;
```

```
        }
```

```
        static void Welcome(string planet)
```

```
        {
```

```
    Console.WriteLine($"Welcome to {planet}!");  
}  
}  
}
```

OUTPUT

Welcome to Earth!

9.2 Expression-bodied Definitions

Expression-bodied definitions are the first “shortcut” for writing methods. They’re great for writing one-line methods, like this one:

```
bool IsEven(int num)  
{  
    return num % 2 == 0;  
}
```

We can rewrite this definition as an expression-bodied definition by:

- removing the curly braces and return keyword, and
- adding the “fat arrow”, or `=>`, which is composed of the equal sign, `=`, and greater than, `>`, symbols

```
bool isEven(int num) => num % 2 == 0;
```

This also works for methods that return nothing, aka void:

```
void Shout(string x) => Console.WriteLine(x.ToUpper());
```

This type of definition can only be used when a method contains one expression. This helps us remember the name: *expression*-bodied definitions are method definitions with one *expression*.

Fun fact: some developers also call the fat arrow notation, `=>`, a squid! 🐙

9.2 Expression-bodied Definitions Instructions

1.

Convert the method DaysToRotations() to an expression-bodied definition.

2.

Convert the method Welcome() to an expression-bodied definition.

Program.cs

```
using System;
```

```
namespace AlternateExpressions
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Welcome("Earth");
```

```
            double days = 500;
```

```
            double rotations = DaysToRotations(days);
```

```
            Console.WriteLine($"In {days} days, the Earth has rotated {rotations} time(s).");
```

```
        }
```

```
        static double DaysToRotations(double days)
```



```

{
    return days / 365;
}

static void Welcome(string planet)
{
    Console.WriteLine($"Welcome to {planet}!");
}
}
}

```

OUTPUT

Welcome to Earth!

In 500 days, the Earth has rotated 1.36986301369863 time(s).

9.3 Methods as Arguments

Before we get into the next shortcut, we need to understand how methods are passed to other methods as arguments. This is possible and sometimes necessary in C#.

For example, say we need to check if there are even values in an array (you don't need to know much about arrays here, except that they are lists of values).

First you need an array of values and the `IsEven()` method that returns true if its argument is even:

```
int[] numbers = {1, 3, 5, 6, 7, 8};
```

```
public static bool IsEven(int num)
```

```
{  
    return num % 2 == 0;  
}
```

Pass both of these as arguments to the method `Array.Exists()`, which returns a boolean value:

```
bool hasEvenNumber = Array.Exists(numbers, IsEven);
```

You can see that `IsEven`, a method, is passed as the second argument to `Array.Exists()`.

In the background, this is what `Array.Exists()` does:

- The `IsEven()` method is called with each value in the array. We can imagine each of these being called:

```
IsEven(1);
```

```
IsEven(3);
```

```
IsEven(5);
```

```
IsEven(6);
```

- If any of these return true, `Array.Exists()` returns true.

By the end, `Array.Exists()` returns true because `isEven(6)` returns true.

There are other methods that accept methods as arguments, which you will encounter later on. For now, you need to understand that we can use a method's name like a variable, e.g. `IsEven` is a variable representing the method `IsEven()`. We pass this variable to another method, like `Array.Exists()`, which will probably invoke that method-argument at least once within its own body.

9.3 Methods as Arguments Instructions

1.

[Array.Find\(\)](#) is another method that takes an array and a method as arguments. `Array.Find()` calls the method on each element of the array and returns the first element for which the method returns true.

An array `adjectives` and method `IsLong()` are defined for you. Call `Array.Find()` with these two arguments to find the first element in `adjectives` that is “long”.

Store the returned string in a variable named `firstLongAdjective`.

Program.cs

```
using System;
```

```
namespace AlternateExpressions
```

```
{
```

```
    class Program
```

```
    {
```

```
        // Method to be used as second argument
```

```
        public static bool IsLong(string word)
```

```
        {
```

```
            return word.Length > 8;
```

```
        }
```

```
static void Main(string[] args)
```

```
{
```

```
    // Array to be used as first argument
```

```
    string[] adjectives = {"rocky", "mountainous", "cosmic", "extraterrestrial"};
```

```
    // Call Array.Find() and
```

```
    // Pass in the array and method as arguments
```

```
    string firstLongAdjective =
```

```

        Console.WriteLine($"The first long word is: {firstLongAdjective}.");
    }
}

```

OUTPUT

The first long word is: mountainous.

9.4 Lambda Expressions

The next shortcut, *lambda expressions*, are great for situations when you need to pass a method as an argument.

In the past exercise, we used `IsEven()` to check that an even value exists in the array numbers:

```
int[] numbers = {1, 3, 5, 6, 7, 8};
```

```

public static bool IsEven(int num)
{
    return num % 2 == 0;
}

```

```
bool hasEvenNumber = Array.Exists(numbers, IsEven);
```

When using the original definition (with curly braces and return), it takes multiple lines to define the `IsEven()` method and other developers will need to jump around our code to find the definition. With a lambda expression, we can define `IsEven()` directly in the method call. We don't even need to give it a name:

```
bool hasEvenNumber = Array.Exists(numbers, (int num) => num % 2 == 0 );
```

This might look similar to an expression-bodied definition. It sort of is! What makes a lambda expression unique is that it is an *anonymous method*: it has no name.

Generally lambda expressions with one expression (like the above example) take this form. They use the fat arrow, no curly braces, and no semicolon (;):

```
(input-parameters) => expression
```

Lambda expressions with more than one expression use curly braces and semicolon:

```
(input-parameters) => { statement; }
```

Here's an example of the second structure, which checks if any element in numbers is a multiple of 12 and greater than 20:

```
bool hasBigDozen = Array.Exists(numbers, (int num) => {  
    bool isDozenMultiple = num % 12 == 0;  
    bool greaterThan20 = num > 20;  
    return isDozenMultiple && greaterThan20;  
});
```

Since this lambda expression includes multiple expressions (3 in this case), then we must use curly braces and semicolons.

9.4 Lambda Expressions Instructions

1.

Find the line where `Array.Exists()` is used.

Replace `HitGround` with a lambda expression that achieves the same result. It should return true if its input equals "meteorite".

Program.cs

```
using System;
```

```

namespace AlternateExpressions
{
    class Program
    {
        static void Main(string[] args)
        {
            string[] spaceRocks = {"meteoroid", "meteor", "meteorite"};

            bool makesContact = Array.Exists(spaceRocks, HitGround);

            if (makesContact)
            {
                Console.WriteLine("At least one space rock has reached the Earth's
surface!");
            }
        }

        static bool HitGround(string s)
        {
            return s == "meteorite";
        }
    }
}

```

OUTPUT

At least one space rock has reached the Earth's surface!

9.5 Shorter Lambda Expressions

Time to put on our detective caps: using deductive reasoning, we can make our lambda expression even shorter. Here's what we have to start:

```
bool hasEvenNumbers = Array.Exists(numbers, (int num) => num % 2 == 0 );
```

The type of num is int. It's great to be explicit like this to avoid errors, but some developers wouldn't include int. To them, it's obvious! Here's their reasoning:

- The modulo operator (%) is only used with numbers, so num must be a number
- The result of the operation num % 2 is compared to the integer 0. We can only compare similar types, so num must also be an integer!

Therefore, we can remove int without causing any errors:

```
bool hasEvenNumbers = Array.Exists(numbers, (num) => num % 2 == 0 );
```

When there is just one parameter in a lambda expression, we don't need the parentheses around the parameter either:

```
bool hasEvenNumbers = Array.Exists(numbers, num => num % 2 == 0 );
```

We just learned two new shortcuts "within" the lambda expression shortcut. Though we don't need to use them all the time, we do need to recognize them in other developers' code:

1. We can remove the parameter type if it can be inferred
2. We can remove the parentheses if there is one parameter

9.5 Shorter Lambda Expressions Instructions

1.

Apply the first shortcut to the lambda expression (remove the parameter type).

2.

Apply the second shortcut to the lambda expression (remove the parentheses).

Program.cs

```
using System;
```

```
namespace AlternateExpressions
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] spaceRocks = {"meteoroid", "meteor", "meteorite"};
```

```
            bool makesContact = Array.Exists(spaceRocks, (string s) => s == "meteorite");
```

```
            if (makesContact)
```

```
            {
```

```
                Console.WriteLine("At least one space rock has reached the Earth's  
surface!");
```

```
            }
```

```
        }
```

```
    }
```



```
}
```

OUTPUT

At least one space rock has reached the Earth's surface!

9.6 Review

Well done! We learned two shortcuts for defining methods:

Expression-bodied definitions can be used for one-line method bodies.

```
bool isEven(int num) => num % 2 == 0;
```

Lambda expressions can be used to create an anonymous method:

```
bool hasEvenNumbers = Array.Exists(numbers, (int num) => num % 2 == 0 );
```

You learned two “sub-shortcuts” within lambda expressions:

- You can remove the parameter type if it can be inferred:

```
bool hasEvenNumbers = Array.Exists(numbers, (num) => num % 2 == 0 );
```

- You can remove the parentheses if there is one parameter:

```
bool hasEvenNumbers = Array.Exists(numbers, num => num % 2 == 0 );
```

9.6 Review Instructions

Check out the alternate expressions in the text editor! Make sure you understand this code before moving on.

Program.cs

```
using System;
```

```

namespace AlternateExpressions
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] nums = {0, 555, 252, 3, 9, 101};

            bool hasBigNum = Array.Exists(nums, IsBig);

            bool hasSmallNum = Array.Exists(nums, IsSmall);

            bool hasMediumNum = Array.Exists(nums, (n) => n >= 10 && n <= 100);

            Console.WriteLine($"Any big #s? {hasBigNum}");
            Console.WriteLine($"Any small #s? {hasSmallNum}");
            Console.WriteLine($"Any medium #s? {hasMediumNum}");

        }

        static bool IsBig(int n) => n > 100;

        static bool IsSmall(int n) => n < 10;
    }
}

```

}

}

OUTPUT

Any big #s? True

Any small #s? True

Any medium #s? False

10. ARRAYS

10.1 Introduction to Arrays

You want to write a program that keeps track of the heights of different plants in your garden. You could do this as a series of variables, but those would become difficult to work with really fast.

```
// height in inches  
int plantOne = 4;  
int plantTwo = 3;  
int plantThree = 6;
```

Luckily, C# provides us with a built-in solution to easily keep track of lots of different pieces of information. *Data structures* are formats designed to store larger amounts of information in an organized fashion.

An *array* is one very basic data structure. Programmers use arrays as a container to store multiple pieces of information that relate to each other in some way. Like a list of the presidents of the United States, types of cheeses in alphabetical order, and the finishing positions of runners in a race.

What makes arrays special is that they order our data in a specific, linear sequence. Since our values are kept in order, it allows us to easily find the information we're looking for; otherwise, we'd have a huge jumbled mess of data! Which means, rather than having to create multiple variables and retrieve information using a variable name, we can use a single array name and retrieve it using its location in that array.

You're already familiar with strings - strings are arrays of characters. Just like we can use syntax to access parts of string, we can do so with arrays.

In this lesson, we'll cover:

- How to build arrays in C#
- How to find the length of an array
- How to access array values
- How to edit arrays
- How to read documentation and use built-in methods

10.1 Introduction to Arrays Instructions

Comic strips function in a similar way to lists. We can think of the strip as the list and each frame as a separate item in the list. The narrative that makes up a comic demonstrates that the items follow a specific order.

We create lists by adding items to an empty list. Right now we have a bunch of comic frames, but they're not in any order and our comic strip is empty. Complete the comic strip by placing the frames in the correct order. Here's how the story should go:

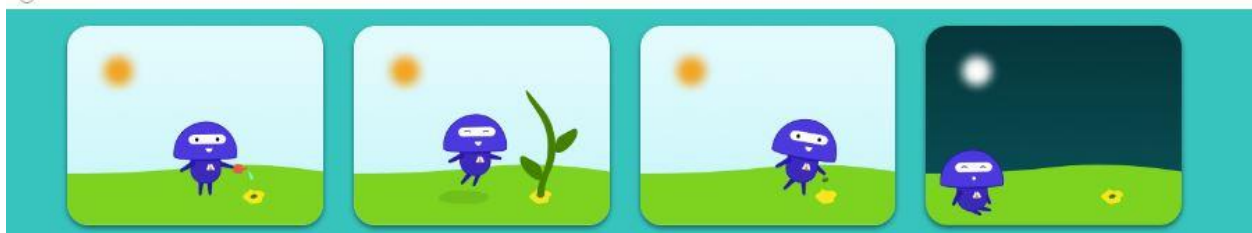
- Codey plants a seed
- Codey waters the seed
- Codey waits
- A sprout grows

Codey plants a seed

[]

length = 0

← Drag a frame into the empty comic strip



10.2 Building Arrays

In C#, arrays are a collection of values that all share the same data type. You could have an array of type `string` that contains a list of your favorite songs, or an array of type `int` that stores a collection of user ids.

Similar to defining a variable for one piece of data, when we define a variable to hold an array we also have to specify the type:

```
// These arrays store ints, strings, and doubles, respectively
int[] x;
string[] s;
double[] d;
```

To declare a variable that holds an array, we first write the type of data that will be stored in an array, then add the square brackets `[]` to signify that it is holding an array (rather than a single value), followed by the name of the array.

Like a variable, we can define and initialize an array at the same time, by specifying the values we want to store in it:

```
int[] plantHeights = { 3, 4, 6 };
```

To declare and initialize an array at the same time, after the array declaration we use the equal sign to denote we're storing a value to the array, then write out the numbers we're putting in the array, separated by commas `,` and enclosed in curly braces `{}`.

You may also see arrays defined and initialized using a new keyword:

```
int[] plantHeights = new int[] { 3, 4, 6 };
```

The new keyword signifies that we are instantiating a new array from the array class. We'll cover classes and instantiation in another lesson, but for now you can just think of it as another way to create an array.

In fact, if you decide to define an array and then initialize it later (rather in one line like above) you **must** use the new keyword:

```
// Initial declaration
int[] plantHeights;

// This works
plantHeights = new int[] { 3, 4, 6 };

// This will cause an error
// plantHeights = { 3, 4, 6 };
```

10.2 Building Arrays Instructions

1.

You want to build a web app where users can create their own playlists. The trick is, they can only create playlists that have eight songs on them. You realize that you can use arrays to store information about each playlist as an array.

First, declare a string array called `summerStrut`. This will be the playlist that we add our songs to.

2.

On a new line, initialize your array with eight song titles as values. If you can't think of any, here are some suggestions:

- *Juice*
- *Missing U*
- *Raspberry Beret*
- *New York Groove*
- *Make Me Feel*
- *Rebel Rebel*
- *Despacito*

- *Los Angeles*

3.

Next, declare and initialize an array named ratings that contains your rating for each song (between 1 - 5).

Program.cs

```
using System;
```

```
namespace BuildingArrays
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

10.3 Array Length

We often want to know how many items an array contains. We can do this with the `.Length` property.


```
int[] plantHeights = { 3, 4, 6 };
```

```
// arrayLength will be 3
```

```
int arrayLength = plantHeights.Length
```

Using the .Length property will return the number of items in an array and zero if the array is empty.

10.3 Array Length Instructions

1.

Each playlist can only have eight songs, so we want to write a program that checks to make sure that there are the right amount of songs.

If there are eight songs in the playlist, have the console print out a message that lets the user know the playlist is complete, like “summerStrut Playlist is ready to go!”

2.

If a user tries to add more than eight songs to a playlist, write them a message that their playlist will be rejected for being too long. You can write something like, “Too many songs!”

To check that it works, add another song to the array.

3.

If there are less than eight songs in the playlist, let the user know that they should add more songs. You can write something like, “Add some songs!”

To check that it works, delete songs from the playlist.

Program.cs

```
using System;
```

```
namespace ArrayLength
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] summerStrut;
```

```
            summerStrut = new string[] { "Juice", "Missing U", "Raspberry Beret", "New  
York Groove", "Make Me Feel", "Rebel Rebel", "Despacito", "Los Angeles" };
```

```
        }
```

```
    }
```

```
}
```

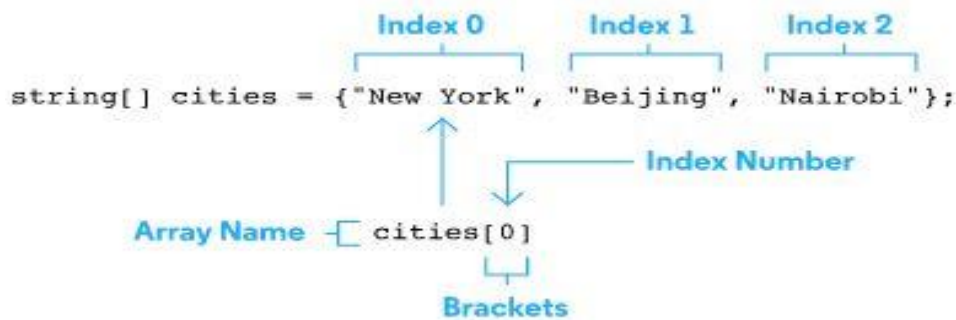
OUTPUT

Add some songs!

10.4 Accessing Array Items

Arrays are useful for storing values, but they're not very useful if they simply stay there — we also need a way to access them.

Arrays order items so that they're in a specific sequence, which makes it helpful for accessing each item. Each value has a specific position in the array, which is known as its *index*. You can think of an index like an address - it's what we use to locate an item in an array. In C#, arrays start their index at 0 and then add one for each value.



To access a value from a list, we write out the name of the array, followed by brackets `[]` and within the brackets, the index number of that value that we want:

```
int[] plantHeights = {3, 4, 6};
```

```
// plantTwoHeight will be 4
```

```
int plantTwoHeight = plantHeights[1];
```

We can now use that value and do something with it, like save it to a variable.

10.4 Accessing Array Items Instructions

1.

Print out a statement that includes the name of the second song in the playlist and the rating that you gave it. The message should read something like "You rated the song Missing U 4 stars."

Program.cs

```
using System;
```

```
namespace AccessingArrays
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] summerStrut;
```

```
            summerStrut = new string[] { "Juice", "Missing U", "Raspberry Beret", "New  
York Groove", "Make Me Feel", "Rebel Rebel", "Despacito", "Los Angeles" };
```

```
            int[] ratings = { 5, 4, 4, 3, 3, 5, 5, 4 };
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

You rated the song Missing U 4 stars.

10.5 Editing Arrays

Once we create an array, the size of that array is fixed. However, it's possible to change the values it contains.

For example, we can initialize an array that has a length of three without specifying what those values are, then later go back and edit the array to include a new value. This is useful if we know how many things we're expecting, but we don't know their specific values yet:

```
// plantHeights will be equal to [0, 0, 0]
int[] plantHeights = new int[3];
```

```
// plantHeights will now be [0, 0, 8]
plantHeights[2] = 8;
```

When we create the array with a known length but no known values, the array stores a *default type value* ([0 for int, null for string](#)). We then edit the array and swap out one of the default values with a new, specific value. In this case, we're saying that at index 2 we want to swap the default value 0 for 8.

We can also edit the values of pre-existing arrays. Again, we can't add to the length of an existing array, but we can swap out values:

```
int[] plantHeights = { 3, 4, 6 };
```

```
// plantHeights will be [3, 5, 6]
plantHeights[1] = 5;
```

In this case, we already have an array with a defined set of values, { 3, 4, 6 }. But what if a plant grows? We'll need to go back in and change its value. So if it's the second plant, we access its value using bracket notation, then change its value to 5.

10.5 Editing Arrays Instructions

1.

You've decided that you want to add Cardi B's *I Like It* to your playlist. But whoops! Only 8 tracks allowed! Swap the last song in the playlist out for *I Like It* (or another song of your choosing).

2.

Since you swapped out the song, you also have to update the rating! Change the rating to reflect the playlist update.

Program.cs

```
using System;
```

```
namespace EditingArrays
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] summerStrut;
```

```
            summerStrut = new string[] { "Juice", "Missing U", "Raspberry Beret", "New  
York Groove", "Make Me Feel", "Rebel Rebel", "Despacito", "Los Angeles" };
```

```
            int[] ratings = { 5, 4, 4, 3, 3, 5, 5, 4 };
```

```
        }
```

```
    }
```

```
}
```

10.6 Built-In Methods

In C#, there are several built-in methods we can use with arrays. The full list can be found in the [Microsoft documentation of the Array class, under methods](#).

SORT

The built-in method `Array.Sort()` ([documentation](#)), as its name suggests, sorts an array. This method is a quick way to further organize array data into a logical sequence:

```
int[] plantHeights = { 3, 6, 4, 1 };
```

```
// plantHeights will be { 1, 3, 4, 6 }
```

```
Array.Sort(plantHeights);
```

`Sort()` takes an array as a parameter and edits the array so its values are sorted. If it is an array of integer values, it will sort them into ascending values (lowest to highest). If it's an array of string values, they would be sorted alphabetically.

INDEX OF

The Array method `Array.IndexOf()` ([documentation](#)) takes a value and returns its index. `IndexOf()` works best when you have a specific value and need to know where it's located in the array (or if it even exists!).

```
int[] plantHeights = { 3, 6, 4, 1, 6, 8 };
```

```
// returns 1
```

```
Array.IndexOf(plantHeights, 6);
```

`IndexOf()` typically takes two parameters: the first is the array and the second is the value whose index we're locating. `IndexOf()` also has several overloads that allow you to search for a specific range of the array. If the value appears more than once in an array, it returns only the first occurrence within the specified range. If it cannot find the value, it returns the lower bound of the array, minus 1 (since most arrays start at 0, it's usually -1).

FIND

The Array method `Array.Find()` ([documentation](#)) searches a one-dimensional array for a specific value or set of values that match a certain condition and returns the first occurrence in the array.

```
int[] plantHeights = { 3, 6, 4, 1, 6, 8 };
```

```
// Find the first occurrence of a plant height that is greater than 5 inches  
int firstHeight = Array.Find(plantHeights, height => height > 5);
```

`Find()` takes two parameters: the first is the array and the second is a *predicate* that defines what we're looking for. A predicate is a method that takes one input and outputs a boolean. Unlike `IndexOf()`, `Find()` returns the actual values that match the condition, instead of their index.

It's customary to use a lambda function for the predicate to determine if the value meets the necessary criteria. If you need a refresher on lambda functions, check out our C# methods lesson.

For more information on using built-in methods, check out our lessons on [C#: Methods](#).

10.6 Built-In Methods Instructions

1.

Using an Array method, find the position for the first 3-star rated song and save it to a variable. Print a message to the console, like "Song number X is rated three stars".

2.

Find the first song that has more than 10 characters in its title. Save it to a variable and print a message to the console, such as "The first song that has more than 10 characters in the title is X."

3.

Organize the playlist alphabetically. To check that it worked, print the first and last song titles to the console.

Program.cs

```
using System;
```

```
namespace BuiltInMethods
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] summerStrut;
```

```
            summerStrut = new string[] { "Juice", "Missing U", "Raspberry Beret", "New  
York Groove", "Make Me Feel", "Rebel Rebel", "Despacito", "Los Angeles" };
```

```
            int[] ratings = { 5, 4, 4, 3, 3, 5, 5, 4 };
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

Song number 4 is rated three stars.

The first song that has more than 10 characters in the title is Raspberry Beret.

The first song in the playlist is now Despacito.

The last song in the playlist is now Rebel Rebel.

10.7 Documentation Hunt

In addition to `Sort()`, `IndexOf()`, and `Find()`, there are several other built-in methods for arrays. You can find them (and you probably guessed it) in the [Microsoft documentation](#).

Let's put your documentation sleuthing to use and hunt down some built-in methods to write some code!

10.7 Documentation Hunt Instructions

1.

In the [Microsoft documentation](#), find the method that allows you to copy your playlist to a new playlist called `summerStrutCopy`. Print the first value of `summerStrutCopy` to the playlist to see if it is the same as `summerStrut`.

2.

In the [Microsoft documentation](#), find the method that reverses the order of the array elements. Use it to reverse the order of the `summerStrut` playlist. Check to see if it worked by printing the first and last songs to the console.

3.

In the [Microsoft documentation](#), find the method that turns every rating in the ratings array to zero. Check to see if it worked by printing out the first value to the console (it should be to 0).

Program.cs

```
using System;
```

```
namespace DocumentationHunt
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] summerStrut;
```

```
            summerStrut = new string[] { "Juice", "Missing U", "Raspberry Beret", "New  
York Groove", "Make Me Feel", "Rebel Rebel", "Despacito", "Los Angeles" };
```

```
            int[] ratings = { 5, 4, 4, 3, 3, 5, 5, 4 };
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

Juice

Los Angeles

Juice

0

10.8 Review

Congratulations, we covered a lot in this lesson! We learned about:

- Data structures and how we can use them to better organize our data
- How to build, access, and edit values in arrays
- How to use Array class built-in methods, including Sort(), IndexOf(), and Find()

If you get stuck or would like to learn more, check out the official Microsoft site:

- For general conceptual information, read [the programming guide on Arrays](#)
- For specific information on methods like Find(), read [the Array class documentation](#)

10.8 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson.

Program.cs

```
using System;
```

```
namespace ArraysReview
{
    class Program
    {
        static void Main(string[] args)
        {
            /* use this space to write your own short program!
            Here's what you learned:

            Building Arrays
            .Length Property
            Accessing and Editing Arrays using bracketNotation[]
            Built-in-Methods: Sort(), IndexOf(), Find()

            Good luck! */
        }
    }
}
```

Concept Review

C# Arrays

In C#, an *array* is a structure representing a fixed length ordered collection of values or objects with the same type.

Arrays make it easier to organize and operate on large amounts of data. For example, rather than creating 100 integer variables, you can just create one array that stores all those integers!

```
// `numbers` array that stores integers
```

```
int[] numbers = { 3, 14, 59 };
```

```
// 'characters' array that stores strings
```

```
string[] characters = new string[] { "Huey", "Dewey", "Louie" };
```

Declaring Arrays

A C# array variable is declared similarly to a non-array variable, with the addition of square brackets ([]) after the type specifier to denote it as an array.

The new keyword is needed when instantiating a new array to assign to the variable, as well as the array length in the square brackets. The array can also be instantiated with values using curly braces ({}). In this case the array length is not necessary.

```
// Declare an array of length 8 without setting the values.
```

```
string[] stringArray = new string[8];
```

```
// Declare array and set its values to 3, 4, 5.
```

```
int[] intArray = new int[] { 3, 4, 5 };
```

Declare and Initialize array

In C#, one way an array can be declared and initialized at the same time is by assigning the newly declared array to a comma separated list of the values surrounded by curly braces ({}). Note how we can omit the type signature and new keyword on the right side of the assignment using this syntax. This is only possible during the array's declaration.

```
// `numbers` and `animals` are both declared and initialized with values.
```

```
int[] numbers = { 1, 3, -10, 5, 8 };
```

```
string[] animals = { "shark", "bear", "dog", "raccoon" };
```

Array Element Access

In C#, the elements of an array are labeled incrementally, starting at 0 for the first element. For example, the 3rd element of an array would be indexed at 2, and the 6th element of an array would be indexed at 5.

A specific element can be accessed by using the square bracket operator, surrounding the index with square brackets. Once accessed, the element can be used in an expression, or modified like a regular variable.

```
// Initialize an array with 6 values.
```

```
int[] numbers = { 3, 14, 59, 26, 53, 0 };
```

```
// Assign the last element, the 6th number in the array (currently 0), to 58.
```

```
numbers[5] = 58;
```

```
// Store the first element, 3, in the variable `first`.
```

```
int first = numbers[0];
```

C# Array Length

The *Length* property of a C# array can be used to get the number of elements in a particular array.

```
int[] someArray = { 3, 4, 1, 6 };
```

```
Console.WriteLine(someArray.Length); // Prints 4
```

```
string[] otherArray = { "foo", "bar", "baz" };
```

```
Console.WriteLine(otherArray.Length); // Prints 3
```

C# For Loops

A C# *for loop* executes a set of instructions for a specified number of times, based on three provided expressions. The three expressions are separated by semicolons, and in order they are:

- *Initialization*: This is run exactly once at the start of the loop, usually used to initialize the loop's iterator variable.
- *Stopping condition*: This boolean expression is checked before each iteration to see if it should run.
- *Iteration statement*: This is executed after each iteration of the loop, usually used to update the iterator variable.

// This loop initializes i to 1, stops looping once i is greater than 10, and increases i by 1 after each loop.

```
for (int i = 1; i <= 10; i++) {
```

```
    Console.WriteLine(i);
```



```
}
```

```
Console.WriteLine("Ready or not, here I come!");
```

C# For Each Loop

A C# foreach loop runs a set of instructions once for each element in a given collection. For example, if an array has 200 elements, then the foreach loop's body will execute 200 times. At the start of each iteration, a variable is initialized to the current element being processed.

A *for each* loop is declared with the foreach keyword. Next, in parentheses, a *variable type* and *variable name* followed by the in keyword and the collection to iterate over.

```
string[] states = { "Alabama", "Alaska", "Arizona", "Arkansas", "California",  
"Colorado" };
```

```
foreach (string state in states) {
```

```
    // The `state` variable takes on the value of an element in `states` and updates  
    every iteration.
```

```
    Console.WriteLine(state);
```

```
}
```

```
// Will print each element of `states` in the order they appear in the array.
```

C# While Loop

In C#, a *while loop* executes a set of instructions continuously while the given boolean expression evaluates to true or one of the instructions inside the loop body, such as the `break` instruction, terminates the loop.

Note that the loop body might not run at all, since the boolean condition is evaluated before the very first iteration of the *while loop*.

The syntax to declare a while loop is simply the `while` keyword followed by a boolean condition in parentheses.

```
string guess = "";
Console.WriteLine("What animal am I thinking of?");

// This loop will keep prompting the user, until they type in "dog".
while (guess != "dog") {
    Console.WriteLine("Make a guess:");
    guess = Console.ReadLine();
}
Console.WriteLine("That's right!");
```

C# Do While Loop

In C#, a *do while* loop runs a set of instructions once and then continues running as long as the given boolean condition is true. Notice how this behavior is nearly identical to a *while* loop, with the distinction that a *do while* runs one or more times, and a *while* loop runs zero or more times.

The syntax to declare a *do while* is the `do` keyword, followed by the code block, then the `while` keyword with the boolean condition in parentheses. Note that a semi-colon is necessary to end a *do while* loop.

```
do {  
    DoStuff();  
} while(boolCondition);
```

// This do-while is equivalent to the following while loop.

```
DoStuff();  
while (boolCondition) {  
    DoStuff();  
}
```

C# Infinite Loop

An *infinite loop* is a loop that never terminates because its stopping condition is always false. An *infinite loop* can be useful if a program consists of continuously executing one chunk of code. But, an unintentional *infinite loop* can cause a program to hang and become unresponsive due to being stuck in the loop.

A program running in a shell or terminal stuck in an infinite loop can be ended by terminating the process.

```
while (true) {  
    // This will loop forever unless it contains some terminating statement such as  
    `break`.  
}
```

C# Jump Statements

Jump statements are tools used to give the programmer additional control over the program's control flow. They are very commonly used in the context of loops to exit from the loop or to skip parts of the loop.

Control flow keywords include `break`, `continue`, and `return`. The given code snippets provide examples of their usage.

```
while (true) {  
    Console.WriteLine("This prints once.");  
    // A `break` statement immediately terminates the loop that contains it.  
    break;  
}  
  
for (int i = 1; i <= 10; i++) {  
    // This prints every number from 1 to 10 except for 7.  
    if (i == 7) {  
        // A `continue` statement skips the rest of the loop and starts another iteration  
        // from the start.  
        continue;  
    }  
    Console.WriteLine(i);  
}  
  
static int WeirdReturnOne() {  
    while (true) {  
        // Since `return` exits the method, the loop is also terminated. Control returns  
        // to the method's caller.  
        return 1;  
    }  
}
```

11. LOOPS

11.1 Introduction to Loops

Imagine you're building a video game and in this game, you want to add 15 aliens to the screen.

How do we use code to tell a computer this: "Create a variable and call the method 15 times"?

We could write it out 15 times:

```
AddAlien();  
AddAlien();  
AddAlien();  
AddAlien();  
...
```

...We'll spare you the rest. This approach takes a long time and it can easily lead to mistakes. Instead, let's give the instructions once and tell the computer how many times to repeat them:

```
for (int i = 0; i < 15; i++)  
{  
    AddAlien();  
}
```

Loops are structures that we can use to repeat instructions until a certain condition is met. That condition could be a change in state ("keep playing music until 10pm"), an end of a list ("say each name out loud"), or a number ("knock three times").

When we see repetition in our code, it's a good sign that we should probably use a loop. By not writing out instructions multiple times, we reduce our chance for errors and save ourselves some time.

There are several kinds of loops that we can use depending on the situation. This lesson will cover:

- while loops
- do...while loops
- for loops
- foreach loops

11.1 Introduction to Loops Instructions

Click the shapes in the diagram to move through each step of the loop structure. What is the condition or count that must be met to stop looping?



11.2 While Loop

Loops are used to repeat a set of instructions based on a set of conditions. If this makes you think of conditional statements, you're not wrong!

The while loop looks very similar to an if statement. Just like an if statement, it executes the code inside of it if the condition, a statement that evaluates to a boolean value, is true.

```
while (condition)
{
    statement;
}
```

However, unlike an if statement that runs once, a while loop will continue to execute the code inside of it, over and over again, *while* the condition is true. The computer is constantly checking to see if the condition is satisfied. For this reason, while loops are useful when you know at what point a program should stop, rather than the number of times it should repeat.

In your video game, you want the player to rise up in the air as long as the user is pressing the spacebar:

```
while (spacebar == "down")
{
    RiseUp();
}
```

In this example, while spacebar is equal to down, the character will keep rising on the screen. It will exit the while loop once the user releases the spacebar and then the rest of the program will continue.

while loops can get dangerous, because they depend on that condition to at some point return false. What if we never take our finger off the spacebar? Sounds ridiculous, but theoretically, the program would run forever! This is known as an *infinite loop*. If you get stuck in an infinite loop while on Codecademy, you can reload the page to stop it.

11.2 While Loop Instructions

1.

You're really trying to step up your "inbox zero" game and want to build a tool that can help you get there. For your first prototype, you're just going to delete all the emails.

Write a while loop that checks to see if there are any emails in your inbox. If there are still emails, decrease the amount of emails by one until there are no emails left.

If the test runs infinitely, you probably ran into an infinite loop! Refresh the page and make sure your loop is able to terminate.

2.

It's a little hard to tell what's happening in our program, so within the loop have it print a message that it's deleting an email and how many emails are left.

3.

When your inbox reaches zero, have your program print out "INBOX ZERO ACHIEVED!" or some other congratulatory message.

Program.cs

```
using System;
```

```
namespace WhileLoop
```

```
{
```

```
    class Program
```

```
    {
```



```
static void Main(string[] args)
{
    int emails = 20;

}
}
}
```

OUTPUT

one email deleted, 19 left
one email deleted, 18 left
one email deleted, 17 left
one email deleted, 16 left
one email deleted, 15 left
one email deleted, 14 left
one email deleted, 13 left
one email deleted, 12 left
one email deleted, 11 left
one email deleted, 10 left
one email deleted, 9 left
one email deleted, 8 left
one email deleted, 7 left
one email deleted, 6 left
one email deleted, 5 left
one email deleted, 4 left

one email deleted, 3 left

one email deleted, 2 left

one email deleted, 1 left

one email deleted, 0 left

INBOX ZERO ACHIEVED!

11.3 Do...While Loop

Similar to the while loop, a do...while loop will continue running until a stopping condition is met. One key difference is that no matter what, a do...while loop will *always run once*.

```
do
{
    statement;
} while (condition);
```

Instead of checking the condition before the code block executes, the program in the block runs once and *then* checks the conditional statement. It will either stop or continue to execute until the condition is no longer true. do...while loops are good for when a program should execute at least once and then depending on the circumstances, continue to execute or stop.

In your video game, you want to show the start screen for at least one frame. If the user does not hit next, you want the start screen to continue to appear until they do so.

```
bool startGame = false;
```

```
do
{
    ShowStartScreen();
} while (!startGame);
```

In this example, we initialize a Boolean value `startGame` to the state `false`. The program runs once, showing the start screen for one frame, then checks to see if the player has started the game. If the player has yet to start the game, it will call `ShowStartScreen()` again and will then continue to show the start screen until the player presses start.

11.3 Do...While Loop Instructions

1.

You've decided to try making another productivity tool. This time you build a [pomodoro timer](#). When the timer finishes, an alarm will ring once. If you don't click the button to turn it off, the alarm will repeat until it is turned off.

Write a program that plays the alarm once, and then if the button isn't clicked (`!buttonClick`), it will repeat the alarm. To simulate an alarm, you can print something like "BLARRRRRRRRR" to the console.

2.

After the loop finishes, the program should print a message telling you to take a break, like: "Time for a five minute break."

Program.cs

```
using System;
```

```
namespace DoWhileLoop
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
{  
    bool buttonClick = true;  
  
}  
  
}  
  
}
```

OUTPUT

BLARRRRRRRRR

Time for a five minute break

11.4 For Loop

What if we want our code to execute a specific number of times? We can use a for loop to do that.

```
for (initialization; stopping condition; iteration statement)  
{  
    statement;  
}
```

The for loop tells the computer how many times to repeat the instructions using the for keyword and three expressions inside of parentheses. Each of these expressions use what's known as an iterator variable, which is a variable that keeps track of how many times the program goes through the loop.

These expressions are:

- *Initialization*: where the loop begins

- *Stopping condition*: the condition that the iterator variable is evaluated against
- *Iteration statement*: used to update the iterator variable on each loop

The for loop is good to use when you know the number of times you'd like to perform a task before you begin, like printing three copies of a document or inserting eight rows into a table.

In our video game, we want ten flags to appear at the start of each level:

```
for (int i = 0; i < 10; i++)  
{  
    DisplayFlag();  
}
```

When a computer receives this program it sets a counter to 0 and executes the instructions in the body of the loop, which in this case instructs the computer to display a flag. After each iteration, or one turn through the loop, the program advances the counter by one (i++). The process repeats until the counter reaches 10, meaning ten iterations are completed and there are ten flags on the screen.

11.4 For Loop Instructions

1.

For your next tool, you want to create a template for your weekly team meeting. Rather than clone a new one each week, you decide to make all of them at once.

Write an empty for loop that runs once for each week in your 16-week long project.

2.

Inside of the loop, call the `CreateTemplate()` method. It takes a number as a parameter that represents what week it is within the project so that when the

template is generated it will say Week X at the top, with X representing which week it is.

Program.cs

```
using System;
```

```
namespace ForLoop
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
    }
```

```
    static void CreateTemplate(int week)
```

```
    {
```

```
        Console.WriteLine($"Week {week}");
```

```
        Console.WriteLine("Announcements: \n \n \n ");
```

```
        Console.WriteLine("Report Backs: \n \n \n");
```

```
        Console.WriteLine("Discussion Items: \n \n \n");
```

```
    }
```

}

}

OUTPUT

Week 1

Announcements:

Report Backs:

Discussion Items:

Week 2

Announcements:

Report Backs:

Discussion Items:

Week 3

Announcements:

Report Backs:

Discussion Items:

Week 4

Announcements:

Report Backs:

Discussion Items:

Week 5

Announcements:

Report Backs:

Discussion Items:

Week 6

Announcements:

Report Backs:

Discussion Items:

Week 7

Announcements:

Report Backs:

Discussion Items:

Week 8

Announcements:

Report Backs:

Discussion Items:

Week 9

Announcements:

Report Backs:

Discussion Items:

Week 10

Announcements:

Report Backs:

Discussion Items:

Week 11

Announcements:

Report Backs:

Discussion Items:

Week 12

Announcements:

Report Backs:

Discussion Items:

Week 13

Announcements:

Report Backs:

Discussion Items:

Week 14

Announcements:

Report Backs:

Discussion Items:

Week 15

Announcements:

Report Backs:

Discussion Items:

Week 16

Announcements:

Report Backs:

Discussion Items:

11.5 For Each Loop

There's one more way to give looping instructions to a computer. We define a sequence of values and tell the computer to repeat the instructions for each item in the sequence.

```
foreach (type element in sequence)
{
    statement;
}
```

The foreach loop is used to iterate over collections, such as an array.

In our video game, we want to play a melody. We can do that by iterating through a list of individual notes, playing one after the other. Here's an example array of notes:

```
string[] melody = { "a", "b", "c", "c", "b" };
```

and the loop would look like:

```
foreach (string note in melody)
{
    PlayMusic(note);
}
```

The sequence we used was an array, but we can use other similar data structures. The umbrella term for those is collection, so we can also call foreach loops *collection loops*.

Use this loop when you need to perform a task for every item in a list, or when the order of things must be maintained. In this case, both are important. A note must be placed for each item in the list and the order of them is essential to the musical pattern.

11.5 For Each Loop Instructions

1.

Now you want to create a To Do list to keep track of your tasks. Write an empty loop that will iterate through each item in your todo array.

2.

Inside of the loop, call the `CreateTodoItem()` method.

Program.cs

```
using System;
```

```
namespace ForEachLoop
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            string[] todo = { "respond to email", "make wireframe", "program feature", "fix  
bugs" };
```

```
        }
```

```
        static void CreateTodoItem(string item)
```

```
        {
```

```
            Console.WriteLine($"{item}");
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

- [] respond to email
- [] make wireframe
- [] program feature
- [] fix bugs

11.6 Comparing Loops

You may have noticed that there are lots of similarities between different types of loops, and you're right!

We just showed how we can use a foreach loop to iterate through an array. But we can also use a for loop to iterate through an array:

```
string[] items = { "potion", "dagger", "shield", "plant" };
```

```
for (int i = 0; i < items.Length; i++)  
{  
    Console.WriteLine(items[i]);  
}
```

We could even write a complicated while loop that starts a counter at 0, then compares that counter to the length of the items array. If the counter is less than the array, the loop will continue. Otherwise, it will stop looping through the statements and the program will move on to the next line of code.

```
int i = 0;  
while (i < items.Length)  
{  
    Console.WriteLine(items[i]);  
    i++;  
}
```

Since a foreach loop does the same thing as the other two but is more concise, it is less prone to errors and the better choice in this circumstance.

```
string[] items = { "potion", "dagger", "shield", "plant" };
```

```
foreach (string item in items)
{
    Console.WriteLine(item);
}
```

In general,

- while loops are good when you know your stopping condition, but not when you know how many times you want a program to loop or if you have a specific collection to loop through.
- do...while loops are only necessary if you definitely want something to run once, but then stop if a condition is met.
- for loops are best if you want something to run a specific number of times, rather than given a certain condition.
- foreach loops are the best way to loop over an array, or any other collection.

11.6 Comparing Loops Instructions

1.

You want to build an app that blocks websites, so you find some code online and copy and paste it into your text editor. You notice that it uses a while loop to iterate through a websites array, but you know a better way to do that!

Re-write the loop so that it uses a loop that better suits the objective.

Program.cs

```
using System;
```

```
namespace ComparingLoops
```

```
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            string[] websites = { "twitter", "facebook", "gmail" };  
            int counter = 0;  
  
            while (counter < websites.Length)  
            {  
                Console.WriteLine(websites[counter]);  
                counter++;  
            }  
        }  
    }  
}
```

OUTPUT

twitter

facebook

gmail

11.7 Jump Statements

There are a few keywords we can use to add further control flow to our loops. Typically, they work with a series of *nested loops*, where one loop is written entirely within the body of another loop. These keywords are often used to limit while loops and prevent them from creating infinite loops.

BREAK

At any point within a loop block, you can end it by using the break keyword.

```
while (playerIsAlive)
{
    // this code will keep running

    if (playerIsAlive == false)
    {
        // eventually if this stopping condition is true,
        // it will break out of the while loop
        break;
    }
}
```

```
// rest of the program will continue
```

You've already seen the break keyword— it's the same keyword that is used in switch statements.

CONTINUE

The continue keyword is used to bypass portions of code. It will ignore whatever comes after it in the loop and then will go back to the top and start the loop again.

```
int bats = 10;

for (int i = 0; i <= 10; i++)
{
    if (i < 9)
    {
        continue;
    }
}
```

```

}
// this will be skipped until i is no longer less than 9
Console.WriteLine(i);
}

```

Here, the program starts in the for loop, then hits the if statement. Since there is a continue in the if statement, it will bypass the Console.WriteLine() statement until the condition in the if statement is no longer true. So while the loop starts at 0, nothing will print to the console until i is equal to 9.

RETURN

The return keyword is another way to exit a loop, specifically loops that are used within a method. When a return is used within such a loop, it breaks out of the loop and returns control to the point in the program where the method was called.

```

class MainClass {
    public static void Main (string[] args) {
        UnlockDoor();

        // after it hits the return statement, it will move on to this method
        PickUpSword();
    }

    static bool UnlockDoor()
    {
        bool doorsLocked = true;

        // this code will keep running
        while (doorsLocked)
        {
            bool keyFound = TryKey();

            // eventually if this stopping condition is true,
            // it will break out of the while loop
            if (keyFound)

```

```

{
    // this return statement will break out of the entire method
    return true;
}
}
return false;
}
}

```

You should only use return if you need to exit a method because it will break out of *all* loops. If you only want to break out of one loop and not exit a method, use break.

11.7 Jump Statements Instructions

1.

You've decided to go back to the pomodoro application. This time, you don't want the alarm to ring endlessly. Once it rings 3 times, it should shut off even if the button has not been clicked.

Create a variable that will keep track of how many times the alarm has gone off.

Note that we have temporarily set the initial value of buttonClick to true.

Otherwise, running the code as is would result in an infinite loop! You will update the starting value of buttonClick in a later step.

2.

Inside the do...while loop, increase the count every time the alarm goes off.

3.

The program should break out of the loop if the count variable reaches three.

Write a statement that checks if the count variable has reached three, and when it does, have it break out of the do...while loop.

Now, you can update the starting value of buttonClick to false to see the alarm ring three times.

Program.cs

```
using System;
```

```
namespace JumpStatements
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            bool buttonClick = true;
```

```
            do
```

```
            {
```

```
                Console.WriteLine("BLARRRRR");
```

```
            } while(!buttonClick);
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

BLARRRRR

BLARRRRR

BLARRRRR

11.8 Review

Well done! In C#, loops are commonly used because they save time, reduce errors, and are easy to read. Being comfortable with each type of loop will make you a better programmer. In review:

- A *loop* is a structure in programming where the instructions are written once, but a computer can execute them multiple times
- Each execution of those instructions is called an *iteration*
- while loops repeat until a condition changes
- do...while loops execute once, and then repeat until a condition changes
- for loops repeat for a specified number of times
- foreach loops repeat for each item in a collection
- *jump statements*, like `break`, `continue`, and `return` are used to add additional control flow to loops

Now that you know a few things about loops, try writing a program that:

- Loops through a piece of text and only prints words that start with the letter “a” to the console to create a poem.
- Loops through a list of numbers and if it is even, print even and if it’s odd, print odd.
- A Choose Your Own Adventure game that uses a while loop to make sure a user chooses a correct option.

11.8 Review Instructions

Time for some practice! Use the code editor to play around with what you learned in this lesson. If you're not sure what to do, try one of the extensions above!

Program.cs

```
using System;
```

```
namespace LoopsReview
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            /* use this space to write your own short program!
```

```
            Here's what you learned:
```

```
            while loop: while(){..}
```

```
            do...while loop: do{...}while();
```

```
            for loop: for(int i=0; i<x; i++){}
```

```
            foreach loop: foreach(int item in list){}
```

```
            jump statements: break, continue, return
```

```
            Good luck! */
```

```
        }
```

```
    }
```

}

Concept Review

C# Arrays

In C#, an *array* is a structure representing a fixed length ordered collection of values or objects with the same type.

Arrays make it easier to organize and operate on large amounts of data. For example, rather than creating 100 integer variables, you can just create one array that stores all those integers!

```
// `numbers` array that stores integers
```

```
int[] numbers = { 3, 14, 59 };
```

```
// 'characters' array that stores strings
```

```
string[] characters = new string[] { "Huey", "Dewey", "Louie" };
```

Declaring Arrays

A C# array variable is declared similarly to a non-array variable, with the addition of square brackets ([]) after the type specifier to denote it as an array.

The `new` keyword is needed when instantiating a new array to assign to the variable, as well as the array length in the square brackets. The array can also be instantiated with values using curly braces ({}). In this case the array length is not necessary.

```
// Declare an array of length 8 without setting the values.
```

```
string[] stringArray = new string[8];
```

```
// Declare array and set its values to 3, 4, 5.
```

```
int[] intArray = new int[] { 3, 4, 5 };
```

Declare and Initialize array

In C#, one way an array can be declared and initialized at the same time is by assigning the newly declared array to a comma separated list of the values surrounded by curly braces ({}). Note how we can omit the type signature and new keyword on the right side of the assignment using this syntax. This is only possible during the array's declaration.

```
// `numbers` and `animals` are both declared and initialized with values.
```

```
int[] numbers = { 1, 3, -10, 5, 8 };
```

```
string[] animals = { "shark", "bear", "dog", "raccoon" };
```

Array Element Access

In C#, the elements of an array are labeled incrementally, starting at 0 for the first element. For example, the 3rd element of an array would be indexed at 2, and the 6th element of an array would be indexed at 5.

A specific element can be accessed by using the square bracket operator, surrounding the index with square brackets. Once accessed, the element can be used in an expression, or modified like a regular variable.

```
// Initialize an array with 6 values.
```



```
int[] numbers = { 3, 14, 59, 26, 53, 0 };
```

```
// Assign the last element, the 6th number in the array (currently 0), to 58.
```

```
numbers[5] = 58;
```

```
// Store the first element, 3, in the variable `first`.
```

```
int first = numbers[0];
```

C# Array Length

The *Length* property of a C# array can be used to get the number of elements in a particular array.

```
int[] someArray = { 3, 4, 1, 6 };
```

```
Console.WriteLine(someArray.Length); // Prints 4
```

```
string[] otherArray = { "foo", "bar", "baz" };
```

```
Console.WriteLine(otherArray.Length); // Prints 3
```

C# For Loops

A C# *for loop* executes a set of instructions for a specified number of times, based on three provided expressions. The three expressions are separated by semicolons, and in order they are:

- *Initialization*: This is run exactly once at the start of the loop, usually used to initialize the loop's iterator variable.

- *Stopping condition*: This boolean expression is checked before each iteration to see if it should run.
- *Iteration statement*: This is executed after each iteration of the loop, usually used to update the iterator variable.

// This loop initializes i to 1, stops looping once i is greater than 10, and increases i by 1 after each loop.

```
for (int i = 1; i <= 10; i++) {
    Console.WriteLine(i);
}
```

```
Console.WriteLine("Ready or not, here I come!");
```

C# For Each Loop

A C# foreach loop runs a set of instructions once for each element in a given collection. For example, if an array has 200 elements, then the foreach loop's body will execute 200 times. At the start of each iteration, a variable is initialized to the current element being processed.

A *for each* loop is declared with the foreach keyword. Next, in parentheses, a *variable type* and *variable name* followed by the in keyword and the collection to iterate over.

```
string[] states = { "Alabama", "Alaska", "Arizona", "Arkansas", "California",
    "Colorado" };
```

```
foreach (string state in states) {
```

```
    // The `state` variable takes on the value of an element in `states` and updates
    every iteration.
```

```
    Console.WriteLine(state);  
}  
  
// Will print each element of `states` in the order they appear in the array.
```

C# While Loop

In C#, a *while loop* executes a set of instructions continuously while the given boolean expression evaluates to true or one of the instructions inside the loop body, such as the `break` instruction, terminates the loop.

Note that the loop body might not run at all, since the boolean condition is evaluated before the very first iteration of the *while loop*.

The syntax to declare a while loop is simply the `while` keyword followed by a boolean condition in parentheses.

```
string guess = "";  
  
Console.WriteLine("What animal am I thinking of?");  
  
// This loop will keep prompting the user, until they type in "dog".  
while (guess != "dog") {  
    Console.WriteLine("Make a guess:");  
    guess = Console.ReadLine();  
}  
  
Console.WriteLine("That's right!");
```

C# Do While Loop

In C#, a *do while* loop runs a set of instructions once and then continues running as long as the given boolean condition is true. Notice how this behavior is nearly identical to a *while* loop, with the distinction that a *do while* runs one or more times, and a *while* loop runs zero or more times.

The syntax to declare a *do while* is the *do* keyword, followed by the code block, then the *while* keyword with the boolean condition in parentheses. Note that a semi-colon is necessary to end a *do while* loop.

```
do {  
    DoStuff();  
} while(boolCondition);
```

// This do-while is equivalent to the following while loop.

```
DoStuff();  
while (boolCondition) {  
    DoStuff();  
}
```

C# Infinite Loop

An *infinite loop* is a loop that never terminates because its stopping condition is always false. An *infinite loop* can be useful if a program consists of continuously executing one chunk of code. But, an unintentional *infinite loop* can cause a program to hang and become unresponsive due to being stuck in the loop.

A program running in a shell or terminal stuck in an infinite loop can be ended by terminating the process.

```
while (true) {
```

```
// This will loop forever unless it contains some terminating statement such as  
`break`.  
}
```

C# Jump Statements

Jump statements are tools used to give the programmer additional control over the program's control flow. They are very commonly used in the context of loops to exit from the loop or to skip parts of the loop.

Control flow keywords include `break`, `continue`, and `return`. The given code snippets provide examples of their usage.

```
while (true) {  
    Console.WriteLine("This prints once.");  
    // A `break` statement immediately terminates the loop that contains it.  
    break;  
}
```

```
for (int i = 1; i <= 10; i++) {  
    // This prints every number from 1 to 10 except for 7.  
    if (i == 7) {  
        // A `continue` statement skips the rest of the loop and starts another iteration  
        from the start.  
        continue;  
    }  
    Console.WriteLine(i);  
}
```

```
}
```

```
static int WeirdReturnOne() {
```

```
    while (true) {
```

```
        // Since `return` exits the method, the loop is also terminated. Control returns  
        to the method's caller.
```

```
        return 1;
```

```
    }
```

```
}
```

11. Quiz

1. Why will this code throw an error?

```
public static void Main (string[] args)
```

```
{
```

```
    bool[] answers = {false, false, true};
```

```
    for (i = 0; i < answers.Length; i++)
```

```
    {
```

```
        Console.WriteLine(answers[i]);
```

```
    }
```

```
}
```

- a) This is an infinite loop.
- b) Length is not a valid property of an array.
- c) The loop will try to access an index outside of the bounds of the array.
- d) The type for variable i is undefined.

2. Define an array of doubles without using new.

_____ temperatures = _____ ;

- double[]
- [8.6, 9.9]
- {8.6, 9.9}
- new
- int[]

Click or drag and drop to fill in the blank

3. Define an empty array of integers with a length of 10.

_____ empty = _____ _____ ;

- new
- int[10]
- int[]

Click or drag and drop to fill in the blank

4. Create a while loop that prints out every number in the array.

```
double[] lapTimes = {4.00, 5.32, 4.68, 3.95};
```

```
int j = _____ ;
```

```
while (j < _____)
```

```
{
```

```
    Console.WriteLine(lapTimes[j]);
```

```
    _____ ;
```

```
}
```

- 1

- lapTimes.Length
- j--
- 0
- lapTime[]
- j++

Click or drag and drop to fill in the blank

5. In this code, what is an *element*?

```
char[] letters = { 'z' };
```

- a) char[]
- b) letters
- c) =
- d) 'z'

6. What is the index of "give"?

```
string[] lyrics = {"Never", "gonna", "give", "you", "up"};
```

- a) 3
- b) 2
- c) 0
- d) 1

7. Define breeds as an array of strings.

```
_____ breeds = _____ {"Dalmatian", "Shiba Inu", "Chihuahua"};
```

- int[]
- new
- double[]

- char[]
- string[]
- string[]

Click or drag and drop to fill in the blank

8. What will be printed to the console?

```
public static void Main (string[] args)
{
    string[] names = {"Ursula", "K.", "Le", "Guin"};
    foreach (string name in names)
    {
        Console.Write(name + " ");
    }
}
```

- a) Ursula K. Le Guin
- b) Le Guin Ursula K.
- c) name name name name
- d) UrsulaK.LeGuin

9. What is an *array*?

- a) A list of ordered data.
- b) A variable.
- c) A type of loop.
- d) A statement.

10. What is a *data structure*?

- a) A variable.
- b) A type of array.
- c) An organized way to manage and access information.

d) The code that defines a loop.

11. When this code is run, how many times will "bzzt" be written to the console?

```
public static void Main (string[] args)
{
    for (int z = 2; z < 3; z++)
    {
        Console.WriteLine("bzzt");
    }
}
```

- a) 1
- b) 2
- c) 3
- d) 0

12. Create a **do...while** loop that makes the player **Dance()** until **keyPressed** is **true**.

```
_____
{
    Player.Dance();
} _____ (_____);
```

- keyPressed = true
- do
- for
- while
- foreach
- keyPressed != true

Click or drag and drop to fill in the blank

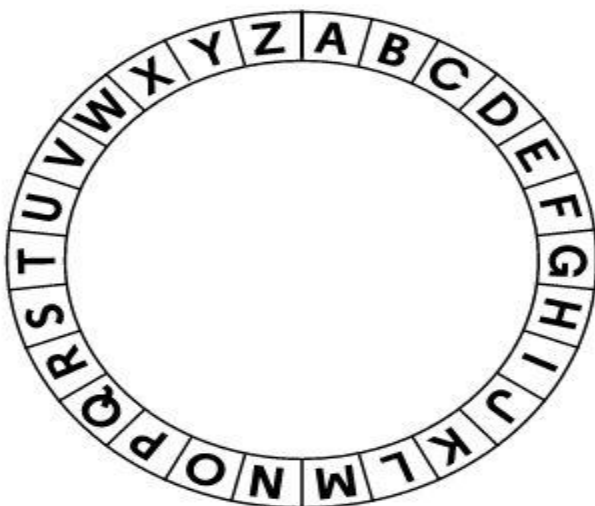
Project-7

Caesar Cipher

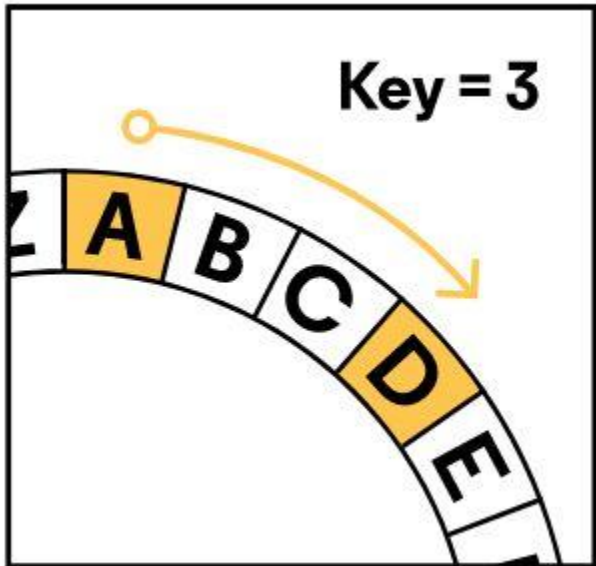
By 6 a.m. on Sunday, your team's project is nearly finished. The Object of Your Affection (the name's a work in progress) is getting attention from other teams in the hackathon. With one day left, they're getting desperate. To steal your project idea, your competitors have been reading your team's emails!

As the team's C# expert, you have been asked to write a cipher: a tool to encrypt text, making it unreadable to other teams. You've decided to implement the [Caesar Cipher](#), which was used by the Roman Empire to encode military secrets.

To use the cipher, draw the alphabet in a circle like so:



Take every letter of your message and shift it three places to the right. For example, A becomes D, B becomes E, C becomes F, and “attack” becomes “dwwdfn”.



Your teammates can decrypt your message by reversing the process: shift each letter three places to the left.

For this project you’ll need to convert strings to arrays of characters with [ToCharArray\(\)](#):

```
string msgString = "brutus";  
char[] msgArray = msgString.ToCharArray();
```

You’ll also need to convert arrays of characters to strings with [Join\(\)](#):

```
char[] msgArray = new char[] { 'b', 'r', 'u', 't', 'u', 's' };  
string msgString = String.Join("", msgArray);
```

The team is counting on you. Let’s get started!

TASKS

Prepare for Encryption

1.

You'll build the encryption tool as an interactive console app.

First, ask the user for a secret message and store the result in a variable.

2.

Convert the captured string to an array of characters. Store the result in a new variable called `secretMessage`.

3.

Create a new, empty array of characters to hold the encrypted message. It should be named `encryptedMessage` and have a length equal to the length of `secretMessage`.

Encrypt

4.

We'll need to perform encryption for every letter in the message.

Create an empty for loop that loops through each character of `secretMessage`.

- The iterator variable should be called `i` and start at 0
- It should continue as long as it is less than `secretMessage.Length`
- Each iteration it should increment by 1

5.

Within the loop, access the character at position `i` in the `secretMessage` array and store it in a variable.

6.

Find the position of the character in the alphabet array using the method `Array.IndexOf()`. Store the value in a variable.

7.

Add 3 to the letter position and store the value in a variable.

8.

Find the new encrypted character by getting the character in the alphabet array with that new position.

9.

Add the encrypted character to the array `encryptedMessage`.

Store the character at the index `i` (the iterator variable).

Test and Improve

10.

Your loop is finished! Now we need to convert our array of encrypted characters back into a readable string that we can print to the console.

Convert the character array to a string using `String.Join()` and print it to the console.

11.

Run your app and try these messages (not all of them may work yet!):

- hello converts to khood
- citizen converts to flwlchq

12.

What went wrong? When the program tried to encrypt the z in citizen, it found its index in the alphabet, 25. It looked up the letter 3 spaces to the right, which would be alphabet[28].

This threw an error because the alphabet is only 26 letters!

We can “wrap around” by using the modulo operator: %. On the line where you add 3 to the letter position, surround the expression letterPosition + 3 with parentheses and take the modulo of 26.

Now the new letter position will never go past 26.

13.

Test the code again with citizen, which converts to flwlchq.

Extensions

14.

Well done! You built an automated encryption engine that your team can use to communicate in secrecy. The hackathon is all but won!

If you’d like, you can keep building on your app:

- The app doesn’t work with uppercase letters. Fix that by converting any message to lowercase.
- The app doesn’t work with symbols, like ! or ?. Skip any symbols in your loop so that they are not encrypted.
- Rewrite the loop as a method Encrypt() which takes a character array and key and returns an encrypted character array .
- Write a Decrypt() method which takes a character array and key and returns a decrypted character array.

```

char[] secretMessage = {'h', 'e', 'l', 'l', 'o'};

// encrypted should equal {'k', 'h', 'o', 'o', 'r'}
string encrypted = Encrypt(secretMessage, 3);

// decrypted should equal {'h', 'e', 'l', 'l', 'o'}
string decrypted = Decrypt(encrypted, 3);

```

Program.cs

```

using System;

namespace CaesarCipher
{
    class Program
    {
        static void Main(string[] args)
        {
            char[] alphabet = new char[] { 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n',
            'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z' };

        }
    }
}

```

OUTPUT

\$ dotnet run

Enter your secret message: ACDF

dfgi

Test Message: hello - Encrypted Message: koor

Test Message: citizen - Encrypted Message: flwlchq

Encrypted: dfgi

Decrypted: acdf

12. BASIC CLASSES AND OBJECTS

12.1 Introduction to Classes

With data types like `int`, `string`, and `bool` we can represent basic data and perform basic operations:

```
int count = 32;  
count++;
```

What if we want to represent something more complex — something in the real world?

Say we are writing a program to manage nature reserves and we need to track forests. A forest has a certain number of trees, it has a name, it can grow, it can burn. Representing many forests with basic data types would mean tracking tons of separate variables and methods.

Instead, we can define our own custom `Forest` data type and use it like any other data type in our program. This process of bundling related data and methods into a type is called *encapsulation*, and it makes code easier to organize and reuse.

In C#, a custom data type is defined with a *class*, and each instance of this type is an *object*. This lesson will teach you how to:

- Define a class
- Add members, like properties and methods, to a class
- Customize access to those members using `public` and `private`
- Create objects from a class

In your coding adventures you may come across the term *struct*. While similar to a class it has a few differences, but we won't be covering those in this lesson.

If you do find yourself in need of a struct, you can [refer to Microsoft's explanation](#). Structs are easy to learn after you understand classes in this lesson.

Let's get started!

12.1 Introduction to Classes Instructions

Here are more examples of custom classes. Testing this kind of code takes longer, so some checkpoints in this lesson will need more time to run. It's worth the wait!



12.2 Making Classes

C# provides built-in data types, like string: each instance of the string type has its own values and functionality.

```
string phrase = "zoinks!";  
Console.WriteLine(phrase.Length);  
Console.WriteLine(phrase.IndexOf("k"));
```

In this case phrase is an *instance* of the string type. Every string has a Length property and IndexOf() method, but the resulting values are different for each instance.

A *class* represents a custom data type. In C#, the class defines the kinds of information and methods included in a custom type.

You can then make *instances* of that class (above, phrase was an instance of string). There may be many instances of the same class, all with unique values.

To begin defining a class, C# uses this structure:

```
class Forest {  
}
```

The code for a class is usually put into a file of its own, named with the name of the class. In this case it's **Forest.cs**. This keeps our code organized and easy to debug.

In other parts of code, like Main() in **Program.cs**, we can use the class. We make instances, or *objects*, of the Forest class with the new keyword:

```
Forest f = new Forest();
```

We could say f is an instance of the Forest class, or f is of type Forest.

The process of creating an instance is called *instantiation*. Today we *instantiate* a class; yesterday they *instantiated* a class, and so on.

12.2 Making Classes Instructions

1.

We will define our class in **Forest.cs** and work with that class in the Main() method in **Program.cs**.

Within the namespace BasicClasses, build an empty Forest class in **Forest.cs**.

2.

In Main() in **Program.cs** make a new instance of the Forest class and store the result in a variable f.

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

Forest.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
}
```

12.3 Fields

We need to associate different pieces of data, like a size and name, to each Forest object. In C#, these pieces of data are called *fields*. Fields are one type of class *member*, which is the general term for the building blocks of a class.

Create fields like this:

```
class Forest {  
    public string name;  
    public int trees;  
}
```

This might look similar to defining a variable. It is! Each field is a variable and it will have a different value for each object.

With the code above, we haven't set the value of either field, so each has a default value. In this case strings default to null, ints to 0, and bools to false. You can find the default values for more types in [Microsoft's default values table](#).

It is common practice to name fields using all lowercase (name instead of Name). This makes fields easy to recognize later on!

Don't worry about public yet: it's explained later in this lesson.

Once we create a Forest instance, we can access and edit each field with dot notation:

```
Forest f = new Forest();  
f.name = "Amazon";  
Console.WriteLine(f.name); // Prints "Amazon"
```

```
Forest f2 = new Forest();  
f2.name = "Congo";  
Console.WriteLine(f2.name); // Prints "Congo"
```

Each instance has a name field, but the value may differ across instances.

12.3 Fields Instructions

1.

In **Forest.cs**, add 4 fields to the Forest class.

Two fields of type string:

- name
- biome

Two fields of type int:

- trees
- age

2.

In **Program.cs** in Main(), a Forest object has already been instantiated. Set values to those four fields.

3.

In Main(), print the name field to the console.

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
static void Main(string[] args)
{
    Forest f = new Forest();
}
}
```

Forest.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
    class Forest
    {

    }

}
```

OUTPUT

Amazon Rainforest

12.4 Properties

As of now, a program can plant any value in a Forest field. For example, if we had an area field of type int, we could set it to 0, 40, or -1249. Can we have a forest of -1249 area? We need a way to define what values are valid and disallow those that are not. C# provides a tool for that: *properties*.

Properties are another type of class member. Each property is like a spokesperson for a field: it controls the access (getting and setting) to that field. We can use this to validate values before they are set to a field. A property is made up of two methods:

- a `get()` method, or getter: called when the property is accessed
- a `set()` method, or setter: called when the property is assigned a value

This shows a basic Area property without validation:

```
public int area;
public int Area
{
    get { return area; }
    set { area = value; }
}
```

The Area property is associated with the area field. It's common to name a property with the title-cased version of its field's name, e.g. age and Age, name and Name.

The `set()` method above uses the keyword `value`, which represents the value we assign to the property. Back in **Program.cs**, when we access the Area property, the `get()` and `set()` methods are called:

```
Forest f = new Forest();
f.Area = -1; // set() is called
Console.WriteLine(f.Area); // get() is called; prints -1
```

In the above example, when `set()` is called, the value variable is -1, so area is set to -1.

Here's the same property with validation in the `set()` method. If we try to set Area to a negative value, it will be changed to 0.

```
public int Area
{
    get { return area; }
    set
    {
        if (value < 0) { area = 0; }
        else { area = value; }
    }
}
```

In **Program.cs**:

```
Forest f = new Forest();
// set() is called
f.Area = -1;
// get() is called; prints 0
Console.WriteLine(f.Area);
```

12.4 Properties Instructions

1.

Define a basic Name property for the name field. The getter and setter will have no validation.

2.

Define a basic Trees property for the trees field. The getter and setter will have no validation.

3.

In **Program.cs**,

- Replace any use of the field f.name with the property f.Name.

- Replace any use of the field `f.trees` with the property `f.Trees`.

4.

Define a `Biome` property for the `biome` field. It will have a getter and setter. The setter should only allow three values: "Tropical", "Temperate", and "Boreal". If any other value is used, set `biome` to "Unknown".

For example, this is how it would work in **Program.cs**:

```
f.Biome = "Tropical";  
// Prints Tropical  
Console.WriteLine(f.Biome);
```

```
f.Biome = "Desert";  
// Prints Unknown  
Console.WriteLine(f.Biome);
```

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Forest f = new Forest();
```

```
            f.name = "Congo";
```

```
            f.trees = 0;
```

```
            f.age = 0;
```

```
f.biome = "Tropical";

    Console.WriteLine(f.name);
}
}
}
```

Forest.cs

```
using System;

namespace BasicClasses
{
    class Forest
    {
        public string name;
        public int trees;
        public int age;
        public string biome;

    }

}
```

OUTPUT

Congo

12.5 Automatic Properties

It might have felt tedious to write the same getter and setter for the Name and Trees properties. C# has a solution for that! The basic getter and setter pattern is so common that there is a short-hand called an *automatic property*. As a reminder, here's the basic pattern for an imaginary size property:

```
public string size;
public string Size
{
    get { return size; }
    set { size = value; }
}
```

This pattern can be written as an *automatic property*:

```
public string Size
{ get; set; }
```

In this form, you don't have to write out the get() and set() methods, and you don't have to define a size field at all! A hidden field is defined in the background for us. All we have to worry about is the Size property.

12.5 Automatic Properties Instructions

1.

Replace the current name field and Name property with an automatic Name property.

2.

Replace the current trees field and Trees property with an automatic Trees property.

Program.cs

using System;

namespace BasicClasses

{

class Program

{

static void Main(string[] args)

{

Forest f = new Forest();

f.Name = "Congo";

f.Trees = 0;

f.age = 0;

f.biome = "Tropical";

Console.WriteLine(f.Name);

}

}

}

Forest.cs

using System;

namespace BasicClasses

```
{  
class Forest  
{  
    public string name;  
    public int trees;  
    public int age;  
    public string biome;  
  
    public string Name  
    {  
        get { return name; }  
        set { name = value; }  
    }  
  
    public int Trees  
    {  
        get { return trees; }  
        set { trees = value; }  
    }  
  
    public string Biome  
    {  
        get { return biome; }  
        set
```

```

{
    if (value == "Tropical" ||
        value == "Temperate" ||
        value == "Boreal")
    {
        biome = value;
    }
    else
    {
        biome = "Unknown";
    }
}
}
}
}

}

```

OUTPUT

Congo

12.6 Public vs. Private

At this point we have built fields to associate data with a class and properties to control the getting and setting of each field. As it is now, any code outside of the Forest class can “sneak past” our properties by directly accessing the field:


```
f.Age = 32; // using property  
f.age = -1; // using field
```

The second line avoids the property's validation by directly accessing the field. We can fix this by using the *access modifiers* public and private:

- public — a public member can be accessed by any class
- private — a private member can only be accessed by code in the same class

For simplicity, we've been adding public to every member so far. That allows code to access the members from the Main() method, which doesn't belong to the Forest class. When we switch a field from public to private it will no longer be accessible from Main(), although code inside the Forest class — like properties — can still access it.

Access modifiers can be applied to all members of a class, including fields, properties, and the rest of the members covered in this lesson.

Remember *encapsulation*? public and private are necessary to encapsulate our classes. Think of it like “defensive coding”: you are protecting the inner mechanisms of a class with private so that other code can't break your class. You only expose what you want to be public.

For example, since a class' properties define how other programs get and set its fields, it's good practice to make fields private and properties public.

C# encourages encapsulation by defaulting class members to private and classes to public.

12.6 Public vs. Private Instructions

1.

Currently the biome field and Biome property are public. In **Program.cs**, the field is directly accessed and set to "Desert", an invalid value.

Run the code to see that “Desert” is the field's value.

2.

By directly accessing the public biome field the code skipped the validation. Let's prevent that by setting the biome field to private in **Forest.cs**.

When you run the code, you should see the error:

error CS0122: 'Forest.biome' is inaccessible due to its protection level

That means C# has prevented us from accessing a private field (which is good).

3.

Address the error by changing the code in **Program.cs**: use the property (Biome) instead of the field (biome). You'll need to change code in two places in the file.

What is printed to the console now?

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Forest f = new Forest();
```

```
            f.Name = "Congo";
```

```
            f.Trees = 0;
```

```
            f.age = 0;
```

```
            f.biome = "Desert";
```

```
        Console.WriteLine(f.Name);  
        Console.WriteLine(f.biome);  
    }  
}  
}
```

Forest.cs

```
using System;  
  
namespace BasicClasses  
{  
    class Forest  
    {  
        public int age;  
        public string biome;  
  
        public string Name  
        { get; set; }  
  
        public int Trees  
        { get; set; }  
  
        public string Biome
```

```
{
  get { return biome; }
  set
  {
    if (value == "Tropical" ||
        value == "Temperate" ||
        value == "Boreal")
    {
      biome = value;
    }
    else
    {
      biome = "Unknown";
    }
  }
}
```

OUTPUT

Congo

Unknown

12.7 Get-Only Properties

Previously we used properties for field validation. By applying public and private, we can also use properties to control access to fields.

Recall our imaginary Area property. Say we want programs to get the value of the property, but we don't want programs to set the value of the property. Then we either:

1. don't include a set() method, or
2. make the set() method private.

This shows approach 1 — don't include a set():

```
public string Area
{
    get { return area; }
}
```

We can still get Area, but if we try to set Area we get an error:

error CS0200: Property or indexer 'Forest.Area' cannot be assigned to (it is read-only)

This shows approach 2 — make set() private:

```
public int Area
{
    get { return area; }
    private set { area = value; }
}
```

We can still get Area, but if we try to set Area in Main() we get an error:

error CS0272: The property or indexer 'Forest.Area' cannot be used in this context because the set accessor is inaccessible

Notice that in approach 1 we get an error for setting Area anywhere. In approach 2 we only get an error for setting Area outside of the Forest class. Generally we prefer approach 2 because it allows other Forest methods to set Area.

12.7 Get-Only Properties Instructions

1.

In **Forest.cs**, define an Age property for the age field. It should have a public getter and a private setter.

2.

In **Program.cs** in Main(), try to set the value of f.Age. You should see an error.

error CS0272: The property or indexer 'Forest.Age' cannot be used in this context because the set accessor is inaccessible

This error means that the private setter prevented us from setting Age outside of the class (which is good!).

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Forest f = new Forest();
```

```
f.Name = "Congo";  
f.Trees = 0;  
f.age = 0;  
f.Biome = "Desert";  
  
Console.WriteLine(f.Name);  
Console.WriteLine(f.Biome);  
}  
}  
}
```

Forest.cs

```
using System;  
  
namespace BasicClasses  
{  
    class Forest  
    {  
        public int age;  
        private string biome;  
  
        public string Name  
        { get; set; }  
    }  
}
```

```
public int Trees
```

```
{ get; set; }
```

```
public string Biome
```

```
{
```

```
    get { return biome; }
```

```
    set
```

```
{
```

```
    if (value == "Tropical" ||
```

```
        value == "Temperate" ||
```

```
        value == "Boreal")
```

```
{
```

```
    biome = value;
```

```
}
```

```
else
```

```
{
```

```
    biome = "Unknown";
```

```
}
```

```
}
```

```
}
```

```
}
```



```
}
```

OUTPUT

Congo

Unknown

12.8 Methods

The third type of member in classes is *methods*. This lesson assumes that you are already familiar with methods, so the syntax should look familiar.

In the past you learned that methods are a useful way to organize chunks of code to perform a task. But most methods belong to a class (even the ones you have written!), so methods are also used to define how an instance of a class behaves. You can think of them as the “actions” that an object can perform.

This code defines a method `IncreaseArea()` that changes the value of the `Area` property:

```
class Forest {  
    public int Area  
    { /* property body omitted */ }  
    public int IncreaseArea(int growth)  
    {  
        Area = Area + growth;  
        return Area;  
    }  
}
```

You would call the method like so:

```
Forest f = new Forest();  
int result = f.IncreaseArea(2);  
Console.WriteLine(result); // Prints 2
```

12.8 Methods Instructions

1.

In the Forest class, define a public method Grow(). It should:

- take zero arguments
- increase the Trees property by 30 and the Age property by 1
- return the updated number of trees

2.

Define a public method Burn(). It should:

- take zero arguments
- decrease the Trees property by 20 and increase the Age property by 1
- return the updated number of trees

Forest.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Forest
```

```
    {
```

```
        public int age;
```

```
private string biome;
```

```
public string Name
```

```
{ get; set; }
```

```
public int Trees
```

```
{ get; set; }
```

```
public string Biome
```

```
{
```

```
    get { return biome; }
```

```
    set
```

```
    {
```

```
        if (value == "Tropical" ||
```

```
            value == "Temperate" ||
```

```
            value == "Boreal")
```

```
        {
```

```
            biome = value;
```

```
        }
```

```
    else
```

```
    {
```

```
        biome = "Unknown";
```

```
    }
```

```
}
```

```

    }

    public int Age
    {
        get { return age; }
        private set { age = value; }
    }

}

}

```

Program.cs

```

using System;

namespace BasicClasses
{
    class Program
    {
        static void Main(string[] args)
        {
            Forest f = new Forest();
            f.Name = "Congo";
            f.Trees = 0;
        }
    }
}

```

```
f.Biome = "Desert";

Console.WriteLine(f.Name);
Console.WriteLine(f.Biome);
}
}
}
```

12.9 Constructors

In each of the examples so far, we created a new Forest object and set the property values one by one. It would be nice if we could write a method that's run every time an object is created to set those values at once.

C# has a special type of method, called a *constructor*, that does just that. It looks like a method, but there is no return type listed and the method name is the name of its enclosing class:

```
class Forest
{
    public Forest()
    {
    }
}
```

We can add code in the constructor to set values to fields:

```
class Forest
{
    public int Area;

    public Forest(int area)
```

```
{  
    Area = area;  
}  
}
```

This constructor method is used whenever we instantiate an object with the new keyword:

```
// Constructor is called here  
Forest f = new Forest(400);
```

But we've been instantiating new objects all day! Why did it work before we defined a constructor?

If no constructor is defined in a class, one is automatically created for us. It takes no parameters, so it's called a *parameterless constructor*. That's why we have been able to instantiate new objects without errors:

```
Forest f = new Forest();
```

12.9 Constructors Instructions

1.

Define a constructor for the Forest class. It should have two parameters:

- name, which sets the Name property
- biome, which sets the Biome property

It should also set the value of Age to 0.

2.

The code in **Program.cs** has been commented out. Un-comment it and run it.

You should see an error:

error CS7036: There is no argument given that corresponds to the required formal parameter 'name' of 'Forest.Forest(string, string)'

This error occurs because you are using the parameterless constructor `Forest()` in **Program.cs**. This no longer works because a constructor `Forest(string, string)` has been defined.

3.

Call the new constructor in `Main()` to create a `Forest` object with the name "Congo" and biome "Tropical".

Delete the lines:

```
f.Name = "Congo";
```

and

```
f.Biome = "Desert";
```

They're no longer useful because those properties are now set in the constructor!

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            /*
```

```
            Forest f = new Forest();
```

```
            f.Name = "Congo";
```

```
            f.Trees = 0;
```

```
f.Biome = "Desert";

Console.WriteLine(f.Name);
Console.WriteLine(f.Biome);
    */
}
}
}
```

Forest.cs

```
using System;

namespace BasicClasses
{
    class Forest
    {
        public int age;
        private string biome;

        public string Name
        { get; set; }

        public int Trees
        { get; set; }
```



```
public string Biome
{
    get { return biome; }
    set
    {
        if (value == "Tropical" ||
            value == "Temperate" ||
            value == "Boreal")
        {
            biome = value;
        }
        else
        {
            biome = "Unknown";
        }
    }
}
```

```
public int Age
{
    get { return age; }
    private set { age = value; }
}
```

```
public int Grow()
{
    Trees += 30;
    Age += 1;
    return Trees;
}
```

```
public int Burn()
{
    Trees -= 20;
    Age += 1;
    return Trees;
}
```

```
}
```

```
}
```

OUTPUT

Congo

Tropical

12.10 this

In the last exercise we assigned the area field in the constructor:

```
class Forest
{
    public int Area
    { /* property omitted */ }

    public Forest(int area)
    {
        Area = area;
    }
}
```

The parameter for the constructor area looks a lot like the old field area and the new property Area. It's good to be explicit when writing code so that there is no room for misinterpretation. We can refer to the current instance of a class with the this keyword.

```
class Forest
{
    public int Area
    { /* property omitted */ }

    public Forest(int area)
    {
        this.Area = area;
    }
}
```

this.Area = area means “when this constructor is used to make a new instance, use the argument area to set the value of this new instance's Area field”.

We would call it the same way:

```
Forest f = new Forest(400);
```

f.Area now equals 400.

The word `this` might seem frustratingly vague. Think back to the “class is to instance as blueprint is to house” analogy. The class/blueprint has to use the generic `this` because the class/blueprint is going to be reused for every instance/house.

12.10 this Instructions

1.

Specify the instance properties by using `this.Name` and `this.Biome`.

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Forest f = new Forest("Congo", "Tropical");
```

```
            f.Trees = 0;
```

```
            Console.WriteLine(f.Name);
```

```
            Console.WriteLine(f.Biome);
```

```
        }
```

```
    }
```

```
}
```

Forest.cs

using System;

namespace BasicClasses

{

class Forest

{

public int age;

private string biome;

public Forest(string name, string biome)

{

 Name = name;

 Biome = biome;

 Age = 0;

}

public string Name

{ get; set; }

public int Trees

{ get; set; }

public string Biome

```

{
    get { return biome; }
    set
    {
        if (value == "Tropical" ||
            value == "Temperate" ||
            value == "Boreal")
        {
            biome = value;
        }
        else
        {
            biome = "Unknown";
        }
    }
}

```

```

public int Age
{
    get { return age; }
    private set { age = value; }
}

```

```

public int Grow()

```

```
{  
    Trees += 30;  
    Age += 1;  
    return Trees;  
}
```

```
public int Burn()  
{  
    Trees -= 20;  
    Age += 1;  
    return Trees;  
}
```

```
}
```

```
}
```

OUTPUT

Congo

Tropical

12.11 Overloading Constructors

Just like other methods, constructors can be overloaded. For example, we may want to define an additional constructor that takes one argument:

```
public Forest(int area, string country)
{
    this.Area = area;
    this.Country = country;
}
```

```
public Forest(int area)
{
    this.Area = area;
    this.Country = "Unknown";
}
```

The first constructor provides values for both fields, and the second gives a default value when the country is not provided. Now you can create a Forest instance in two ways:

```
Forest f = new Forest(800, "Hungary");
Forest f2 = new Forest(400);
```

Notice how we've written duplicate code for our second constructor: `this.Area = area;`. Later on, if we need to adjust the constructor, we'll need to find every copy of the code and make the exact same change. That means more work and chances for errors.

We have two options to resolve this. In either case we will remove the duplicated code:

1. Use default arguments. This is useful if you are using C# 4.0 or later (which is fairly common) and the only difference between constructors is default values.

```
public Forest(int area, string country = "Unknown")
{
    this.Area = area;
    this.Country = country;
}
```

2. Use `: this()`, which refers to another constructor in the same class. This is useful for old C# programs (before 4.0) and when your second constructor has additional

functionality. This example has an additional functionality of announcing the default value.

```
public Forest(int area, string country)
{
    this.Area = area;
    this.Country = country;
}
```

```
public Forest(int area) : this(area, "Unknown")
{
    Console.WriteLine("Country property not specified. Value defaulted to
'Unknown'.");
}
```

Remember that `this.Area` refers to the current instance of a class. When we use `this()` like a method, it refers to another constructor in the current class. In this example, the second constructor calls `this()` — which refers to the first `Forest()` constructor — AND it prints information to the console.

12.11 Overloading Constructors Instructions

1.

Define a second constructor for the `Forest` class:

- It should take one parameter, `name`.
- It should use `: this()` with the `name` variable as the first argument and `"Unknown"` as the second.
- It should also print a warning to the console about the defaulted value.

2.

In `Main()`, call the second constructor to create a `Forest` object named `"Rendlesham"`.

3.

Below the constructor call, print the Biome property of this new instance.

When you run the code, you should see the warning message and “Unknown” printed to the console. Why are these two things printed?

Program.cs

```
using System;

namespace BasicClasses
{
    class Program
    {
        static void Main(string[] args)
        {
            Forest f = new Forest("Congo", "Tropical");
            f.Trees = 0;

            Console.WriteLine(f.Name);
            Console.WriteLine(f.Biome);
        }
    }
}
```

Forest.cs

```
using System;
```

```

namespace BasicClasses
{
    class Forest
    {
        public int age;
        private string biome;

        public Forest(string name, string biome)
        {
            this.Name = name;
            this.Biome = biome;
            Age = 0;
        }

        public string Name
        { get; set; }

        public int Trees
        { get; set; }

        public string Biome
        {
            get { return biome; }
        }
    }
}

```

```

set
{
    if (value == "Tropical" ||
        value == "Temperate" ||
        value == "Boreal")
    {
        biome = value;
    }
    else
    {
        biome = "Unknown";
    }
}

```

```

public int Age
{
    get { return age; }
    private set { age = value; }
}

```

```

public int Grow()
{
    Trees += 30;
}

```

```
    Age += 1;
    return Trees;
}
```

```
public int Burn()
{
    Trees -= 20;
    Age += 1;
    return Trees;
}
```

```
}
```

```
}
```

OUTPUT

Congo

Tropical

Biome not specified. Value defaulted to 'Unknown'.

Unknown

12.12 Review

Congrats! You've finished a lot of content and some of the most important concepts in C#. When someone asks you, "How do I make a custom data type in C#?" you can talk all about it! In this lesson, you learned how to:

- Define a *class*
- Instantiate an *object* using `new`
- Define *fields*, the pieces of data for each class
- Define *properties*, the spokespeople for each field
- Define *automatic properties*, the shorthand for making properties
- Define *methods*, the actions a class can take
- Define *constructors*, the special methods called when a class is instantiated
- Overload *constructors* and reuse code with this
- Control access to class members using `public` and `private`

12.12 Review Instructions

1.

Try out your complete Forest class in `Main()`!

- Instantiate a new object with the name "Amazon". Store the result in a variable.
- Print the `Trees` property to the console.
- Call the `Grow()` method.
- Print the `Trees` property again to the console to confirm that the `Grow()` method works.

Program.cs

```
using System;
```

```
namespace BasicClasses
{
    class Program
    {
        static void Main(string[] args)
        {

        }

    }
}
```

Forest.cs

```
using System;

namespace BasicClasses
{
    class Forest
    {
        public int age;
        private string biome;

        public Forest(string name, string biome)
        {
```

```
this.Name = name;  
this.Biome = biome;  
Age = 0;  
}
```

```
public Forest(string name) : this(name, "Unknown")  
{ }
```

```
public string Name  
{ get; set; }
```

```
public int Trees  
{ get; set; }
```

```
public string Biome  
{  
    get { return biome; }  
    set  
    {  
        if (value == "Tropical" ||  
            value == "Temperate" ||  
            value == "Boreal")  
        {  
            biome = value;
```



```
    }  
    else  
    {  
        biome = "Unknown";  
    }  
}  
}
```

```
public int Age  
{  
    get { return age; }  
    private set { age = value; }  
}
```

```
public int Grow()  
{  
    Trees += 30;  
    Age += 1;  
    return Trees;  
}
```

```
public int Burn()  
{  
    Trees -= 20;
```

```
        Age += 1;
        return Trees;
    }

}

}
```

OUTPUT

```
0
30
```

Concept Review

C# Classes

In C#, *classes* are used to create custom types. The class defines the kinds of information and methods included in a custom type.

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Forest {
```

```
        public string name;
```

```
        public int trees;
    }
}
```

// Here we have the Forest class which has two pieces of data, called fields. They are the "name" and "trees" fields.

C# Constructor

In C#, whenever an instance of a class is created, its *constructor* is called. Like methods, a constructor can be overloaded. It must have the same name as the enclosing class. This is useful when you may want to define an additional constructor that takes a different number of arguments.

// Takes two arguments

```
public Forest(int area, string country)
{
    this.Area = area;
    this.Country = country;
}
```

// Takes one argument

```
public Forest(int area)
{
    this.Area = area;
    this.Country = "Unknown";
}
```

```
}
```

// Typically, a constructor is used to set initial values and run any code needed to “set up” an instance.

// A constructor looks like a method, but its return type and method name are reduced to the name of the enclosing type.

C# Parameterless Constructor

In C#, if no constructors are specified in a class, the compiler automatically creates a parameterless constructor.

```
public class Freshman
```

```
{
```

```
    public string FirstName
```

```
    { get; set; }
```

```
}
```

```
public static void Main (string[] args)
```

```
{
```

```
    Freshman f = new Freshman();
```

```
    // name is null
```

```
    string name = f.FirstName;
```

```
}
```

// In this example, no constructor is defined in Freshman, but a parameterless constructor is still available for use in Main().

C# Access Modifiers

In C#, members of a class can be marked with *access modifiers*, including public and private. A public member can be accessed by other classes. A private member can only be accessed by code in the same class.

By default, fields, properties, and methods are private, and classes are public.

```
public class Speech
{
    private string greeting = "Greetings";

    private string FormalGreeting()
    {
        return $"{greeting} and salutations";
    }

    public string Scream()
    {
        return FormalGreeting().ToUpper();
    }
}
```

```

public static void Main (string[] args)
{
    Speech s = new Speech();
    //string sfg = s.FormalGreeting(); // Error!
    //string sg = s.greeting; // Error!
    Console.WriteLine(s.Scream());
}

```

// In this example, greeting and FormalGreeting() are private. They cannot be called from the Main() method, which belongs to a different class. However the code within Scream() can access those members because Scream() is part of the same class.

C# Field

In C#, a *field* stores a piece of data within an object. It acts like a variable and may have a different value for each instance of a type.

A field can have a number of modifiers, including: public, private, static, and readonly. If no access modifier is provided, a field is private by default.

```

public class Person
{
    private string firstName;
    private string lastName;
}

```

```
// In this example, firstName and lastName are private fields of the Person class.
```

```
// For effective encapsulation, a field is typically set to private, then accessed using a property. This ensures that values passed to an instance are validated (assuming the property implements some kind of validation for its field).
```

C# this Keyword

In C#, the `this` keyword refers to the current instance of a class.

```
// We can use the this keyword to refer to the current class's members hidden by similar names:
```

```
public NationalPark(int area, string state)
{
    this.area = area;
    this.state = state;
}
```

```
// The code below requires duplicate code, which can lead to extra work and errors when changes are needed:
```

```
public NationalPark(int area, string state)
{
    area = area;
    state = state;
}
```

```
public NationalPark(int area)
{
    area = area;
    state = "Unknown";
}
```

// Use this to have one constructor call another:

```
public NationalPark(int area) : this (state, "Unknown")
{ }
```

C# Members

In C#, a class contains *members*, which define the kind of data stored in a class and the behaviors a class can perform.

```
class Forest
{
    public string name;
    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```


// A member of a class can be a field (like name), a property (like Name) or a method (like get()/set()). It can also be any of the following:

// Constants

// Constructors

// Events

// Finalizers

// Indexers

// Operators

// Nested Types

C# Dot Notation

In C#, a member of a class can be accessed with dot notation.

```
string greeting = "hello";
```

```
// Prints 5
```

```
Console.WriteLine(greeting.Length);
```

```
// Returns 8
```

```
Math.Min(8, 920);
```

C# Class Instance

In C#, an object is an *instance* of a class. An object can be created from a class using the new keyword.

```
Burger cheeseburger = new Burger();
```

// If a class is a recipe, then an object is a single meal made from that recipe.

```
House tudor = new House();
```

// If a class is a blueprint, then an object is a house made from that blueprint.

C# Property

In C#, a *property* is a member of an object that controls how one field may be accessed and/or modified. A property defines two methods: a `get()` method that describes how a field can be accessed, and a `set()` method that describes how a field can be modified.

One use case for properties is to control access to a field. Another is to validate values for a field.

```
public class Freshman
{
    private string firstName;

    public string FirstName
    {
        get { return firstName; }
        set { firstName = value; }
    }
}
```

```
}
```

```
public static void Main (string[] args) {
```

```
    Freshman f = new Freshman();
```

```
    f.FirstName = "Louie";
```

```
    // Prints "Louie"
```

```
    Console.WriteLine(f.FirstName);
```

```
}
```

```
// In this example, FirstName is a property
```

C# Auto-Implemented Property

In C#, an *auto-implemented* property reads and writes to a private field, like other properties, but it does not require explicit definitions for the accessor methods nor the field. It is used with the { get; set; } syntax. This helps your code become more concise.

```
public class HotSauce
```

```
{
```

```
    public string Title
```

```
    { get; set; }
```

```
    public string Origin
```

```
    { get; set; }
```

```
}
```

// In this example, Title and Origin are auto-implemented properties. Notice that a definition for each field (like private string title) is no longer necessary. A hidden, private field is created for each property during runtime.

C# Static Constructor

In C#, a *static constructor* is run once per type, not per instance. It must be parameterless. It is invoked before the type is instantiated or a static member is accessed.

```
class Forest
{
    static Forest()
    {
        Console.WriteLine("Type Initialized");
    }
}
```

// In this class, either of the following two lines would trigger the static constructor (but it would not be triggered twice if these two lines followed each other in succession):

```
Forest f = new Forest();
Forest.Define();
```

C# Static Class

In C#, a *static* class cannot be instantiated. Its members are accessed by the class name.

This is useful when you want a class that provides a set of tools, but doesn't need to maintain any internal data.

Math is a commonly-used static class.

//Two examples of static classes calling static methods:

```
Math.Min(23, 97);
```

```
Console.WriteLine("Let's Go!");
```

12. Quiz

1. What is an *object*?

- a) The definition of a custom type
- b) A class
- c) An instance of a class
- d) A member of a class

2. Given the SpacInvader class, what is SpacInvader()?

```
class SpacInvader
{
    public SpacInvader()
    {
        this.Speed = 5;
    }
}
```

```

public bool IsMothership
{ get; set; }

public int Speed
{ get; set; }
}

```

- a) A non-constructor method
- b) A property
- c) A field
- d) A constructor method

3. When Main() is run, what will be printed to the console?

```

class Program {
    public static void Main (string[] args) {
        Player p = new Player();
    }
}

class Player
{
    public Player(string name)
    {
        Console.WriteLine($"Player named: {name}");
    }

    public Player() : this("n/a")
    {}
}

```

- a) "Player named: name"
- b) "Player named:"

- c) "Player named: n/a"
- d) Nothing will be printed

4. Define *encapsulation*.

- a) Encapsulation is a type.
- b) Encapsulation means bundling related data and methods into one logical group.
- c) Encapsulation is an instance of a class.
- d) Encapsulation means all class members are private.

5. Which of these is NOT a class member?

- a) Property
- b) Constructor
- c) Method
- d) Object

6. Given the Breakfast class, what is ingredients?

```
class Breakfast
{
    string[] ingredients;
}
```

- a) A field
- b) A method
- c) A property
- d) A constructor

7. Given the Unicorn class, which line in Main() will cause an error?

```

class Program {
    public static void Main (string[] args) {
        Unicorn u = new Unicorn();
        Console.WriteLine(u.HornLength);
        u.HornLength = 5;
        int len = u.HornLength;
    }
}

```

```

class Unicorn
{
    public int HornLength
    { get; private set; }
}

```

- a) Console.WriteLine(u.HornLength);
- b) Unicorn u = new Unicorn();
- c) int len = u.HornLength;
- d) u.HornLength = 5;

8. What is a *class*?

- a) A file in C#
- b) The Main() method
- c) The definition of a custom type
- d) An object

9. Create an instance of the Chainsaw class, which has a parameterless constructor.

Chainsaw cutter = _____ ;

- new
- Chainsaw()

- chainsaw()

Click or drag and drop to fill in the blank

10. An instance of the Random class is constructed below. Call the instance method Next(), which returns a random integer.

```
Random generator = new Random();
int guess = _____ ;
```

- generator
- Random
- .
- Next()

Click or drag and drop to fill in the blank

11. Given the SpacInvader class, what is Speed?

```
class SpacInvader
{
    public SpacInvader()
    {
        this.Speed = 5;
    }

    public bool IsMothership
    { get; set; }

    public int Speed
    { get; set; }
}
```

- a) A field
- b) A property
- c) A constructor
- d) A method

13. STATIC MEMBERS

13.1 Introduction to Static

At this point, you may recall:

- A custom data type is defined by a *class*.
- An *instance* of a class is called an *object*. Multiple, unique objects can be instantiated from one class.
- This process of bundling related data and methods into a type is called *encapsulation*, and it makes code easier to organize and reuse.

What if we needed to do something related to the type itself, not instances of that type? For example, where do we store the count of all Forest objects, or an explanation of forests in general?

To keep with the rules of encapsulation, these shouldn't be associated with one instance (because this information is related to the Forest class, not a single Forest instance).

These facts and behaviors should be associated with the class itself! We call these types of members *static*.

This lesson will cover the meaning of *static*, how to apply it to different types of class members, and its typical uses cases.

13.1 Introduction to Static Instructions

The Forest class is defined in **Forest.cs**. Make sure you're familiar with it before moving on! We'll be adding to this class throughout the lesson.

Testing this kind of code takes longer, so some checkpoints in this lesson will need more time to run. It's worth the wait!

Program.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

Forest.cs

```
using System;
```

```
namespace BasicClasses
```

```
{
```

```
    class Forest
```

```
    {
```

```
        // FIELDS
```

```
public int age;
private string biome;

// CONSTRUCTORS

public Forest(string name, string biome)
{
    this.Name = name;
    this.Biome = biome;
    Age = 0;
}

public Forest(string name) : this(name, "Unknown")
{ }

// PROPERTIES

public string Name
{ get; set; }

public int Trees
{ get; set; }
```

```

public string Biome
{
    get { return biome; }
    set
    {
        if (value == "Tropical" ||
            value == "Temperate" ||
            value == "Boreal")
        {
            biome = value;
        }
        else
        {
            biome = "Unknown";
        }
    }
}

```

```

public int Age
{
    get { return age; }
    private set { age = value; }
}

```

```
// METHODS
```

```
public int Grow()  
{  
    Trees += 30;  
    Age += 1;  
    return Trees;  
}
```

```
public int Burn()  
{  
    Trees -= 20;  
    Age += 1;  
    return Trees;  
}
```

```
}
```

```
}
```

13.2 Static Fields and Properties

You already know how to create a field and property, like:

```

class Forest
{
    private string definition;
    public string Definition
    {
        get { return definition; }
        set { definition = value; }
    }
}

```

The definition of what a forest is applies to all Forest objects, not just one — there should only be one value for the whole class. This is a good use case for a static field/property.

To make a static field and property, just add static after the access modifier (public or private).

```

class Forest
{
    private static string definition;
    public static string Definition
    {
        get { return definition; }
        set { definition = value; }
    }
}

```

Remember that static means “associated with the class, not an instance”. Thus any static member is accessed from the class, not an instance:

```

static void Main(string[] args)
{
    Console.WriteLine(Forest.Definition);
}

```

If you tried to access a static member from an instance (like `f.Definition`) you would get an error like:

error CS0176: Static member 'Forest.Definition' cannot be accessed with an instance reference, qualify it with a type name instead

13.2 Static Fields and Properties Instructions

1.

In the previous exercise we mentioned storing the count of all Forest objects. We'll use a static field and property to store that. Define a private static field named `forestsCreated`.

2.

Define a public static property named `ForestsCreated`. Give it a public getter and private setter.

3.

In the first constructor, increment `ForestsCreated`. This will add 1 to the property every time an object is constructed.

Forest.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Forest
```

```
    {
```

```
        // FIELDS
```

```
public int age;  
private string biome;
```

```
// CONSTRUCTORS
```

```
public Forest(string name, string biome)  
{  
    this.Name = name;  
    this.Biome = biome;  
    Age = 0;  
}
```

```
public Forest(string name) : this(name, "Unknown")  
{ }
```

```
// PROPERTIES
```

```
public string Name  
{ get; set; }
```

```
public int Trees  
{ get; set; }
```

```
public string Biome
```

```

{
    get { return biome; }
    set
    {
        if (value == "Tropical" ||
            value == "Temperate" ||
            value == "Boreal")
        {
            biome = value;
        }
        else
        {
            biome = "Unknown";
        }
    }
}

```

```

public int Age
{
    get { return age; }
    private set { age = value; }
}

```

```

// METHODS

```

```
public int Grow()
{
    Trees += 30;
    Age += 1;
    return Trees;
}
```

```
public int Burn()
{
    Trees -= 20;
    Age += 1;
    return Trees;
}
```

```
}
```

```
}
```

Program.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
class Program
{
    static void Main(string[] args)
    {

    }
}
}
```

13.3 Static Methods

You already know how to create an instance method, like:

```
class Forest
{
    private string definition;
    public void Define()
    {
        Console.WriteLine(definition);
    }
}
```

This behavior (printing a general definition) isn't specific to any one instance — it applies to the class itself, so it should be made static.

To make a static method, just add `static` after the access modifier (public or private).

```
class Forest
{
    private static string definition;
    public static void Define()
```

```

{
    Console.WriteLine(definition);
}
}

```

Notice that we added static to both the field definition and method Define().

This is because a static method can only access other static members. It cannot access instance members:

```

class Forest
{
    private string definition;
    public static void Define()
    {
        // Throws error because definition is not static
        Console.WriteLine(definition);
    }
}

```

Otherwise, static methods work like any other method.

13.3 Static Methods Instructions

1.

Earlier we mentioned storing an explanation of forests in general. We'll use a field and property to define the explanation. Define a private static string field named treeFacts.

2.

Define a public static property named TreeFacts with just a getter (no setter).

3.

Define a public static method name PrintTreeFacts() that writes the value of TreeFacts to the console.

Note that TreeFacts is never assigned a value: we'll resolve that in the next exercise.

Forest.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Forest
```

```
    {
```

```
        // FIELDS
```

```
        public int age;
```

```
        private string biome;
```

```
        private static int forestsCreated;
```

```
        // CONSTRUCTORS
```

```
        public Forest(string name, string biome)
```

```
        {
```

```
            this.Name = name;
```

```
            this.Biome = biome;
```

```
            Age = 0;
```

```
            ForestsCreated++;
```

```
        }
```

```
public Forest(string name) : this(name, "Unknown")
```

```
{ }
```

```
// PROPERTIES
```

```
public string Name
```

```
{ get; set; }
```

```
public int Trees
```

```
{ get; set; }
```

```
public string Biome
```

```
{
```

```
    get { return biome; }
```

```
    set
```

```
    {
```

```
        if (value == "Tropical" ||
```

```
            value == "Temperate" ||
```

```
            value == "Boreal")
```

```
        {
```

```
            biome = value;
```

```
        }
```

```
    else
```



```
{  
    biome = "Unknown";  
}  
}  
}
```

```
public int Age  
{  
    get { return age; }  
    private set { age = value; }  
}
```

```
public static int ForestsCreated  
{  
    get { return forestsCreated; }  
    private set { forestsCreated = value; }  
}
```

```
// METHODS
```

```
public int Grow()  
{  
    Trees += 30;  
    Age += 1;
```

```

        return Trees;
    }

    public int Burn()
    {
        Trees -= 20;
        Age += 1;
        return Trees;
    }

}

}

```

Program.cs

```

using System;

namespace StaticMembers
{
    class Program
    {
        static void Main(string[] args)
        {

```

```
}  
}  
}
```

13.4 Static Constructors

An instance constructor is run before an instance is used, and a *static constructor* is run once before a class is used:

```
class Forest  
{  
    static Forest()  
    { /* ... */ }  
}
```

This constructor is run when either one of these events occurs:

- Before an object is made from the type.
- Before a static member is accessed.

In other words, if this was the first line in `Main()`, a static constructor for `Forest` would be run:

```
Forest f = new Forest();
```

It would also be run if this was the first line in `Main()`:

```
Forest.Define();
```

Typically we use static constructors to set values to static fields and properties.

A static constructor does not accept an access modifier.

13.4 Static Constructors Instructions

1.

In the previous exercises our `treeFacts` and `forestsCreated` fields were never given values! We'll fix that.

First, create a static constructor for `Forest`.

2.

In the body of the static constructor, set the `treeFacts` field to this string:

```
"Forests provide a diversity of ecosystem services including:\r\n    aiding in  
regulating climate.\r\n    purifying water.\r\n    mitigating natural hazards such as  
floods.\n"
```

3.

In the body of the static constructor, set the `ForestsCreated` property to 0.

4.

In **Program.cs**, call `Forest.PrintTreeFacts()` to check that the `TreeFacts` property was set.

Program.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
}  
}  
}
```

Forest.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Forest
```

```
    {
```

```
        // FIELDS
```

```
        public int age;
```

```
        private string biome;
```

```
        private static int forestsCreated;
```

```
        private static string treeFacts;
```

```
        // CONSTRUCTORS
```

```
        public Forest(string name, string biome)
```

```
        {
```

```
            this.Name = name;
```

```
this.Biome = biome;  
Age = 0;  
ForestsCreated++;  
}
```

```
public Forest(string name) : this(name, "Unknown")  
{ }
```

```
// PROPERTIES
```

```
public string Name  
{ get; set; }
```

```
public int Trees  
{ get; set; }
```

```
public string Biome  
{  
    get { return biome; }  
    set  
    {  
        if (value == "Tropical" ||  
            value == "Temperate" ||
```

```

        value == "Boreal")
    {
        biome = value;
    }
    else
    {
        biome = "Unknown";
    }
}
}

```

```

public int Age
{
    get { return age; }
    private set { age = value; }
}

```

```

public static int ForestsCreated
{
    get { return forestsCreated; }
    private set { forestsCreated = value; }
}

```

```

public static string TreeFacts

```

```
{  
    get { return treeFacts; }  
}  
  
// METHODS  
  
public int Grow()  
{  
    Trees += 30;  
    Age += 1;  
    return Trees;  
}  
  
public int Burn()  
{  
    Trees -= 20;  
    Age += 1;  
    return Trees;  
}  
  
public static void PrintTreeFacts()  
{  
    Console.WriteLine(TreeFacts);  
}
```



```
}
```

```
}
```

OUTPUT

Forests provide a diversity of ecosystem services including:

aiding in regulating climate.

purifying water.

mitigating natural hazards such as floods.

13.5 Static Classes

We covered a few static members: field, property, method, and constructor.
What if we made the whole class static?

```
static class Forest {}
```

A static class cannot be instantiated, so you only want to do this if you are making a utility or library, like Math or Console.

These two common classes are static because they are just tools — they don't need specific instances and they don't store new information.

Now when you see something like:

```
Math.Min(34, 54);
```

```
Console.WriteLine("yeehaw!");
```

You know that these are two static classes calling two static methods.

13.5 Static Classes Instructions

1.

We rarely create static classes of our own, so let's practice using other static classes. First print the value of pi — a commonly-used value in geometry —, which is stored in Math.PI.

2.

Find the absolute value of -32 using the method Math.Abs(). This method returns the absolute value, or “positive version”, of the argument.

Print the result to the console.

Program.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

OUTPUT

3.14159265358979

32

13.6 Common Static Errors

With great power comes great responsibility. If you plan on using static, you must be familiar with the errors you might cause! Here a few common ones:

This error usually means you labeled a static constructor as public or private, which is not allowed:

error CS0515: 'Forest.Forest()': static constructor cannot have an access modifier

This usually means you tried to reference a non-static member from a class, instead of from an instance:

error CS0120: An object reference is required to access non-static field, method, or property 'Forest.Grow()'

This usually means that you tried to reference a static member from an instance, instead of from the class:

error CS0176: Member 'Forest.TreeFacts' cannot be accessed with an instance reference; qualify it with a type name instead

13.6 Common Static Errors Instructions

1.

Fix error CS0515.

2.

Fix error CS0120.

3.

Fix error CS0176.

Program.cs

using System;

namespace StaticMembers

{

class Program

{

static void Main(string[] args)

{

Forest f = new Forest("Congo", "Tropical");

Forest.Grow();

Console.WriteLine(f.TreeFacts);

}

}

}

Forest.cs

using System;

```

namespace StaticMembers
{
    class Forest
    {
        // FIELDS

        public int age;
        private string biome;
        private static int forestsCreated;
        private static string treeFacts;

        // CONSTRUCTORS

        public Forest(string name, string biome)
        {
            this.Name = name;
            this.Biome = biome;
            Age = 0;
            ForestsCreated++;
        }

        public Forest(string name) : this(name, "Unknown")
        { }
    }
}

```

```

public static Forest()
{
    treeFacts = "Forests provide a diversity of ecosystem services including:\r\n
aiding in regulating climate.\r\n purifying water.\r\n mitigating natural hazards
such as floods.\n";

    ForestsCreated = 0;

}

// PROPERTIES

public string Name
{ get; set; }

public int Trees
{ get; set; }

public string Biome
{
    get { return biome; }
    set
    {
        if (value == "Tropical" ||
            value == "Temperate" ||

```

```
        value == "Boreal")
    {
        biome = value;
    }
    else
    {
        biome = "Unknown";
    }
}
}
```

```
public int Age
{
    get { return age; }
    private set { age = value; }
}
```

```
public static int ForestsCreated
{
    get { return forestsCreated; }
    private set { forestsCreated = value; }
}
```

```
public static string TreeFacts
```

```
{
    get { return treeFacts; }
}

// METHODS

public int Grow()
{
    Trees += 30;
    Age += 1;
    return Trees;
}

public int Burn()
{
    Trees -= 20;
    Age += 1;
    return Trees;
}

public static void PrintTreeFacts()
{
    Console.WriteLine(TreeFacts);
}
```



```
}
```

```
}
```

OUTPUT

Forests provide a diversity of ecosystem services including:

aiding in regulating climate.

purifying water.

mitigating natural hazards such as floods.

13.7 Main()

Now that you're familiar with classes, you're ready to tackle the Main() method, the entry point for any program. You've seen it many times, but now you can explain every part!

```
class Program
{
    public static void Main (string[] args)
    {
    }
}
```

- Main() is a method of the Program class.
- public — The method can be called outside the Program class.
- static — The method is called from the class name: Program.Main().
- void — The method means returns nothing.

- `string[] args` — The method has one parameter named `args`, which is an array of strings.

`Main()` is like any other method you've encountered. It has a special use for C#, but that doesn't mean you can't treat it like a plain old method!

13.7 Main() Instructions

1.

Each time we run `dotnet run`, the `Main()` method is called. We can include arguments on the command line, like `dotnet run arg1 arg2 arg3` that will be converted into an array as the `args` parameter. In the console, enter:

```
dotnet run mango pineapple lychee
```

Based on this new information, how is your text printed to the console?

Program.cs

```
using System;
```

```
namespace ApplyingClasses
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            if (args.Length > 0)
```

```
            {
```

```
                string mainPhrase = String.Join(" and ", args);
```

```

        Console.WriteLine($"My favorite fruits are {mainPhrase}!");
    }

}
}
}

```

OUTPUT

```

$ dotnet run mango pineapple lychee
My favorite fruits are mango and pineapple an
d lychee!

```

13.8 Review

Congrats! You are now ready to use static throughout your classes:

- In general, *static* means “associated with the class, not an instance”.
- A static member is always accessed by the class name, rather than the instance name, like `Forest.Area`.
- A static method cannot access non-static members.
- A static constructor is run once per type, not per instance. It is invoked before the type is instantiated or a static member is accessed.
- Either of these would trigger the static constructor of `Forest`:

```

public static void Main() {
    Forest f = new Forest();
}

```

or

```
public static void Main() {  
    Forest.Define();  
}
```

- A static class cannot be instantiated. Its members are accessed by the class name, like Math.PI.

13.8 Review Instructions

1.

From **Program.cs**, print the number of forests created.

2.

Instantiate two Forest objects.

3.

Print the number of forests created again. Before moving on, make sure you can explain how this value was changed.

Program.cs

```
using System;
```

```
namespace StaticMembers
```

```
{
```

```
    class Program
```

```
{
```

```
static void Main(string[] args)
{

}
}
}
```

Forest.cs

```
using System;

namespace StaticMembers
{
    class Forest
    {
        // FIELDS

        public int age;
        private string biome;
        private static int forestsCreated;
        private static string treeFacts;

        // CONSTRUCTORS

        public Forest(string name, string biome)
```

```

{
    this.Name = name;
    this.Biome = biome;
    Age = 0;
    ForestsCreated++;
}

```

```

public Forest(string name) : this(name, "Unknown")
{ }

```

```

static Forest()
{
    treeFacts = "Forests provide a diversity of ecosystem services including:\r\n
aiding in regulating climate.\r\n purifying water.\r\n mitigating natural hazards
such as floods.\n";
    ForestsCreated = 0;
}

```

```

// PROPERTIES

```

```

public string Name
{ get; set; }

```

```

public int Trees

```

```
{ get; set; }
```

```
public string Biome
```

```
{
```

```
    get { return biome; }
```

```
    set
```

```
{
```

```
    if (value == "Tropical" ||
```

```
        value == "Temperate" ||
```

```
        value == "Boreal")
```

```
{
```

```
    biome = value;
```

```
}
```

```
else
```

```
{
```

```
    biome = "Unknown";
```

```
}
```

```
}
```

```
}
```

```
public int Age
```

```
{
```

```
    get { return age; }
```

```
    private set { age = value; }
```

```
}
```

```
public static int ForestsCreated  
{  
    get { return forestsCreated; }  
    private set { forestsCreated = value; }  
}
```

```
public static string TreeFacts  
{  
    get { return treeFacts; }  
}
```

```
// METHODS
```

```
public int Grow()  
{  
    Trees += 30;  
    Age += 1;  
    return Trees;  
}
```

```
public int Burn()  
{
```



```
Trees -= 20;  
Age += 1;  
return Trees;  
}
```

```
public static void PrintTreeFacts()  
{  
    Console.WriteLine(TreeFacts);  
}  
  
}  
  
}
```

OUTPUT

0
2

13. Quiz

1. Why will this class definition cause an error?

```
class DollHouse  
{  
    private int numberOfRooms;
```

```

public static string Describe()
{
    return $"Each dollhouse has {numberOfRooms} rooms.";
}
}

```

- a) No constructor is defined.
- b) The static method Describe() attempts to access the instance field numberOfRooms.
- c) numberOfRooms is private.
- d) numberOfRooms is not assigned a value.

2. Call the static method ToBoolean(), which is defined in the Convert class. It should take the argument answer.

```

string answer = "true";
bool convertedAnswer = _____ ( _____ );

```

- .
- ToBoolean
- answer
- Convert

Click or drag and drop to fill in the blank

3. What is true about a *static class*?

- a) All of its members are private.
- b) It cannot be instantiated.
- c) It must be instantiated with a static constructor.
- d) It has no members.

4. Given this definition of Main(), which of these statements is TRUE?

```
class MainClass
{
    public static void Main (string[] args)
    {
    }
}
```

- a) Main() returns nothing.
- b) Main() can only be accessed by code within MainClass.
- c) Main() has no parameters.
- d) Main() is an instance method.

5. When Main() is run, what is printed FIRST to the console?

```
class MainClass
{
    public static void Main (string[] args) {
        SpaceInvader.Beep();
        SpaceInvader enemy = new SpaceInvader();
    }
}
```

```
class SpaceInvader
{
    static SpaceInvader()
    {
        Console.WriteLine("Aliens detected!");
    }

    public SpaceInvader()
    {
        Console.WriteLine("SpaceInvader instantiated.");
    }
}
```

```

public static void Beep()
{
    Console.WriteLine("* beep beep *");
}
}

```

- a) "SpaceInvader instantiated."
- b) Nothing is printed.
- c) "Aliens detected!"
- d) "* beep beep *"

6. The Forest class has:

- a public static ForestsCreated property
- a public static Define() method
- no instance constructors explicitly defined

Which of these statements will cause an error?

```

class MainClass
{
    public static void Main (string[] args) {
        Forest.Define();
        int count = Forest.ForestsCreated;
        Forest jungle = new Forest();
        jungle.Define();
    }
}

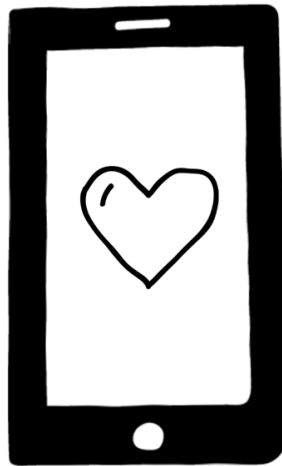
```

- a) jungle.Define();
- b) int count = Forest.ForestsCreated;
- c) Forest.Define();
- d) Forest jungle = new Forest();

Project-8

The Object of Your Affection

Your friend is building a new match-making service: The Object of Your Affection or OOYA for short (don't worry, you still have time to convince them to change the name).



With your new understanding of C# objects and classes, your friend thought you could build a pretty well-organized system of dating profiles.

Your first step? Build a Profile class that allows users to generate profile objects.

The Profile should store the following information:

- User's name
- User's age
- User's city

- User's country
- User's pronouns
- User's hobbies

And this is how users should be able to interact with their own profiles:

- Create a new profile with some information
- Add hobbies
- View profile

Let's get started!

TASKS

The Fields of a Classy Profile

1.

Tab over to **Profile.cs** and set up the skeleton of the Profile class.

2.

Add the following fields to Profile:

- a string name
- an int age
- a string city
- a string country
- a string pronouns
- a string[] hobbies

We could implement these as properties, but we'll use fields. Properties are used to:

- validate values
- control external access

Later in this project you'll see how we achieve the same result with methods.

3.



Tab over to **Program.cs**. In `Main()`:

- Instantiate a new Profile object called `sam`. (Your friend Sam is looking for love.)
- Try to give `sam` a name: "Sam Drakkila".
- Then try to run the code using `dotnet run`.

Going Public

4.

Yikes, what was that error message all about? Something to do with Profile.name being private?

Oh that's right! All the members in a class (including name) are automatically set to private.

To make this more clear for ourselves and others, make the access level explicit: Add private to all the fields you created in Profile.

5.

Users should be able to add their profile information in a constructor.

Below the fields, declare a constructor for Profile that allows you to set:

- name
- age
- city
- country
- pronouns (give this a default value of "they/them" just in case it's ever left blank)

Define the constructor in **Profile.cs** and set the fields to the values passed in. Make sure to also set hobbies to an empty array of strings.

Use this to differentiate parameters from instance fields. For example, this will work:

```
public Profile(int population)
{ this.population = population; }
```

But this won't:

```
public Profile(int population)
{ population = population; }
```

6.

Time to test your code out!

If you assigned sam a name in **Program.cs** before, remove that line. Where sam is constructed, pass in the following information:

- a name of "Sam Drakkila"
- an age of 30
- city and country of "New York" and "USA"
- pronouns of "he/him"

Then run your code.

7.

Nice work! But how can you access profile information once it's been added?

We could use properties, but we'd like users to see all of the information in a single, formatted string. Time to add a `ViewProfile()` method.

In **Profile.cs** define a `ViewProfile()` method under the constructor. It should have:

- public access
- a return type of string
- no parameters

It should return a string containing all of the profile's information.

8.

In `Main()`, test out the new method on sam and print out the result.

Hobbies

9.

You still need to give users a way to set hobbies.

In **Profile.cs**, declare a new method `SetHobbies()` with:

- public access
- no return value
- a `string[]` parameter named `hobbies`

In the method body, set the field `this.hobbies` equal to the `hobbies` argument.

10.

Great! Go back into `ViewProfile()` and modify the method so that you display a profile's hobbies.

(Remember, you can loop through `this.hobbies` to access each element.)

11.

Before you show this all to your friend, be sure to test your work.

Above where you print sam's profile information out, add some hobbies to sam:

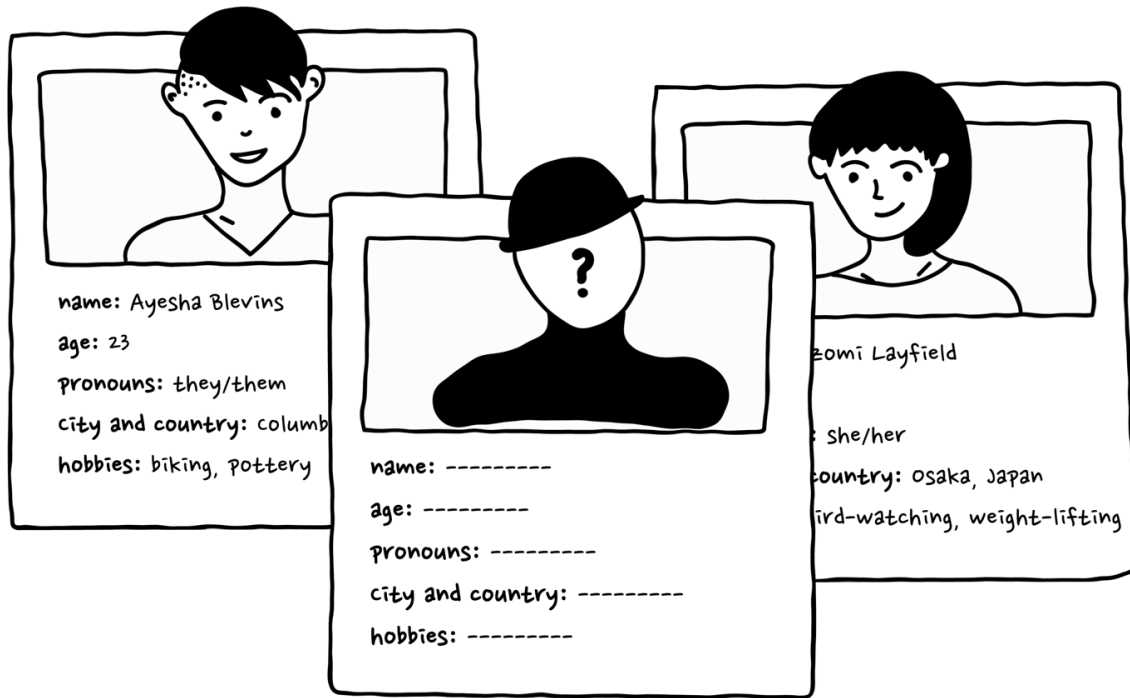
- "listening to audiobooks and podcasts"
- "playing rec sports like bowling and kickball"
- "writing a speculative fiction novel"
- "reading advice columns"

Now run your code!

Tweaks

12.

Your friend is super impressed with the `Profile` class you've created!



Here are a few suggestions to make the Profile class even better:

- If you call `ViewProfile()` before calling `SetHobbies()`, you'll get an error because the hobbies field isn't set to any value. Fix the class so that you can call `ViewProfile()` without adding hobbies.
- Convert the fields into private properties and add validation. For example, users must be at least 18 years of age.
- Some users may create a profile with just a name and age. Use optional parameters or create a constructor overload to handle those issues.

Program.cs

```
using System;
```

```
namespace DatingProfile
```

```
{
```

```
    class Program
```

```
{  
    static void Main(string[] args)  
    {  
  
    }  
}  
}
```

Profile.cs

```
using System;
```

```
namespace DatingProfile
```

```
{  
  
}
```

OUTPUT

```
$ dotnet run  
Name: Sam Drakkila  
Age: 30  
City: New York  
Country: USA  
Pronouns: he/him  
Hobbies:  
- listening to audiobooks and podcasts  
- playing rec sports like bowling and kickba  
ll  
- writing a speculative fiction novel  
- reading advice columns
```

14. INTERFACES

14.1 Introduction to Interfaces

Since programming is complex, it's easy to make mistakes. For example, maybe we try to use an `int` like a `string`. This would cause a type error. C# has rules built-in that check for those mistakes before we use that code in the real world.

The diagram below estimates the number of bugs in our code at each stage of development. We discover and fix some of those bugs at each stage, so the number decreases from left to right. Sadly we can't fix every bug. But if we can catch bugs earlier, then we save ourselves work and we save users frustration.

We say that C# is "type-safe" because it helps us catch bugs at one of the earliest stages: the type checking stage.

The difference between an `int` and a `string` is clear, but what about custom-defined classes? Say we made `Fruit` and `Vegetable` types. Can we use `PlantSeed()` on both? Do they both have a `Seed` property? The computer running the program needs more information.

In this lesson we'll dive into *interfaces*, which are sets of actions and values that describe how a class can be used. This lets the computer check that we are using each type legally, thus preventing a whole group of type errors!

In this lesson you'll learn how to:

- build your own interface
- write classes that implement an interface

14.1 Introduction to Interfaces Instructions

Based on the code in **Program.cs**, we can expect the Sedan class to have:

- a Speed property
- a LicensePlate property
- a Wheels property
- a Honk() method

By the end of this lesson you'll build an interface that guarantees that all Sedans have these behaviors and attributes.

Program.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Sedan s = new Sedan(60);
```

```
            Console.WriteLine($"Sedan with license plate {s.LicensePlate} and {s.Wheels} wheels, driving at {s.Speed} km/h.");
```

```
            s.Honk();
```

```
        }
```

```
    }
```

```
}
```

Sedan.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
    class Sedan
```

```
    {
```

```
        public string LicensePlate
```

```
        { get; }
```

```
        public double Speed
```

```
        { get; private set; }
```

```
        public int Wheels
```

```
        { get; }
```

```
        public Sedan(double speed)
```

```
        {
```

```
            Speed = speed;
```

```
            LicensePlate = Tools.GenerateLicensePlate();
```

```
            Wheels = 4;
```

```
        }
```

```
public void Honk()
{
    Console.WriteLine("HONK!");
}
```

```
public void SpeedUp()
{
    Speed += 5;
}
```

```
public void SlowDown()
{
    Speed -= 5;
}
```

```
}
```

```
}
```

OUTPUT

Sedan with license plate VV534HBM and 4 wheels, driving at 60 km/h.

HONK!

14.2 Build an Interface

For this lesson we will be designing a new set of transportation machines that satisfy the requirements of BOTH car designers and the highway patrol. First the highway patrol tells us: “Every automobile on the road must have these properties and methods accessible to us:”

- speed
- license plate number
- number of wheels
- ability to honk

The patrol needs this information to write speeding tickets and prevent bad behavior on the highway.

In other words, the patrol makes these requirements so that it can interact with automobiles in a certain way. In C#, this group of interactions is called an *interface*. The interface is a set of properties, methods, and other members. They are declared with a signature but their behaviors are not defined. A class *implements* an interface if it defines those properties, methods, and other members.

For example, if the patrol requires automobiles to have a license plate, then the `IAutomobile` interface contains a `LicensePlate` property. A class implements this interface if it defines a `LicensePlate` property.

The skeleton of an interface looks a bit like a class:

```
interface IAutomobile
{
}
```

Every interface should have a name starting with “I”. This is a useful reminder to other developers and our future selves that this is an interface, not a class. We can add members, like properties and methods, to the interface. Here’s an example of a fake property and method:

```
interface IAutomobile
{
    string Id { get; }
```

```
void Vroom();  
}
```

Notice that the property and method bodies are not defined. An interface is a set of actions and values, but it doesn't specify how they work.

In our highway example, the highway patrol doesn't care HOW the license plate property and honk method work, they just care whether every automobile has it.

14.2 Build an Interface Instructions

1.

Just like classes, interfaces are best organized in their own files.

In **IAutomobile.cs** within the LearnInterfaces namespace, create an empty IAutomobile interface.

2.

Add these three properties and one method to the interface:

- a string called LicensePlate
- a double called Speed
- an int called Wheels
- a void method called Honk()

The properties only need a getter. Use the get shorthand demonstrated in the narrative for Id.

IAutomobile.cs

```
using System;
```

```
namespace LearnInterfaces
{

}
```

14.3 Implementing an Interface

Our interface is complete! Pretty easy, right?

As we design our automobile-like classes, we'll need to implement this `IAutomobile` interface. In C#, we must first clearly announce that a class implements an interface using the colon syntax:

```
class Sedan : IAutomobile
{
}
```

This empty Sedan class “promises” to implement the `IAutomobile` interface. In other words, it must have the properties and methods the highway patrol asked for (Speed, LicensePlate, Wheels, and Honk()).

If we don't, we get a type error like this:

```
error CS0535: Sedan does not implement interface member
'IAutomobile.LicensePlate'
```

To fix this we'll need to define the members in the interface:

```
class Sedan : IAutomobile
{
    public string LicensePlate
    { get; }

    // and so on...
}
```

Remember that these members must be public. How else will the highway patrol be able to access them?

14.3 Implementing an Interface Instructions

1.

In **Sedan.cs**, create an empty Sedan class that implements the IAutomobile interface. Use colon (:) notation.

2.

You should see the error CS0535 telling you that the Sedan needs to implement the interface! Implement the interface by adding the three properties and one method defined in IAutomobile, which you can check in **IAutomobile.cs**.

Sedan.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
}
```

IAutomobile.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
interface IAutomobile
{
    string LicensePlate { get; }
    double Speed { get; }
    int Wheels { get; }
    void Honk();
}
}
```

14.4 What Interfaces Cannot Do

The Sedan needs to satisfy more than the highway patrol's rules (the IAutomobile interface). The car designers have asked that sedans are built and move in certain ways — it must have constructors and methods that aren't required by the IAutomobile interface. This is okay in C#! The interface says what a class **MUST** have. It does not say what a class **MUST NOT** have.

In fact, interfaces cannot specify two types of members that are commonly found in classes:

- Constructors
- Fields

14.4 What Interfaces Cannot Do Instructions

1.

Add a constructor to the Sedan class with one parameter, speed, of type double. It should

- set the Speed property to speed

- set a random LicensePlate value
- set Wheels to 4

To make a random license plate, a utility class is provided for you. Use it in the constructor like so: Tools.GenerateLicensePlate().

2.

Add a void SpeedUp() method that increases the Speed property by 5.

3.

Did you get an error? There is no setter for the Speed property. Add a private setter such as private set to the Speed property.

4.

Add a void SlowDown() method that decreases the Speed by 5.

Sedan.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
    class Sedan : IAutomobile
```

```
    {
```

```
        public string LicensePlate
```

```
        { get; }
```

```
public double Speed
{ get; }

public int Wheels
{ get; }

public void Honk()
{
    Console.WriteLine("HONK!");
}

}

}
```

14.5 Implementing an Interface Again

We've completed a Sedan class that satisfies both car designers and highway patrol: it can be constructed and change speed, and it implements the `IAutomobile` interface.

But sedans aren't the only automobile on the road. There can be multiple classes that implement an interface.

For example, we can create a Truck class that also implements the interface.

This is where we start to see the power of interfaces. Even though Sedan and Truck are different types, we can assume that they behave similarly because they share an interface. Car designers can build different vehicle classes, but the highway patrol can treat them all the same.

14.5 Implementing an Interface Again Instructions

1.

In **Truck.cs**, create an empty Truck class that implements the IAutomobile interface.

2.

You should see the error CS0535 telling you that the Truck needs to implement the interface! Implement the interface by adding the three properties and one method defined in IAutomobile, which you can check in **IAutomobile.cs**.

Truck.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
}
```

IAutomobile.cs

```
using System;
```

```
namespace LearnInterfaces
```

```
{
```

```
    interface IAutomobile
```

```
    {
```

```
        string LicensePlate { get; }
```



```
double Speed { get; }  
int Wheels { get; }  
void Honk();  
}  
}
```

14.6 Finish Truck Class

The car designers have asked that trucks act a bit differently from sedans. Trucks need a new property called `Weight`. Whenever a truck is constructed, its number of wheels will depend on its weight. For example, a heavier truck might need 12 instead of 8 wheels to support itself.

Just like sedans, trucks will also `SpeedUp()` and `SlowDown()`.

14.6 Finish Truck Class Instructions

1.

Add a public double `Weight` property with just a getter.

2.

Add a constructor to the `Truck` class with two parameters: double speed and double weight. It should:

- Set the `Speed` property using `speed`
- Set the `Weight` property using `weight`
- Set a random `LicensePlate` value using `Tools.GenerateLicensePlate()`
- Set `Wheels` to 8 if `Weight` is less than 400 and set `Wheels` to 12 otherwise

3.

Add a void SpeedUp() method that increases the Speed property by 5.

4.

Did you get an error? There is no setter for the Speed property. Add a private setter to that property.

5.

Add a void SlowDown() method that decreases the Speed by 5.

Truck.cs

using System;

namespace LearnInterfaces

{

class Truck : IAutomobile

{

public string LicensePlate

{ get; }

public double Speed

{ get; }

public int Wheels

```

    { get; }

    public void Honk()
    {
        Console.WriteLine("HONK!");
    }

}
}

```

14.7 Testing Interfaces

Now we have a Sedan class and Truck class that implement the IAutomobile interface. Though they have some different behaviors, they both have the properties and method defined in the interface:

- double Speed
- string LicensePlate
- int Wheels
- void Honk()

At this point we can be confident that we won't cause any errors if we try to access these members in either the Sedan or Truck class.

14.7 Testing Interfaces Instructions

1.

Create two sedans and a truck:

- a sedan with speed 60
- a sedan with speed 70
- a truck with speed 45 and weight 500

2.

Write three Console.WriteLine() statements that print the automobiles' Speed, Wheels, and LicensePlate.

3.

Call SpeedUp() on all three automobiles.

4.

Using Console.WriteLine(), print out the three automobile's new speeds.

Program.cs

using System;

namespace LearnInterfaces

{

class Program

{

static void Main(string[] args)

{

}

}

}

IAutomobile.cs

using System;

namespace LearnInterfaces

{

interface IAutomobile

{

string LicensePlate { get; }

double Speed { get; }

int Wheels { get; }

void Honk();

}

}

Sedan.cs

using System;

namespace LearnInterfaces

{

class Sedan : IAutomobile

{

public string LicensePlate

{ get; }

```
public double Speed
```

```
{ get; private set; }
```

```
public int Wheels
```

```
{ get; }
```

```
public Sedan(double speed)
```

```
{
```

```
    Speed = speed;
```

```
    LicensePlate = Tools.GenerateLicensePlate();
```

```
    Wheels = 4;
```

```
}
```

```
public void Honk()
```

```
{
```

```
    Console.WriteLine("HONK!");
```

```
}
```

```
public void SpeedUp()
```

```
{
```

```
    Speed += 5;
```

```
}
```

```
public void SlowDown()
```

```
{  
    Speed -= 5;  
}  
  
}  
}
```

Truck.cs

```
using System;  
  
namespace LearnInterfaces  
{  
    class Truck : IAutomobile  
    {  
        public string LicensePlate  
        { get; }  
  
        public double Speed  
        { get; private set; }  
  
        public int Wheels  
        { get; }  
  
        public double Weight
```

```
{ get; }
```

```
public Truck(double speed, double weight)
{
    Speed = speed;
    LicensePlate = Tools.GenerateLicensePlate();
    Weight = weight;
```

```
    if (weight < 400)
    {
        Wheels = 8;
    }
    else
    {
        Wheels = 12;
    }
}
```

```
public void Honk()
{
    Console.WriteLine("HONK!");
}
```

```
public void SpeedUp()
```



```
{  
    Speed += 5;  
}  
  
public void SlowDown()  
{  
    Speed -= 5;  
}  
  
}  
}
```

OUTPUT

Sedan with license plate GSQJMA1Q and 4 wheels, driving at 60 km/h.

Sedan's faster speed: 65

Sedan with license plate KD69NHIL and 4 wheels, driving at 70 km/h.

Sedan's faster speed: 75

Truck with license plate DGNVK3ZH and 12 wheels, driving at 45 km/h.

Truck's faster speed: 50

14.8 Review

Well done! In this lesson, you:

- Learned that interfaces are useful to guarantee certain functionality across multiple classes
- Built an interface using the interface keyword
- Defined properties and methods (but not constructors or fields) in the interface
- Built classes that implemented the interface
- Added members to the classes that weren't specified in the interface

As a last note: a class can implement multiple interfaces. For example, Sedan could implement IAutomobile and a new IRecyclable interface. It would be useful to separate interfaces if they aren't related, i.e. not all automobiles are recyclable.

With this lesson you completed, you might be asking yourself this question:

We have duplicated code, like SpeedUp() and SlowDown(), in two classes, and we know that duplicated code is hard to maintain. Is there a way to avoid duplication?

The answer has to do with *inheritance*. The concept won't be covered in this lesson, but now you have one good reason to learn it.

Your C# skills are growing. Keep up the good work!

14.8 Review Instructions

The completed code is provided for you here. Make sure you are comfortable with interfaces before you move on from this lesson.

- **IAutomobile.cs** defines the interface
- **Sedan.cs** and **Truck.cs** define two classes that implement the interface
- **Program.cs** demonstrates those classes in action

IAutomobile.cs

using System;

namespace LearnInterfaces

{

interface IAutomobile

{

string LicensePlate { get; }

double Speed { get; }

int Wheels { get; }

void Honk();

}

}

Sedan.cs

using System;

namespace LearnInterfaces

{

class Sedan : IAutomobile

{

public string LicensePlate

{ get; }

```
public double Speed
```

```
{ get; private set; }
```

```
public int Wheels
```

```
{ get; }
```

```
public Sedan(double speed)
```

```
{
```

```
    Speed = speed;
```

```
    LicensePlate = Tools.GenerateLicensePlate();
```

```
    Wheels = 4;
```

```
}
```

```
public void Honk()
```

```
{
```

```
    Console.WriteLine("HONK!");
```

```
}
```

```
public void SpeedUp()
```

```
{
```

```
    Speed += 5;
```

```
}
```

```
public void SlowDown()
```

```
{  
    Speed -= 5;  
}  
  
}  
}
```

Truck.cs

```
using System;  
  
namespace LearnInterfaces  
{  
    class Truck : IAutomobile  
    {  
        public string LicensePlate  
        { get; }  
  
        public double Speed  
        { get; private set; }  
  
        public int Wheels  
        { get; }  
  
        public double Weight
```

```
{ get; }
```

```
public Truck(double speed, double weight)
{
    Speed = speed;
    LicensePlate = Tools.GenerateLicensePlate();
    Weight = weight;
```

```
    if (weight < 400)
    {
        Wheels = 8;
    }
    else
    {
        Wheels = 12;
    }
}
```

```
public void Honk()
{
    Console.WriteLine("HONK!");
}
```

```
public void SpeedUp()
```

```

    {
        Speed += 5;
    }

    public void SlowDown()
    {
        Speed -= 5;
    }

}

```

Program.cs

```

using System;

namespace LearnInterfaces
{
    class Program
    {
        static void Main(string[] args)
        {
            Sedan s = new Sedan(60);

            Console.WriteLine($"Sedan with license plate {s.LicensePlate} and {s.Wheels}
wheels, driving at {s.Speed} km/h.");

            s.SpeedUp();

```

```
Console.WriteLine($"Sedan's faster speed: {s.Speed}");
```

```
Sedan s2 = new Sedan(70);
```

```
Console.WriteLine($"Sedan with license plate {s2.LicensePlate} and  
{s2.Wheels} wheels, driving at {s2.Speed} km/h.");
```

```
s2.SpeedUp();
```

```
Console.WriteLine($"Sedan's faster speed: {s2.Speed}");
```

```
Truck t = new Truck(45, 500);
```

```
Console.WriteLine($"Truck with license plate {t.LicensePlate} and {t.Wheels}  
wheels, driving at {t.Speed} km/h.");
```

```
t.SpeedUp();
```

```
Console.WriteLine($"Truck's faster speed: {t.Speed}");
```

```
}
```

```
}
```

```
}
```

Concept Review

C# Interface

In C#, an *interface* contains definitions for a group of related functionalities that a class can implement.

Interfaces are useful because they guarantee how a class behaves. This, along with the fact that a class can implement multiple interfaces, helps organize and modularize components of software.

It is best practice to start the name of an interface with “I”.

```
interface IAutomobile
```

```
{  
    string LicensePlate { get; }  
    double Speed { get; }  
    int Wheels { get; }  
}
```

// The IAutomobile interface has three properties. Any class that implements this interface must have these three properties.

```
public interface IAccount
```

```
{  
    void PayInFunds ( decimal amount );  
    bool WithdrawFunds ( decimal amount );  
    decimal GetBalance ();  
}
```

// The IAccount interface has three methods to implement.

```
public class CustomerAccount : IAccount
```

```
{ }
```

// This CustomerAccount class is labeled with : IAccount, which means that it will implement that interface.

C# Inheritance

In C#, *inheritance* is the process by which one class inherits the members of another class. The class that inherits is called a *subclass* or *derived* class. The other class is called a *superclass*, or a *base* class.

When you define a class that inherits from another class, the derived class implicitly gains all the members of the base class, except for its constructors and finalizers. The derived class can thereby reuse the code in the base class without having to re-implement it. In the derived class, you can add more members. In this manner, the derived class extends the functionality of the base class.

```
public class Honeymoon : TripPlanner  
{ }
```

// Similar to an interface, inheritance also uses the colon syntax to denote a class inherited super class. In this case, Honeymoon class inherits from TripPlanner class.

// A derived class can only inherit from one base class, but inheritance is transitive. That base class may inherit from another class, and so on, which creates a class hierarchy.

C# protected Keyword

In C#, a protected member can be accessed by the current class and any class that inherits from it. This is designated by the protected access modifier.

```
public class BankAccount
{
    protected decimal balance = 0;
}

public class StudentAccount : BankAccount
{
}
```

// In this example, the BankAccount (superclass) and StudentAccount (subclass) have access to the balance field. Any other class does not.

C# override/virtual Keywords

In C#, a derived class (subclass) can modify the behavior of an inherited method. The method in the derived class must be labeled override and the method in the base class (superclass) must be labeled virtual.

The virtual and override keywords are useful for two reasons:

1. Since the compiler treats “regular” and virtual methods differently, they must be marked as such.
2. This avoids the “hiding” of inherited methods, which helps developers understand the intention of the code.

```
class BaseClass
{
```

```

public virtual void Method1()
{
    Console.WriteLine("Base - Method1");
}
}

class DerivedClass : BaseClass
{
    public override void Method1()
    {
        Console.WriteLine("Derived - Method1"); }
}

```

C# abstract Keyword

In C#, the abstract modifier indicates that the thing being modified has a missing or incomplete implementation. It must be implemented completely by a derived, non-abstract class.

The abstract modifier can be used with classes, methods, properties, indexers, and events. Use the abstract modifier in a class declaration to indicate that a class is intended only to be a base class of other classes, not instantiated on its own.

If at least one member of a class is abstract, the containing class must also be marked abstract.

The complete implementation of an abstract member must be marked with override.

```

abstract class Shape

```

```
{  
    public abstract int GetArea();  
}
```

```
class Square : Shape
```

```
{  
    int side;  
    public Square(int n) => side = n;  
    // GetArea method is required to avoid a compile-time error.  
    public override int GetArea() => side * side;  
}
```

// In this example, GetArea() is an abstract method within the abstract Shape class. It is implemented by the derived class Square.

15. INHERITANCE

15.1 Introduction to Inheritance

Duplicated code leads to errors. Say you have two classes Sedan and Truck. They're different types, but they share a few properties and methods, like SpeedUp() and SlowDown(). If one of those members (say it's SpeedUp()) has to change, then we would have to change the code in every location where SpeedUp() is defined.

In this case it's two classes, but in other programs it may be many more! There are two reasons we don't want to make the same change on code across multiple files:

- It's a waste of time
- More importantly, it is a big risk for making mistakes

In this lesson you'll learn about a solution to this problem: *inheritance*. With inheritance, you can define one superclass that contains the shared members (like SpeedUp() and SlowDown()). All classes that need those members can *inherit* them from the superclass.

15.1 Introduction to Inheritance Instructions

Check out **Sedan.cs** and **Truck.cs**. What code is duplicated across these types?

Sedan.cs

```
using System;
```

```

namespace LearnInheritance
{
    class Sedan : IAutomobile
    {
        public string LicensePlate
        { get; }

        public double Speed
        { get; private set; }

        public int Wheels
        { get; }

        public Sedan(double speed)
        {
            Speed = speed;
            LicensePlate = Tools.GenerateLicensePlate();
            Wheels = 4;
        }

        public void Honk()
        {
            Console.WriteLine("HONK!");
        }
    }
}

```

```

    }

    public void SpeedUp()
    {
        Speed += 5;
    }

    public void SlowDown()
    {
        Speed -= 5;
    }

}
}

```

Truck.cs

```

using System;

namespace LearnInheritance
{
    class Truck : IAutomobile
    {
        public string LicensePlate
        { get; }
    }
}

```



```
public double Speed
```

```
{ get; private set; }
```

```
public int Wheels
```

```
{ get; }
```

```
public double Weight
```

```
{ get; }
```

```
public Truck(double speed, double weight)
```

```
{
```

```
    Speed = speed;
```

```
    LicensePlate = Tools.GenerateLicensePlate();
```

```
    Weight = weight;
```

```
    if (weight < 400)
```

```
    {
```

```
        Wheels = 8;
```

```
    }
```

```
    else
```

```
    {
```

```
        Wheels = 12;
```

```
    }
```

```

    }

    public void Honk()
    {
        Console.WriteLine("HONK!");
    }

    public void SpeedUp()
    {
        Speed += 5;
    }

    public void SlowDown()
    {
        Speed -= 5;
    }

}
}

```

OUTPUT

Sedan with license plate J9K2GYIN and 4 wheels, driving at 60 km/h.

Sedan's faster speed: 65

Sedan with license plate 2GIRDN25 and 4 wheels, driving at 70 km/h.

Sedan's faster speed: 75

Truck with license plate 7S3MPH MV and 12 wheels, driving at 45 km/h.

Truck's faster speed: 50

15.2 Superclass and Subclass

In inheritance, one class inherits the members of another class. The class that inherits is called a *subclass* or *derived class*. The other class is called a *superclass* or *base class*.

In our car example, Sedan and Truck are subclasses (or derived classes). They will inherit members from a new class called Vehicle, which is the superclass (or base class).

Before using inheritance, both classes had:

- Wheels, LicensePlate, and Speed properties
- Honk(), SpeedUp(), and SlowDown() methods
- Similar constructors

We can pull these out of both classes and put it in a Vehicle class. Sedan and Truck will still have access to those members, but we only need to write them in one place.

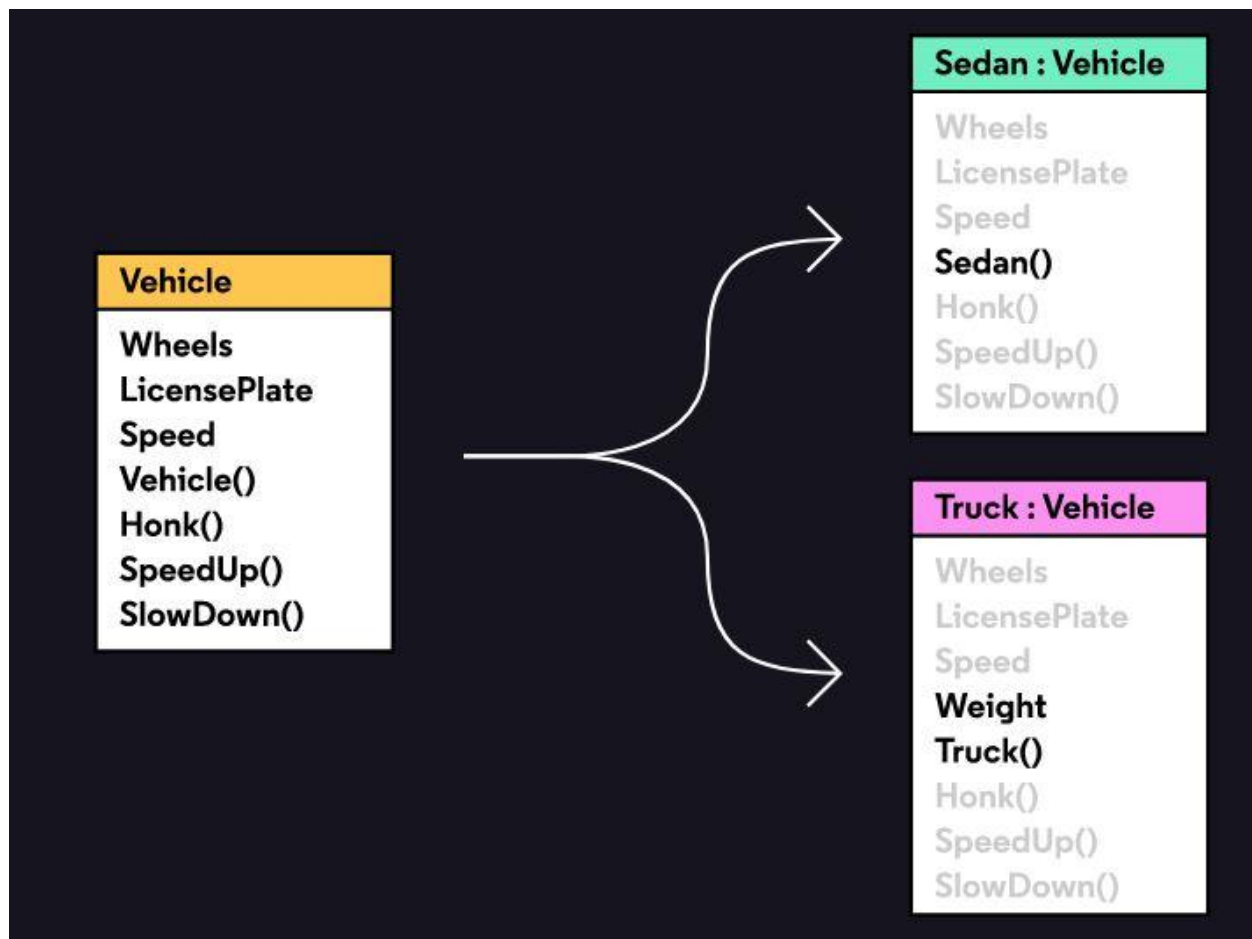
By the way, this inheritance hierarchy can extend either way: a new PickupTruck class could inherit from Truck, which inherits from Vehicle, which inherits from a new Machine class. The only rule is that a class can only inherit from one base class, e.g. Vehicle can't inherit from Machine and Contraption.

15.2 Superclass and Subclass Instructions

Take a look at this diagram representing inheritance.

- Sedan and Truck inherit from Vehicle
- Members in black font are defined in that class
- Members in grey font have been inherited from a superclass

For example, Wheels is defined in the Vehicle class and inherited by Sedan and Truck. Truck() is defined only in the Truck class.



15.3 Create a Superclass

A superclass is defined just like any other class:

```
class Vehicle
{
}
```

And a subclass inherits, or “extends”, a superclass using colon syntax (:):

```
class Sedan : Vehicle
{
}
```

A class can extend a superclass and implement an interface with the same syntax. Separate them with commas and make sure the superclass comes before any interfaces:

```
class Sedan : Vehicle, IAutomobile
{
}
```

The above code means that Sedan will inherit all the functionality of the Vehicle class, and it “promises” to implement all the functionality in the IAutomobile interface.

15.3 Create a Superclass Instructions

1.

In **Vehicle.cs**, build an empty Vehicle class.

2.

In **Vehicle.cs**, define:

- string LicensePlate property (getter only)
- double Speed property (getter and private setter)
- int Wheels property (getter only)
- void Honk() method

- SpeedUp() method
- SlowDown() method

3.

In **Sedan.cs**, use colon syntax to announce that Sedan inherits the Vehicle class.

4.

Sedan now inherits the members you defined in Vehicle. Remove them from **Sedan.cs**. You may still see errors and that's okay! We'll fix those in the next exercise.

Vehicle.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
}
```

Sedan.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
    class Sedan : IAutomobile
```

```
{
```

```
        public string LicensePlate
    { get; }

    public double Speed
    { get; private set; }

    public int Wheels
    { get; }

    public Sedan(double speed)
    {
        Speed = speed;
        LicensePlate = Tools.GenerateLicensePlate();
        Wheels = 4;
    }

    public void Honk()
    {
        Console.WriteLine("HONK!");
    }

    public void SpeedUp()
    {
        Speed += 5;
    }
}
```

```
}

public void SlowDown()
{
    Speed -= 5;
}

}

}
```

Program.cs

```
using System;

namespace LearnInheritance
{
    class Program
    {
        static void Main(string[] args)
        {

        }

    }
}
```


15.4 Access Inherited Members with Protected

While working on Vehicle and Sedan, you may have seen this error:

Sedan.cs(11,13): error CS0200: Property or indexer 'Vehicle.Wheels' cannot be assigned to -- it is read only

Remember public and private? A public member can be accessed by any code outside of the enclosing class. A private member can only be accessed by code within the same class.

The above error comes up because either:

1. There is no setter for Vehicle.Wheels, or
2. The setter is private

How do we fix this problem? Making the setter public is not secure. Making it private is too restrictive – we only want the subclass Sedan to access the property. C# has another access modifier to solved that: protected!

A *protected* member can be accessed by the current class and any class that inherits from it. In this case, if the setter for Vehicle.Wheels is protected, then any Vehicle, Truck, and Sedan instance can call it.

15.4 Access Inherited Members with Protected Instructions

1.

In Vehicle, add a protected setter to each of these properties:

- LicensePlate
- Speed
- Wheels

Vehicle.cs

using System;

namespace LearnInheritance

{

class Vehicle

{

public string LicensePlate

{ get; }

public double Speed

{ get; private set; }

public int Wheels

{ get; }

public void SpeedUp()

{

Speed += 5;

}

public void SlowDown()

{

Speed -= 5;

```

    }

    public void Honk()
    {
        Console.WriteLine("HONK!");
    }

}
}

```

Sedan.cs

```

using System;

namespace LearnInheritance
{
    class Sedan : Vehicle, IAutomobile
    {
        public Sedan(double speed)
        {
            Speed = speed;
            LicensePlate = Tools.GenerateLicensePlate();
            Wheels = 4;
        }
    }
}

```

```
}  
}
```

OUTPUT

Sedan with license plate 79286GH0 and 4 wheels, driving at 60 km/h.

Sedan's faster speed: 65

Sedan with license plate 1F8ODHEZ and 4 wheels, driving at 70 km/h.

Sedan's faster speed: 75

Truck with license plate LRJ6OGJU and 12 wheels, driving at 45 km/h.

Truck's faster speed: 50

15.5 Remove Duplicate Code

At the start of this lesson we had duplicate code in Sedan and Truck. We know that duplicated code leads to errors, so we created a superclass Vehicle to contain that code.

But one version of the duplicated code lives on in Truck! Once we have Truck inherit from Vehicle we can remove that code as well. At that point, we'll have three classes that have Speed, LicensePlate, SlowDown(), etc. but we'll have it written in only one place.

15.5 Remove Duplicate Code Instructions

1.

Make Truck inherit from Vehicle:

- Use colon syntax to announce that Truck inherits the Vehicle class.
- Remove the duplicated properties and methods from Truck.

Truck.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
    class Truck : IAutomobile
```

```
    {
```

```
        public string LicensePlate
```

```
        { get; }
```

```
        public double Speed
```

```
        { get; private set; }
```

```
        public int Wheels
```

```
        { get; }
```

```
        public double Weight
```

```
        { get; }
```

```
        public Truck(double speed, double weight)
```

```
        {
```

```
            Speed = speed;
```

```
            LicensePlate = Tools.GenerateLicensePlate();
```

```
Weight = weight;
```

```
if (weight < 400)
```

```
{
```

```
    Wheels = 8;
```

```
}
```

```
else
```

```
{
```

```
    Wheels = 12;
```

```
}
```

```
}
```

```
public void Honk()
```

```
{
```

```
    Console.WriteLine("HONK!");
```

```
}
```

```
public void SpeedUp()
```

```
{
```

```
    Speed += 5;
```

```
}
```

```
public void SlowDown()
```

```
{
```

```
    Speed -= 5;
}

}

}
```

15.6 Access Inherited Members with Base

To construct a Sedan, we must first construct an instance of its superclass Vehicle.

We can refer to a superclass inside a subclass with the base keyword.

For example, in Sedan:

```
base.SpeedUp();
```

refers to the SpeedUp() method in Vehicle.

There's special syntax for calling the superclass constructor:

```
class Sedan : Vehicle
{
    public Sedan (double speed) : base(speed)
    {
    }
}
```

The above code shows a Sedan that inherits from Vehicle. The Sedan constructor calls the Vehicle constructor with one argument, speed. This works as long as Vehicle has a constructor with one argument of type double.

Even if we don't use base() in Sedan, it will call a Vehicle constructor. Without an explicit call to base(), any subclass constructor will implicitly call the default parameterless constructor for its superclass. For example, this code implicitly calls Vehicle():

```

class Sedan : Vehicle
{
    // Implicitly calls base(), aka Vehicle()
    public Sedan (double speed)
    {
    }
}

```

The above code is equivalent to this:

```

{
    public Sedan (double speed) : base()
    {
    }
}

```

This code will ONLY work if the constructor Vehicle() exists. If it doesn't, then an error will be thrown.

15.6 Access Inherited Members with Base Instructions

1.

Currently, the Sedan constructor implicitly calls the Vehicle default parameterless constructor, also known as Vehicle().

Let's explicitly define a constructor in Vehicle. It should look similar to the Sedan constructor, with a few differences:

- It has one parameter, double speed
- Within the constructor, it sets Speed and LicensePlate

After doing this you may see the error below, which is good! It proves to us that the Sedan constructor is calling the parameterless constructor Vehicle(). Now that we have explicitly defined a constructor Vehicle(double speed), there is no more Vehicle(), hence the error.

error CS7036: There is no argument given that corresponds to the required formal parameter 'speed' of 'Vehicle.Vehicle(double)'

2.

Back in **Sedan.cs**:

- Delete the lines in the constructor that set Speed and LicensePlate
- Call the superclass constructor using : base(speed).

3.

Repeat the process in **Truck.cs**:

- Delete the lines in the constructor that set Speed and LicensePlate
- Call the superclass constructor using : base(speed)

4.

Since the LicensePlate and Speed properties defined in Vehicle are no longer accessed in Sedan or Truck, they no longer need to be protected. Switch those two setters to private.

Vehicle.cs

using System;

namespace LearnInheritance

{

class Vehicle

{

public string LicensePlate

```
{ get; protected set; }
```

```
public double Speed  
{ get; protected set; }
```

```
public int Wheels  
{ get; protected set; }
```

```
public void SpeedUp()  
{  
    Speed += 5;  
}
```

```
public void SlowDown()  
{  
    Speed -= 5;  
}
```

```
public void Honk()  
{  
    Console.WriteLine("HONK!");  
}  
  
}
```

```
}
```

Sedan.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
    class Sedan : Vehicle, IAutomobile
```

```
    {
```

```
        public Sedan(double speed)
```

```
        {
```

```
            Speed = speed;
```

```
            LicensePlate = Tools.GenerateLicensePlate();
```

```
            Wheels = 4;
```

```
        }
```

```
    }
```

```
}
```

Truck.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
class Truck : Vehicle, IAutomobile
{
    public double Weight
    { get; }

    public Truck(double speed, double weight)
    {
        Speed = speed;
        LicensePlate = Tools.GenerateLicensePlate();
        Weight = weight;

        if (weight < 400)
        {
            Wheels = 8;
        }
        else
        {
            Wheels = 12;
        }
    }
}
```

OUTPUT

Sedan with license plate B27Y9J1C and 4 wheels, driving at 60 km/h.

Sedan's faster speed: 65

Sedan with license plate UA9Y3KG9 and 4 wheels, driving at 70 km/h.

Sedan's faster speed: 75

Truck with license plate QR7PPC6U and 12 wheels, driving at 45 km/h.

Truck's faster speed: 50

15.7 Override Inherited Members

Say that we wanted to make one more vehicle that operates a bit differently than a sedan or truck. We want to use most of the members in `Vehicle`, but we need to write new versions of `SpeedUp()` and `SlowDown()`.

What we want is to *override* an inherited method. To do that, we use the override and virtual modifiers.

In the superclass, we mark the method in question as virtual, which tells the computer “this member might be overridden in subclasses”:

```
public virtual void SpeedUp()
```

In the subclass, we mark the method as override, which tells the computer “I know this member is defined in the superclass, but I’d like to override it with this method”:

```
public override void SpeedUp()
```

As an aside: there’s another way to solve this problem. Instead of using virtual and override to override a member, [we can define a new member with the same name](#). Essentially, the inherited member still exists, but it is “hidden” by the member in the subclass. If this sounds confusing, that’s okay! We also think it leads to unnecessary confusion, and that leads to errors. We’re going to stick with the virtual - override approach in this lesson.

15.7 Override Inherited Members Instructions

1.

Our new Bicycle class will access the Wheels and Speed properties in Vehicle, so make both setters protected again in **Vehicle.cs**.

Wheels should already be protected — we set that a few exercises ago.

2.

In **Bicycle.cs**, create an empty Bicycle class that inherits Vehicle.

3.

Define a constructor that:

- has one double parameter for setting the Speed property
- calls base() with that parameter
- in its body, sets Wheels to 2

4.

Define a public void SpeedUp() method that limits the Speed to 15. In other words, in the method body:

- Add 5 to Speed
- If Speed is greater than 15, set it to 15

An error might occur here, which is okay.

5.

You probably saw the warning:

warning CS0108: 'Bicycle.SpeedUp()' hides inherited member 'Vehicle.SpeedUp()'. Use the new keyword if hiding was intended.

The computer suggests using the new approach, but we prefer override for its clarity.

In **Bicycle.cs**, label SpeedUp() with override.

6.

Now you probably saw the error:

error CS0506: 'Bicycle.SpeedUp()': cannot override inherited member 'Vehicle.SpeedUp()' because it is not marked virtual, abstract, or override

In **Vehicle.cs**, label SpeedUp() with virtual.

7.

Repeat the process with SlowDown() in **Bicycle.cs** (let's assume that only sedans and trucks can go in reverse). It should override the inherited version and limit the Speed to 0. In other words, the method:

- Subtracts 5 from Speed
- Sets it to 0 if Speed is less than 0
- Is labeled override

8.

In **Vehicle.cs**, label SlowDown() with virtual.

Vehicle.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
    class Vehicle
```

```
    {
```

```
public string LicensePlate
```

```
{ get; private set; }
```

```
public double Speed
```

```
{ get; private set; }
```

```
public int Wheels
```

```
{ get; protected set; }
```

```
public Vehicle(double speed)
```

```
{
```

```
    Speed = speed;
```

```
    LicensePlate = Tools.GenerateLicensePlate();
```

```
}
```

```
public void SpeedUp()
```

```
{
```

```
    Speed += 5;
```

```
}
```

```
public void SlowDown()
```

```
{
```

```
    Speed -= 5;
```

```
}
```



```
public void Honk()
{
    Console.WriteLine("HONK!");
}

}

}
```

Bicycle.cs

```
using System;
```

```
namespace LearnInheritance
{

}
```

OUTPUT

Sedan with license plate K02WJXDR and 4 wheels, driving at 60 km/h.

Sedan's faster speed: 65

Sedan with license plate TRQOMD6J and 4 wheels, driving at 70 km/h.

Sedan's faster speed: 75

Truck with license plate K9DO7U1Y and 12 wheels, driving at 45 km/h.

Truck's faster speed: 50

15.8 Make Inherited Members Abstract

Now we want to add one more method to `Vehicle` called `Describe()`. It will be different for every subclass, so there's no point in defining a default one in `Vehicle`. Regardless, we want to make sure that it is implemented in each subclass.

This might sound similar to an interface. Why not add this method to the `IAutomobile` interface? We want `Describe()` to be available to all vehicles, not just automobiles.

To do this we need one more modifier: `abstract`. This line would go into the `Vehicle` class:

```
public abstract string Describe();
```

This is like the `Vehicle` class telling its subclasses: "If you inherit from me, you must define a `Describe()` method because I won't be giving you any default functionality to inherit." In other words, abstract members have no implementation in the superclass, but they must be implemented in all subclasses.

If one member of a class is abstract, then the class itself can't really exist as an instance. Imagine calling `Vehicle.Describe()`. It doesn't make sense because it doesn't exist! This means that the entire `Vehicle` class must be abstract. Label it with `abstract` as well:

```
abstract class Vehicle
```

If you don't do this, you'll get an error message like this:

```
error CS0513: 'Vehicle.Describe()' is abstract but it is contained in non-abstract class 'Vehicle'
```

Once we write the abstract method and mark the class as abstract, we'll need to actually implement it in each subclass. For example in `Sedan`:

```
public override string Describe()  
{
```

```
    return $"This Sedan is moving on {Wheels} wheels at {Speed} km/h, with license  
    plate {LicensePlate}.";  
}
```

To make it clear that this `Describe()` method in `Sedan` is overriding the `Describe()` method in `Vehicle`, we will need to label it `override`.

15.8 Make Inherited Members Abstract Instructions

1.

Add the abstract method `Describe()` to the `Vehicle` class.

- `Describe()` should be public and return a string
- `Vehicle` will also need to be labeled abstract

You might see an error after this.

2.

You probably saw this error:

error CS0534: 'Bicycle' does not implement inherited abstract member 'Vehicle.Describe()'

Fix this by implementing `Describe()` methods in `Bicycle`, `Sedan`, and `Truck`. Each method should:

- mention the type, e.g. the bicycle version of the method returns a string containing "bicycle"
- mention the license plate, speed, and wheels
- be labeled with `override`

For bicycles, the returned string might look like this:

This bicycle is moving on 2 wheels at 10 km/h, with license plate ABCD1234.

3.

In **Program.cs**, call Describe on each instance and print the result to the console.

Vehicle.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
    class Vehicle
```

```
    {
```

```
        public string LicensePlate
```

```
        { get; private set; }
```

```
        public double Speed
```

```
        { get; protected set; }
```

```
        public int Wheels
```

```
        { get; protected set; }
```

```
        public Vehicle(double speed)
```

```
        {
```

```
            Speed = speed;
```

```
            LicensePlate = Tools.GenerateLicensePlate();
```

```
        }
```

```
public virtual void SpeedUp()
{
    Speed += 5;
}

public virtual void SlowDown()
{
    Speed -= 5;
}

public void Honk()
{
    Console.WriteLine("HONK!");
}

}

}
```

Bicycle.cs

```
using System;
```

```
namespace LearnInheritance
```

```
{
```

```
class Bicycle : Vehicle
{

    public Bicycle(double speed) : base(speed)
    {
        Wheels = 2;
    }

    public override void SpeedUp()
    {
        Speed += 5;

        if (Speed > 15)
        {
            Speed = 15;
        }
    }

    public override void SlowDown()
    {
        Speed -= 5;

        if (Speed < 0)
        {
```

```
        Speed = 0;
    }
}

}
}
```

Sedan.cs

```
using System;

namespace LearnInheritance
{
    class Sedan : Vehicle, IAutomobile
    {
        public Sedan(double speed) : base(speed)
        {
            Wheels = 4;
        }

    }
}
```

Truck.cs

```
using System;
```

```

namespace LearnInheritance
{
    class Truck : Vehicle, IAutomobile
    {
        public double Weight
        { get; }

        public Truck(double speed, double weight) : base(speed)
        {
            Weight = weight;

            if (weight < 400)
            {
                Wheels = 8;
            }
            else
            {
                Wheels = 12;
            }
        }
    }
}

```


Program.cs

using System;

namespace LearnInheritance

{

class Program

{

static void Main(string[] args)

{

Sedan s = new Sedan(60);

Truck t = new Truck(45, 500);

Bicycle b = new Bicycle(10);

}

}

}

OUTPUT

This Sedan is moving on 4 wheels at 60 km/h, with license plate 3TJO26AN.

This Truck is moving on 12 wheels at 45 km/h, with license plate L8D3FIH4.

This Bicycle is moving on 2 wheels at 10 km/h, with license plate 2V8467CQ.

15.9 Review

Well done! You learned a lot very quickly, so let's do a review:

- *Inheritance* is a way to avoid duplication across multiple classes.
- In inheritance, one class inherits the members of another class.
- The class that inherits is called a *subclass* or *derived class*. The other class is called a *superclass* or *base class*.
- We can access a superclass' members using `base`. This is very useful when calling the superclass' constructor.
- We can restrict access to a superclass and its subclasses using `protected`.
- We can override a superclass member using `virtual` and `override`.
- We can make a member in a superclass without defining its implementation using `abstract`. This is useful if every subclass' implementation will be different.

15.9 Review Instructions

The completed code is provided for you here.

Make sure you are comfortable with inheritance before you move on from this lesson. Here are a few questions to test yourself:

- In **Program.cs**, `Bicycle.Describe()` is called. Find the definition for that method in the `Bicycle` class, then find the abstract definition of that method in `Vehicle`.
- In **Program.cs**, a `Sedan` is instantiated. Find the constructor definition in the `Sedan` class. What happens when that constructor calls `base()`?
- In **Bicycle.cs**, `SpeedUp()` is defined. How is it different from `SpeedUp()` in the `Vehicle` class?

Program.cs

using System;

```

namespace LearnInheritance
{
    class Program
    {
        static void Main(string[] args)
        {
            Sedan s = new Sedan(60);
            // Call SpeedUp() here
            Console.WriteLine(s.Describe());

            Truck t = new Truck(45, 500);
            // Call SpeedUp() here
            Console.WriteLine(t.Describe());

            Bicycle b = new Bicycle(10);
            // Call SpeedUp() here
            Console.WriteLine(b.Describe());
        }
    }
}

```

Vehicle.cs

```

using System;

```

```
namespace LearnInheritance
{
    abstract class Vehicle
    {
        public string LicensePlate
        { get; private set; }

        public double Speed
        { get; protected set; }

        public int Wheels
        { get; protected set; }

        public Vehicle(double speed)
        {
            Speed = speed;
            LicensePlate = Tools.GenerateLicensePlate();
        }

        public virtual void SpeedUp()
        {
            Speed += 5;
        }
    }
}
```

```
public virtual void SlowDown()
{
    Speed -= 5;
}
```

```
public void Honk()
{
    Console.WriteLine("HONK!");
}
```

```
public abstract string Describe();

}

}
```

Sedan.cs

```
using System;
```

```
namespace LearnInheritance
{
    class Sedan : Vehicle, IAutomobile
    {
        public Sedan(double speed) : base(speed)
```

```

{
    Wheels = 4;
}

public override string Describe()
{
    return $"This Sedan is moving on {Wheels} wheels at {Speed} km/h, with
license plate {LicensePlate}.";
}

}
}

```

Bicycle.cs

```

using System;

namespace LearnInheritance
{
    class Bicycle : Vehicle
    {

        public Bicycle(double speed) : base(speed)
        {
            Wheels = 2;
        }
    }
}

```

```
public override void SpeedUp()
{
    Speed += 5;

    if (Speed > 15)
    {
        Speed = 15;
    }
}

public override void SlowDown()
{
    Speed -= 5;

    if (Speed < 0)
    {
        Speed = 0;
    }
}

public override string Describe()
{
    return $"This Bicycle is moving on {Wheels} wheels at {Speed} km/h, with
license plate {LicensePlate}.";
```

}

}

}

15. Quiz

1. Fill in the code so that the **WorkerBee** class inherits from the **Bee** class.

```
class ____ ____ ____  
{
```

- WorkerBee
- :
- Bee

Click or drag and drop to fill in the blank

2. Fill in the code so that Calculator extends the Computer class and implements the IClickable interface.

```
class ____ ____ ____ , ____
```

- :
- IClickable
- Computer
- Calculator

Click or drag and drop to fill in the blank

3. What is *inheritance*?

- a) The process by which one class inherits the members of another class
- b) The process of writing duplicate code in multiple classes
- c) A definition for a group of related functionalities that a class can implement
- d) A type of class

4. Pick the true statement.

- a) A derived class inherits from a base class.
- b) A base class inherits from a derived class.
- c) A superclass inherits from a subclass.
- d) A base class inherits from a superclass.

5. Fill in the code so that WorkerBee implements the IFlyable interface.

class _____

- :
- IFlyable
- WorkerBee

Click or drag and drop to fill in the blank

6. Given that IMachine is an interface, what does the first line mean?

```
class Tractor : IMachine
{}
```

- a) The Tractor class implements the IMachine interface.
- b) The IMachine interface implements the Tractor class.
- c) There is one class named Tractor : IMachine.

d) Calling Tractor() will call IMachine().

7. Fill in the code so that Giganotosaurus.Warn() overrides the Dinosaur.Warn() method.

```
class Dinosaur
{
    public _____ void Warn()
    {
        Console.WriteLine("Wow, a dino!");
    }
}

class Giganotosaurus : Dinosaur
{
    public _____ void Warn()
    {
        Console.WriteLine("Wow, a GIANT dino!");
    }
}
```

- override
- virtual
- abstract

Click or drag and drop to fill in the blank

8. Why will this code cause an error?

```
interface IPersonable
{
    DateTime BirthDate { get; }
    string FullName { get; }
```

```
}
```

```
class Passport : IPersonable  
{
```

- a) The Passport class does not correctly implement the interface.
- b) The interface is missing member implementations.
- c) IPersonable is not a legal interface name.
- d) Interfaces cannot contain properties.

9. What does it mean for a class member to be abstract?

- a) The member has a missing or incomplete implementation. It must be implemented by derived classes.
- b) The member cannot be accessed by derived classes.
- c) The member can be accessed by any class.
- d) The member overrides an inherited member.

10. This code will throw errors. What change will fix those errors?

```
class Building  
{  
    private int x;  
    private int y;  
}
```

```
class Garage : Building  
{  
    public string ListValues()  
    {  
        return $"x: {this.x}, y: {this.y}";  
    }  
}
```

- a) Change both private fields to abstract.
- b) Change both private fields to virtual.
- c) Change both private fields to override.
- d) Change both private fields to protected.

WEB ARCHITECTURE

16.WHAT IS THE BACK-END?

16.1 Front and Back

In this lesson, we'll explain what makes up the back-end of a web application or website. The back-end can feel very abstract, but it becomes clearer when we explain it in terms of the front-end! To oversimplify a bit, the front-end is the parts of a webpage that a visitor can interact with and see.

Various tools and frameworks can be used to make a webpage, but, at its core, the front-end is composed of JavaScript, CSS, HTML, and other *static assets*, such as images or videos. Static assets are files that don't change. When a visitor navigates to a webpage, these assets are sent to their browser.

Visiting a simple website is like ordering delivery from a restaurant: we place an order for our meal, and, once it's delivered to us, we have it entirely in our possession. In this analogy, we can think of the front-end as everything that's dropped off with the delivery: the containers, the utensils, and the food itself.

You'll sometimes hear front-end development referred to as *client-side* development. Our instinct might be to think of the client as the human visitor or user of a website, but when referring to the client in web development, we're usually referring to the non-human requester of content. In the case of visiting a website, the client is the browser, but in other circumstances, a client might be another application, a mobile device, or even a "smart" appliance!

While the front-end is the part of the website that makes it to the browser, the back-end consists of all the behind-the-scenes processes and data that make a website function and send resources to clients.

16.1 Front and Back Instructions

Watch the video to get a better understanding of the front-end.

https://content.codecademy.com/courses/server-side-web-dev/FrontEndCut_v1.mp4

16.2 The Web Server

We talked about how the front-end consists of the information sent to a client so that a user can see and interact with a website, but where does the information come from? The answer is a *web server*.

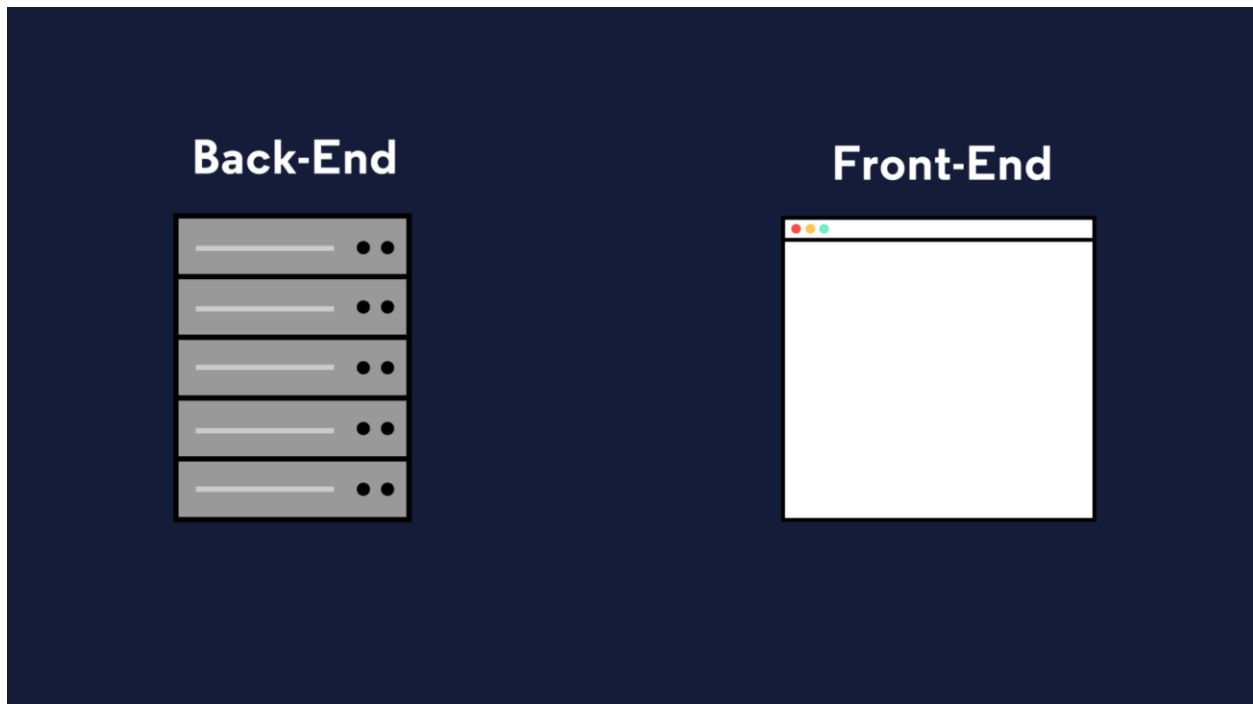
The word “server” can mean a lot of things in computing, but we’re going to focus on web servers specifically. A *web server* is a process running on a computer that listens for incoming requests for information over the internet and sends back responses. Each time a user navigates to a website on their browser, the browser makes a request to the web server of that website. Every website has at least one web server. A large company like Facebook has thousands of powerful computers running web servers in facilities located all around the world which are listening for requests, but we could also run a simple web server from our own computer!

The specific format of a request (and the resulting response) is called the *protocol*. You might be familiar with the protocol used to access websites: HTTP. When a visitor navigates to a website on their browser, similarly to how one places an order for takeout, they make [an HTTP request](#) for the resources that make up that site.

For the simplest websites, a client makes a single request. The web server receives that request and sends the client a response containing everything needed to view the website. This is called a *static website*. This doesn’t mean the website is not interactive. As with the individual static assets, a website is static because once those files are received, they don’t change or move. A static website might be a good choice for a simple personal website with a short bio and

family photos. A user navigating Twitter, however, wants access to new content as it's created, which a static website couldn't provide.

A static website is like ordering takeout, but modern web applications are like dining in person at a sit-down restaurant. A restaurant patron might order drinks, different courses, make substitutions, or ask questions of the waiter. To accomplish this level of complexity, an equally complex back-end is required.

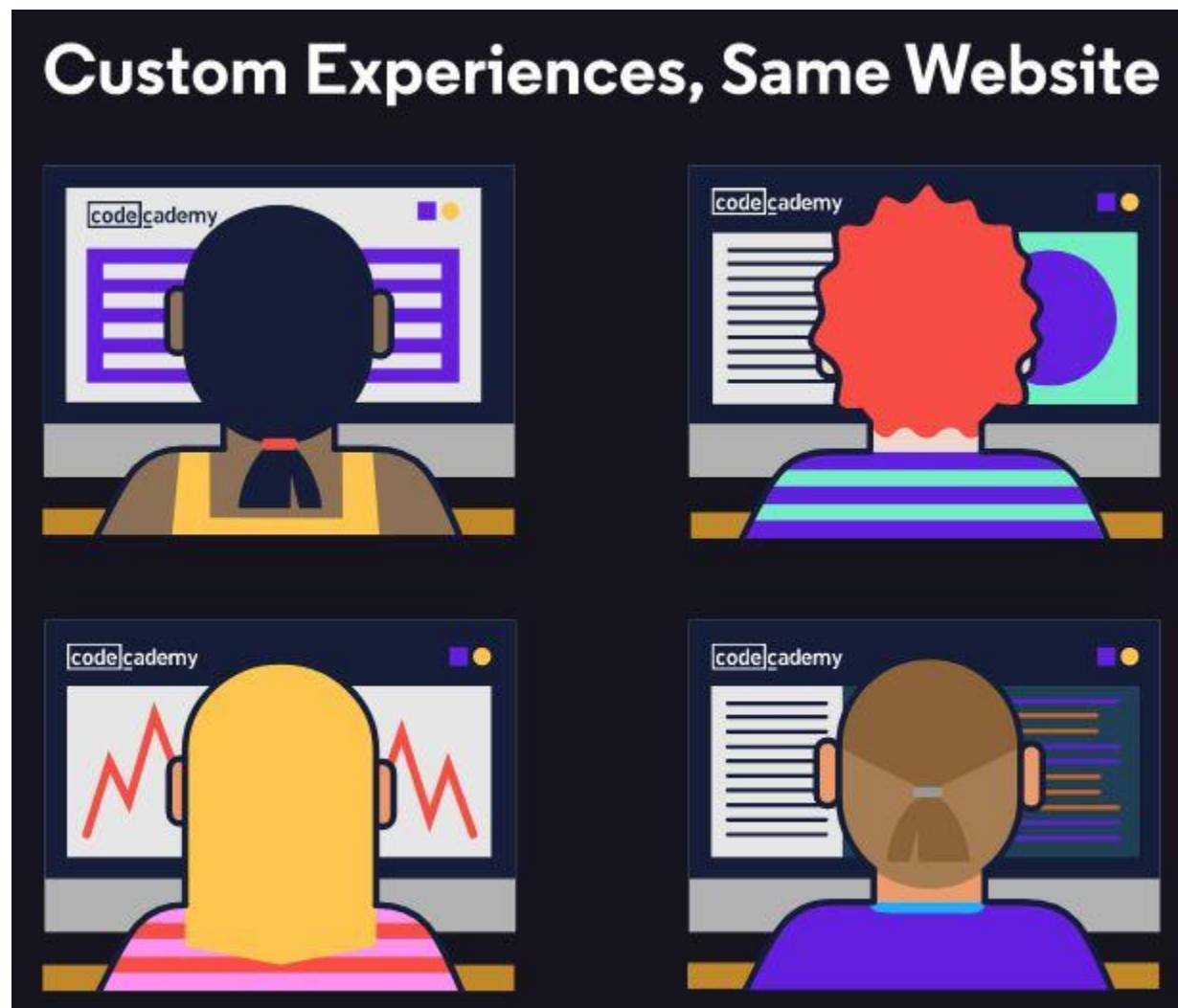


16.3 So What is the Back-end?

When a user navigates to [google.com](https://www.google.com), their request specifies the URL but not the filename for today's [Google Doodle](https://www.google.com/doodles). The web application's back-end will need to hold the logic for deciding which assets to send. Moreover, modern web applications often cater to the specific user rather than sending the same files to every visitor of a webpage. This is known as *dynamic content*.

When we eat at a restaurant, we'll order to our tastes, make substitutions, etc; the result is a dining experience unique to us. Aside from that, there's a lot happening behind the scenes to make a restaurant work: ingredients are ordered from suppliers, new menus are designed, and employees' schedules are created. Similarly, to make a web application that runs smoothly, the back-end is doing a lot more than simply sending assets to browsers.

The collection of programming logic required to deliver dynamic content to a client, manage security, process payments, and myriad other tasks is sometimes known as the “application” or *application server*. The application server can be responsible for anything from sending an email confirmation after a purchase to running the complicated algorithms a search engine uses to give us meaningful results.



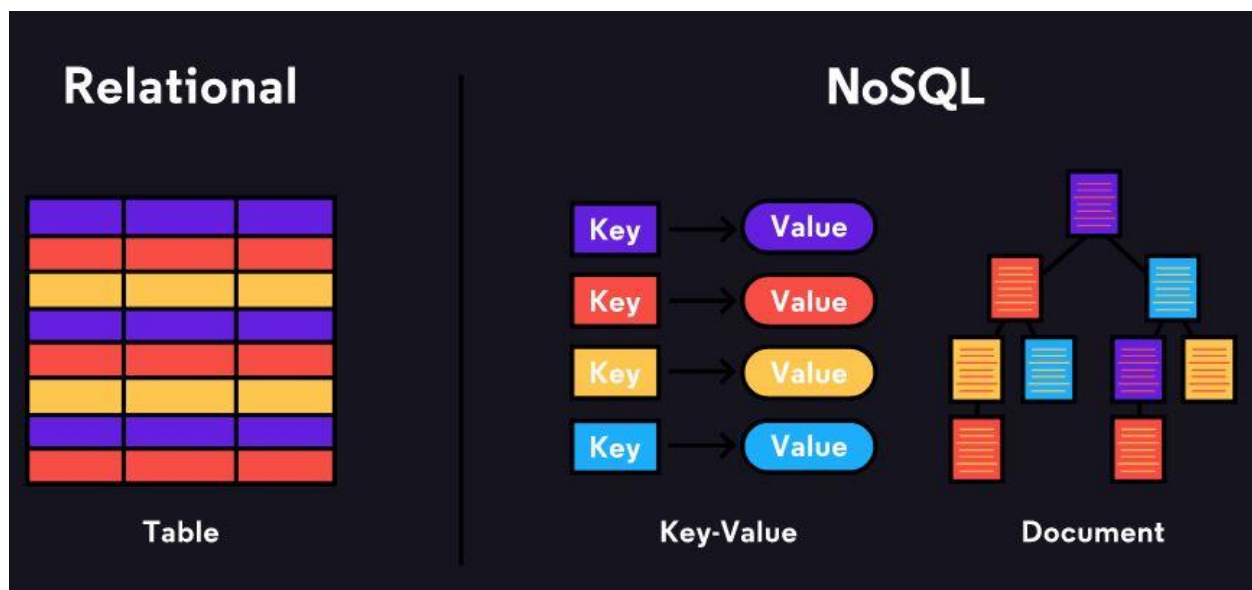
16.4 Storing Data

You've probably heard that data is a big deal. By some measures, 90% of the world's data has been generated in just the past two years! From a stored credit card number on an e-commerce site to the timestamp when you hit pause on Netflix, modern web applications collect a lot of data. For that data to be useful, it has to be organized and stored somewhere.

The back-ends of modern web applications include some sort of *database*, often more than one. Databases are collections of information. There are many different databases, but we can divide them into two types: [relational databases](#) and [non-relational databases \(also known as NoSQL databases\)](#).

Whereas relational databases store information in tables with columns and rows, non-relational databases might use other systems such as key-value pairs or a document storage model. **SQL**, **Structured Query Language**, is a programming language for accessing and changing data stored in relational databases. Popular relational databases include [MySQL](#) and [PostgreSQL](#) while popular NoSQL databases include [MongoDB](#) and [Redis](#).

In addition to the database itself, the back-end needs a way to programmatically access, change, and analyze the data stored there.



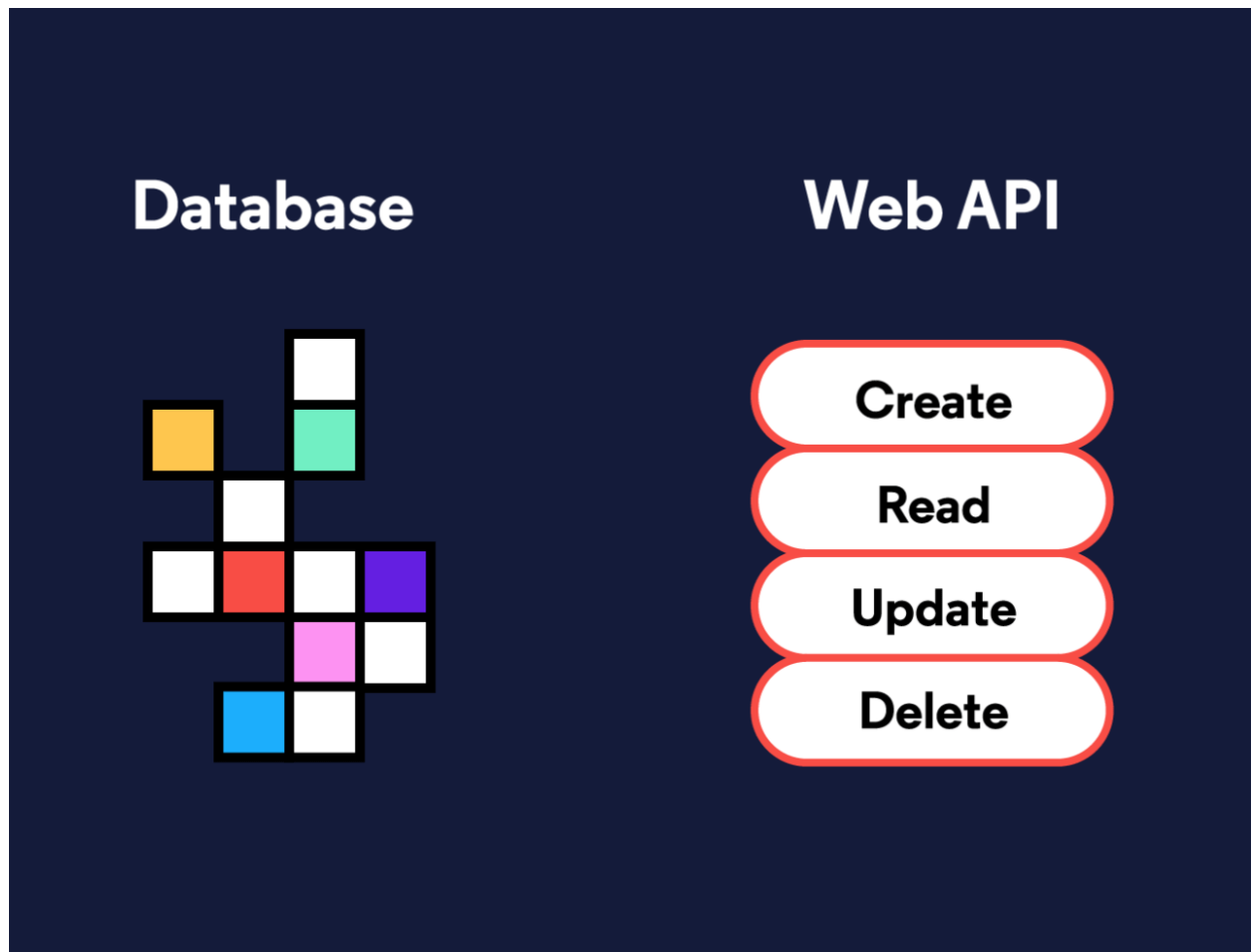
16.5 What is an API?

When a user navigates to a specific item for sale on an e-commerce site, the price listed for that item is stored in a database, and when they purchase it, the database will need to be updated with the correct inventory for that item type. In fact, much of what the back-end entails is reading, updating, or deleting information stored in a database.

In order to have consistent ways of interacting with data, a back-end will often include a *web API*. API stands for **A**pplication **P**rogramming **I**nterface and can mean a lot of different things, but a *web API* is a collection of predefined ways of, or rules for, interacting with a web application's data, often through an HTTP request-response cycle. Unlike the HTTP requests a client makes when a user navigates to a website's URL, this type of request indicates how it would like to interact with a web application's data (create new data, read existing data, update existing data, or delete existing data), and it receives some data back as a response.

Let's walk through the example of making an online purchase again, but this time, we'll imagine how the application's web API might be used. When a user presses the button to submit an order, that will trigger a request to send them to a different view of the website, an order confirmation page, but an additional request will be triggered from the front-end, unseen by the user, to the web API so that the database can be updated with the information from the order.

Some web APIs are open to the public. [Instagram](#), for example, has an API that other developers can use to access some of the data Instagram stores. Others are only used by the web application internally— Codecademy, for example, has a web API that employees use to accomplish internal tasks.



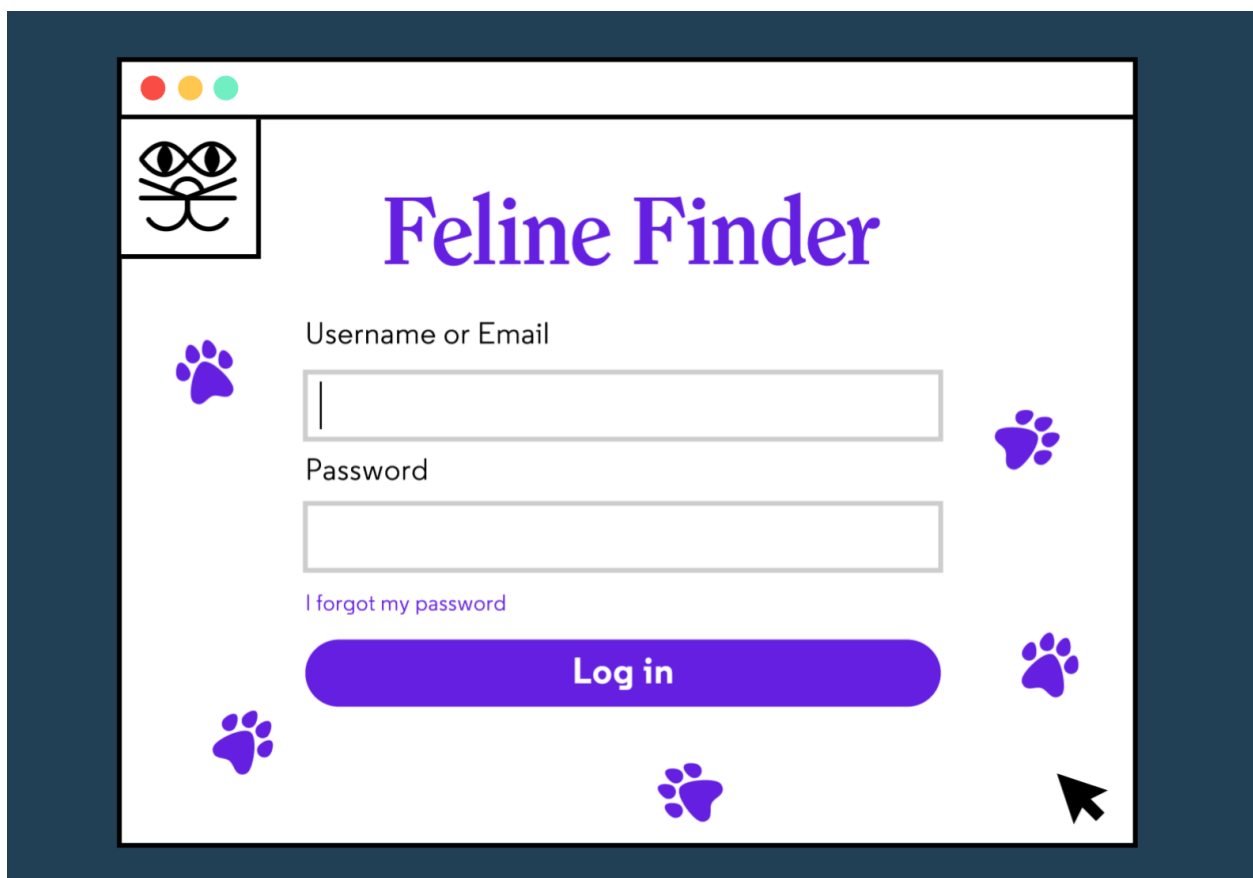
16.6 Authorization and Authentication

Two other concepts we'll want our server-side logic to handle are *authentication* and *authorization*.

Authentication is the process of validating the identity of a user. One technique for authentication is to use logins with usernames and passwords. These credentials need to be securely stored in the back-end on a database and checked upon each visit. Web applications can also use external resources for authentication. You've likely logged into a website or application using your Facebook, Google, or Github credentials; that's also an authentication process.

Authorization controls which users have access to which resources and actions. Certain application views, like the page to edit a social media personal profile, are only accessible to that user. Other activities, like deleting a post, are often similarly restricted.

When building a robust web application back-end, we need to incorporate both authentication (Who is this user? Are they who they claim to be?) and authorization (Who is allowed to do and see what?) into our server-side logic to make sure we're creating secure, personalized, and dynamic content.



16.7 Different Back-end Stacks

Unlike the front-end, which must be built using HTML, CSS, and JavaScript, there's a lot of flexibility in which technologies can be used in order to create the back-end of a web application. Developers can construct back-ends in many different languages like PHP, Java, JavaScript, Python, and more.

You don't need to reinvent the wheel to create a robust back-end. Instead, most developers make use of *frameworks* which are collections of tools that shape the organization of your back-end and provide efficient ways of accomplishing otherwise difficult tasks.

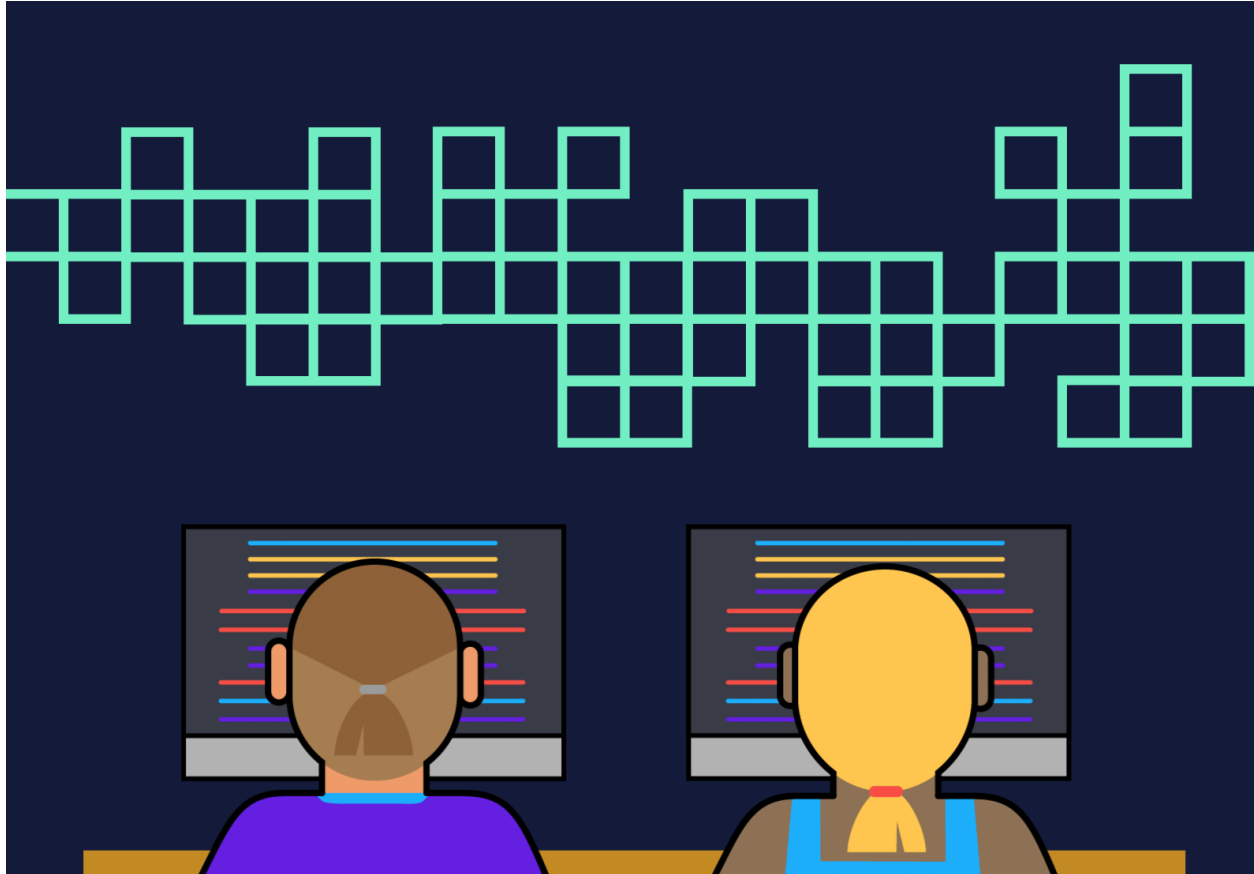
There are numerous [back-end frameworks](#) from which developers can choose. Here are a few examples:

Framework	Language
-----------	----------

Laravel	PHP
Express.js	JavaScript (runs in the Node environment)
Ruby on Rails	Ruby
Spring	Java
JSF	Java
Flask	Python
Django	Python
ASP.NET	C#

The collection of technologies used to create the front-end and back-end of a web application is referred to as a *stack*. This is where the term *full-stack developer* comes from; rather than working in either the front-end or the back-end exclusively, a full-stack developer works in both.

For example, [the MEAN stack](#) is a technology stack for building web applications that uses **M**ongoDB, **E**xpress.js, **A**ngularJS, and **N**ode.js: MongoDB is used as the database, Node.js with Express.js for the rest of the back-end, and Angular is used as a front-end framework. While the [LAMP Stack](#), sometimes considered the archetypal stack, uses **L**inux, **A**pache, **M**ySQL, and **P**HP.



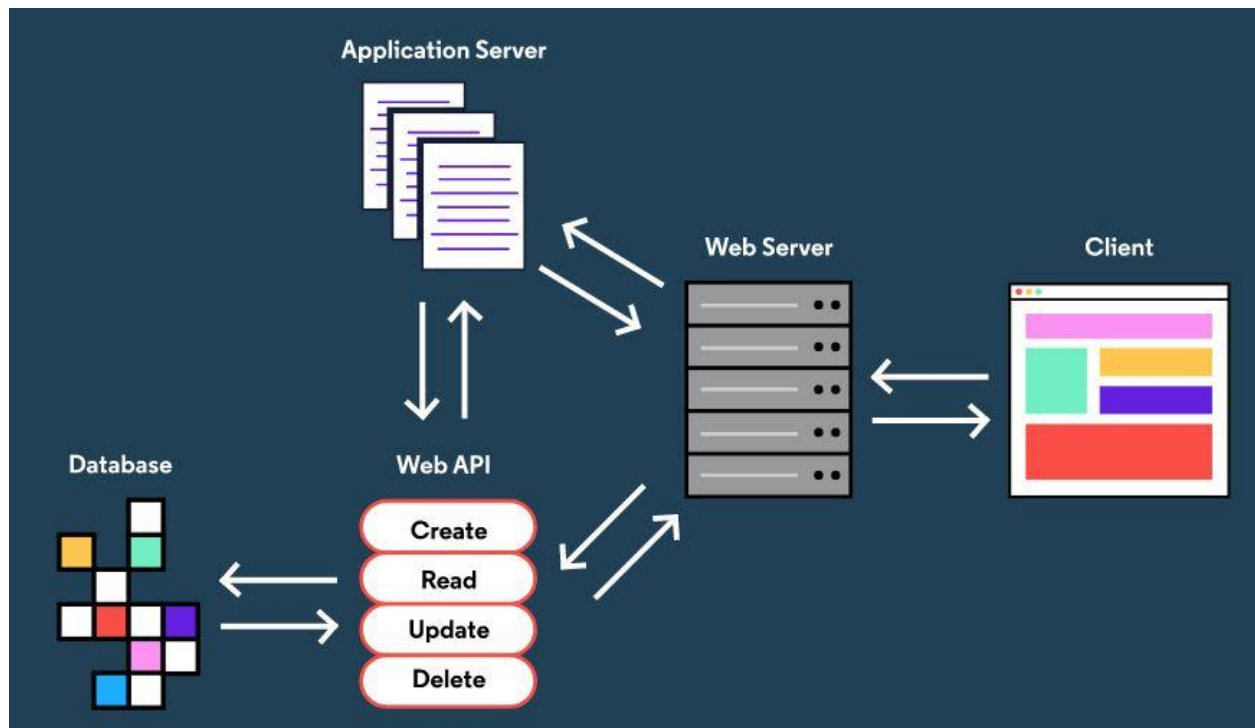
16.8 Review

In order to deliver the front-end of a website or web application to a user, a lot needs to happen behind the scenes on the back-end! Understanding what makes up the back-end can be overwhelming because the back-end has a lot of different

parts, and different websites or web applications can have dramatically different back-ends. We covered a lot in this lesson, so let's review what we learned:

- The front-end of a website or application consists of the HTML, CSS, JavaScript, and static assets sent to a client, like a web browser.
- A web server is a process running on a computer somewhere that listens for incoming requests for information over the internet and sends back responses.
- Storing, accessing, and manipulating data is a large part of a web application's back-end
- Data is stored in databases which can be relational databases or NoSQL databases.
- The server-side of a web application, sometimes called the application server, handles important tasks such as authorization and authentication.
- The back-end of web application often has a web API which is a way of interacting with an application's data through HTTP requests and responses.
- Together the technologies used to build the front-end and back-end of a web application are known as the stack, and many different languages and frameworks can be used to build a robust back-end.

Now that you have a sense for server-side web development and what the back-end is, you're ready to dive in and learn about the different parts in more depth!



4.Article

Back-End Web Architecture

This article provides an overview of servers, databases, routing, and anything else that happens between when a client makes a request and receives a response.

Software engineers seem to always be discussing the front-end and the back-end of their apps. But what exactly does this mean?

The front-end is the code that is executed on the client side. This code (typically HTML, CSS, and JavaScript) runs in the user's browser and creates the user interface.

The back-end is the code that runs on the server, that receives requests from the clients, and contains the logic to send the appropriate data back to the client. The back-end also includes the database, which will persistently store all of the data for the application. This article focuses on the hardware and software on the server-side that make this possible.

Review [HTTP](#) and [REST](#) if you want to refresh your memory on these topics. These are the main conventions that provide structure to the request-response cycle between clients and servers.

Let's start by reviewing the client-server relationship, and then we can start to put the pieces all together!

What are the clients?

The clients are anything that send requests to the back-end. They are often browsers that make requests for the HTML and JavaScript code that they will execute to display websites to the end user. However, there are many different kinds of clients: they might be a mobile application, an application running on another server, or even a web enabled smart appliance.

What is a back-end?

The back-end is all of the technology required to process the incoming request and generate and send the response to the client. This typically includes three major parts:

- The server. This is the computer that receives requests.
- The app. This is the application running on the server that listens for requests, retrieves information from the database, and sends a response.
- The database. Databases are used to organize and persist data.

What is a server?

A server is simply a computer that listens for incoming requests. Though there are machines made and optimized for this particular purpose, any computer that is connected to a network can act as a server. In fact, you will often use your very own computer as server when developing apps.

What are the core functions of the app?

The server runs an app that contains logic about how to respond to various requests based on the [HTTP verb](#) and the [Uniform Resource Identifier \(URI\)](#). The pair of an HTTP verb and a URI is called a *route* and matching them based on a request is called *routing*.

Some of these handler functions will be *middleware*. In this context, middleware is any code that executes between the server receiving a request and sending a response. These middleware functions might modify the request object, query the database, or otherwise process the incoming request. Middleware functions typically end by passing control to the next middleware function, rather than by sending a response.

Eventually, a middleware function will be called that ends the request-response cycle by sending an HTTP response back to the client.

Often, programmers will use a framework like Express or [Ruby on Rails](#) to simplify the logic of routing. For now, just think that each route can have one or many handler functions that are executed whenever a request to that route (HTTP verb and URI) is matched.

What kinds of responses can a server send?

The data that the server sends back can come in different forms. For example, a server might serve up an HTML file, send data as JSON, or it might send back only an [HTTP status code](#). You've probably seen the status code "404 - Not Found" whenever you've tried navigating to a URI that doesn't exist, but there are many more status codes that indicate what happened when the server received the request.

What is a database, and why do we need to use them?

Databases are commonly used on the back-end of web applications. These databases provide an interface to save data in a persistent way to memory. Storing the data in a database both reduces the load on the main memory of the server [CPU](#) and allows the data to be retrieved if the server crashes or loses power.

Many requests sent to the server might require a database query. A client might request information that is stored in the database, or a client might submit data with their request to be added to the database.

What is a Web API, really?

An API is a collection of clearly defined methods of communication between different software components.

More specifically, a *Web API* is the interface created by the back-end: the collection of endpoints and the resources these endpoints expose.

A Web API is defined by the types of requests that it can handle, which is determined by the routes that it defines, and the types of responses that the clients can expect to receive after hitting those routes.

One Web API can be used to provide data for different front-ends. Since a Web API can provide data without really specifying how the data is viewed, multiple different HTML pages or mobile applications can be created to view the data from the Web API.

Other principles of the request-response cycle:

- The server typically cannot initiate responses without requests!
- Every request needs a response, even if it's just a 404 status code indicating that the content was not found. Otherwise your client will be left *hanging* (indefinitely waiting).
- The server should not send more than one response per request. This will throw errors in your code.

Mapping out a request

Let's make all of this a bit more concrete, by following an example of the main steps that happen when a client makes a request to the server.

1. Alice is shopping on SuperCoolShop.com. She clicks on a picture of a cover for her smartphone, and that click event makes a GET request to **`http://www.SuperCoolShop.com/products/66432`**.

Remember, GET describes the kind of request (the client is just asking for data, not changing anything). The URI (uniform resource identifier) **`/products/66432`** specifies that the client is looking for more information about a product, and that product, has an id of 66432.

SuperCoolShop has an huge number of products, and many different categories for filtering through them, so the actual URI would be more complicated than

this. But this is the general principle for how requests and resource identifiers work.

2. Alice's request travels across the internet to one of SuperCoolShop's servers. This is one of the slower steps in the process, because the request cannot go faster than the speed of light, and it might have a long distance to travel. For this reason, major websites with users all over the world will have many different servers, and they will direct users to the server that is closest to them!

3. The server, which is actively listening for requests from all users, receives Alice's request!

4. Event listeners that match this request (the HTTP verb: GET, and the URI: **/products/66432**) are triggered. The code that runs on the server between the request and the response is called *middleware*.

5. In processing the request, the server code makes a database query to get more information about this smartphone case. The database contains all of the other information that Alice wants to know about this smartphone case: the name of the product, the price of the product, a few product reviews, and a string that will provide a path to the image of the product.

6. The database query is executed, and the database sends the requested data back to the server. It's worth noting that database queries are one of the slower steps in this process. Reading and writing from static memory is fairly slow, and the database might be on a different machine than the original server. This query itself might have to go across the internet!

7. The server receives the data that it needs from the database, and it is now ready to construct and send its response back to the client. This response body has all of the information needed by the browser to show Alice more details (price, reviews, size, etc) about the phone case she's interested in. The response header will contain an HTTP status code 200 to indicate that the request has succeeded.

8. The response travels across the internet, back to Alice's computer.

9. Alice's browser receives the response and uses that information to create and render the view that Alice ultimately sees!

Make your first ASP.NET App

5.Article

What is ASP.NET Razor Pages?

Discover how the C# language and the ASP.NET framework are used to build websites and web apps

[According to Microsoft](#), the makers of the software, ASP.NET is: “an open source web framework...for building modern web apps and services with .NET.”

If you’re not familiar with .NET, then start with [this article on the .NET platform](#). In short, .NET is the engine that lets you build programs with C# (and a few other languages).

So, ASP.NET is something that goes on top of .NET to build a specific type of program: web apps and services. “Web apps” and “web services” are involved in nearly everything that you encounter on the internet. Web sites, e-mail, downloading an app on your phone, anything that deals with HTTP requests and responses — those all use some kind of web app or web service. This article focuses on using ASP.NET to build websites, which we’ll also call web apps (there’s a difference, but we won’t get into it now).

Let’s say we want to build a web app of our own. We can choose nearly any programming language, but let’s say we pick C#. If we build a program with C#,

we'll probably use the .NET platform. Since our program is a web app, we'll use ASP.NET!

If you'd like more details, here are some of the tools included in ASP.NET:

- A base framework for processing web requests in C# or F#
- A front-end syntax, known as Razor, for building dynamic web pages using C#
- Libraries for common web patterns, such as [Model View Controller \(MVC\)](#)
- Authentication system that includes libraries, a database, and template pages for handling logins
- Editor extensions to provide syntax highlighting, code completion, and other functionality specifically for developing web pages

Using ASP.NET, you can build web apps in a number of different ways — the same way a “flying machine” can be built as a jet plane or a helicopter, or a “floor-cleaning machine” can be built as a broom or a vacuum. We call these different ways *architectures* or *patterns*. We'll focus on one: the Razor Pages architecture.

The Razor Pages architecture goes like this: Every page of our website is represented by two files:

- A *view page*, that displays information, and
- A *page model*, which handles the accessing and processing of information.

For example, say that you have a bakery website that handles online orders. When users visit the site, they see a page containing a menu and a form to input an order. The way that the menu and form are displayed is defined in the view page. How the menu is accessed and how the order is handled are defined in the corresponding page model.

Razor Pages can also be used to make things other than web pages, like APIs and microservices. (Remember when Microsoft said ASP.NET is good for web apps AND services?) In those cases we wouldn't need a view page, we'd just use page models.

Let's recap:

- To build programs with C#, you need the .NET platform
- To build programs like web apps and web services, you need the ASP.NET framework
- ASP.NET provides a number of ways to build web apps/services. We focused on one called Razor Pages, which uses a page-focused (View Page + Page Model) pattern.

Now you know the tools; it's time to get building!

6.Article

Getting Started with ASP.NET and Visual Studio

Get acquainted with Visual Studio and build your first ASP.NET web app on your own computer

In this video, you'll learn how to build a basic ASP.NET Razor Pages application on your own computer. If you don't already have Visual Studio, follow [this set-up article](#).

VIDEO:

<https://youtu.be/eMpWKC8Q0o8>

Here's a quick recap of the skills covered:

- Create an ASP.NET Core Razor Pages template in Visual Studio
- Explain the purpose of each file and folder in the generated template
- Run an ASP.NET application from the **dotnet** CLI

When you feel comfortable with these, you're ready to move on!

17. RAZOR PAGES SYNTAX I

17.1 Introduction to Razor Pages

Wouldn't it be great if you could get started creating a web application by just running a few commands? Well, that's possible now with Razor Pages!!

When it comes to building web applications with ASP.NET we're usually able to take two approaches: we can take an [MVC \(Model-View-Controller\) approach](#), or more recently, use *Razor Pages*. Whether one approach is better than the other really depends on what you're building and its complexity, but let's look at some reasons why we should use Razor Pages.

Compared to Razor Pages, the MVC pattern uses a more complex folder structure and requires a deeper understanding of more web framework concepts in order to get started.

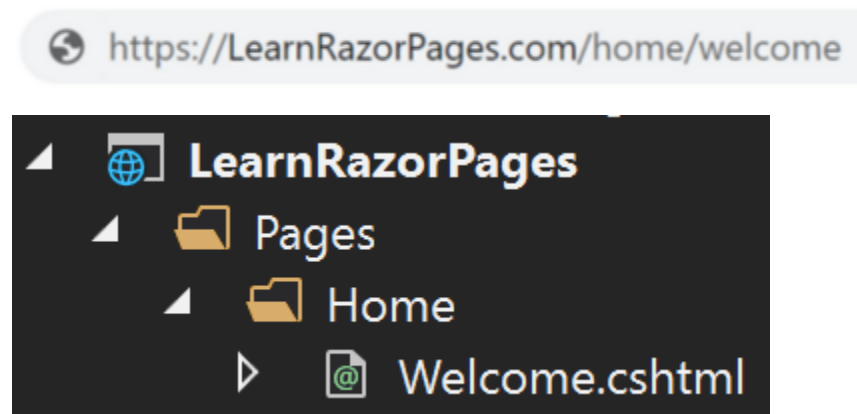
With Razor Pages, we don't have to worry about Controllers, Actions, and Views the same way we would in an MVC app. Now, we use what we call *Pages* and *PageModels*. Everything related to a specific "Page" is in one place and *PageModels* takes care of the processing logic for the said page (We will go more in-depth into these later in the course). This makes applications very cohesive and requires fewer additional concepts to learn. Throughout this course, we will be referring to *Pages* as "view pages" and they will have a *cshtml* extension to their filename (i.e. **Home.cshtml**). *PageModels* will be referred to as a "model page" and will have a *cshtml.cs* extension to their filename (i.e. **Home.cshtml.cs**).

Razor Pages also has a much more organized file structure; instead of having a whole chain of events in different files when working with MVC, we now simply have a Razor View file, where we can write C# code together with HTML, and a code-behind file (the *PageModel*) to handle the logic of what the user interacts with.

What this essentially means is that now, there is a direct link between the URL of a web page and the physical file location of that page on the server.

The URL `https://LearnRazorPages.com/Home/Welcome`

will use the **Welcome.cshtml** file located in the **/Home** folder of the root **/Pages** folder.



In essence, Razor is a view templating engine that's currently supported in ASP.NET. With Razor, we're able to create dynamic web pages and write C# and HTML together using Razor syntax.

17.1 Introduction to Razor Pages Instructions

We've provided you with a demo app so you can explore how the folder structure of a Razor Pages app looks. Click **Run** to launch the application.

- Take a look at **Index.cshtml**. This is what we call a “view page”, can you find what parts of the markup is actually C# code?
- Navigate to **Index.cshtml.cs**. This is a “model page”, can you figure out how it's related to **Index.cshtml**? Given what you know about HTTP requests, when do you think the `OnGet()` method is called?

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="text-center">
```

```
    <h1 class="display-4">@Model.Greet</h1>
```

```
    <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building  
Web apps with ASP.NET Core</a>.</p>
```

```
</div>
```

Index.cshtml.cs

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Threading.Tasks;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
using Microsoft.Extensions.Logging;
```

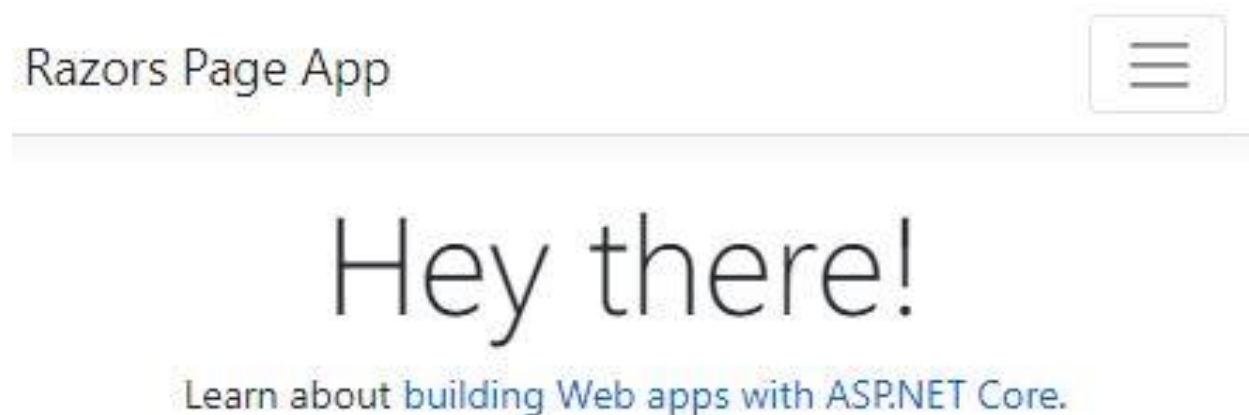
```
namespace demo_app.Pages
```

```
{
```

```
    public class IndexModel : PageModel
```

```
{  
    public string Greet { get; set; }  
  
    public void OnGet()  
    {  
        Greet = "Hey there!";  
    }  
}
```

OUTPUT



17.2 Structure of a Razor View Page

So how does a view page in a Razor app work? In order for a file to behave as a Razor view page, the first line in the file must be `@page`. The `@page` directive indicates that the file is a Razor Page and the ASP.NET compiler will treat it as

such. This means that we can now write C# code in our view page and HTML markup.

When a user navigates to a web page, the web browser submits a “request” to the web server to return the contents of the web page so that it can display it to the user. That request is handled by Razor Pages, and specifically, the Razor engine that looks at the view file (.cshtml) of the specific page the user wishes to see and produces the HTML content based on the code implemented in that view.

In order for the Razor engine to interpret code as C# instead of HTML, we must prepend the keyword with the “@” symbol, otherwise, it will read the code as HTML. We’ll look at a few different ways we can write C# in our view page in the next exercise.

17.2 Structure of a Razor View Page Instructions

1.

Add the @page directive on the first line so our file will be read as a Razor view page.

2.

Add an HTML <h1> header with your name on it to display HTML content.

Index.cshtml

```
<!-- Add the page directive above this line -->
```

```
<div class="text-center">
```

```
    <h2>Hey there! My name is:</h2>
```

<!-- Insert a header with your name below -->

<p>Learn about building
Web apps with ASP.NET Core.</p>

</div>

OUTPUT



17.3 Writing C# in a Razor Page

All C# expressions are preceded with the character “@”. For example:

<h1>@DateTime.Now.ToShortDateString()</h1>

If your C# code needs spaces, then it must be wrapped in parentheses:

<p>Last week this time: @(DateTime.Now - TimeSpan.FromDays(7))</p>

We can use code blocks:

@{ // C# code }

if our code exceeds one line or we want to declare variables.

```
@{  
    int num1 = 6;  
    int num2 = 4;  
    int result = num1 + num2;  
}
```

<h3> The result of @num1 + @num2 is: @result</h3>

Result:

<h3> The result of 6 + 4 is: 10</h3>

17.3 Writing C# in a Razor Page Instructions

1.

Open a code block below @model IndexModel so that we can start writing C# code in them.

2.

Within your code block, start by creating a string variable named firstName and assign it to your first name.

3.

Let's now create an int variable named age and assign it your age.

4.

Now that we have a few assigned variables, let's display their values on the page! Fill out our header describing your name and age.

Index.cshtml

@page

@model IndexModel

<!-- Write your code block below -->

```
<div class="text-center">
```

```
  <h1 class="display-4">Welcome</h1>
```

```
  <br>
```

```
  <h2 class="user-info">My name is <!-- name --> and I am <!-- age --> years  
  old!</h2>
```

```
</div>
```

OUTPUT

Razors Page App



Welcome

**My name is John and I am 25
years old!**

17.4 Conditionals in Razor Pages: If Statements

You can use the if statement in your Razor code pretty much the same way you would in regular C# code — just prefix the keyword with the “@” sign:

```
@{  
    int value = 4;  
}
```

```
@if (value % 2 == 0)  
{  
    <p>The value is even!</p>  
}
```

When writing if/else statements, we don’t need to prepend the else if or else lines with an “@” sign:

```
@if (value % 2 == 0)  
{  
    <p>The value was even.</p>  
}  
else if (value >= 1337)  
{  
    <p>The value is large.</p>  
}  
else  
{  
    <p>The value is odd and small.</p>  
}
```

17.4 Conditionals in Razor Pages: If Statements Instructions

- 1.

Let's imagine we're at a stoplight and we want to display a message depending on the color of the streetlight.

Below the commented line in the code editor, open a code block and create a variable of type string called stopLight. Assign it to the string "green".

2.

Open an if statement and check if the stopLight is green if so, display an <h5> heading, with the message: The stoplight is green, go!.

3.

Write an else if statement, check if the stopLight is red and if so, display an <h5> heading, with the message: The stoplight is red, stop!

Make sure to change the variable stopLight to ensure the case works!

4.

Write another else if statement to check if the stopLight is yellow and if so, display an <h5> heading, with the message: The stoplight is yellow, slow down!

Make sure to change the variable stopLight to ensure the else case works!

5.

For the last else of the series of if/else statements, display an <h5> heading with the message: That stoplight doesn't exist!

Make sure to change the variable stopLight to a string that is not "green", "yellow", or "red" to ensure the else case works!

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<!-- Create variable below -->
```

```
<div class="text-center">
```

```
    <h1 class="display-5">Let's work with conditionals!</h1>
```

```
    <hr>
```

```
    <br>
```

```
    <h4 style="color:blue;">Results from your if statement:</h4>
```

```
    <br>
```

```
    <!-- If Statement below -->
```

```
</div>
```

OUTPUT

Let's work with conditionals!

Results from your if statement:

That stoplight doesn't exist!

17.5 Conditionals in Razor Pages: Switch Statements

Similar to if statements, switch cases operate the same way — by prefixing the keyword with the “@” sign:

```
@{ int number = 2 }
```

```
@switch (number)
{
    case 1: <h1>The value is 1!</h1>
        break;
    case 2: <h1>The value is 2!</h1>
        break;
    default: ...
}
```

In the code above we’re evaluating an expression, number, and simply comparing the values to each case. If a value matches the expression, then the associated code block will be executed.

In this case, since the variable number is equal to 2, then the following HTML would be executed:

```
<h1>The value is 2!</h1>
```

17.5 Conditionals in Razor Pages: Switch Statements

Instructions

1.

You've received some exam results but there's some useful feedback missing.

We'll be evaluating the cases based on the value of your grade.

Open a switch statement and use the variable grade as the expression you'll be comparing to each case.

2.

For the first case, we want to check whether the grade is equal to A. If so, display the following message in an <h4> heading:

Excellent job!

3.

Continue switch cases and display the following feedback <h4> headings depending on the grade:

Grade	Feedback
"A"	"Excellent job!"
"B"	"Well done!"
"C"	"Needs some work!"
"D"	"You passed"

Grade	Feedback
"F"	"You failed, better try next time"
default case	"Invalid grade!"

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
    string grade = "A";
```

```
}
```

```
<div class="text-center">
```

```
    <h1 class="display-5">Let's work with conditionals!</h1>
```

```
    <hr>
```

```
    <h3>Exam results:</h3>
```

```
    <br>
```

```
    <!-- Switch statement below -->
```

```
    <h4>Your grade is, @grade</h4>
```

</div>

OUTPUT

**Let's work with
conditionals!**

Exam results:

**You failed, better try next time
Your grade is, F**

17.6 Working with Loops

Let's not forget about loops!

There are a number of different types of loops in C# that can be written in Razor syntax. Let's say we have a list of names we'll be looping over:

```
@{  
    List<string> names = new List<string>()  
    {  
        "Scott Allen",  
        "James Dorf",  
        "Tim Alston",  
        "Jane Rashid",  
        "John Doe"  
    };  
}
```


We can iterate over the names with a for loop, which is useful when we need to keep track of how far in the looping process we are:

```
<ul>
  @for (int i = 0; i < names.Count; i++)
  {
    <li>@names[i]</li>
  }
</ul>
```

Resulting HTML from the above example:

```
<ul>
  <li>Scott Allen</li>
  <li>James Dorf</li>
  <li>Tim Alston</li>
  <li>Jane Rashid</li>
  <li>John Doe</li>
</ul>
```

Similarly, we can use a foreach loop:

```
<ul>
  @foreach(string name in names)
  {
    <li>@name</li>
  }
</ul>
```

Resulting HTML from the above example:

```
<ul>
  <li>Scott Allen</li>
  <li>James Dorf</li>
  <li>Tim Alston</li>
  <li>Jane Rashid</li>
  <li>John Doe</li>
</ul>
```

Lastly, we can iterate over the names with a while loop:

```
<ul>
  @{
    int counter = 0;
  }
  @while(counter < names.Count)
  {
    <li>@names[counter++]</li>
  }
</ul>
```

Resulting HTML from the above example:

```
<ul>
  <li>Scott Allen</li>
  <li>James Dorf</li>
  <li>Tim Alston</li>
  <li>Jane Rashid</li>
  <li>John Doe</li>
</ul>
```

17.6 Working with Loops Instructions

1.

Create a list of 4 of your favorite foods and assign them to a variable called favoriteFoods.

2.

You're going to iterate through the favoriteFoods list and display each food item in an unordered list. For each item in the loop, you'll be adding an unordered list tag.

Open some tags for an unordered list under the <h4> heading with the text: "Results from your for loop".

3.

Inside the unordered list, you'll be looping over 4 of your favorite foods and display them as list items. Start by creating a for loop with the following conditions:

1. An initial variable, `i`, with a count starting at 0
2. A terminating condition that ends when we reach the count of our `favoriteFoods` length.
3. An iteration statement that increases our initial value by 1.

After the for loop expression, open some curly brackets where you'll write the rest of your C# code.

4.

Within your code blocks, open a list item tag. Inside it, access each element in the `favoriteFoods` list using the initial variable, `i`.

5.

Now you have your `favoriteFoods` displayed! Let's rewrite this loop in a different way. Instead of creating a for loop, open a pair of `<ul>` tags and use a foreach loop and call each item on the list.

6.

Awesome! Now, try using a while loop. Start out by creating a variable called `counter` and assign it to 0. You'll iterate over your favorite foods while the counter is less than the length of `favoriteFoods`.

Index.cshtml

@page

@model IndexModel

<!-- Create favoriteFoods below -->

<div class="text-center">

<h1 class="display-5">Let's work with loops!</h1>

<h4>Results from your for loop:</h4>

<h4>Results from your foreach loop:</h4>

<h4>Results from your while loop:</h4>

</div>

OUTPUT

Let's work with loops!

Results from your for loop:

- Cake
- Lasagna
- Pizza
- Bananas

Results from your foreach loop:

- Cake
- Lasagna
- Pizza
- Bananas

Results from your while loop:

- Cake
- Lasagna
- Pizza
- Bananas

17.7 Review

Great job! We learned a lot in this lesson. We were able to re-create certain HTML markup using Razor syntax and saw how Razor Pages can be useful by reducing repetitive code.

Let's summarize what we've learned so far:

- Razor is a markup syntax for embedding server-based code into webpages. It consists of Razor markup, C#, and HTML. Files containing Razor generally have a .cshtml file extension.
- In order to use the Razor Pages project type in ASP.NET, each Razor page (file ending in .cshtml) must start with the @page directive.
- In ASP.NET Razor syntax, C# code is denoted with an “at” symbol: @.
 - If the code is a single expression, no spaces, no additional syntax is needed. The expression immediately follows the @.
 - If the code is a single line with spaces, it should be surrounded by parentheses, ().
 - If the code is multiple lines or contains a mix of C# and HTML, it should be surrounded by curly braces, { }
- In ASP.NET Razor syntax, an if control structure is written with a code block, where the first line contains @if and a set of conditions. Any following else-if and else conditions do not need @ symbols.
- In ASP.NET Razor syntax, a @for control structure is written with a code block, where the first line contains @for and a set of conditions.
- In ASP.NET Razor syntax, a @foreach control structure is written with a code block, where the first line contains @foreach and a set of conditions.
- In ASP.NET Razor syntax, a @while control structure is written with a code block, where the first line contains @while and a condition.

In the following lesson, we'll explore the page model and its relation to the view files we've been using throughout this lesson.

17.7 Review Instructions

Provided is the generated template from ASP.NET when creating a new Razor App. Use the main view page as your playground to write C# and HTML and display whatever you'd like on the view pages! Make sure to click **Run** to see the changes displayed!

Index.cshtml

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Home page";
}

<div class="text-center">
    <h1 class="display-4">@Model.Greet</h1>
    <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building
Web apps with ASP.NET Core</a>.</p>
</div>
```

Concept Review

Razor Syntax

Razor Syntax allows you to embed code (C#) into page views through the use of a few keywords (such as “@”), and then have the C# code be processed and converted at runtime to HTML. In other words, rather than coding static HTML syntax in the page view, a user can code in the view in C# and have the Razor engine convert the C# code into HTML at runtime, creating a dynamically generated HTML web page.

```
@page
```

```
@model IndexModel
```

```
<h2>Welcome</h2>
```

```
<ul>
```

```
    @for (int i = 0; i < 3; i++)
```

```
    {
```

```
        <li>@i</li>
```

```
    }
```

```
</ul>
```

The @page Directive

In a Razor view page (**.cshtml**), the @page directive indicates that the file is a Razor Page.

In order for the page to be treated as a Razor Page, and have ASP.NET parse the view syntax with the Razor engine, the directive @page should be added at the top of the file.

There can be empty space before the @page directive, but there cannot be any other characters, even an empty code block.

```
@page
```



```
@model IndexModel
```

```
<h1>Welcome to my Razor Page</h1>
```

```
<p>Title: @Model.Title</p>
```

The @model Directive

The page model class, i.e. the data and methods that hold the functionality associated with a view page, is made available to the view page via the @model directive.

By specifying the model in the view page, Razor exposes a Model property for accessing the model passed to the view page. We can then access properties and functions from that model by using the keyword Model or render its property values on the browser by prefixing the property names with @Model, e.g. @Model.PropertyName.

```
@page
```

```
@model PersonModel
```

```
// Rendering the value of FirstName in PersonModel
```

```
<p>@Model.FirstName</p>
```

```
<ul>
```

```
    // Accessing the value of FavoriteFoods in PersonModel
```

```
    @foreach (var food in Model.FavoriteFoods)
```

```
    {
```

```
        <li>@food</li>
```

```
}  
</ul>
```

Razor Markup

Razor pages use the @ symbol to transition from HTML to C#. C# expressions are evaluated and then rendered in the HTML output. You can use Razor syntax under the following conditions:

1. Anything immediately following the @ is assumed to be C# code.
2. Code blocks must appear within @{ ... } brackets.
3. A single line of code that uses spaces should be surrounded by parentheses, ().

@page

@model PersonModel

// Using the `@` symbol:

<h1>My name is @Model.FirstName and I am @Model.Age years old </h1>

// Using a code block:

```
@{  
    var greet = "Hey threre!";  
    var name = "John";  
    <h1>@greet I'm @name!</h1>  
}
```

// Using parentheses:

```
<p>Last week this time: @(DateTime.Now - TimeSpan.FromDays(7))</p>
```

Razor Conditionals

Conditionals in Razor code can be written pretty much the same way you would in regular C# code. The only exception is to prefix the keyword if with the @ symbol. Afterward, any else or else if conditions doesn't need to be prepended with the @ symbol.

// if-else if-else statment:

```
@{ var time = 9; }
```

```
@if (time < 10)
```

```
{
```

```
<p>Good morning, the time is: @time</p>
```

```
}
```

```
else if (time < 20)
```

```
{
```

```
<p>Good day, the time is: @time</p>
```

```
}
```

```
else
```

```
{
```

```
<p>Good evening, the time is: @time</p>
```

```
}
```

Razor Switch Statements

In Razor Pages, a switch statement begins with the @ symbol followed by the keyword switch. The condition is then written in parentheses and finally the switch cases are written within curly brackets, {}.

```
@{ string day = "Monday"; }

@switch (day)
{
    case "Saturday":
        <p>Today is Saturday</p>
        break;
    case "Sunday":
        <p>Today is Sunday</p>
        break;
    default:
        <p>Today is @day... Looking forward to the weekend</p>
        break;
}
```

Razor For Loops

In Razor Pages, a for loop is prepended by the @ symbol followed by a set of conditions wrapped in parentheses. The @ symbol must be used when referring to C# code.

```
@{
```

```
List<string> avengers = new List<string>()
{
    "Spiderman",
    "Iron Man",
    "Hulk",
    "Thor",
};
}
```

```
<h1>The Avengers Are:</h1>
```

```
@for (int i = 0; i < @avengers.Count; i++)
{
    <p>@avengers[i]</p>
}
```

Razor Foreach Loops

In Razor Pages, a foreach loop is prepended by the @ symbol followed by a set of conditions wrapped in parentheses. Within the conditions, we can create a variable that will be used when rendering its value on the browser.

```
@{
```

```
List<string> avengers = new List<string>()
{
    "Spiderman",
    "Iron Man",
    "Hulk",
    "Thor",
};
}
```

<h1>The Avengers Are:</h1>

```
@foreach (var avenger in avengers)
{
    <p>@avenger</p>
}
```

Razor While Loops

A while loop repeats the execution of a sequence of statements as long as a set of conditions is true, once the condition becomes false we break out of the loop.

When writing a while loop, we must prepend the keyword while with the @ symbol and write the condition within parentheses.

```
@{ int i = 0; }
```

```

@while (i < 5)
{
    <p>@i</p>
    i++;
}

```

Razor View Data

In Razor Pages, you can use the ViewData property to pass data from a Page Model to its corresponding view page, as well as share it with the layout, and any partial views.

ViewData is a dictionary that can contain key-value pairs where each key must be a string. The values can be accessed in the view page using the @ symbol.

A huge benefit of using ViewData comes when working with layout pages. We can easily pass information from each individual view page such as the title, into the layout by storing it in the ViewData dictionary in a view page:

```
@{ ViewData["Title"] = "Homepage" }
```

We can then access it in the layout like so: ViewData["Title"]. This way, we don't need to hardcode certain information on each individual view page.

```
// Page Model: Index.cshtml.cs
```

```

public class IndexModel : PageModel
{
    public void OnGet()
    {
        ViewData["Message"] = "Welcome to my page!";
        ViewData["Date"] = DateTime.Now();
    }
}

```

```

    }
}

// View Page: Index.cshtml

@page
@model IndexModel

<h1>@ViewData["Message"]</h1>
<h2>Today is: @ViewData["Date"]</h2>

```

Razor Shared Layouts

In Razor Pages, you can reduce code duplication by sharing layouts between view pages. A default layout is set up for your application in the **_Layout.cshtml** file located in **Pages/Shared/**.

Inside the **_Layout.cshtml** file there is a method call: `RenderBody()`. This method specifies the point at which the content from the view page is rendered relative to the layout defined.

If you want your view page to use a specific Layout page you can define it at the top by specifying the filename without the file extension: `@{ Layout = "LayoutPage" }`

```

// Layout: _LayoutExample.cshtml

<body>

    ...

<div class="container body-content">

    @RenderBody()

```



```

<footer>

    <p>@DateTime.Now.Year - My ASP.NET Application</p>

</footer>

</div>

</body>

// View Page: Example.cshtml

@page

@model ExampleModel

@{ Layout = "_LayoutExample" }

<h1>This content will appear where @RenderBody is called!</h1>

```

Razor Tag Helpers

In Razor Pages, Tag Helpers change and enhance existing HTML elements by adding specific attributes to them. The elements they target are based on the element name, the attribute name, or the parent tag.

ASP.NET provides us with numerous built-in Tag Helpers that can be used for common tasks - such as creating forms, links, loading assets, and more.

```

// Page Model: Example.cshtml.cs

public class ExampleModel : PageModel
{
    public string Language { get; set; }
}

```

```

public List<SelectListItem> Languages { get; } = new List<SelectListItem>
{
    new SelectListItem { Value = "C#", Text = "C#" },
    new SelectListItem { Value = "Javascript", Text = "Javascript" },
    new SelectListItem { Value = "Ruby", Text = "Ruby" },
};
}

```

// View Page: Example.cshtml

```

<h1>Select your favorite language!</h1>
<form method="post">
    // asp-for: The name of the specified model property.
    // asp-items: A collection of SelectListItem options that appear in the select list.
    <select asp-for="Language" asp-items="Model.Languages"></select>
    <br />
    <button type="submit">Register</button>
</form>

```

// HTML Rendered:

```

<form method="post">
    <select id="Language" name="Language">
        <option value="C#">C#</option>

```

```
<option value="Javascript">Javascript</option>
<option value="Ruby">Ruby</option>
<br>
</select>
<button type="submit">Register</button>
</form>
```

Razor View Start File

When creating a template with ASP.NET, a **ViewStart.cshtml** file is automatically generated under the **/Pages** folder.

The **ViewStart.cshtml** file is generally used to define the layout for the website but can be used to define common view code that you want to execute at the start of each View's rendering. The generated file contains code to set up the main layout for the application.

```
// ViewStart.cshtml
@{
    Layout: "_Layout"
}
```

Razor View Imports

The **_ViewImports.cshtml** file is automatically generated under **/Pages** when we create a template with ASP.NET.

Just like the **_ViewStart.cshtml** file, **_ViewImports.cshtml** is invoked for all your view pages before they are rendered.

The purpose of the file is to write common directives that our view pages need. ASP.NET currently supports a few directives that can be added such as: `@namespace`, `@using`, `@addTagHelpers`, and `@inject` amongst a few other ones. Instead of having to add them individually to each page, we can place the directives here and they'll be available globally throughout the application.

```
// _ViewImports.cshtml

@using YourProject

@namespace YourProject.Pages

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Razor Partial

Partial views are an effective way of breaking up large views into smaller components and reduce complexity. A partial consists of fragments of HTML and server-side code to be included in any number of pages or layouts.

We can use the [Partial Tag Helper](#), `<partial>`, in order to render a partial's content in a view page.

```
// _MyPartial.cshtml

<form method="post">

  <input type="email" name="emailaddress">

  <input type="submit">

</form>

// Example.cshtml
```

<h1> Welcome to my page! </h1>

<h2> Fill out the form below to subscribe!:</h2>

<partial name="_MyPartial" />

18. RAZOR PAGES SYNTAX II

18.1 Introduction to the Page Model

Now that we have an understanding of how the view page renders C# and HTML, let's take a look at how we can work with the logic behind it.

In order to add functionality to the view layer, we must add a `PageModel` class where all the logic will belong. This class is usually found in the **`cshtml.cs`** file placed in the same folder as the view page. The class file shares the same file name as the view page, albeit with an additional `.cs` on the end to denote the fact that it's actually a C# class file. So if we have a view page at **`Pages/Index.cshtml`**, the model page will be available in **`Pages/Index.cshtml.cs`**.

Our model for the view page will be responsible for exposing methods, commands, and other properties that help maintain the state of the view, and trigger events in the view itself.

Below are some of the advantages that come when separating the logic from the view page:

- Smaller and more reusable units of code allow for flexibility to create scalable apps with more ease.
- View pages will be easier to maintain if the model needs to be updated or requires changes. If the view page and the model weren't separated, and the model changed frequently, the view would be flooded with update requests leading you to spend extra time fixing and tweaking small changes like method calls or property names.
- Distinct sections also allow for code to be unit tested — a practice of creating tests to test the written code, which ultimately allows for automated testing — a concept that becomes ever more important as a project grows in size and expands functionality.

18.1 Introduction to the Page Model Instructions

Looking at the bottom right diagram, we can see how properties are created in the page model — **Hello.cshtml.cs**, and how they're used in the view page — **Hello.cshtml**.

Each file has its own distinct use; whereas the model page takes care of the logic behind the properties and methods, the view page is simply making use of said properties and methods by calling them and/or exposing them to the user.

In the model page, we can see how a property is defined and assigned a value:

Hello.cshtml.cs

```
public string Title = "Hello";
```

And in the view page, we're displaying the value to the user:

Hello.cshtml

```
<div>@Model.Title</div>
```

We can also see how a method is created and used.

In the model, we're defining how the method works. What will the method be named? What will it do when it's called? What UI elements will change?

Hello.cshtml.cs

```
public void OnPost()
{
    Title = "Hi!";
}
```

Here we're updating the value of the property Title to "Hi!" once the method, OnPost() is called.

In the view page, we're making use of this method by submitting a form.

Hello.cshtml

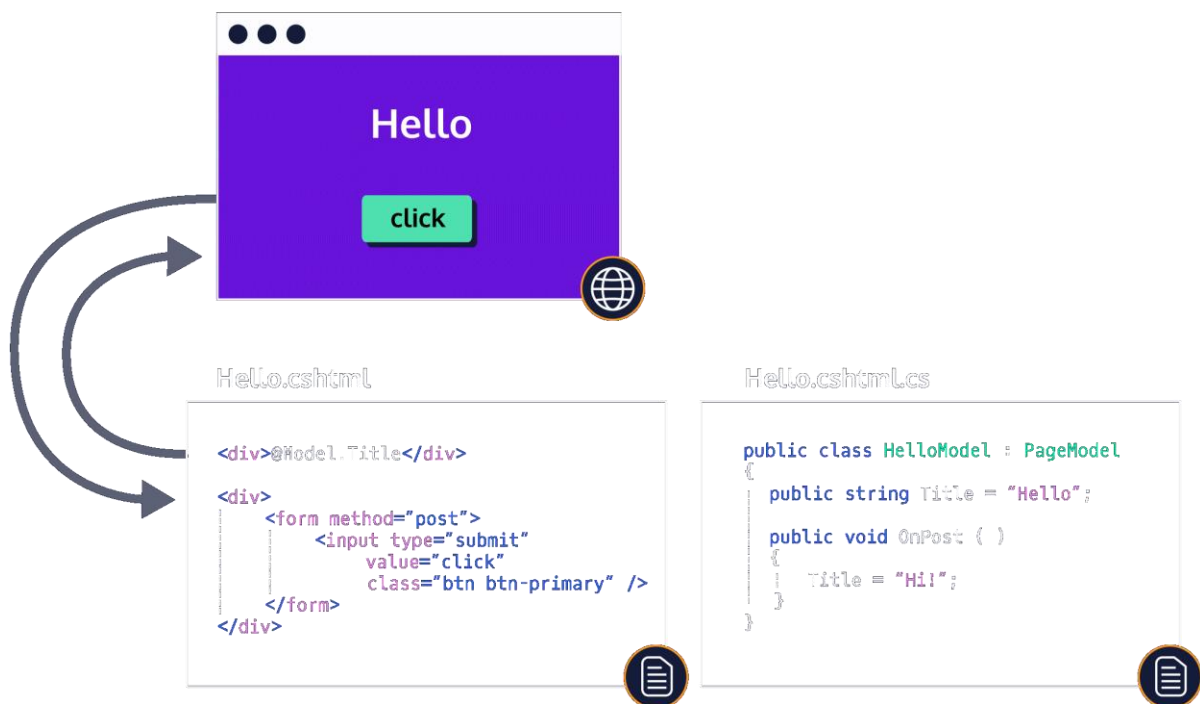
```
<form method="post">
```

```
...
```

```
</form>
```

Once the submit button is clicked, the method `OnPost()` will be called, and the value of `Title` will be updated from “Hello” to “Hi!”.

Make sure to watch a few loops of the attached gif in order for this to make sense!



18.2 Working with the Page Model

With a page model, you can create certain data and behaviors that you’ll be able to call from the view page. Let’s say you want to display information about a user in a user profile page; you can create a class holding properties of interest for that user (username, age, birthday, etc) and simply call the properties in the view page

instead of hardcoding them. Moreover, what if the user wants to update their username? The class can hold a method that updates the username property.

The model you'll create must inherit from the `PageModel` class to have access to various properties that will be useful when working with HTTP requests. Let's look at an example:

```
using System;
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace RazorTest.Pages
{
    public class UserModel : PageModel
    {
        public string ConvertToUsername (string firstName)
        {
            return "l33t_user_" + firstName;
        }

        public int Age { get; set; }
        public string Username { get; set; }
        public bool Status { get; set; }
        public DateTime Birthday { get; set; }

        public void OnGet()
        {
            Age = 25;
            Username = ConvertToUsername("John");
            Status = true;
            Birthday = DateTime.Now;
        }
    }
}
```

In the above model, there are 4 properties that will be accessible in the view page. Through the `OnGet()` method, you're able to assign values to those properties on page load. But how exactly does the `OnGet()` method work?

The `OnGet()` method is what we call a handler method in Razor Pages. These are methods that are executed as a result of a request. Let's say a user wants to view their profile page on a website they're registered at. Once the profile page is reached, a GET request will be sent to the server to fetch all the information about the said user. The GET request invokes the `OnGet()` method. In this case, we're assigning values to the properties in our `UserModel` once the page is loaded.

Once a model has been created you can access its properties and methods in the view page using the `@model` directive followed by the model you're referring to:

```
@model ModelName
```

After the model has been specified, Razor exposes a *Model* property for accessing the model passed to the view. So you can then call these properties using `@Model` in the view page:

```
@page
@model UserModel
<div>
    Age: @Model.Age
    <br />
    Username: @Model.Username
    <br />
    Status: @Model.Status
    <br />
    Birthday: @Model.Birthday.ToString("MM/dd/yyyy")
</div>
```

You can also create methods and call them in the `OnGet()` method. Let's create one where we generate a Username by prepending a string before the user's firstName:

```
public string ConvertToUsername (string firstName)
{
```

```
return "l33t_user_" + firstName;  
}
```

We can create a property called Username:

```
public string Username {get; set;}
```

Then we would call it in our OnGet() method as so:

```
public void OnGet()  
{  
    Username = ConvertToUsername("John");  
}
```

18.2 Working with the Page Model Instructions

1.

Suppose you're building an application to take orders at a pizza restaurant. You will be filling out properties in **Pizza.cshtml.cs** to complete the app.

In order for the view page, **Pizza.cshtml**, to know what model it will have access to, you need to specify it.

Use the @model directive to specify the model: PizzaModel.

2.

Navigate to **Pizza.cshtml.cs**

You're currently provided with a method, PizzaTotal(), and a double property, Total, but some information is missing!

Add the following properties:

Property	DataType
Customer	String
Order	String

Property	DataType
----------	----------

ExtraCheese	Boolean
-------------	---------

Make sure they are public properties and to provide them with a getter and a setter!

3.

Once properties have been defined in the class, we can assign some values to them. Since we want the app to be initialized with some state and have the values be rendered on the page load, we'll be assigning values in the `OnGet()` method.

Within the `OnGet()` method, assign the following values to the properties below:

Property	Value
----------	-------

Customer	<i>your name</i>
----------	------------------

Order	"Cheese"
-------	----------

ExtraCheese	false
-------------	-------

Total	PizzaTotal("Cheese")
-------	----------------------

4.

Now we can work on our view page, take a look at **Pizza.cshtml**. There will be placeholders where we can call the properties.

Call each property in the appropriate place and click *Run* afterward. You should now see the values being rendered on the page.

To render the Total property correctly, use the dollar sign \$ and the `String.Format()` method. Here's an example of `String.Format()` in plain C#:

```
double cost = 15.00;  
string formattedCost = String.Format("{0:0.00}", cost);
```

Pizza.cshtml

@page

<!-- Specify the model below -->

@{
}

<h1 class="text-center">Codecademy Pizza</h1>

<div class="card mx-auto" style="width: 18rem;">

<div class="card-body">

<h4 class="card-title text-center">Confirm your order</h4>

<h5>Pizza for: <!-- Customer Name --></h5>

<h5>Order: <!-- Customer Order --></h5>

<h5>Extra Cheese: <!-- Extra Cheese Bool --></h5>

<h5>Total: <!-- Total Cost --></h5>

Place Order

</div>

</div>

Pizza.cshtml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace starting_app.Pages
{
    public class PizzaModel : PageModel
    {
        public double PizzaTotal (string pizzaType)
        {
            Dictionary<string, double> PizzaCost = new Dictionary<string, double>()
            {
                { "Cheese", 10 },
                { "Pepperoni", 11 },
                { "Vegetarian", 12 },
            };

            return PizzaCost[pizzaType];
        }

        public double Total {get; set;}
    }
}

```

```
public void OnGet()  
{  
  
}  
}  
}
```

OUTPUT

Codecademy Pizza



18.3 ViewData

ViewData allows us to pass information from the page model into the view page. ViewData is of type ViewDataDictionary, which acts as a generic dictionary. What exactly is a dictionary in C#? Well, you can think of a dictionary in the same way as you would think of an English dictionary: a collection of words and their definitions. In a similar fashion, a Dictionary in C# is a collection of keys and values, where the key is typically a string and the value is its definition. For more information please refer to the [following documentation](#).

We can add key/value pairs to our ViewData in the PageModel as so:

```
public class IndexModel : PageModel
{
    public void OnGet()
    {
        ViewData["MyDogsName"] = "Alfie";
        ViewData["MyDogsAge"] = 7;
    }
}
```

ViewData is accessible in the view page and we can refer to the values stored in them by calling the key names:

```
@page
@model RazorTest.Pages.ExampleModel
<div>
    <p>My dog @ViewData["MyDogsName"] is @ViewData["MyDogsAge"] years
    old</p>
    ...
</div>
```

This might look similar to the way you would access properties in the class from the page model, however, ViewData becomes very useful when we're working with layout pages or partials which we'll cover in the following exercise.

18.3 ViewData Instructions

1.

Let's take a look at **Profile.cshtml.cs**. There's currently an empty OnGet() method. Let's add certain information for a user's profile.

Add a key/value pair to ViewData for each of the following:

- myName: a string containing your full name.
- username: A string containing your personal username (can be as simple as your first name).
- myOccupation: Your occupation or study.
- myAge: Your age.
- currentDate: A string with today's date in the following format: "MM/DD/YY".

2.

In Profile.cshtml there are commented lines for you to fill out. Go ahead and access each of the properties in ViewData and fill out the blanks!

Profile.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MyApp.Namespace
{
    public class ProfileModel : PageModel
```

```

{
    public void OnGet()
    {
    }
}

```

Profile.cshtml

```

@page
@model MyApp.Namespace.ProfileModel
@{
}

```

```

<div class="container">
    <div class="row">
        <div class="col-12">
            <div class="card">
                <div class="card-body">
                    <div class="card-title mb-4">
                        <div class="d-flex justify-content-start">
                            <div class="image-container">
                                
                                </div>

```

```

<div class="userData ml-3">
    <h2 class="d-block"
style="font-size: 1.5rem; font-weight: bold">
        <!-- Username Here -->
    </h2>
</div>
</div>
</div>

<div class="row">
    <div class="col-12">
        <ul class="nav nav-tabs mb-4" id="myTab"
role="tablist">
            <li class="nav-item">
                <a class="nav-link active"
href="#">Basic Info</a>
            </li>
        </ul>

        <div class="tab-content ml-1"
id="myTabContent">
            <div class="tab-pane fade show
active" id="basicInfo" aria-labelledby="basicInfo-tab">
                <div class="row">
                    <div class="col-
sm-3 col-md-2 col-5">

```

```
                                <label
style="font-weight:bold;">Full Name</label>
```

```
</div>
```

```
<div class="col-md-8 col-6">
```

```
<!-- Your Name Here -->
```

```
</div>
```

```
</div>
```

```
<hr />
```

```
<div class="row">
```

```
<div class="col-sm-3 col-md-2 col-5">
```

```
<label style="font-weight:bold;">Age</label>
```

```
</div>
```

```
<div class="col-md-8 col-6">
```

```
<!-- Your Age Here -->
```

```
</div>
```

```
</div>
```

```
<hr />
```

```
<div class="row">
```

```
<div class="col-sm-3 col-md-2 col-5">
```

```
<label style="font-weight:bold;">Occupation</label>
```

```
</div>
```

```
<div class="col-md-8 col-6">
  <!-- Your Occupation Here -->
```

```
</div>
```

```
</div>
```

```
<hr />
```

```
<div class="row">
```

```
  <div class="col-sm-3 col-md-2 col-5">
```

```
    <label style="font-weight:bold;">Current Date</label>
```

```
  </div>
```

```
  <div class="col-md-8 col-6">
```

```
    <!-- Current Time Here -->
```

```
  </div>
```

```
</div>
```

```
<hr/>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

</div>

OUTPUT

 **John123**

Basic Info

Full Name	John Doe
Age	24
Occupation	Codecademy Student
Current Date	01/01/20

18.4 Sharing Pages

Most web apps have a common layout that provides the user with a consistent experience as they navigate from page to page. Certain layouts may include a header, footer, menus, and so on, that may be shared amongst different pages.

Let's say we're building an application that contains a header throughout most (if not all) of your pages. If we're creating the markup for all these pages, the header will still need to be added to each one. That's a lot of copy-pasting if the header

does not change. We can avoid redundancy by essentially storing the header somewhere separate and rendering its content on each page. If our website needs an update we can simply change a single file and the changes will be reflected everywhere the content has been inserted.

Standard practice is to specify the location of the main layout page in the **_ViewStart.cshtml** file. This file is automatically generated under **/Pages** when we create a template with ASP.NET and it looks like this:

```
@{  
    Layout = "_Layout";  
}
```

Here, we're directing our app to use the **_Layout** file as the main layout for all of our content. But where is **_Layout** coming from? Well, Razor Pages searches by walking up the directory tree from the location of the calling page looking for the filename specified, followed by the **/Pages/Shared** folder. In this case, our generated template has a **_Layout.cshtml** file under **/Pages/Shared**.

Once we specify the location of the layout page, that layout will affect all content pages. You can think of this layout as the main layout that's used throughout your whole application.

By convention, layout pages use a leading underscore in their filename: **_Layout.cshtml**.

In the generated Razor Pages template the **_Layout.cshtml** file will contain a method call, **@RenderBody()** within the **<body>** tags:

```
<div class="container">  
    <main role="main" class="pb-3">  
        @RenderBody()  
    </main>  
</div>
```

When we work on our own view pages, the content will be rendered wherever **@RenderBody()** is being called on the layout page.

One other advantage of sharing pages this way is that we can use our handy ViewData dictionary and pass data down into our layout page! Let's say

that different pages in our app will have their own title tags. We can create a ViewData key/value pair set to title as the property, and unique titles as the values for different pages:

About.cshtml

```
@{  
    ViewData["Title"] = "About";  
}
```

With our key/value pair defined, all we need to do is simply call the property in our Layout page as so:

```
<title>@ViewData["Title"] - Razor App</title>
```

Our Layout will render the value for the title of the page depending on where it's defined, this allows us to avoid creating unnecessary properties in our PageModel and save us some space!

We don't always have to use the main layout provided for all of our content. If we want to specify our own layout pages we can do this at the top of our content page. Razor is smart enough to search through a set of predefined locations so we don't need to provide the whole path of where our layout is located. The filename (.cshtml not needed) should suffice:

```
@{  
    Layout = "_NewLayout";  
}
```

18.4 Sharing Pages Instructions

1.

If we look at both the Privacy and Index page, there's still some code being duplicated! Let's consolidate this and add it to our layout page.

1. Remove the footer and header content from **Privacy.cshtml**
2. Remove the footer and header content from **Index.cshtml**

3. Add the footer and header content in the appropriate place in **_Layout.cshtml**.

2.

Now that we have refactored our code, we can define what Layout the application should use.

Navigate to **_ViewStart.cshtml**

Open some code blocks and assign the Layout property to "_Layout" so it's executed upon each rendered view page.

3.

The application crashed! Let's go ahead and fix that.

At the moment the layout page, **_Layout.cshtml** is not running because it's missing the required `RenderBody()` method.

Add a `@RenderBody()` method between the footer element and the header element. Otherwise, it won't render any content from other pages!

4.

Let's make use of `ViewData` to see how its properties can be shared amongst pages.

In **_Layout.cshtml**, below your `RenderBody()` method call, create an `<h1>` heading and access the `ViewData["Title"]` property in order to display the following in the Privacy and Home page.

Greetings from the Home page!

Greetings from the Privacy Policy page!

NOTE: You'll only be adding the line once.

5.

We can now run our application and see what occurred. Run the app and try clicking on the navigation links you added in the header and the footer. You'll notice that even when changing pages, we're still having the same content being produced! This is because our main layout page is being executed on every rendered view page. Take note that the only content changing is the actual markup that was provided in **Privacy.cshtml** and **Index.cshtml**. This is being rendered wherever `RenderBody()` is called.

layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>@ViewData["Title"] - Starting App</title>
  <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <!-- Header Content Below-->

  <div class="container">
    <main role="main" class="pb-3">
      <!-- Add Render Body Below -->
```

```

<!-- Add h1 Heading Below -->

</main>
</div>
<!-- Footer Content Below-->

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>

@RenderSection("Scripts", required: false)
</body>
</html>

```

Index.cshtml

```

@page
@model IndexModel
@{
    ViewData["Title"] = "Home";
}
<!-- Header Start -->
<header>
    <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-
white border-bottom box-shadow mb-3">
        <div class="container">

```

```

<a class="navbar-brand" asp-area="" asp-page="/Index">Starting App</a>
  <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target=".navbar-collapse" aria-controls="navbarSupportedContent"
    aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>
<div class="navbar-collapse collapse d-sm-inline-flex flex-sm-row-reverse">
  <ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
      <a class="nav-link text-dark" asp-page="/Index">Home</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-page="/Privacy">Privacy</a>
    </li>
  </ul>
</div>
</div>
</nav>
</header>
<!-- Header End -->

```

```

<div class="text-center">
  <h1 class="display-4">Home</h1>
  <hr>
</div>

```

```

<!-- Footer Start -->
<footer class="border-top footer text-muted">
  <div class="container">
    &copy; 2019 - RazorTest - <a asp-area="" asp-page="/Privacy">Privacy</a>
  </div>
</footer>
<!-- Footer End -->

```

Privacy.cshtml

```

@page
@model PrivacyModel
@{
  ViewData["Title"] = "Privacy Policy";
}

<!-- Header Start -->
<header>
  <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-
white border-bottom box-shadow mb-3">
    <div class="container">
      <a class="navbar-brand" asp-area="" asp-page="/Index">Starting App</a>
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target=".navbar-collapse" aria-controls="navbarSupportedContent"
        aria-expanded="false" aria-label="Toggle navigation">

```

```

        <span class="navbar-toggler-icon"></span>
    </button>
    <div class="navbar-collapse collapse d-sm-inline-flex flex-sm-row-reverse">
        <ul class="navbar-nav flex-grow-1">
            <li class="nav-item">
                <a class="nav-link text-dark" asp-page="/Index">Home</a>
            </li>
            <li class="nav-item">
                <a class="nav-link text-dark" asp-page="/Privacy">Privacy</a>
            </li>
        </ul>
    </div>
</div>
</nav>
</header>
<!-- Header End -->

<div class="text-center">
    <h1 class="display-4">Privacy</h1>
    <hr>
</div>

<!-- Footer Start -->
<footer class="border-top footer text-muted">

```

```
<div class="container">

    &copy; 2019 - RazorTest - <a asp-area="" asp-page="/Privacy">Privacy</a>

</div>

</footer>

<!-- Footer End -->
```

ViewStart.cshtml

OUTPUT

Starting App 

- [Home](#)
- [Privacy](#)

Home

© 2019 - RazorTest - [Privacy](#)

18.5 Working with ViewImports

On top of sharing layouts, we're also provided with a file where we can include directives that will become available globally throughout the rest of our pages. This saves us work in typing and makes maintenance easier.

There are a number of directives that you'll be able to include in **_ViewImports.cshtml**, three of the most common ones that are added by default are `@namespace`, `@using`, and `@addTagHelpers`.

- `@namespace`: Namespaces help us organize our code into groups by giving them some context. With namespaces, we're able to avoid any naming collision with other classes of the same name. In this case, the namespace

is used to declare the root namespace for your Pages. The default generated **_ViewImports.cshtml** file for most projects will have:

```
@namespace YourProjectName.Pages
```

You can only have *one* @namespace directive per **_ViewImports.cshtml** file, the directive is necessary for the models to work.

- @using: Allows us to add imports to all of our Pages instead of having to add them individually per page. For example, if our **_ViewImports.cshtml** file contains:

```
@using Microsoft.Extensions.Localization;
```

then any pages under that folder will have access to members under Microsoft.Extensions.Localization namespace without having to explicitly import them with @using.

- @addTagHelper: This directive is used to give access to Tag Helpers throughout our pages. The default directive includes:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

which imports all Tag Helpers to our Pages. We will look more into Tag Helpers later in the lesson.

You might be wondering why not combine both **_ViewStart.cshtml** file and **_ViewImports.cshtml** into a single file and share that globally? Well this comes back to the idea of separation of concerns; ViewImports cares about imports, ViewStart cares about logic required for the view pages. Using ViewImports allows us to keep our code more concise and help us create a more scalable application.

18.5 Working with ViewImports Instructions

1.

The current app looks like it's missing a few things! If you take a look at **Index.cshtml** and **Privacy.cshtml** you can see that we are still including our `@namespace`, `@using`, and `@addTagHelpers` here. Let's consolidate this code.

1. Remove the three directives from **Privacy.cshtml**
2. Remove the three directives from **Index.cshtml**
3. Add the three directives at the top of **_ViewImports.cshtml**.

Index.cshtml

```
@page
```

```
<!-- Imports start -->
```

```
@using starting_app
```

```
@namespace starting_app.Pages
```

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

```
<!-- Imports end -->
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="text-center">
```

```
    <h1 class="display-4">Welcome</h1>
```

```
    <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building  
Web apps with ASP.NET Core</a>.</p>
```

</div>

<!-- Footer start -->

<!-- Footer end -->

Privacy.cshtml

@page

<!-- Imports start -->

@using starting_app

@namespace starting_app.Pages

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

<!-- Imports end -->

@model PrivacyModel

@{

 ViewData["Title"] = "Privacy Policy";

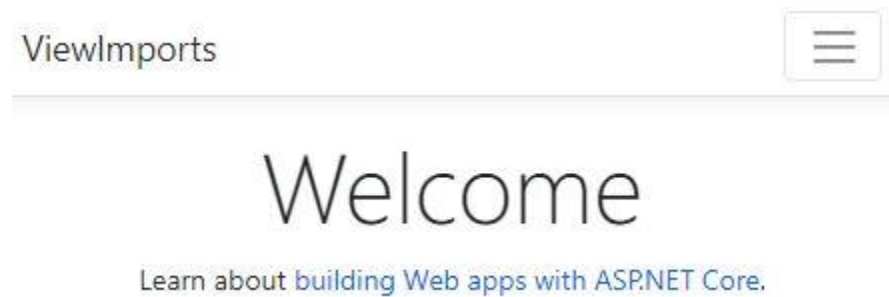
}

<h1>@ViewData["Title"]</h1>

<p>Use this page to detail your site's privacy policy.</p>

_ViewImports.cshtml

OUTPUT



18.6 Using Partial

In certain cases, we might end up reusing snippets of HTML, such as forms, on different pages. We can also use these “partials” to break up more complex pages into smaller sections. This is another way to reduce duplication of common markup content across different files and keep our code DRY. [“D.R.Y” \(Don’t Repeat Yourself\), is a software development principle, the main aim of which is to reduce the repetition of code.](#)

Partial file names also begin with an underscore in their name (through convention). And in order to render the content we can use the following syntax in our content page:

```
<partial name="_PartialName" />
```

Where name refers to the filename of the partial.

Let’s look at an example:

Picture an application with the following folder structure:

```
Razor App
| Startup.cs
| Program.cs
| RazorApp.sln
```

```

|   ...
|
└── Pages
    |   _ViewStart.cshtml
    |   MyPage.cshtml
    |   MyPage.cshtml.cs
    |   ...
    |
    └── Shared
        |   _Layout.cshtml
        |   _MyPartial.cshtml.
        |   ...
    └── Properties

```

If our partial has the following content:

_MyPartial.cshtml:

```
<div>Here's some content from a partial!</div>
```

We can use the partial in a separate page as follows:

MyPage.cshtml:

```

<div>
  <h1>Welcome to My Page!</h1>
  <partial name="_MyPartial"/>
</div>

```

Similarly to Layouts, Razor Pages will search all registered partial locations for a file with the supplied name and if it finds it, it will render its content resulting in:

```

<div>
  <h1>Welcome to My Page!</h1>
  <div>Here's some content from a partial!</div>
</div>

```

18.6 Using Partial Instructions

1.

A very reusable component in many applications is a `<form>`. Let's imagine we're working on an app that will use a simple form to register and log in users. The form will be used throughout different pages. If we explore our folder structure, you'll see that we have a "Registration" page: **Register.cshtml**, and a "Login" page: **Login.cshtml**.

Copy the content in the `<form>` tags and paste it in the partial page: **_Form.cshtml**

2.

Remove the `<form>` content from the **Login.cshtml** and **Register.cshtml** pages.

3.

Use the `<partial>` tag to add the form in both **Register.cshtml** and **Login.cshtml**.

Login.cshtml

@page

@model MyApp.Namespace.LoginModel

@{

}

`<div class="jumbotron">`

`<h1 class="display-4">Log in!</h1>`

`<p class="lead">Welcome back!</p>`

`<hr class="my-4">`

```

<p>Provide your information below to log in</p>
    <!-- Form partial -->
<form>
    <div class="form-group">
        <label for="exampleInputEmail1">Email address</label>
        <input type="email" class="form-control" id="exampleInputEmail1" aria-
describedby="emailHelp" placeholder="Enter email">
        <small id="emailHelp" class="form-text text-muted">We'll never share your
email with anyone else.</small>
    </div>
    <div class="form-group">
        <label for="exampleInputPassword1">Password</label>
        <input type="password" class="form-control" id="exampleInputPassword1"
placeholder="Password">
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
</form>
</div>

```

Register.cshtml

```

@page
@model MyApp.Namespace.RegisterModel
@{
}

```

```

<div class="jumbotron">
  <h1 class="display-4">Register!</h1>
  <p class="lead">Don't have an account? Registration is quick and easy!</p>
  <hr class="my-4">
  <p>Provide your information below to create an account</p>
  <!-- Form partial -->
  <form>
    <div class="form-group">
      <label for="exampleInputEmail1">Email address</label>
      <input type="email" class="form-control" id="exampleInputEmail1" aria-
describedby="emailHelp" placeholder="Enter email">
      <small id="emailHelp" class="form-text text-muted">We'll never share your
email with anyone else.</small>
    </div>
    <div class="form-group">
      <label for="exampleInputPassword1">Password</label>
      <input type="password" class="form-control" id="exampleInputPassword1"
placeholder="Password">
    </div>
    <button type="submit" class="btn btn-primary">Submit</button>
  </form>
</div>

```

_Form.cshtml

OUTPUT

Log in!

Welcome back!

Provide your information below to log in

Email address

We'll never share your email with anyone else.

Password

Submit

18.7 Tag Helpers

Tag Helpers are very useful ways to create HTML elements with server-side attributes. For example, when we use an anchor tag, `<a>`, we usually have an href attribute to direct the user to a specific page when they click it. Razor Pages provides handy Tag Helpers that help us generate links and other useful information.

So instead of writing an anchor tag as so:

```
<a href="/Attendee?attendeeid=10">View Attendee</a>
```

We can rewrite it with a Tag Helper like this:

```
<a asp-page="/Attendee"  
  asp-route-attendeeid="10">View Attendee</a>
```


The syntax allows developers to easily customize the UI using HTML/CSS knowledge.

There are many available predefined Tag Helpers to help us generate links, create forms, load assets, etc. There is even one to help us render partials which we just went over! So let's go ahead and take a look at how some of them work.

```
<partial name="_ItemPartial" ... />
```

In the tag above, we're using one of the predefined Tag Helpers provided by ASP.NET: `<partial ...>`, the partial tag will take a required name attribute that searches all registered partial locations for the file with that name, and once the page is loaded, the server will render the HTML from that file. Certain tag helpers have optional attributes that they can take. The partial one has the following optional attributes:

- **for**: Assigns an expression to be evaluated against the current model. This is one way to pass data to the partial.

```
<partial name="_ItemPartial" for="Item" />
```

Note: The model syntax is inferred so we don't need to use `@Model.Item`

- **model**: The model the partial references. This is another way to pass data into our partial. The main difference between **model** and **for**, is that **model** allows you to use a more specific model instead of the inferred one that **for** provides. HOWEVER, we can't use it in conjunction with **for**. We must use one or the other:

```
<partial name="_ItemPartial" model="Model.Item" />
```

In this case, we could pass in a brand new instance of our model:

```
<partial name="_ItemPartial"
model='new Item { Number = 1, Name: "Test Item, Price: 10"}' />
```

- **view-data**: Assigns a key/value pair of type `ViewDataDictionary` that we can pass directly into the partial view:

```
@{
    ViewData["IsItemReadOnly"] = true;
}
```

```
<partial name="_ItemPartial" view-data="ViewData" />
```

During runtime, the Razor Application will process the markup and when it reads out certain Tag Helpers it will convert them into plain HTML before sending the page to the user. For reference, [here is a list of all the tag helpers provided by ASP.NET!](#)

18.7 Tag Helpers Instructions

1.

You're in the early stages of developing a travel agency application and you're currently working on the homepage.

Take a look at the documentation for the [Select Tag Helper](#). It's ok if not everything is 100% clear! We'll go over how to use it in this exercise. Once you've gone over the documentation, navigate over to the **Explorella.cshtml.cs** file. There's a model with a property called Country.

Create another property of type List that stores items of data type [SelectListItem](#). Call this property Countries and give it a setter and a getter.

2.

Each instantiation of a SelectListItem will take a few optional properties. You'll be using the Text and Value properties in this case.

There are two ways to create an instance of a SelectListItem:

1. By creating a property and assigning it values:

```
new SelectListItem { Value = "1", Text = "Admin" },
```

2. By giving it two parameters, where the first one is the Value and the second is the Text:

```
new SelectListItem("1", "Admin")
```

In your OnGet() method, assign the property Countries to a new List of 5 separate SelectListItem instances.

Set their Value and Text as follows:

Value	Text
"AR"	"Argentina"
"FR"	"France"
"BR"	"Brazil"
"GER"	"Germany"
"CHI"	"China"

3.

Now that you have the property Countries set up with some values, you can use those values with the Select tag helper in the view page. Remove all the hardcoded option tags.

4.

Within the select tag in **Explorella.cshtml**, we'll be making use of two attributes.

- asp-for: Refers to the model property that will be assigned the selected value.
- asp-items: The collection of SelectListItem that provides the options for the list.

Add those two attributes and assign them to the appropriate property and Items.

The countries should now populate on the drop-down menu when selecting your next destination! Imagine we had a list of 50 countries? That would be a lot of <option> tags to write! With the use of Tag Helpers, we were able to consolidate our code from 8 lines, down to 2!

Explorella.cshtml

@page

@model MyApp.Namespace.ExplorellaModel

@{

}

```
<div class="jumbotron jumbotron text-white" style="background-image:
url(https://c1.staticflickr.com/1/725/20835879729_66b87b0759_b.jpg);
background-repeat: no-repeat;
```

```
background-position: center center;
```

```
background-size: cover;">
```

```
<div class="container">
```

```
<h2 class="text-center display-3 mb-4">Welcome to Explorella!</h2>
```

```
</div>
```

```
</div>
```

```
<h1 class="display-5">Pick your next destination!</h1>
```

```
<form class="form-inline">
```

```
<label class="my-1 mr-2" for="inlineFormCustomSelectPref">Preference</label>
```

```
<!-- Select Tag Helper -->
```

```
<select class="custom-select my-1 mr-sm-2">
```

```
<option selected>Choose...</option>
```

```
<option value="AR">Argentina</option>
```

```
<option value="FR">France</option>
```

```

    <option value="BR">Brazil</option>
    <option value="GER">Germany</option>
    <option value="CHI">China</option>
</select>

<div class="custom-control custom-checkbox my-1 mr-sm-2">
    <input type="checkbox" class="custom-control-input"
id="customControlInline">
    <label class="custom-control-label" for="customControlInline">Remember my
preference</label>
</div>

<button type="submit" class="btn btn-primary my-1">Submit</button>
</form>

```

Explorella.cshtml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MyApp.Namespace
{

```

```
public class ExplorellaModel : PageModel
{
    public string Country { get; set; }
    public void OnGet()
    {
    }
}
}
```

OUTPUT



Pick your next destination!

Preference

☐ Remember my preference

Submit

18.8 Review

We learned a lot in the lesson! Now we know the relationship between the page model and our view page and how to attach functionality. Let's summarize what we've learned so far:

- In ASP.NET Razor Pages, a view page (.cshtml file) is attached to a page model (.cs file) by the @model directive. Any members defined in the page model class are accessible on the Razor page.
- In ASP.NET, loosely typed data can be passed from a controller/page model to a shared view/Razor page using the ViewData property.
- In ASP.NET, a shared layout page can be used by multiple views/Razor pages in order to reduce code duplication. The layout page must use the @RenderBody() method to render each view's specific content. Each view can specify a layout by assigning the Layout property. Although not covered in this lesson, layouts can also optionally refer to other sections by calling the @RenderSection() method. This method allows you to carve out sections of the page, and then allow the main body of the content to emerge naturally wherever the @RenderBody() declaration is placed. For more information on @RenderSection(), [refer to its documentation](#).
- In ASP.NET, partials can be used to render HTML output within another markup file's rendered output. They're helpful to reduce repetitive code.
- In ASP.NET, Razor pages support Tag Helpers, which are used to add server-side attributes to HTML elements.

For more information regarding Razor Pages reference the [main documentation](#).

18.8 Review Instructions

Provided is the generated template from ASP.NET when creating a new Razor Pages app. Use the model page, **Index.cshtml.cs**, as a playground to create new properties and methods, and display them on the view page!

Index.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;
```

```
namespace demo_app.Pages
{
    public class IndexModel : PageModel
    {
        public string Greet { get; set; }

        public void OnGet()
        {
            Greet = "Hey there!";
        }
    }
}
```

Index.cshtml

```
@page
```



```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="text-center">
```

```
    <h1 class="display-4">@Model.Greet</h1>
```

```
    <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building  
Web apps with ASP.NET Core</a>.</p>
```

```
</div>
```

18. Quiz

1. What are Tag Helpers?

- a) Tag Helpers are files used to break up large and complex pages into smaller units.
- b) Tag Helpers are reserved methods by Razor to handle HTTP requests.
- c) Tag Helpers allow you to replace Razor's syntax with a more natural HTML-like syntax to run server-side code.
- d) Tag Helpers are reserved Razor directives used to access properties and methods from the model.

2. Fill in the code in order to iterate over all the items in the list fruits.

```
@{
```

```
    List<string> fruits = new List<string>()
```

```
{
```

```
"Apple",  
"Pear",  
"Banana",  
"Grapes"  
};  
}
```

<p>My favorite fruits are:</p>


```
@for(int i = ____; i < fruits.____; i++)  
{  
  @fruits[i]  
}
```


- Count
- Size
- 1
- 0
- Length

Click or drag and drop to fill in the blank

3. This Layout page currently defines a header and footer. Fill in the code so that it renders a content page in-between the two.

<p>This is header text</p>

<p>© 2020 Codecademy. All rights reserved.</p>

- @RenderBody()
- @RenderSection()

- @RenderContent()

Click or drag and drop to fill in the blank

4. What will be displayed on the view page given the following code?

```
@page
```

```
@{  
    int i = 0;  
}
```

```
@while (i < 3)  
{  
    i += 1;  
    <p>@i</p>  
}
```

a) 1

2

3

b) 0

1

2

c) 0

0

0

d) 1

1

1

5. What is Razor Pages?

- a) Razor Pages is a programming language developed to create dynamic web pages.

- b) Razor is a language to query and maintain data in relational database management systems.
- c) Razor Pages is a framework based on ASP.NET Core, it consists of Razor markup, C#, and HTML.
- d) Razor Pages is a templating engine, which uses C++ and HTML to develop “page-based” focused websites.

6. If this code was included in a Razor view page, how would you display the value of the Title property inside a <header> tag?

```
@{
    ViewData["Title"] = "About";
}
```

- a) <header>@ViewData["Title"]</header>
- b) <header>@{ViewData["Title"]};</header>
- c) <header>@ViewData[Title]</header>
- d) <header>ViewData["Title"]</header>

7. Use a foreach loop to iterate over animals and display each element in an tag.

```
@{
    List<string> animals = new List<string>()
    {
        "Dog",
        "Cat",
        "Cow",
        "Mouse"
    };
}
```

```
<ul>
    @foreach(string _____ in animals)
    {
```

```
<li>_____</li>
}
</ul>
```

- creature
- animal
- @animals
- @animal

Click or drag and drop to fill in the blank

8. Fill in Profile.cshtml so that it uses the model defined in Profile.cshtml.cs.

Profile.cshtml.cs:

```
namespace App.Pages
{
    public class ProfileModel : PageModel
    {
        ...
    }
}
```

Profile.cshtml:

@page

- Profile.cshtml.cs
- @Model
- Profile
- @RazorModel
- @model

- ProfileModel

Click or drag and drop to fill in the blank

9. In order for a file to act as a Razor view page, what directive must be added on the first line of the file?

- a) @model
- b) @page
- c) @razor
- d) @using

10. A model, UserModel, with a property named FullName is passed into a view page. How would you access and display that property?

@page

@model UserModel

<div>

<h1>Welcome back, _____ </h1>

</div>

- @Model.FullName
- @model.FullName
- @UserModel.FullName
- @FullName

Click or drag and drop to fill in the blank

11. What is the resulting HTML when this code runs on a view page?

@{ int num = 3; }

@if(num < 5)

```

{
  <h5>The number is less than 5!</h5>
}
else
{
  <h5>The number is more than 5!</h5>
}
<h5>The number is @num</h5>

```

- a) The number is less than 5!
The number is num
- b) The number is less than 5!
The number is @num
- c) The number is less than 5!
The number is 3
- d) The number is more than 5!
The number is num

12. Complete the switch case below. Each case will be compared to the value of day.

```

@{int day = 1;}
@switch (____) {
  ____ 1:
    <p>Saturday</p>
    break;
  case 2:
    <p>Sunday</p>
    break;
  default:
    <p>Weekday</p>
    ____;
}

```

- break

- case
- end
- day
- int day

Click or drag and drop to fill in the blank

13. What is the difference between `_ViewImports.cshtml` and `_ViewStart.cshtml`?

a) `_ViewImports.cshtml` is used to provide using statements throughout all your view pages.

`_ViewStart.cshtml` is used for code to be executed as soon as a page is rendered.

b) `_ViewImports.cshtml` is used to provide using statements available throughout all your view pages.

`_ViewStart.cshtml` is a file used to store all the partials you'll be using throughout the application.

c) `_ViewImports.cshtml` is used to provide using statements throughout all your view pages.

`_ViewStart.cshtml` is used to set up the layout page only.

d) `_ViewImports.cshtml` is used to store code to be executed as soon as a page is rendered.

`_ViewStart.cshtml` is used to provide using statements throughout all your views.

Add a Razor Page in Visual Studio

Apply your knowledge of Razor Syntax in Visual Studio

In this video, you'll learn how to add pages to an existing ASP.NET Razor Pages application on your own computer. If you don't already have Visual Studio, follow [this set-up article](#).

<https://youtu.be/wHZ4FvoDm8s>

Here's a quick recap of the skills covered:

- Add a Razor Page (with page model) to an existing app using the “Add...” feature in Visual Studio
- Use default routing to access the newly created page
- Find and edit the layout file
- Pass data from a Razor Page to the layout file using **ViewData**

When you feel comfortable with these, you're ready to move on!

19.PAGE MODEL BASICS

19.1 Re-Introduction To Page Models

When you want to take your website to the next level, you need to do more than display text and images. You need to handle data and route requests. That's where page models come in!

Let's review some points about ASP.NET Core and the page-focused Razor Pages framework:

- In view pages (**.cshtml** files), Razor syntax lets us mix C# and HTML code in one file so that we can make flexible templates.
- In page models (**.cshtml.cs** files), we use C# code and special PageModel classes to maintain data and handle different actions, like GET and POST requests.

This lesson will focus on page models and some of the things we can do with them, such as:

- Respond to requests using the OnGet() and OnPost() methods
- Respond to requests using the asynchronous OnGetAsync() and OnPostAsync() methods
- Use model binding to share data between view pages and page models

19.1 Re-Introduction To Page Models Instructions

By the end of this lesson, you'll understand nearly all of the code used in this page model. Run the code to see it working.

Remember that, by default, the name of the file determines its URL segment.
Check
that **Index.cshtml** matches localhost:8000/ and **Archive.cshtml** matches localhost:
8000/Archive.

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
```

```
    <div id="request-info">
```

```
        <em>RequestMethod</em>: @Model.RequestMethod
```

```
<em>RequestInfo</em>: @Model.RequestValues
```

```
    </div>
```

```
    <div id="jumbotron-main-text">
```

```
        <h1>Chef's Blog</h1>
```

```
        <p>A place for the best recipes!</p>
```

```
    </div>
```

```
</div>
```

```
<div class="row">
```

```
    <div class="col">
```

```
        <h1>Write a new post</h1>
```

```
<hr>
<!-- Form start -->
<form method="post">
  <div class="form-group">
    <label for="Title">Title</label>
    <!-- Use asp-for here -->
    <input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">
  </div>
  <div class="form-group">
    <label for="Date">Date</label>
    <!-- Use asp-for here -->
    <input type="date" class="form-control" id="Date" name="Date">
  </div>
  <div class="form-group">
    <label for="Body">Your post</label>
    <!-- Use asp-for here -->
    <textarea class="form-control" id="Body" name="Body"
rows="3"></textarea>
  </div>
  <button type="submit" class="btn" id="submit">Submit</button>
</form>
<!-- Form end -->
</div>
```

```
<div class="col">
  <h1>Recent posts</h1>
  <hr>
  <!-- Page model properties displayed here -->
  <div class="blog-post">
    <h2 class="blog-post-title">@Model.Title</h2>
    <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
    <p>@Model.Body</p>
  </div>
</div>
</div>
```

Index.cshtml.cs

```
using System;
using System.IO;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages
{
    public class IndexModel : PageModel
    {
        public string RequestMethod
```

```
{ get; set; }
```

```
public string RequestValues
```

```
{ get; set; }
```

```
[BindProperty]
```

```
public string Title { get; set; }
```

```
[BindProperty]
```

```
public DateTime Date { get; set; }
```

```
[BindProperty]
```

```
public string Body { get; set; }
```

```
public async Task OnGet()
```

```
{
```

```
    // For debugging
```

```
    RequestMethod = "GET";
```

```
    RequestValues = "n/a";
```

```
    // Assign property values here
```

```
    Title = "Cuban Midnight Sandwich";
```

```
    Date = new DateTime(2000, 3, 24);
```

```
    Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'
```

```
    It makes a wonderful dinner sandwich because it is served hot. A nice side dish is  
    black bean soup or black beans and rice, and plaitain chips.";
```

```
// Asynchronous task
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
{
    await writer.WriteLineAsync($"OnGetAsync() called at {DateTime.Now}.");
}
}
```

```
// Define OnPostAsync() here
```

```
public async Task OnPostAsync()
```

```
{
```

```
    // For debugging
```

```
    RequestMethod = "POST";
```

```
    RequestValues = "n/a";
```

```
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
```

```
{
```

```
    await writer.WriteLineAsync($"OnPostAsync() called at {DateTime.Now}.");
```

```
}
```

```
}
```

```
// For debugging
```

```
private string GetFormValues(bool ignoreRequestVerificationToken = true)
```

```
{
```

```
    string formData = "";
```

```

string separator = " | ";

foreach (var pair in this.Request.Form)
{
    if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")
    {
        continue;
    }
    else
    {
        formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
    }
}

if (formData.EndsWith(separator))
{
    formData = formData.Substring(0, formData.Length - separator.Length);
}

return formData;
}
}
}

```


@page

@model ArchiveModel

@{

}

<div class="jumbotron jumbotron-fluid" id="archive-jumbotron">

<div id="jumbotron-main-text">

<h1>Archive</h1>

<p>The best past recipes!</p>

</div>

</div>

<div class="row">

<div class="col">

<!-- Page model properties displayed here -->

<div class="blog-post">

<h3 class="blog-post-title">@Model.Displayed.Title</h3>

<p class="blog-post-meta">Posted on
@Model.Displayed.Date.ToShortDateString()</p>

<p>@Model.Displayed.Body</p>

</div>

</div>

<div class="col">

<h1>Older posts</h1>

<hr>

```

    @for (int i = 0; i < Model.Archive.Count; i++)
    {
        <div>
            <h3>
                <a asp-page="/Archive" asp-route-index="@i">@Model.Archive[i].Title</a>
            </h3>
            <p>Posted on @Model.Archive[i].Date.ToShortDateString()</p>
        </div>
    }
</div>
</div>

```

Archive.cshtml

```

@page
@model ArchiveModel
@{
}

<div class="jumbotron jumbotron-fluid" id="archive-jumbotron">
    <div id="jumbotron-main-text">
        <h1>Archive</h1>
        <p>The best past recipes!</p>
    </div>
</div>

```

```

<div class="row">
  <div class="col">
    <!-- Page model properties displayed here -->
    <div class="blog-post">
      <h3 class="blog-post-title">@Model.Displayed.Title</h3>
      <p class="blog-post-meta">Posted on
@Model.Displayed.Date.ToShortDateString()</p>
      <p>@Model.Displayed.Body</p>
    </div>
  </div>
  <div class="col">
    <h1>Older posts</h1>
    <hr>
    @for (int i = 0; i < Model.Archive.Count; i++)
    {
      <div>
        <h3>
          <a asp-page="/Archive" asp-route-index="@i">@Model.Archive[i].Title</a>
        </h3>
        <p>Posted on @Model.Archive[i].Date.ToShortDateString()</p>
      </div>
    }
  </div>
</div>

```

Archive.cshtml.cs

```
using System;

using System.Collections.Generic;

using System.Linq;

using Microsoft.AspNetCore.Mvc.RazorPages;

using Blog.Models;

namespace Blog.Pages
{
    public class ArchiveModel : PageModel
    {
        public readonly List<BlogPost> Archive = new List<BlogPost>
        {
            new BlogPost("One Pot Thai-Style Rice", new DateTime(2020, 1, 24), "This is a famous dish popularized in Thailand. Although the recipe varies from cook to cook and region to region, this is a good attempt at recreating what I ate from Thai-owned hole-in-the-wall restaurants in Okinawa, Japan. Key to the flavor are the sugar levels, unsalted peanuts, peanut oil, and either oyster or fish sauce."),
            new BlogPost("Balsamic Glazed Salmon Fillets", new DateTime(2020, 1, 20), "A glaze featuring balsamic vinegar, garlic, honey, white wine and Dijon mustard makes baked salmon fillets extraordinary."),
            new BlogPost("Spicy Garlic Lime Chicken", new DateTime(2019, 9, 4), "A delightful chicken dish with a little spicy kick. Serve with rice and your favorite vegetable."),
            new BlogPost("Mushroom Pork Chops", new DateTime(2019, 6, 14), "Quick and easy, but very delicious. One of my family's favorites served over brown rice."),
        }
    }
}
```

```
};
```

```
public BlogPost Displayed { get; set; }
```

```
public void OnGet(int index)
```

```
{
```

```
    if (index >= Archive.Count || index < 0)
```

```
    {
```

```
        Displayed = new BlogPost("n/a", new DateTime(0001, 01, 01), "n/a");
```

```
    }
```

```
    else
```

```
    {
```

```
        Displayed = Archive[index];
```

```
    }
```

```
}
```

```
}
```

```
}
```

OUTPUT

RequestMethod: GET RequestInfo: n/a

Chef's Blog

A place for the best recipes!



Write a new post

Title

Date

Your post

Submit

Recent posts

Cuban Midnight Sandwich

Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight.' It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice, and plaitain chips.

19.2 OnGet()

Page models handle [HTTP requests](#), like GET and POST. We define their response using handler methods.

One of those handler methods is the `OnGet()` method. Whenever a user makes a GET request to a page, the page model's `OnGet()` method is invoked.

For example, if a user navigates to `localhost:8000/Archive` in their browser:

1. The browser sends a GET request to that URL.
2. `localhost:8000/Archive` corresponds to **Archive.cshtml** so it receives the request.

3. **Archive.cshtml** page references the `ArchiveModel`, so that class' `OnGet()` method will be invoked.

It might sound complicated now, but it will feel natural by the end of this lesson. (You can even check this for yourself by exploring the relationship between `localhost:8000/`, **Index.cshtml**, and **Index.cshtml.cs** in the browser and editor.)

By default, an empty `OnGet()` method returns the corresponding view page. In the example above, the `ArchiveModel.OnGet()` corresponds to **Archive.cshtml**.

Since an empty `OnGet()` method has no return statement, the return type is `void`:

```
public void OnGet()  
{ }
```

Within the method we can set values to properties, which can be displayed in the view page.

19.2 `OnGet()` Instructions

1.

In **Index.cshtml.cs**, you can see that the properties for a blog post are defined: `Title`, `Date`, and `Body`.

In **Index.cshtml**, the property values are displayed.

Go to **Index.cshtml.cs** and, within `OnGet()`, set the properties to some placeholder values:

- `Title`: "Cuban Midnight Sandwich"
- `Date`: `new DateTime(2000, 3, 24)`
- `Body`: "This sandwich is called a 'Media Noche' which translates to 'Midnight.'"

When you navigate to `localhost:8000/`, you should see the values displayed.

Index.cshtml

@page

@model IndexModel

@{

 ViewData["Title"] = "Home page";

}

<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">

 <div id="request-info">

 RequestMethod: @Model.RequestMethod

 RequestInfo: @Model.RequestValues

 </div>

 <div id="jumbotron-main-text">

 <h1>Chef's Blog</h1>

 <p>A place for the best recipes!</p>

 </div>

</div>

<div class="row">

 <div class="col">

 <h1>Write a new post</h1>

 <hr>

 <!-- Form start -->

 <!-- Form end -->


```
</div>
```

```
<div class="col">
```

```
<h1>Recent posts</h1>
```

```
<hr>
```

```
<!-- Page model properties displayed here -->
```

```
<div class="blog-post">
```

```
<h2 class="blog-post-title">@Model.Title</h2>
```

```
<p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
```

```
<p>@Model.Body</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

Index.cshtml.cs

```
using System;
```

```
using System.Threading.Tasks;
```

```
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages
```

```
{
```

```
    public class IndexModel : PageModel
```

```
    {
```

```
        public string RequestMethod
```

```
{ get; set; }
```

```
public string RequestValues
```

```
{ get; set; }
```

```
public string Title { get; set; }
```

```
public DateTime Date { get; set; }
```

```
public string Body { get; set; }
```

```
public void OnGet()
```

```
{
```

```
    // For debugging
```

```
    RequestMethod = "GET";
```

```
    RequestValues = "n/a";
```

```
    // Assign property values here
```

```
}
```

```
public void OnPost()
```

```
{
```

```
    // For debugging
```

```
    RequestMethod = "POST";
```

```
    RequestValues = GetFormValues();
```

```

// Assign property values here

}

// For debugging
private string GetFormValues(bool ignoreRequestVerificationToken = true)
{
    string formData = "";
    string separator = " | ";

    foreach (var pair in this.Request.Form)
    {
        if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")
        {
            continue;
        }
        else
        {
            formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
        }
    }

    if (formData.EndsWith(separator))
    {

```

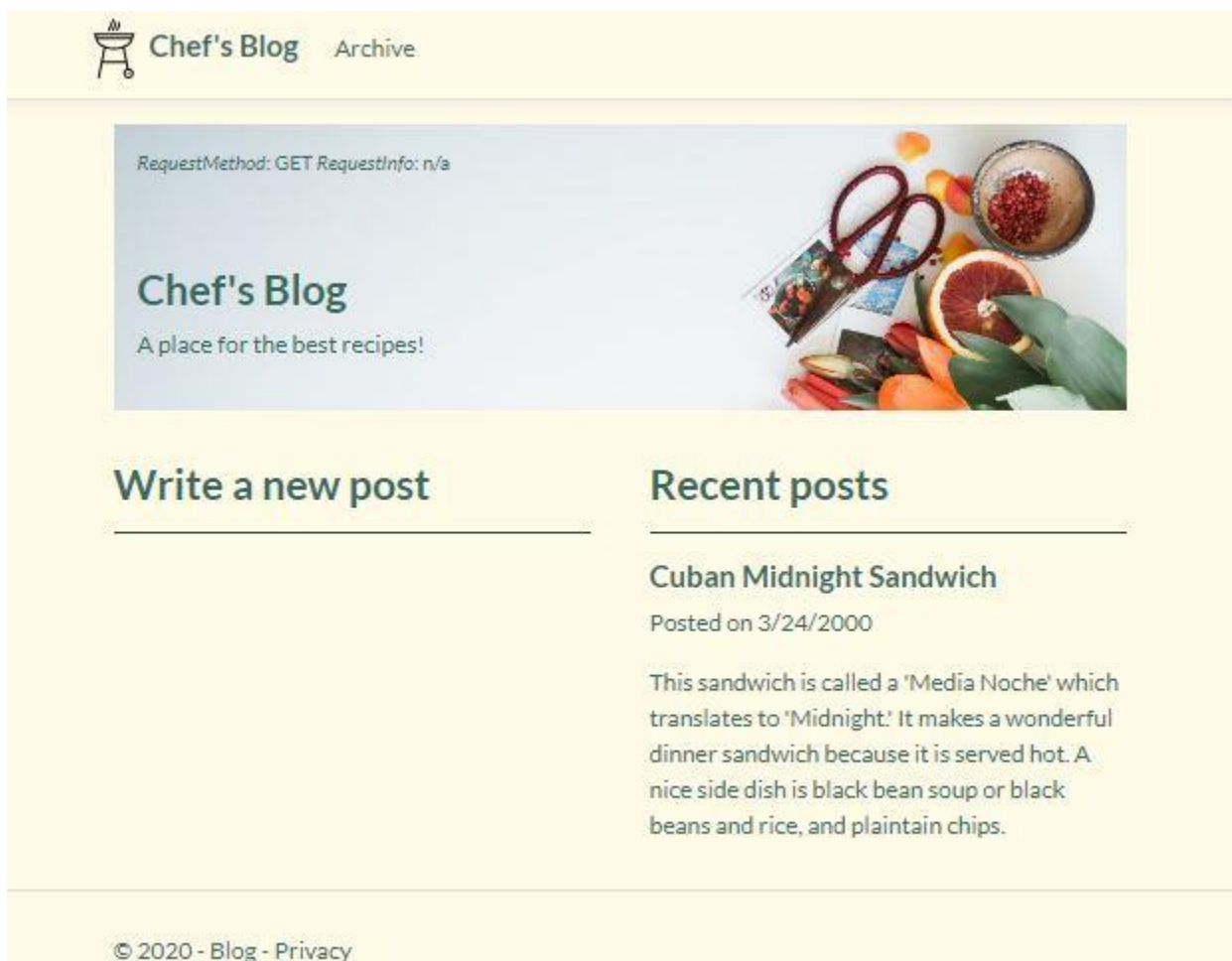
```

        formData = formData.Substring(0, formData.Length - separator.Length);
    }

    return formData;
}
}
}

```

OUTPUT



19.3 OnPost()

The `OnPost()` handler method is invoked when a POST request is sent to a page. This usually happens when a user submits a form (an HTML `<form>` element).

Just like with `OnGet()`, the default behavior of an empty `OnPost()` method is to send the corresponding page. Without a return statement, this method also returns `void`.

```
public void OnPost()
{ }
```

Usually POST requests come with some information in the form of a query string. For example, say that a form at `www.library.com/favorite` asks for a book and an author:

```
<form method="post">
  <div class="form-group">
    <label for="Title">Title</label>
    <input type="text" class="form-control" id="Title" name="Title">
  </div>
  <div class="form-group">
    <label for="Author">Author</label>
    <input type="text" class="form-control" id="Author" name="Author">
  </div>
  <button type="submit" class="btn btn-primary">Submit</button>
</form>
```

Notice that each `<input>` has a `name` attribute — `Title` and `Author`, respectively.

In a browser it would look like:

Title

Title

Author

Author

Submit

If a user provided the input Where The Wild Things Are and Maurice Sendak, the URL would look like this (+ represents a space):

`www.library.com/favorite?Title=Where+The+Wild+Things+Are&Author=Maurice+Sendak`

OnPost() can capture the values in the query string via matching method parameters. To capture the above values, the method would look like this:

```
public void OnPost(string title, string author)
{
    Title = title;
    Author = author;
}
```

Those method parameters are matched case-insensitively; author or Author or aUTHOR would have worked.

19.3 OnPost() Instructions

- 1.

Create a form by copying and pasting this code into **Index.cshtml**:

```
<form method="post">
  <div class="form-group">
    <label for="Title">Title</label>
    <input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">
  </div>
  <div class="form-group">
    <label for="Date">Date</label>
    <input type="date" class="form-control" id="Date" name="Date">
  </div>
  <div class="form-group">
    <label for="Body">Your post</label>
    <textarea class="form-control" id="Body" name="Body" rows="3"></textarea>
  </div>
  <button type="submit" class="btn" id="submit">Submit</button>
</form>
```

The place to paste is denoted with comments.

If done correctly, submitting the form will produce a query string with the values: Title, Date, and Body. For example:

```
localhost:8000?Title=Example+Title&Date=2020-03-
24&Body=just+some+words+here
```

2.

In **Index.cshtml.cs**, modify the `OnPost()` method by adding matching method parameters.

They should match the name attributes used in the `<form>`.

3.

Within the method, assign each value to its corresponding property.

If done correctly, you can submit the form and see your data displayed on the page.

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
```

```
    <div id="request-info">
```

```
        <em>RequestMethod</em>: @Model.RequestMethod
```

```
<em>RequestInfo</em>: @Model.RequestValues
```

```
    </div>
```

```
    <div id="jumbotron-main-text">
```

```
        <h1>Chef's Blog</h1>
```

```
        <p>A place for the best recipes!</p>
```

```
    </div>
```

```
</div>
```

```
<div class="row">
```

```
    <div class="col">
```

```
        <h1>Write a new post</h1>
```

```
        <hr>
```



```
<!-- Form start -->
```

```
<!-- Form end -->
```

```
</div>
```

```
<div class="col">
```

```
<h1>Recent posts</h1>
```

```
<hr>
```

```
<!-- Page model properties displayed here -->
```

```
<div class="blog-post">
```

```
<h2 class="blog-post-title">@Model.Title</h2>
```

```
<p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
```

```
<p>@Model.Body</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

Index.cshtml.cs

```
using System;
```

```
using System.Threading.Tasks;
```

```
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages
```

```
{
```

```

public class IndexModel : PageModel
{
    public string RequestMethod
    { get; set; }

    public string RequestValues
    { get; set; }

    public string Title { get; set; }
    public DateTime Date { get; set; }
    public string Body { get; set; }

    public void OnGet()
    {
        // For debugging
        RequestMethod = "GET";
        RequestValues = "n/a";

        // Assign property values here
        Title = "Cuban Midnight Sandwich";
        Date = new DateTime(2000, 3, 24);

        Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is
black bean soup or black beans and rice, and plaitain chips.";
    }
}

```

```

public void OnPost()
{
    // For debugging
    RequestMethod = "POST";
    RequestValues = GetFormValues();

    // Assign property values here

}

// For debugging
private string GetFormValues(bool ignoreRequestVerificationToken = true)
{
    string formData = "";
    string separator = " | ";

    foreach (var pair in this.Request.Form)
    {
        if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")
        {
            continue;
        }
        else

```

```
{  
    formData += pair.Key + ": " + Request.Form[pair.Key] + separator;  
}  
}  
  
if (formData.EndsWith(separator))  
{  
    formData = formData.Substring(0, formData.Length - separator.Length);  
}  
  
return formData;  
}  
}  
}
```

OUTPUT



RequestMethod: GET RequestInfo: n/a

Chef's Blog

A place for the best recipes!



Write a new post

Title

Date



Your post

Recent posts

Cuban Midnight Sandwich

Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight'. It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice, and plantain chips.

Submit

19.4 Model Binding

In the previous exercise we typed out each parameter name in the form, then copied them over to the OnPost() method, then set them equal to properties. In some cases, we would need a fourth step to convert each value into a valid .NET type...

ASP.NET Core provides a feature that takes care of all that for us: *model binding*. In model binding, data is retrieved from an HTTP request, it is converted into .NET types, and it is stored in corresponding model properties.

To activate model binding in a page model, make sure that the desired property name matches the name attribute in the <form>, then label it with the attribute [BindProperty].

In the view page:

```
<form method="post">
  <div class="form-group">
    <label for="Title">Title</label>
    <input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">
  </div>
  <div class="form-group">
    <label for="Author">Author</label>
    <input type="text" class="form-control" id="Author" name="Author"
placeholder="Author">
  </div>
  <button type="submit" class="btn btn-primary">Submit</button>
</form>
```

In the page model:

```
[BindProperty]
string Title { get; set; }
```

```
[BindProperty]
string Author { get; set; }
```

We can then remove the method parameters and assume that the submitted form values are set to their corresponding properties:

```
// No more method parameters!
// No more Author = author!
public void OnPost()
{ }
```

You might not be familiar with *attributes* in C#, but that's okay. All you need to know is that the line [BindProperty] goes directly above each property.

Technically, you also need to use the Microsoft.AspNetCore.Mvc namespace, but that is included in default ASP.NET templates. If you'd like to learn more, [read about model binding in the Microsoft documentation](#).

19.4 Model Binding Instructions

1.

We'll repeat the same steps as the previous exercise, but this time using model binding. Create a form by copying and pasting this code into **Index.cshtml**:

```
<form method="post">
  <div class="form-group">
    <label for="Title">Title</label>
    <input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">
  </div>
  <div class="form-group">
    <label for="Date">Date</label>
    <input type="date" class="form-control" id="Date" name="Date">
  </div>
  <div class="form-group">
    <label for="Body">Your post</label>
    <textarea class="form-control" id="Body" name="Body" rows="3"></textarea>
  </div>
  <button type="submit" class="btn" id="submit">Submit</button>
</form>
```

2.

Instead of adding method parameters, use model binding:

1. Mark the three properties in **Index.cshtml.cs** with the [BindProperty] attribute.

2. Remove the method parameters from OnPost().
3. Delete the lines assigning the method arguments to properties.
4. Run the code and submit the form. Check that your input shows up in the resulting page.

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
```

```
    <div id="request-info">
```

```
        <em>RequestMethod</em>: @Model.RequestMethod
```

```
<em>RequestInfo</em>: @Model.RequestValues
```

```
    </div>
```

```
    <div id="jumbotron-main-text">
```

```
        <h1>Chef's Blog</h1>
```

```
        <p>A place for the best recipes!</p>
```

```
    </div>
```

```
</div>
```

```
<div class="row">
```

```
    <div class="col">
```



```

<h1>Write a new post</h1>
<hr>
<!-- Form start -->

<!-- Form end -->
</div>

<div class="col">
  <h1>Recent posts</h1>
  <hr>
  <!-- Page model properties displayed here -->
  <div class="blog-post">
    <h2 class="blog-post-title">@Model.Title</h2>
    <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
    <p>@Model.Body</p>
  </div>
</div>
</div>

```

Index.cshtml.cs

```

using System;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

```

```

namespace Blog.Pages
{
    public class IndexModel : PageModel
    {
        public string RequestMethod
        { get; set; }

        public string RequestValues
        { get; set; }

        public string Title { get; set; }
        public DateTime Date { get; set; }
        public string Body { get; set; }

        public void OnGet()
        {
            // For debugging
            RequestMethod = "GET";
            RequestValues = "n/a";

            // Assign property values here
            Title = "Cuban Midnight Sandwich";
            Date = new DateTime(2000, 3, 24);
        }
    }
}

```

Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is
black bean soup or black beans and rice, and plaintain chips.";

}

public void OnPost()

{

// For debugging

RequestMethod = "POST";

RequestValues = GetFormValues();

}

// For debugging

private string GetFormValues(bool ignoreRequestVerificationToken = true)

{

string formData = "";

string separator = " | ";

foreach (var pair in this.Request.Form)

{

if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")

{

continue;

}

```

else
{
    formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
}
}

if (formData.EndsWith(separator))
{
    formData = formData.Substring(0, formData.Length - separator.Length);
}

return formData;
}
}
}

```



Chef's Blog
A place for the best recipes!

Write a new post

Title

Date
 

Your post

Recent posts

Cuban Midnight Sandwich
Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight' It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice, and plaitain chips.

OUTPUT

19.5 asp-for

So far, we've built an HTML form in the view page. When submitted it sends a URL query string to our app according to the name attributes of the form. We captured the query values first using method parameters, then using model binding.

We can make that first step (writing a form) even easier using [Tag Helpers](#). We expect you to be familiar with the general concept of Tag Helpers, but we'll show you a new use for them here.

Instead of using the name and id attributes in each <input> element, we'll use the asp-for attribute:

Take this Input Tag Helper, which uses the asp-for attribute:

```
<input asp-for="Author">
```

It will render as this HTML:

```
<input type="text" name="Author" id="Author">
```

Why is this Tag Helper better than basic HTML? The benefits will become clearer as we create more advanced properties, but:

- It saves you the hassle of writing HTML attributes.
- Integrated Development Environments (IDEs), like Visual Studio, can check for errors before you run the code.
- Changes to the property (in a model) are automatically carried into the view page.
- Advanced settings on properties, such as data validation, are converted into additional HTML attributes. We won't be covering those yet, so we won't see this benefit in this lesson.

19.5 asp-for Instructions

1.

In **Index.cshtml**, edit the form so that it uses asp-for instead of id and name attributes.

Make sure that you change all three <input> tags.

2.

In **Index.cshtml.cs**, label each property with [BindProperty].

Run the code and submit the form. Check that your input shows up in the resulting page.

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
```

```
    <div id="request-info">
```

```
        <em>RequestMethod</em>: @Model.RequestMethod
```

```
<em>RequestInfo</em>: @Model.RequestValues
```

```
    </div>
```

```
<div id="jumbotron-main-text">
```

```
    <h1>Chef's Blog</h1>
```

```

    <p>A place for the best recipes!</p>
</div>
</div>

<div class="row">
  <div class="col">
    <h1>Write a new post</h1>
    <hr>
    <!-- Form start -->
    <form method="post">
      <div class="form-group">
        <label for="Title">Title</label>
        <!-- Use asp-for here -->
        <input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">
      </div>
      <div class="form-group">
        <label for="Date">Date</label>
        <!-- Use asp-for here -->
        <input type="date" class="form-control" id="Date" name="Date">
      </div>
      <div class="form-group">
        <label for="Body">Your post</label>
        <!-- Use asp-for here -->

```

```

        <textarea class="form-control" id="Body" name="Body"
rows="3"></textarea>

        </div>

        <button type="submit" class="btn" id="submit">Submit</button>

    </form>

    <!-- Form end -->
</div>


<div class="col">
    <h1>Recent posts</h1>
    <hr>
    <!-- Page model properties displayed here -->
    <div class="blog-post">
        <h2 class="blog-post-title">@Model.Title</h2>
        <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
        <p>@Model.Body</p>
    </div>
</div>
</div>

```

Index.cshtml.cs

```

using System;

using System.Threading.Tasks;

using Microsoft.AspNetCore.Mvc;

```



```
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages
```

```
{
```

```
    public class IndexModel : PageModel
```

```
    {
```

```
        public string RequestMethod
```

```
        { get; set; }
```

```
        public string RequestValues
```

```
        { get; set; }
```

```
        public string Title { get; set; }
```

```
        public DateTime Date { get; set; }
```

```
        public string Body { get; set; }
```

```
        public void OnGet()
```

```
        {
```

```
            // For debugging
```

```
            RequestMethod = "GET";
```

```
            RequestValues = "n/a";
```

```
            // Assign property values here
```

```
            Title = "Cuban Midnight Sandwich";
```

```
Date = new DateTime(2000, 3, 24);
```

```
Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'  
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is  
black bean soup or black beans and rice, and plaintain chips.";
```

```
}
```

```
public void OnPost()
```

```
{
```

```
    // For debugging
```

```
    RequestMethod = "POST";
```

```
    RequestValues = GetFormValues();
```

```
}
```

```
    // For debugging
```

```
private string GetFormValues(bool ignoreRequestVerificationToken = true)
```

```
{
```

```
    string formData = "";
```

```
    string separator = " | ";
```

```
    foreach (var pair in this.Request.Form)
```

```
    {
```

```
        if (ignoreRequestVerificationToken && pair.Key ==  
        "__RequestVerificationToken")
```

```
        {
```

```
            continue;
```

```
        }
```

```
        else
```

```
        {
```

```

        formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
    }
}
if (formData.EndsWith(separator))
{
    formData = formData.Substring(0, formData.Length - separator.Length);
}
return formData;
}
}
}

```

OUTPUT

RequestMethod: GET RequestInfo: n/a

Chef's Blog

A place for the best recipes!



Write a new post

Title

Date

Your post

Recent posts

Cuban Midnight Sandwich

Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight.' It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice, and plaitain chips.

19.6 asp-route-{value}

The Input Tag Helper, with `asp-for`, allows us to easily create a form that submits data with a POST request.

What Tag Helper and attribute(s) would help us with GET requests?

Similarly, the Anchor Tag Helper, with `asp-page` and `asp-route-{value}` attributes, allows us to create `<a>` links that submit data with a GET request.

- `asp-page` — Sets an anchor tag's `href` attribute value to a specific page.
- `asp-route-{value}` — Adds route values to an `href`. The `{value}` placeholder is interpreted as a potential route parameter.

This markup in a view page...

```
<a asp-page="./Authors" asp-route-fullname="Roald Dahl">Roald</a>
```

...would render as this HTML...

```
<a href="./Authors?fullname=Roald+Dahl">Roald</a>
```

Like before, we can capture the query values via method parameters:

```
public void OnGet(string fullname)
{ }
```

Technically we could also use model binding with the attribute `[BindProperty(SupportsGet=true)]` but we generally avoid that because allowing users to set values in GET requests can be dangerous. If GET requests can set attributes, then it would be too easy to mistakenly edit our app's data by browsing to a URL. We prefer to set attributes in POST requests because they have built-in security from ASP.NET.

To recap: Anchor Tag Helpers generate GET requests. They use `asp-page` and `asp-route-{value}`:

```
<a asp-page="./Authors" asp-route-fullname="Roald Dahl">Roald</a>
```

Input Tag Helpers (along with a form) generate POST requests. They use `asp-for`:

<input asp-for="Title">

19.6 asp-route-{value} Instructions

1.

In **Archive.cshtml**, the <a> links just go to /Archive.

Use Anchor Tag Helpers to formulate the correct, unique link for each post:

- Replace the href attribute with an asp-page attribute of the same value
- Add an asp-route-index and use the @for loop index as the value

If done correctly, the “One Pot Thai-Style Rice” should link to /Archive?index=0, “Balsamic Glazed Salmon Fillets” should link to /Archive?index=1, etc.

2.

We need to access that index value in the page model.

In **Archive.cshtml.cs**, add an index method parameter to OnGet().

3.

Add this code to the end of OnGet(). It uses the index to search the Archive:

```
if (index >= Archive.Count || index < 0)
{
    Displayed = new BlogPost("n/a", new DateTime(0001, 01, 01), "n/a");
}
else
{
    Displayed = Archive[index];
}
```

Try clicking the links. They should bring up different blog posts.

Archive.cshtml

@page

@model ArchiveModel

@{

}

<div class="jumbotron jumbotron-fluid" id="archive-jumbotron">

<div id="jumbotron-main-text">

<h1>Archive</h1>

<p>The best past recipes!</p>

</div>

</div>

<div class="row">

<div class="col">

<!-- Page model properties displayed here -->

<div class="blog-post">

<h3 class="blog-post-title">@Model.Displayed.Title</h3>

<p class="blog-post-meta">Posted on
@Model.Displayed.Date.ToShortDateString()</p>

<p>@Model.Displayed.Body</p>

</div>

</div>

<div class="col">

<h1>Older posts</h1>

<hr>

```

    @for (int i = 0; i < Model.Archive.Count; i++)
    {
        <div>
            <h3>
                <!-- Use asp-page and asp-route-index here -->
                <a href="/Archive">@Model.Archive[i].Title</a>
            </h3>
            <p>Posted on @Model.Archive[i].Date.ToShortDateString()</p>
        </div>
    }
</div>
</div>

```

Archive.cshtml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Blog.Models;

namespace Blog.Pages
{
    public class ArchiveModel : PageModel
    {

```

```

public readonly List<BlogPost> Archive = new List<BlogPost>
{
    new BlogPost("One Pot Thai-Style Rice", new DateTime(2020, 1, 24), "This is a
famous dish popularized in Thailand. Although the recipe varies from cook to cook
and region to region, this is a good attempt at recreating what I ate from Thai-
owned hole-in-the-wall restaurants in Okinawa, Japan. Key to the flavor are the
sugar levels, unsalted peanuts, peanut oil, and either oyster or fish sauce."),
    new BlogPost("Balsamic Glazed Salmon Fillets", new DateTime(2020, 1, 20), "A
glaze featuring balsamic vinegar, garlic, honey, white wine and Dijon mustard
makes baked salmon fillets extraordinary."),
    new BlogPost("Spicy Garlic Lime Chicken", new DateTime(2019, 9, 4), "A
delightful chicken dish with a little spicy kick. Serve with rice and your favorite
vegetable."),
    new BlogPost("Mushroom Pork Chops", new DateTime(2019, 6, 14), "Quick
and easy, but very delicious. One of my family's favorites served over brown
rice."),
};

public BlogPost Displayed { get; set; }

public void OnGet()
{
    Displayed = Archive[0];
}
}

```

OUTPUT



One Pot Thai-Style Rice

Posted on 1/24/2020

This is a famous dish popularized in Thailand. Although the recipe varies from cook to cook and region to region, this is a good attempt at recreating what I ate from Thai-owned hole-in-the-wall restaurants in Okinawa, Japan. Key to the flavor are the sugar levels, unsalted peanuts, peanut oil, and either oyster or fish sauce.

Older posts

One Pot Thai-Style Rice

Posted on 1/24/2020

Balsamic Glazed Salmon Fillets

Posted on 1/20/2020

Spicy Garlic Lime Chicken

Posted on 9/4/2019

Mushroom Pork Chops

Posted on 6/14/2019

19.7 OnGetAsync()

An asynchronous operation is one that allows the computer to “move on” to other tasks while waiting for the asynchronous operation to complete.

Asynchronous programming means that time-consuming operations don’t have to bring everything else in our programs to a halt.

There are countless examples of asynchronicity in our everyday lives. Cooking, for example, involves asynchronous operations such as a toaster toasting our bread or a rice cooker making our rice. While we wait on the completion of those operations, we’re free to do other tasks.

Similarly, web development makes use of asynchronous operations. Operations like making a network request or querying a database are done asynchronously so that we can work on other tasks.

C# enables this using `async`, `await`, and the `Task` type.

For example, this code uses an asynchronous operation, `ReadLineAsync()`, to read from the file **storage.txt**. We use the keyword `await` to tell our app to “wait” for that operation to complete:

```
if (System.IO.File.Exists("storage.txt"))
{
    using (StreamReader reader = System.IO.File.OpenText("storage.txt"))
    {
        string content = await reader.ReadToEndAsync();
    }
}
```

To do this within a request handler like `OnGet()`, we have to label it as an asynchronous method using `async`, switch the return type to `Task`, and rename it `OnGetAsync()`:

```
public async Task OnGetAsync()
{
    if (System.IO.File.Exists("storage.txt"))
    {
        using (StreamReader reader = System.IO.File.OpenText("storage.txt"))
        {
            string content = await reader.ReadToEndAsync();
        }
    }
}
```

The last step — renaming it to `OnGetAsync()` — is optional but conventional.

In summary, if you are performing some asynchronous task within your GET handler method:

1. Add `async` to the signature

2. Change the return type to Task
3. Rename the method to OnGetAsync()

You cannot have both OnGet() and OnGetAsync(). Use one or the other.

By the way, our code used using, which we won't require you to learn, but if you're interested, [read this Microsoft article on using](#).

19.7 OnGetAsync() Instructions

1.

In **Index.cshtml.cs**, the app isn't working because we are trying to do an asynchronous action within a synchronous method (OnGet()).

Run the code to see the error.

2.

Fix the code by adding an async prefix, changing the return type to Task, and renaming the method to OnGetAsync().

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home page";
```

```
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
```

```
    <div id="request-info">
```

```
<em>RequestMethod</em>: @Model.RequestMethod  
<em>RequestInfo</em>: @Model.RequestValues
```

```
</div>
```

```
<div id="jumbotron-main-text">
```

```
<h1>Chef's Blog</h1>
```

```
<p>A place for the best recipes!</p>
```

```
</div>
```

```
</div>
```

```
<div class="row">
```

```
<div class="col">
```

```
<h1>Write a new post</h1>
```

```
<hr>
```

```
<!-- Form start -->
```

```
<form method="post">
```

```
<div class="form-group">
```

```
<label for="Title">Title</label>
```

```
<!-- Use asp-for here -->
```

```
<input type="text" class="form-control" id="Title" name="Title"  
placeholder="Title">
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="Date">Date</label>
```

```
<!-- Use asp-for here -->
```

```
<input type="date" class="form-control" id="Date" name="Date">
```

```

</div>
<div class="form-group">
  <label for="Body">Your post</label>
  <!-- Use asp-for here -->
  <textarea class="form-control" id="Body" name="Body"
rows="3"></textarea>
</div>
<button type="submit" class="btn" id="submit">Submit</button>
</form>
<!-- Form end -->
</div>

<div class="col">
  <h1>Recent posts</h1>
  <hr>
  <!-- Page model properties displayed here -->
  <div class="blog-post">
    <h2 class="blog-post-title">@Model.Title</h2>
    <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
    <p>@Model.Body</p>
  </div>
</div>
</div>

```

Index.cshtml.cs

```
using System;  
using System.IO;  
using System.Threading.Tasks;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages  
{  
    public class IndexModel : PageModel  
    {  
        public string RequestMethod  
        { get; set; }  
  
        public string RequestValues  
        { get; set; }  
  
        [BindProperty]  
        public string Title { get; set; }  
  
        [BindProperty]  
        public DateTime Date { get; set; }  
  
        [BindProperty]  
        public string Body { get; set; }  
    }  
}
```

```

public void OnGet()
{
    // For debugging
    RequestMethod = "GET";
    RequestValues = "n/a";

    // Assign property values here
    Title = "Cuban Midnight Sandwich";
    Date = new DateTime(2000, 3, 24);
    Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is
black bean soup or black beans and rice, and plaintain chips.";

    // Asynchronous task
    using (StreamWriter writer = new StreamWriter("log.txt", append: true))
    {
        await writer.WriteLineAsync($"OnGetAsync() called at {DateTime.Now}.");
    }
}

public void OnPost()
{
    // For debugging
    RequestMethod = "POST";

```

```

    RequestValues = GetFormValues();
}

// For debugging
private string GetFormValues(bool ignoreRequestVerificationToken = true)
{
    string formData = "";
    string separator = " | ";

    foreach (var pair in this.Request.Form)
    {
        if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")
        {
            continue;
        }
        else
        {
            formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
        }
    }

    if (formData.EndsWith(separator))
    {
        formData = formData.Substring(0, formData.Length - separator.Length);
    }
}

```



```

    }

    return formData;
}
}
}

```

OUTPUT


Chef's Blog
[Archive](#)

RequestMethod: GET RequestInfo: n/a



Chef's Blog

A place for the best recipes!

Write a new post

Title

Date

Your post

Recent posts

Cuban Midnight Sandwich

Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight.' It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice,

19.8 OnPostAsync()

Just as `OnGet()` has an asynchronous version, `OnGetAsync()`, `OnPost()` has its own asynchronous version named `OnPostAsync()`.

The same `async/await` / `Task` concepts apply to this method:

- Mark the method with `async`
- Define the return type as `Task`
- Rename it `OnPostAsync()`

19.8 `OnPostAsync()` Instructions

1.

In **`Index.cshtml.cs`**, define an empty `OnPostAsync()` method.

2.

In the body of the method, use the below code to asynchronously write to a text file:

```
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
{
    await writer.WriteLineAsync($"OnPostAsync() called at {DateTime.Now}.");
}
```

`Index.cshtml.cs`

```
using System;
using System.IO;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```

namespace Blog.Pages
{
    public class IndexModel : PageModel
    {
        public string RequestMethod
        { get; set; }

        public string RequestValues
        { get; set; }

        [BindProperty]
        public string Title { get; set; }
        [BindProperty]
        public DateTime Date { get; set; }
        [BindProperty]
        public string Body { get; set; }

        public async Task OnGet()
        {
            // For debugging
            RequestMethod = "GET";
            RequestValues = "n/a";

            // Assign property values here

```

```
Title = "Cuban Midnight Sandwich";
```

```
Date = new DateTime(2000, 3, 24);
```

```
Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'  
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is  
black bean soup or black beans and rice, and plaintain chips.";
```

```
// Asynchronous task
```

```
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
```

```
{
```

```
    await writer.WriteLineAsync($"OnGetAsync() called at {DateTime.Now}.");
```

```
}
```

```
}
```

```
// Define OnPostAsync() here
```

```
// For debugging
```

```
private string GetFormValues(bool ignoreRequestVerificationToken = true)
```

```
{
```

```
    string formData = "";
```

```
    string separator = " | ";
```

```
    foreach (var pair in this.Request.Form)
```

```
{
```

```

        if (ignoreRequestVerificationToken && pair.Key ==
"__RequestVerificationToken")
        {
            continue;
        }
        else
        {
            formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
        }
    }

    if (formData.EndsWith(separator))
    {
        formData = formData.Substring(0, formData.Length - separator.Length);
    }

    return formData;
}
}
}

```

Index.cshtml

@page

@model IndexModel

@{

```
ViewData["Title"] = "Home page";  
}
```

```
<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">  
  <div id="request-info">  
    <em>RequestMethod</em>: @Model.RequestMethod  
    <em>RequestInfo</em>: @Model.RequestValues  
  </div>  
  <div id="jumbotron-main-text">  
    <h1>Chef's Blog</h1>  
    <p>A place for the best recipes!</p>  
  </div>  
</div>
```

```
<div class="row">  
  <div class="col">  
    <h1>Write a new post</h1>  
    <hr>  
    <!-- Form start -->  
    <form method="post">  
      <div class="form-group">  
        <label for="Title">Title</label>  
        <!-- Use asp-for here -->  
        <input type="text" class="form-control" id="Title" name="Title"  
placeholder="Title">
```

```

</div>
<div class="form-group">
  <label for="Date">Date</label>
  <!-- Use asp-for here -->
  <input type="date" class="form-control" id="Date" name="Date">
</div>
<div class="form-group">
  <label for="Body">Your post</label>
  <!-- Use asp-for here -->
  <textarea class="form-control" id="Body" name="Body"
rows="3"></textarea>
</div>
<button type="submit" class="btn" id="submit">Submit</button>
</form>
<!-- Form end -->
</div>

<div class="col">
  <h1>Recent posts</h1>
  <hr>
  <!-- Page model properties displayed here -->
  <div class="blog-post">
    <h2 class="blog-post-title">@Model.Title</h2>
    <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
    <p>@Model.Body</p>

```

</div>

</div>

</div>

OUTPUT



Chef's Blog
A place for the best recipes!

Write a new post

Title

Date
 

Your post

Submit

Recent posts

Cuban Midnight Sandwich
Posted on 3/24/2000

This sandwich is called a 'Media Noche' which translates to 'Midnight.' It makes a wonderful dinner sandwich because it is served hot. A nice side dish is black bean soup or black beans and rice, and plantain chips.

19.9 Review

Well done! We've covered a lot of page model concepts. Let's review:

- When a GET request is made to a page, the OnGet() or OnGetAsync() method from its page model is invoked.

- When a POST request is made to a page, the `OnPost()` or `OnPostAsync()` method from its page model is invoked.
- URL query string values can be accessed via a method's parameters.
- URL query string values can also be accessed via model binding, which is activated using `[BindProperty]`.
- The `asp-for` attribute in Input Tag Helpers connects an `<input>` element to a model property.
- The `asp-route-{value}` attribute in Anchor Tag Helpers adds additional information to a link.
- Synchronous handler methods, like `OnGet()` and `OnPost()`, with no return statement will have a return type of `void`.
- Asynchronous handler methods, like `OnGetAsync()` and `OnPostAsync()`, with no return statement will have a return type of `Task`.

If you'd like to see other examples of handler methods, look at [the first few sections of this introduction to Razor Pages by Microsoft](#) and check out [the Handler Methods section on Learn Razor Pages](#).

19.9 Review Instructions

Before you move on, make sure you can answer the following questions. You can poke around the app code for hints:

- When are `OnGet()` and `OnPost()` called?
- Within an Input Tag Helper what does the `asp-for` attribute do?
- How is `[BindProperty]` used?

Index.cshtml.cs

```
using System;
```

```
using System.IO;
```

```
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Blog.Pages
{
    public class IndexModel : PageModel
    {
        public string RequestMethod
        { get; set; }

        public string RequestValues
        { get; set; }

        [BindProperty]
        public string Title { get; set; }
        [BindProperty]
        public DateTime Date { get; set; }
        [BindProperty]
        public string Body { get; set; }

        public async Task OnGetAsync()
        {
            // For debugging

```

```

RequestMethod = "GET";
RequestValues = "n/a";

// Assign property values here
Title = "Cuban Midnight Sandwich";
Date = new DateTime(2000, 3, 24);

Body = "This sandwich is called a 'Media Noche' which translates to 'Midnight.'
It makes a wonderful dinner sandwich because it is served hot. A nice side dish is
black bean soup or black beans and rice, and plaitain chips.";

// Asynchronous task
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
{
    await writer.WriteLineAsync($"OnGetAsync() called at {DateTime.Now}.");
}

// Define OnPostAsync() here
public async Task OnPostAsync()
{
    // For debugging
    RequestMethod = "POST";
    RequestValues = "n/a";

    using (StreamWriter writer = new StreamWriter("log.txt", append: true))

```

```

    {
        await writer.WriteLineAsync($"OnPostAsync() called at {DateTime.Now}.");
    }
}

// For debugging
private string GetFormValues(bool ignoreRequestVerificationToken = true)
{
    string formData = "";
    string separator = " | ";

    foreach (var pair in this.Request.Form)
    {
        if (ignoreRequestVerificationToken && pair.Key ==
            "__RequestVerificationToken")
        {
            continue;
        }
        else
        {
            formData += pair.Key + ": " + Request.Form[pair.Key] + separator;
        }
    }

    if (formData.EndsWith(separator))

```

```

    {
        formData = formData.Substring(0, formData.Length - separator.Length);
    }

    return formData;
}
}
}

```

Index.cshtml

```

@page
@model IndexModel
@{
    ViewData["Title"] = "Home page";
}

<div class="jumbotron jumbotron-fluid" id="chef-jumbotron">
    <div id="request-info">
        <em>RequestMethod</em>: @Model.RequestMethod
        <em>RequestInfo</em>: @Model.RequestValues
    </div>
    <div id="jumbotron-main-text">
        <h1>Chef's Blog</h1>
        <p>A place for the best recipes!</p>
    </div>

```

</div>

<div class="row">

<div class="col">

<h1>Write a new post</h1>

<hr>

<!-- Form start -->

<form method="post">

<div class="form-group">

<label for="Title">Title</label>

<!-- Use asp-for here -->

<input type="text" class="form-control" id="Title" name="Title"
placeholder="Title">

</div>

<div class="form-group">

<label for="Date">Date</label>

<!-- Use asp-for here -->

<input type="date" class="form-control" id="Date" name="Date">

</div>

<div class="form-group">

<label for="Body">Your post</label>

<!-- Use asp-for here -->

<textarea class="form-control" id="Body" name="Body"
rows="3"></textarea>

</div>

```
<button type="submit" class="btn" id="submit">Submit</button>
</form>
<!-- Form end -->
</div>
```

```
<div class="col">
  <h1>Recent posts</h1>
  <hr>
  <!-- Page model properties displayed here -->
  <div class="blog-post">
    <h2 class="blog-post-title">@Model.Title</h2>
    <p class="blog-post-meta">Posted on @Model.Date.ToShortDateString()</p>
    <p>@Model.Body</p>
  </div>
</div>
</div>
```

Archive.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Blog.Models;
```

```

namespace Blog.Pages
{
    public class ArchiveModel : PageModel
    {
        public readonly List<BlogPost> Archive = new List<BlogPost>
        {
            new BlogPost("One Pot Thai-Style Rice", new DateTime(2020, 1, 24), "This is a famous dish popularized in Thailand. Although the recipe varies from cook to cook and region to region, this is a good attempt at recreating what I ate from Thai-owned hole-in-the-wall restaurants in Okinawa, Japan. Key to the flavor are the sugar levels, unsalted peanuts, peanut oil, and either oyster or fish sauce."),
            new BlogPost("Balsamic Glazed Salmon Fillets", new DateTime(2020, 1, 20), "A glaze featuring balsamic vinegar, garlic, honey, white wine and Dijon mustard makes baked salmon fillets extraordinary."),
            new BlogPost("Spicy Garlic Lime Chicken", new DateTime(2019, 9, 4), "A delightful chicken dish with a little spicy kick. Serve with rice and your favorite vegetable."),
            new BlogPost("Mushroom Pork Chops", new DateTime(2019, 6, 14), "Quick and easy, but very delicious. One of my family's favorites served over brown rice."),
        };

        public BlogPost Displayed { get; set; }

        public void OnGet(int index)
        {
            if (index >= Archive.Count || index < 0)

```



```

{
    Displayed = new BlogPost("n/a", new DateTime(0001, 01, 01), "n/a");
}
else
{
    Displayed = Archive[index];
}
}
}
}
}

```

Archive.cshtml

@page

@model ArchiveModel

@{

}

```
<div class="jumbotron jumbotron-fluid" id="archive-jumbotron">
```

```
<div id="jumbotron-main-text">
```

```
<h1>Archive</h1>
```

```
<p>The best past recipes!</p>
```

```
</div>
```

```
</div>
```

```

<div class="row">
  <div class="col">
    <!-- Page model properties displayed here -->
    <div class="blog-post">
      <h3 class="blog-post-title">@Model.Displayed.Title</h3>
      <p class="blog-post-meta">Posted on
@Model.Displayed.Date.ToShortDateString()</p>
      <p>@Model.Displayed.Body</p>
    </div>
  </div>
  <div class="col">
    <h1>Older posts</h1>
    <hr>
    @for (int i = 0; i < Model.Archive.Count; i++)
    {
      <div>
        <h3>
          <a asp-page="/Archive" asp-route-index="@i">@Model.Archive[i].Title</a>
        </h3>
        <p>Posted on @Model.Archive[i].Date.ToShortDateString()</p>
      </div>
    }
  </div>
</div>

```

Concept Review

asp-for

The asp-for attribute is used to connect an <input> element to a page model property.

The above example would be connected to a Username property in the page model:

```
public class IndexModel : PageModel
{
    [BindProperty]
    public string Username { get; set; }
}
```

In the rendered HTML, attributes like type, id, and name will be added to the <input> tag:

Enter a username:

```
<input type="text" id="Username" name="Username" >
```

```
<!-- In .cshtml file -->
```

```
Enter a username: <input asp-for="Username" />
```

URLs in Razor Pages

In a default Razor Pages application, each view page's URL route is its path minus .cshtml.

For example, given the folder structure:

Pages

|-- About.cshtml

|-- About.cshtml.cs

|-- Index.cshtml

|-- Index.cshtml.cs

|-- Businesses/

 |-- Codecademy.cshtml

 |-- Codecademy.cshtml.cs

- **About.cshtml** is available at <https://localhost:8000/About>
- **Codecademy.cshtml** is available at <https://localhost:8000/Businesses/Codecademy>
- **Index.cshtml** is available at <https://localhost:8000/Index> OR <https://localhost:8000>

OnGet() & OnGetAsync()

When a page model receives a GET request, its `OnGet()` or `OnGetAsync()` method is invoked. This typically happens when a user navigates to the page model's corresponding view page.

A page model must have either `OnGet()` or `OnGetAsync()`. It cannot have both.

```
public class ZooModel : PageModel
{
```

```
public string FavoriteAnimal { get; set; }

// Sets FavoriteAnimal when page is requested
public void OnGet()
{
    FavoriteAnimal = "Hippo";
}
}
```

OnPost() & OnPostAsync()

When a page model receives a POST request, its `OnPost()` or `OnPostAsync()` method is invoked. This typically happens when a user submits a form on the page model's corresponding view page.

A page model must have either `OnPost()` or `OnPostAsync()`. It cannot have both.

```
public class IndexModel : PageModel
{
    public string Message { get; set; }

    public void OnPost()
    {
        Message = "OnPost() called!";
    }
}
```

Void Handler Methods

In a page model, synchronous handler methods like `OnGet()` and `OnPost()` that have no return statement will have a return type of `void`.

This results in the current page being rendered in response to every request.

```
public class IndexModel : PageModel
```

```
{
```

```
    public string Username { get; set; }
```

```
    public void OnGet()
```

```
{
```

```
        Username = "n/a";
```

```
}
```

```
    public void OnPost(string username)
```

```
{
```

```
        Username = username;
```

```
}
```

```
}
```

Task Handler Methods

In a page model, asynchronous handler methods like `OnGetAsync()` and `OnPostAsync()` that have no return statement will have a return type of `Task`.

This results in the current page being rendered in response to every request.

```
public class IndexModel : PageModel
{
    public string Users { get; set; }
    private UserContext _context { get; set; }

    public IndexModel(UserContext context)
    {
        _context = context;
    }

    // Task return type
    public async Task OnGetAsync()
    {
        Users = await _context.Users.ToListAsync();
    }

    // Task return type
    public async Task OnPostAsync(string username)
    {
        _context.Users.Add(username);
        await _context.SaveChangesAsync();
    }
}
```

```
}  
}
```

Handler Method Parameters

Page model handler methods, like `OnGet()`, `OnGetAsync()`, `OnPost()`, and `OnPostAsync()`, can access an incoming HTTP request's query string via its own method parameters.

The name of the method parameter(s) must match (case-insensitive) the name(s) in the query string.

// Example GET request

// https://localhost:8000/Songs?id=1

```
public async Task OnGetAsync(int id)
```

```
{
```

```
    // id is 1
```

```
}
```

// Example POST request

// https://localhost:8000/Songs?songName=Say%20It%20Loud

```
public async Task OnPostAsync(string songName)
```

```
{
```

```
    // songName is "Say It Loud"
```

```
}
```

Model Binding

In *model binding*, a page model retrieves data from an HTTP request, converts the data to .NET types, and updates the corresponding model properties. It is enabled with the [BindProperty] attribute.

```
public class IndexModel : PageModel
```

```
{
```

```
    [BindProperty]
```

```
    public string Username { get; set; }
```

```
    [BindProperty]
```

```
    public bool IsActive { get; set; }
```

```
    // Example POST
```

```
    // https://localhost:8000?username=muhammad&IsActive=true
```

```
    public void OnPost()
```

```
    {
```

```
        // Username is "muhammad"
```

```
        // IsActive is true
```

```
    }
```

```
}
```

Append URL Segments

In Razor view pages (**.cshtml** files), the @page directive can be used to add segments to a page's default route. Use this feature by typing a string after @page.

For example, imagine the below code is from **About.cshtml**. Instead of /About, the new route would be /About/Me:

```
@page "Me"
```

```
@page "segment"
```

Append URL Parameters

In Razor view pages (**.cshtml** files), the `@page` directive can be used to add parameters to a page's route.

- The parameter(s) must be in between curly braces { } after `@page`.
- Constraints, like `int` or `alpha`, can be added using colons `:`.
- A parameter can be marked optional using a question mark `?`.

Imagine the below code is from **Book.cshtml**. Instead of /Book, the new route could be /Book/0 or Book/1 or Book/2 etc.:

```
@page "{id:int}"
```

Imagine the below code is from **House.cshtml**. The new route could be /House or House/small or House/big etc.:

```
@page "{size?}"
```

Imagine the below code is from **Song.cshtml**. Instead of /Song, the new route would be /Song or Song/0 or Song/1 etc.:

```
@page "{song:int?}"
```

```
@page {param}
```

asp-route-{value}

The `asp-route-{value}` attribute is used in `<a>` elements to add additional information to a URL route.

- `{value}` typically matches a property in a page model.
- The provided value will be added as a route segment or a query string, depending on how the route is defined.

If the above `<a>` tag is in a `.cshtml` file, it would be rendered as this HTML:

```
<a href="localhost:8000/About?name=Joanne">About Joanne</a>
```

However, if the About page has a route parameter, like `@page {name}`, then the same tag would be rendered as this HTML:

```
<a href="localhost:8000/About/Joanne">About Joanne</a>
```

```
<a asp-page="About" asp-route-name="Joanne">About Joanne</a>
```

Default Responses

In page models, a handler method with no return statement will respond to HTTP requests by sending back the associated page.

In the above example, `IndexModel` is associated with **`Index.cshtml`**.

Neither `OnGet()` nor `OnPostAsync()` have return statements, so they both return **`Index.cshtml`**.

```
public class IndexModel : PageModel
{
    // Sends Index.cshtml

    public void OnGet()
    {}

    // Sends Index.cshtml
```

```
public void OnPost()
{
}
}
```

Page()

To return the view page associated with a page model, use `Page()` in the page model's handler methods.

- This happens implicitly if the handler method has a void return type
- If a handler method calls `Page()`, its return type is typically `ActionResult` or `Task<ActionResult>` (although others exist).

```
public class AboutModel : PageModel
{
    // Sends About.cshtml
    public IActionResult OnGet()
    {
        return Page();
    }
}
```

RedirectToPage()

To redirect users to a different Razor page within the application, use `RedirectToPage()` in the page model's handler methods.

- If a handler method calls `RedirectToPage()`, its return type is typically `IActionResult` or `Task<IActionResult>` (although others exist).
- The string argument passed to this method is a file path. `"/Index"` is a relative path and `"/Index"` is an absolute path.

```
public class IndexModel : PageModel
{
    // Sends Privacy.cshtml
    public IActionResult OnPost()
    {
        return RedirectToPage("./Privacy");
    }
}
```

NotFound()

To send a “Status 404 Not Found” response, use `NotFound()` in the page model’s handler methods.

- If a handler method calls `NotFound()`, its return type is typically `IActionResult` or `Task<IActionResult>` (although others exist).

```
public class EditModel : PageModel
{
    public async Task<IActionResult> OnGetAsync(int? id)
    {
        if (id == null)
        {
```

```
    return NotFound();  
}  
  
// do something with id here  
  
return Page();  
}  
}
```

20. ROUTING

20.1 Introduction to Routing

Imagine that you were building an exercise app to track your activity. You could store each of your day's steps and workouts in the app and you could view both your total activity for the week and individual activity for each day.

With the page-based framework of Razor Pages,

- How would you duplicate the individual-day view? Would you need to write a new page for every day?
- How could you customize your URLs to match each day?

These questions can be solved with *routing*. Instead of URLs like /Activity you can rename it to Days and make variable route parameters like /Days/1, /Days/2, etc. In this lesson you'll do the following:

- Define custom URL segments
- Define route templates
- Add Constraints on route templates
- Understand how Tag Helpers adapt to custom routing

20.1 Introduction to Routing Instructions

In the browser to the right, navigate to the Daily Activity tracker. Check out how the URL changes with each link.

Activity.cshtml

@page "/Days/{day?}"

@model ActivityModel

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-  
row">
```

```
    <div class="col-1">
```

```
</div>
```

```
<div class="col">
```

```
    @if (Model.IsWeeklyDisplay)
```

```
    {
```

```
        <a id="weekly-total-link" class="active-day day-week-links" asp-  
page="/Activity" asp-route-day="">Weekly Total</a>
```

```
    }
```

```
    else
```

```
    {
```

```
        <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-  
route-day="">Weekly Total</a>
```

```
    }
```

```
</div>
```

```
@for (int i = 0; i < Model.Days.Count; i++)
```

```
{
```

```
    <div class="col">
```

```
        @if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
```

```
        {
```



```

        <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
    }
    else
    {
        <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
        @i</a>
    }
</div>
}

```

```

<div class="col-1">
</div>
</div>
<div class="row">
<div class="col circle-col">
    @{
        var stepsData = new ProgressCirclePartialModel
        {
            BackgroundStroke = "#D9EFF6",
            ForegroundStroke = "#3EAED5",
            PercentProgress = @Model.PercentProgressSteps,
            DisplayNumber = @Model.DisplaySteps,
            Unit = "Steps",

```

```
        IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-  
icon.png"
```

```
    };
```

```
    }
```

```
    <partial name="_ProgressCirclePartial" model=@stepsData />
```

```
</div>
```

```
</div>
```

```
<div class="row">
```

```
    <div class="col circle-col">
```

```
        @{
```

```
            var exerciseData = new ProgressCirclePartialModel
```

```
            {
```

```
                BackgroundStroke = "#FBD7E5",
```

```
                ForegroundStroke = "#DA387D",
```

```
                PercentProgress = @Model.PercentProgressMinutesExercise,
```

```
                DisplayNumber = @Model.DisplayMinutesExercise,
```

```
                Unit = "Mins of Exercise",
```

```
                IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-  
icon.png"
```

```
            };
```

```
        }
```

```
        <partial name="_ProgressCirclePartial" model=@exerciseData />
```

```
</div>
```

```
</div>
```

OUTPUT



20.2 Customize the URL

By default, each page's URL is defined by its filename:

- **Index.cshtml** is at localhost:8000
- **Privacy.cshtml** is at localhost:8000/Privacy

What if we want to override those defaults?

We can add and/or change URL segments by adding a string after the `@page` directive. For example, if we used this line at the top of **Privacy.cshtml**...

```
@page "/Pirates"
```

...then **Privacy.cshtml** would now be available at localhost:8000/Pirates.

If we remove the forward slash (/), then we append a segment. Using this line...

```
@page "Pirates"
```

...makes **Privacy.cshtml** available at localhost:8000/Privacy/Pirates.

Aside from practical jokes, custom URLs are useful when you have a page deep in your folder hierarchy and you want a shorter URL. For example, if we used this line at the top of **Movies/Horror/Create.cshtml**...

```
@page "/AddScaries"
```

...then that file would be available at localhost:8000/AddScaries instead of localhost:8000/Movies/Horror/Create.

Just make sure to use that forward slash, /. If you don't, it will append the segment to the end of the existing URL.

20.2 Customize the URL Instructions

1.

Currently, the contents of **Activity.cshtml** is available at /Activity.

Edit the @page directive to make **Activity.cshtml** available at /Days.

Activity.cshtml

```
@page
```

```
@model ActivityModel
```

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-row">
```

```
<div class="col-1">
```

```
</div>
```

```
<div class="col">
```

```
@if (Model.IsWeeklyDisplay)
```

```

{
    <a id="weekly-total-link" class="active-day day-week-links" asp-
page="/Activity" asp-route-day="">Weekly Total</a>
}
else
{
    <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-
route-day="">Weekly Total</a>
}
</div>

```

```

@for (int i = 0; i < Model.Days.Count; i++)
{
    <div class="col">
        @if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
        {
            <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
        }
        else
        {
            <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
@i</a>
        }
    </div>
}

```

```

<div class="col-1">

</div>

</div>

<div class="row">

<div class="col circle-col">

    @{
        var stepsData = new ProgressCirclePartialModel
        {
            BackgroundStroke = "#D9EFF6",
            ForegroundStroke = "#3EAED5",
            PercentProgress = @Model.PercentProgressSteps,
            DisplayNumber = @Model.DisplaySteps,
            Unit = "Steps",
            IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
icon.png"
        };
    }

    <partial name="_ProgressCirclePartial" model=@stepsData />

</div>

</div>

<div class="row">

<div class="col circle-col">

    @{

```

```
var exerciseData = new ProgressCirclePartialModel
{
    BackgroundStroke = "#FBD7E5",
    ForegroundStroke = "#DA387D",
    PercentProgress = @Model.PercentProgressMinutesExercise,
    DisplayNumber = @Model.DisplayMinutesExercise,
    Unit = "Mins of Exercise",
    IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
icon.png"
};
}

<partial name="_ProgressCirclePartial" model=@exerciseData />
</div>
</div>
```

OUTPUT

Weekly Total Day 0 Day 1 Day 2 Day 3



20.3 Route Templates

So far, we've seen two ways to pass data from the browser to a page model. Once a form is submitted we can:

1. Capture the data using method parameters, like `OnPost(string title)`, or
2. Bind the data to properties using `[BindProperty]`

In both cases the POST request is made to a URL with a query string, like:

`localhost:8000/Movies?title=Inception`

We can reformat the URL so that the data is provided in URL segments instead, like:

```
localhost:8000/Movies/Inception
```

This format makes the URL more readable and more search-engine friendly. Here's the generalized format:

```
localhost:8000/Movies/{title}
```

Inside a **.cshtml** file, we can specify this with the `@page` directive, using a kind of variable, known as a route value or route parameter, wrapped in curly braces. Assuming the page lives in **Movies.cshtml**, then the first line of the file would be:

```
@page "{title}"
```

Make sure that you use double quotes and curly braces: "{ }".

The way to capture these values is no different than before.

Capture with method parameters:

```
OnPost(string title)
```

Or with model-binding:

```
[BindProperty]  
string Title { get; set; }
```

20.3 Route Templates Instructions

1.

Currently, URL data for `/Days` is passed by query string. Try clicking the links. You should see your URL as something like:

```
localhost:8000/Days?day=1
```

Let's convert the query string to a URL segment. So that it looks like this:

```
localhost:8000/Days/1
```

Activity.cshtml

@page "/Days"

@model ActivityModel

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-  
row">
```

```
    <div class="col-1">
```

```
    </div>
```

```
    <div class="col">
```

```
        @if (Model.IsWeeklyDisplay)
```

```
        {
```

```
            <a id="weekly-total-link" class="active-day day-week-links" asp-  
page="/Activity" asp-route-day="">Weekly Total</a>
```

```
        }
```

```
        else
```

```
        {
```

```
            <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-  
route-day="">Weekly Total</a>
```

```
        }
```

```
    </div>
```

```
@for (int i = 0; i < Model.Days.Count; i++)
```

```
{
```

```
    <div class="col">
```

```

@if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
{
    <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
}
else
{
    <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
    @i</a>
}
</div>
}

```

```

<div class="col-1">
</div>
</div>
<div class="row">
<div class="col circle-col">
@{
    var stepsData = new ProgressCirclePartialModel
    {
        BackgroundStroke = "#D9EFF6",
        ForegroundStroke = "#3EAED5",
        PercentProgress = @Model.PercentProgressSteps,
        DisplayNumber = @Model.DisplaySteps,

```

```

        Unit = "Steps",
        IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
icon.png"
    };
}
<partial name="_ProgressCirclePartial" model=@stepsData />
</div>
</div>

<div class="row">
    <div class="col circle-col">
        @{
            var exerciseData = new ProgressCirclePartialModel
            {
                BackgroundStroke = "#FBD7E5",
                ForegroundStroke = "#DA387D",
                PercentProgress = @Model.PercentProgressMinutesExercise,
                DisplayNumber = @Model.DisplayMinutesExercise,
                Unit = "Mins of Exercise",
                IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
icon.png"
            };
        }
        <partial name="_ProgressCirclePartial" model=@exerciseData />
    </div>

```

</div>

20.4 Add Optional Route Templates

We can make route values optional with a question mark ?.

For example, this defines an optional title template:

```
@page "{title?}"
```

In order to capture that in a method parameter, you'll need to also use the question mark there:

```
public void OnGet(string? title) { }
```

When combining with an if - else statement, your handler method can change behavior based on whether the value was provided:

```
public void OnGet(string? title)
{
    if (String.IsNullOrEmpty(title))
    {
        IsGeneralDisplay = true;
    }
    else
    {
        Title = title.Value;
        IsGeneralDisplay = false;
    }
}
```

In the if condition, we use `String.IsNullOrEmpty()` to check the value of `title`. It returns true if the title is not provided in the URL or it is empty (""). For non-string types, we use the `HasValue` property. For example:

```
public void OnPost(bool? b)
{
```

```
if (b.HasValue)
{
    // Access b.Value
}
else
{
    // b is null
}
}
```

You can [read more about nullable types in the Microsoft documentation](#).

20.4 Add Optional Route Templates Instructions

1.

In **Activity.cshtml**, the current `@page` directive specifies a required route value. If you navigate to a URL without one, like `localhost:8000/Days`, the app returns an error.

Add a question mark to the route value to make it optional.

2.

In **Activity.cshtml.cs** in `OnGet()`, add a question mark to the method parameter's type. This makes it optional, or nullable.

(If you do this correctly you'll get an error CS0266. We'll fix that next.)

3.

Within the method, we need to adjust our code to handle the new parameter type.

Set `CurrentDay` to `day.Value`.

This converts the nullable `int?` type to `int`.

4.

Within the method, create an if statement that is executed if day is not null.

Move all of the method's existing code into that if block.

5.

Add an else statement below that will be used when a route value is not provided.

Add this code within the else block:

```
CurrentDay = 0;
IsWeeklyDisplay = true;
DisplaySteps = Days.Sum(d => d.Steps);
DisplayMinutesExercise = Days.Sum(d => d.MinutesExercise);
PercentProgressSteps = PercentProgress(DisplaySteps, idealSteps * Days.Count);
PercentProgressMinutesExercise = PercentProgress(DisplayMinutesExercise,
idealMinutesExercise * Days.Count);
```

You should see that navigating to localhost:8000/Days gives you the weekly total and localhost:8000/Days/0 gives you specific exercise information for day 0.

Activity.cshtml

```
@page "/Days/{day}"
```

```
@model ActivityModel
```

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-
row">
```

```
<div class="col-1">
```

```
</div>
```

```

<div class="col">
    @if (Model.IsWeeklyDisplay)
    {
        <a id="weekly-total-link" class="active-day day-week-links" asp-
page="/Activity" asp-route-day="">Weekly Total</a>
    }
    else
    {
        <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-
route-day="">Weekly Total</a>
    }
</div>

```

```

@for (int i = 0; i < Model.Days.Count; i++)
{
    <div class="col">
        @if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
        {
            <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
        }
        else
        {
            <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
@i</a>
        }
    }
}

```



```

</div>
}

<div class="col-1">
</div>
</div>
<div class="row">
<div class="col circle-col">
    @{
        var stepsData = new ProgressCirclePartialModel
        {
            BackgroundStroke = "#D9EFF6",
            ForegroundStroke = "#3EAED5",
            PercentProgress = @Model.PercentProgressSteps,
            DisplayNumber = @Model.DisplaySteps,
            Unit = "Steps",
            IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
icon.png"
        };
    }
    <partial name="_ProgressCirclePartial" model=@stepsData />
</div>
</div>

<div class="row">

```

```

<div class="col circle-col">
    @{
        var exerciseData = new ProgressCirclePartialModel
        {
            BackgroundStroke = "#FBD7E5",
            ForegroundStroke = "#DA387D",
            PercentProgress = @Model.PercentProgressMinutesExercise,
            DisplayNumber = @Model.DisplayMinutesExercise,
            Unit = "Mins of Exercise",
            IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
            icon.png"
        };
    }
    <partial name="_ProgressCirclePartial" model=@exerciseData />
</div>
</div>

```

Activity.cshtml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

```

```

namespace ActivityTracker.Pages
{
    public class ActivityModel : PageModel
    {
        private int idealSteps = 10000;
        private int idealMinutesExercise = 30;

        public List<Day> Days = new List<Day>
        {
            new Day(3000, 20),
            new Day(6000, 12),
            new Day(10000, 40),
            new Day(7000, 25)
        };
        public int CurrentDay { get; set; }
        public int DisplaySteps { get; set; }
        public int DisplayMinutesExercise { get; set; }
        public double PercentProgressSteps { get; set; }
        public double PercentProgressMinutesExercise { get; set; }
        public bool IsWeeklyDisplay { get; set; }

        // Edit this method
        public void OnGet(int day)
        {

```

```

    CurrentDay = day;
    DisplaySteps = Days[CurrentDay].Steps;
    DisplayMinutesExercise = Days[CurrentDay].MinutesExercise;
    PercentProgressSteps = PercentProgress(DisplaySteps, idealSteps);
    PercentProgressMinutesExercise = PercentProgress(DisplayMinutesExercise,
idealMinutesExercise);
}

```

```

private static double PercentProgress(double actual, double expected)
{
    return Math.Clamp(actual / expected, 0, 1);
}
}

```

```

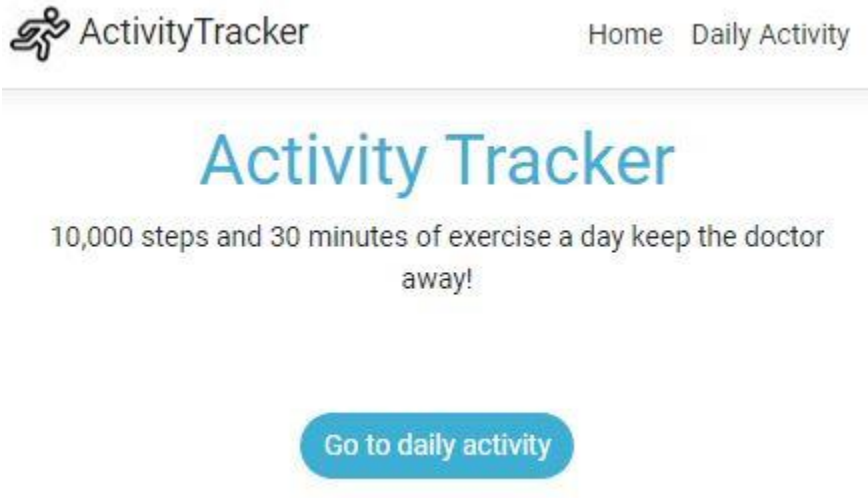
public class Day
{
    public int Steps { get; set; }
    public int MinutesExercise { get; set; }

    public Day(int steps = 0, int minutes = 0)
    {
        Steps = steps;
        MinutesExercise = minutes;
    }
}

```

}

OUTPUT



20.5 Add Constrained Route Templates

Like everywhere else in C#, strict type constraints help us avoid errors. Imagine if someone sent a POST request to this URL:

localhost:8000/veggies/YES/fruits/NO/grains/IDUNNO/protein/SORTA/dairy/NEVER

It would break our application, which expects integers for each of those route values. Within our `@page` directive we can specify that constraint like the below, where `int` stands for “integer” (we’ll show an abbreviated version here):

```
@page "/veggies/{veggies:int}..."
```

The general format is:

```
@page "{routevalue:constraint}"
```

If you want the route value to remain optional, use the question mark after the constraint, like:

```
@page "/veggies/{veggies:int?}..."
```

There are a lot of constraints out there, but here are a few to get started:

- int — value must be any integer
- alpha — value must consist of one or more alphabetical characters (a-z, case-insensitive)
- bool — value must be true or false (case-insensitive)

A longer (but not exhaustive) [list of constraints is available in the Microsoft documentation](#).

20.5 Add Constrained Route Templates Instructions

1.

The current @page directive specifies a route value without constraints. For example, what happens when you navigate to the below “illegal” URL?

```
localhost:8000/Days/all
```

Add the proper constraint so that it only accepts integers (and is still optional).

Activity.cshtml

```
@page "/Days/{day?}"
```

```
@model ActivityModel
```

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-row">
```

```
<div class="col-1">
```

```
</div>
```

```

<div class="col">
    @if (Model.IsWeeklyDisplay)
    {
        <a id="weekly-total-link" class="active-day day-week-links" asp-
page="/Activity" asp-route-day="">Weekly Total</a>
    }
    else
    {
        <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-
route-day="">Weekly Total</a>
    }
</div>

```

```

@for (int i = 0; i < Model.Days.Count; i++)
{
    <div class="col">
        @if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
        {
            <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
        }
        else
        {
            <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
@i</a>

```

```

    }
</div>
}

<div class="col-1">
</div>
</div>
<div class="row">
<div class="col circle-col">
    @{
        var stepsData = new ProgressCirclePartialModel
        {
            BackgroundStroke = "#D9EFF6",
            ForegroundStroke = "#3EAED5",
            PercentProgress = @Model.PercentProgressSteps,
            DisplayNumber = @Model.DisplaySteps,
            Unit = "Steps",
            IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
            icon.png"
        };
    }
    <partial name="_ProgressCirclePartial" model=@stepsData />
</div>
</div>

```



```

<div class="row">
  <div class="col circle-col">
    @{
      var exerciseData = new ProgressCirclePartialModel
      {
        BackgroundStroke = "#FBD7E5",
        ForegroundStroke = "#DA387D",
        PercentProgress = @Model.PercentProgressMinutesExercise,
        DisplayNumber = @Model.DisplayMinutesExercise,
        Unit = "Mins of Exercise",
        IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
icon.png"
      };
    }
    <partial name="_ProgressCirclePartial" model=@exerciseData />
  </div>
</div>

```

OUTPUT



Weekly Total Day 0 Day 1 Day 2
Day 3



20.6 asp-route-{value} revisited

Remember the Anchor Tag Helper? It's the one with `asp-page` and `asp-route-{value}` attributes that allows us to create `<a>` links that submit data with a GET request.

Previously we saw how the resulting `asp-route-{value}` appended a query string to the URL:

```
<a asp-page="/Blogpost" asp-route-id="4">ID 4</a>
```

The above Razor syntax would render as this HTML:

```
<a href="/Blogpost?id=4">ID 4</a>
```

What happens if our URL template accepts URL segments instead of a query string? It would be defined like this in **Blogpost.cshtml**:

```
@page {id}
```

Here we find another advantage of Tag Helpers over plain HTML: it automatically formats URLs to match the defined template.

When we use URL segments instead of query strings, the old Anchor Tag Helper...

```
<a asp-page="/Blogpost" asp-route-id="4">ID 4</a>
```

...would render as this HTML...

```
<a href="/Blogpost/4">ID 4</a>
```

In review:

- `asp-page` — Sets an anchor tag's href attribute value to a specific page
- `asp-route-{value}` — Appends route values to the generated href attribute, either as query string values or URL segments (depending on the route template). {value} is the name of the parameter.

20.6 asp-route-{value} revisited Instructions

1.

We removed the route template for **Activity.cshtml** (it just uses `@page "/Days"` now), meaning it again uses query parameters. The Anchor Tag Helpers within the file use the `asp-route-day` attribute to generate URLs like this:

```
localhost:8000/Days?day=1
```

You can try it for yourself by clicking the links.

Now change the `@page` line to specify the route template with one optional {day?} parameter. The `asp-route-day` attribute should now generate URLs like this:

```
localhost:8000/Days/1
```

Activity.cshtml

@page "/Days"

@model ActivityModel

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-  
row">
```

```
    <div class="col-1">
```

```
    </div>
```

```
    <div class="col">
```

```
        @if (Model.IsWeeklyDisplay)
```

```
        {
```

```
            <a id="weekly-total-link" class="active-day day-week-links" asp-  
page="/Activity" asp-route-day="">Weekly Total</a>
```

```
        }
```

```
        else
```

```
        {
```

```
            <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-  
route-day="">Weekly Total</a>
```

```
        }
```

```
    </div>
```

```
@for (int i = 0; i < Model.Days.Count; i++)
```

```
{
```

```
    <div class="col">
```

```

@if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
{
    <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
}
else
{
    <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
    @i</a>
}
</div>
}

```

```

<div class="col-1">
</div>
</div>
<div class="row">
<div class="col circle-col">
@{
    var stepsData = new ProgressCirclePartialModel
    {
        BackgroundStroke = "#D9EFF6",
        ForegroundStroke = "#3EAED5",
        PercentProgress = @Model.PercentProgressSteps,
        DisplayNumber = @Model.DisplaySteps,

```

```

        Unit = "Steps",
        IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
icon.png"
    };
}
<partial name="_ProgressCirclePartial" model=@stepsData />
</div>
</div>

<div class="row">
    <div class="col circle-col">
        @{
            var exerciseData = new ProgressCirclePartialModel
            {
                BackgroundStroke = "#FBD7E5",
                ForegroundStroke = "#DA387D",
                PercentProgress = @Model.PercentProgressMinutesExercise,
                DisplayNumber = @Model.DisplayMinutesExercise,
                Unit = "Mins of Exercise",
                IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
icon.png"
            };
        }
        <partial name="_ProgressCirclePartial" model=@exerciseData />
    </div>

```

</div>

20.7 Review

Well done! Instead of ugly URLs like:

localhost:8000/Movies/Horror/Create?title=Spooky+Ghosts&length=120

You can now re-define them like:

localhost:8000/AddScaries/Spooky+Ghosts/120

In this lesson you learned:

- Typing a string after the @page directive will edit the page's default route.
- Typing parameters within curly braces after the @page directive will define a route template.
- Route values can be made optional with a question mark ?.
- Route values can be constrained using the colon syntax and [keyword constraints](#).
- The asp-route-{value} attribute can add additional information to an <a> element's href attribute, either in the form of an additional route segment or query string, depending on how the route template is defined.

<!-- No URL segments -->

@page

<!-- Edit the default route -->

@page "/Days"

<!-- Add a route template -->

@page "/Days/{day}"

<!-- Constrain route value -->

@page "/Days/{day:int}"

<!-- Make route value optional -->

@page "/Days/{day:int?}"

20.7 Review Instructions

Experiment with the links at localhost:8000/Days/. Check that the URLs match your expectations and the code in **Activity.cshtml**.

Activity.cshtml

```
@page "/Days/{day?}"
```

```
@model ActivityModel
```

```
<div class="row d-flex align-items-center justify-content-between mx-2" id="links-  
row">
```

```
    <div class="col-1">
```

```
    </div>
```

```
<div class="col">
```

```
    @if (Model.IsWeeklyDisplay)
```

```
    {
```

```
        <a id="weekly-total-link" class="active-day day-week-links" asp-  
page="/Activity" asp-route-day="">Weekly Total</a>
```

```
    }
```

```
    else
```

```
    {
```

```
        <a id="weekly-total-link" class="day-week-links" asp-page="/Activity" asp-  
route-day="">Weekly Total</a>
```

```
    }
```

```
</div>
```



```

@for (int i = 0; i < Model.Days.Count; i++)
{
    <div class="col">
        @if (!Model.IsWeeklyDisplay && i == Model.CurrentDay)
        {
            <a class="active-day day-week-links" asp-page="/Activity" asp-route-
day=@i>Day @i</a>
        }
        else
        {
            <a class="day-week-links" asp-page="/Activity" asp-route-day=@i>Day
@i</a>
        }
    </div>
}

<div class="col-1">
</div>
</div>
<div class="row">
    <div class="col circle-col">
        @{
            var stepsData = new ProgressCirclePartialModel
            {
                BackgroundStroke = "#D9EFF6",

```

```

        ForegroundStroke = "#3EAED5",
        PercentProgress = @Model.PercentProgressSteps,
        DisplayNumber = @Model.DisplaySteps,
        Unit = "Steps",
        IconUrl = "https://content.codecademy.com/courses/asp-dot-net/shoe-
icon.png"
    };
}
<partial name="_ProgressCirclePartial" model=@stepsData />
</div>
</div>

```

```

<div class="row">
    <div class="col circle-col">
        @{
            var exerciseData = new ProgressCirclePartialModel
            {
                BackgroundStroke = "#FBD7E5",
                ForegroundStroke = "#DA387D",
                PercentProgress = @Model.PercentProgressMinutesExercise,
                DisplayNumber = @Model.DisplayMinutesExercise,
                Unit = "Mins of Exercise",
                IconUrl = "https://content.codecademy.com/courses/asp-dot-net/weight-
icon.png"
            };

```

```
    }  
    <partial name="_ProgressCirclePartial" model=@exerciseData />  
</div>  
</div>
```

Activity.cshtml.cs

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Threading.Tasks;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace ActivityTracker.Pages  
{  
    public class ActivityModel : PageModel  
    {  
        private int idealSteps = 10000;  
        private int idealMinutesExercise = 30;  
  
        public List<Day> Days = new List<Day>  
        {  
            new Day(3000, 20),  
            new Day(6000, 12),  
        }  
    }  
}
```

```

    new Day(10000, 40),
    new Day(7000, 25)
};

public int CurrentDay { get; set; }
public int DisplaySteps { get; set; }
public int DisplayMinutesExercise { get; set; }
public double PercentProgressSteps { get; set; }
public double PercentProgressMinutesExercise { get; set; }
public bool IsWeeklyDisplay { get; set; }

public void OnGet(int? day)
{
    if (day.HasValue)
    {
        CurrentDay = day.Value;
        DisplaySteps = Days[CurrentDay].Steps;
        DisplayMinutesExercise = Days[CurrentDay].MinutesExercise;
        PercentProgressSteps = PercentProgress(DisplaySteps, idealSteps);
        PercentProgressMinutesExercise = PercentProgress(DisplayMinutesExercise,
idealMinutesExercise);
    }
    else
    {
        CurrentDay = 0;
        IsWeeklyDisplay = true;
    }
}

```

```

        DisplaySteps = Days.Sum(d => d.Steps);
        DisplayMinutesExercise = Days.Sum(d => d.MinutesExercise);
        PercentProgressSteps = PercentProgress(DisplaySteps, idealSteps *
Days.Count);

        PercentProgressMinutesExercise = PercentProgress(DisplayMinutesExercise,
idealMinutesExercise * Days.Count);
    }
}

private static double PercentProgress(double actual, double expected)
{
    return Math.Clamp(actual / expected, 0, 1);
}

public class Day
{
    public int Steps { get; set; }
    public int MinutesExercise { get; set; }
    public Day(int steps = 0, int minutes = 0)
    {
        Steps = steps;
        MinutesExercise = minutes;
    }
}

```

21. REDIRECTION

21.1 Introduction to Redirection

So far, every handler method has rendered the same view page. For example, `OnGet()` in **Index.cshtml.cs** will always send a response with **Index.cshtml**, and `OnPost()` in **About.cshtml.cs** will always send a response with **About.cshtml**.

Sometimes we want to redirect users to a different page.

- A blog website has a “Posts” page showing blog posts and a separate “Create” page. After submitting on “Create”, the user should be redirected to the list of blog posts on “Posts”.
- A page on your education website has become outdated, and you add a new one to replace it. Learners requesting the old page should be redirected to the newer version of the page (assuming you have to keep the old page for those who are half-way through the content).
- A user submitted a request for non-existent data on your site and you want to show them a “Not Found” page.

This lesson will show you some useful methods to satisfy those examples, including:

- `RedirectToPage()`
- `NotFound()`
- `Page()`

21.1 Introduction to Redirection Instructions

- Go to /Search and enter “all”. Where are you redirected?
- Try searching in /OldSearch. Where are you redirected?

Index.cshtml.cs

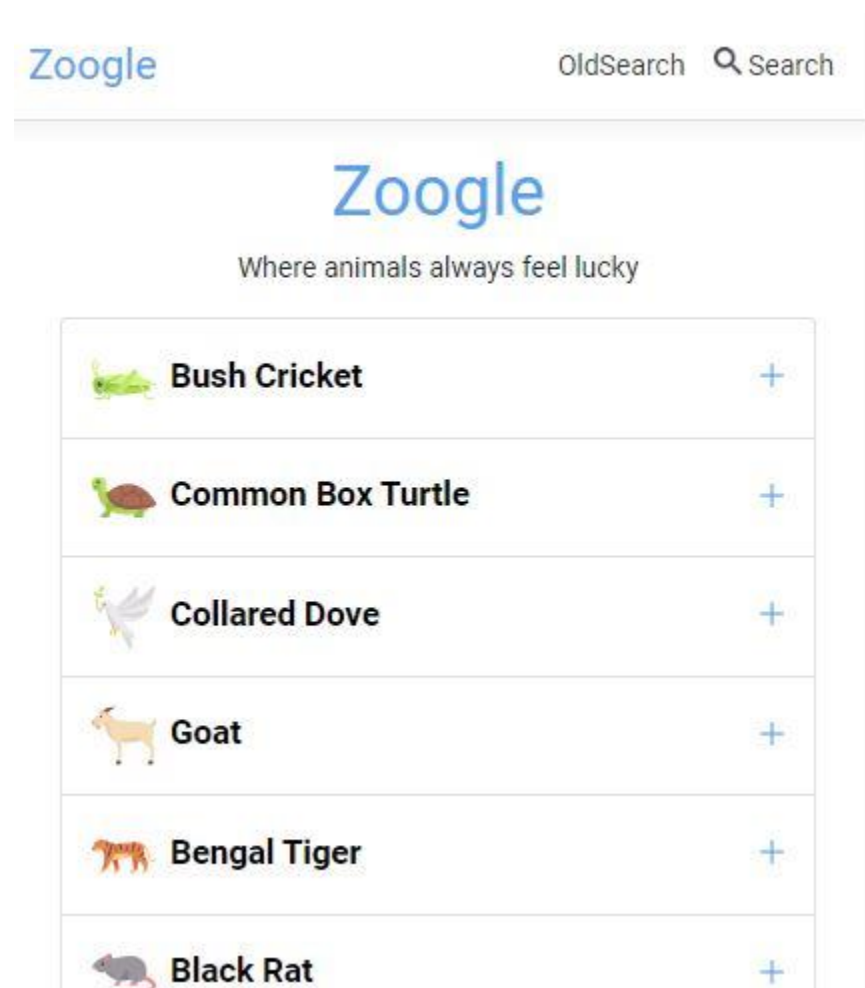
```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;
using Zoogle.Models;

namespace Zoogle.Pages
{
    public class IndexModel : PageModel
    {
        public List<Animal> Animals { get; set; }

        public IActionResult OnGet()
        {
            Animals = Zoo.Animals;
        }
    }
}
```

```
    return Page();  
  }  
}  
}
```

OUTPUT



	Emerald Tree Boa	+
	Armhook Squid	+
	Unicorn	+

© 2020 - Zoogle - [Privacy](#)

21.2 RedirectToPage()

In the past, our handler methods had a void return type:

```
public void OnGet() {}
```

To perform redirection we need our methods to return an object. This object will determine what kind of response is sent back to the browser.

This object can be any type that implements the `IActionResult` interface. There are a few types that do, but we don't need to dive into those. All we need to know is that the methods in this lesson return something that implements `IActionResult`.

Let's start with `RedirectToPage()`. For basic redirection, call it with a string argument and return the result. The string will represent the destination page, written like the URL segment, like `"/Privacy"` or `"/About/Me"`.

Here's an example handler method that always redirects learners to the checkout page:

```
public IActionResult OnPost()
{
    return RedirectToPage("/Checkout");
}
```

Note that the / (slash) makes the path relative to the **Pages** folder. In other words, this takes the user to the contents in **Pages/Checkout.cshtml**. Removing the slash makes the path relative to the current page.

Optional information: For those of you already familiar with HTTP status codes, RedirectToPage() produces an HTTP status code of 302, and the browser knows where to browse to by using information in the response header.

21.2 RedirectToPage() Instructions

1.

Currently, learners can navigate to /OldSearch.

- In **OldSearch.cshtml.cs**, change the return type of OnGet() to IActionResult.
- In the method body, redirect the user to the /Search page.
- Try navigating to /OldSearch and confirm that you are redirected.

OldSearch.cshtml.cs

```
using System;  
using System.Threading.Tasks;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Zoogle.Pages  
{  
    public class OldSearchModel : PageModel  
    {  
        public void OnGet()
```

```

{
}
}
}

```

Zoogle

OldSearch  Search

Zoogle

Where animals always feel lucky


Bush Cricket

Scientific Name

Tettigonia viridissima

Description

Tettigonia viridissima is distinguished by its very long and thin antennae, which can sometimes reach up to three times the length of the body, thus differentiating them from grasshoppers, which always carry short antennae.

Habitat

- Meadows

21.3 Page()

Say that we only want to redirect in some cases. Otherwise we want to have the original response, which included the current page.

Now that our handler methods are returning IActionResult values, we need a way to return the current page. The answer is calling and returning the Page() method.

Returning Page() will lead to the same behavior we saw when we had void handler methods (although we now have the flexibility to redirect as well).

These two methods have the same behavior:

```
public void OnGet()
{
}

public IActionResult OnGet()
{
    return Page();
}
```

21.3 Page() Instructions

1.

In **Search.cshtml.cs**, change the OnGet() method so that it returns Page(). Don't forget to update the return type as well.

2.

Let's add another redirection method: if the searchString argument is "all", redirect the user back to the home page.

Search.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Zoogle.Models;

namespace Zoogle.Pages
{
    public class SearchModel : PageModel
    {
        public List<Animal> Animals { get; set; }
        public Animal FoundAnimal { get; set; }
        public string SearchString { get; set; }

        #nullable enable

        public void OnGet(string? searchString)
        {
            Animals = Zoo.Animals;

            if (!string.IsNullOrEmpty(searchString))
            {
                // Check if searchString is "all" here

                FoundAnimal = Animals
                    .Where(a => a.CommonName == searchString)

```

```

        .FirstOrDefault();
    }

    // Call Page() here

}
#nullable disable
}
}

```

21.4 NotFound()

An HTTP response includes more than just HTML — it also includes an HTTP status code. A popular one is 404, which means that a requested resource was not found. For example, a user requests a profile page of a non-existent person.

We can generate that kind of response using `NotFound()`.

In this example, a request with the username "Machiavelli" will lead to a 404 response:

```

public IActionResult OnPost(string username)
{
    if (username == "Machiavelli")
    {
        return NotFound();
    }

    return Page();
}

```

21.4 NotFound() Instructions

1.

In a previous exercise, we redirected users from /OldSearch to /Search. This time, instead of redirecting, let's show a 404 Not Found page when a user tries to submit the form on /OldSearch.

Edit the OnPost() method so that it returns a 404 response.

OldSearch.cshtml.cs

```
using System;  
using System.Threading.Tasks;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace Zoogle.Pages  
{  
    public class OldSearchModel : PageModel  
    {  
        public void OnGet()  
        {  
        }  
  
        public void OnPost()  
        {  
  
        }  
    }  
}
```

```
}  
}
```

21.5 Async Redirects

Before we were redirecting users, we were using `void` and `Task` as our return types.

`void` was for synchronous methods:

```
public void OnGet()
```

`Task` was for asynchronous methods:

```
public async Task OnGetAsync()
```

With redirection, we will now return `ActionResult` and `Task<ActionResult>`.

`ActionResult` is for synchronous methods:

```
public ActionResult OnGet()
```

`Task<ActionResult>` is for asynchronous methods:

```
public async Task<ActionResult> OnGetAsync()
```

We don't need to get into the details of `Task<ActionResult>` now, but you can imagine it as some kind of operation that will eventually return an `ActionResult`.

21.5 Async Redirects Instructions

1.

In **Index.cshtml.cs**, `OnGet()` method is synchronous, but contains some asynchronous code. Change the method to its asynchronous sibling.

Make sure to:

- Add `async` to the method signature

- Change the return type
- Change the method name
- Return the result of Page()

Index.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;
using Zoogoo.Models;

namespace Zoogoo.Pages
{
    public class IndexModel : PageModel
    {
        public List<Animal> Animals { get; set; }

        public IActionResult OnGet()
        {
            Animals = Zoo.Animals;
        }
    }
}
```

```
// Asynchronous code

using (StreamWriter writer = new StreamWriter("log.txt", append: true))
{
    await writer.WriteLineAsync($"Zoogle visit recorded at {DateTime.Now}.");
}

return Page();
}
}
```

21.6 Review

You're on your way to becoming an ASP.NET master! In this lesson we learned some methods and types that are used to redirect users to pages:

- When a handler method has no return statement, it will send the associated page as a response.
- When a redirect method is used in a handler method, it must return the type `IActionResult` (or some type that implements the interface) or `Task<IActionResult>` if asynchronous.
- `Page()` causes the server to render the Razor view page associated with the current page model.
- `RedirectToPage()` causes the server to render the Razor view page described in the method argument. Generates a 302 HTTP status code.
- `NotFound()` causes the server to send a "Status 404 Not Found" response.

These are just a few common redirection methods. You can find a [full list of them defined within the PageModel type](#). A good one to start with is `Redirect()`, which works similarly to `RedirectToPage()`, but can redirect to URLs outside of your own application.

You can find [further examples of redirection methods in the Microsoft tutorials](#) and a [nice list of common redirection methods on Learn Razor Pages](#).

21.6 Review Instructions

Try out the redirections that you tested at the beginning of the lesson:

- Go to `/Search` and enter “all”. Where are you redirected?
- Try searching in `/OldSearch`. Where are you redirected?

Now you know how it’s done!

Index.cshtml

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;
using Zoogle.Models;

namespace Zoogle.Pages
{
```

```

public class IndexModel : PageModel
{
    public List<Animal> Animals { get; set; }

    public async Task<IActionResult> OnGet()
    {
        Animals = Zoo.Animals;

        // Asynchronous code
        using (StreamWriter writer = new StreamWriter("log.txt", append: true))
        {
            await writer.WriteLineAsync($"Zoogle visit recorded at {DateTime.Now}.");
        }

        return Page();
    }
}

```

Search.cshtml.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Zoogle.Models;
```

```
namespace Zoogle.Pages
{
    public class SearchModel : PageModel
    {
        public List<Animal> Animals { get; set; }
        public Animal FoundAnimal { get; set; }
        public string SearchString { get; set; }
```

```
#nullable enable
```

```
    public IActionResult OnGet(string? searchString)
    {
        Animals = Zoo.Animals;

        if (!string.IsNullOrEmpty(searchString))
        {
            if (searchString.ToLower() == "all")
            {
                return RedirectToPage("/Index");
            }
        }
    }
}
```

```

        FoundAnimal = Animals
            .Where(a => a.CommonName == searchString)
            .FirstOrDefault();
    }

    return Page();
}
}
#nullable disable

}

```

OldSearch.cshtml.cs

```

using System;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace Zoogle.Pages
{
    public class OldSearchModel : PageModel
    {
        public void OnGet()
        {

```

```
}

public IActionResult OnPost()
{
    return NotFound();
}
}
}
```

21. Quiz

1. Say you have a default Razor pages app with the below files. Which URL would you use to access the contents of Pages/About.cshtml?

Pages/

|-- About.cshtml

|-- About.cshtml.cs

|-- Index.cshtml

|-- Index.cshtml.cs

a) /About.cshtml

b) /Index

c) /Pages/About

d) /About

2. The Razor page Pages/Contact.cshtml is shown below. Fill in the code so that the page is made available at /Contact/Me.

@page _____

```
@{  
    ViewData["Title"] = "Contact";  
}
```

- "Me"
- "Contact/Me"
- "/Me"

Click or drag and drop to fill in the blank

3. Complete this method definition correctly.

```
public class AboutModel  
{  
    public string TeamMember { get; set; }  
  
    public _____ OnGet(string member)  
    {  
        TeamMember = member;  
    }  
}
```

- void
- Task
- IActionResult

Click or drag and drop to fill in the blank

4. Razor page Pages/Contact.cshtml is shown below. Fill in the code so that the URL has an additional parameter, like Contact/Sheila or Contact/Sueko or Contact/James.

@page _____

```
@{
    ViewData["Title"] = "Contact";
}
```

- @Model.Name
- "{name}"
- "Sheila"
- "name"

Click or drag and drop to fill in the blank

5. Say that ContactModel is associated with the page /Contact. Fill in the code so that it will send the /Contact page in response to every GET request.

```
public class ContactModel
{
    public IActionResult OnGet()
    {
        return _____;
    }
}
```

- Page()
- NotFound()
- this
- RedirectToPage()

Click or drag and drop to fill in the blank

6. Say that a user is browsing pages on my-portfolio.com, built with ASP.NET. They are about to go to the /Projects page, which is associated with the ProjectsModel. What method will be invoked when they enter this URL in their browser?

`https://my-portfolio.com/Projects/`

- a) `ProjectsModel.OnGetAsync()`
- b) `ProjectsModel.OnPostAsync()`
- c) `ProjectsModel.OnPost()`
- d) `IndexModel.OnGet()`

7. Imagine that this code is in `Pages/Rugs.cshtml.cs`. What will be in the response when a GET request is handled by this page model?

```
public class RugsModel
{
    public void OnGet()
    {
        Debug.Log("OnGet() called");
    }
}
```

- a) The response will include the logged text: "OnGet() called".
- b) The response will include the text within `OnGet()`, in this case, `Debug.Log("OnGet() called");`.
- c) Nothing is included in the response.
- d) The response will include the Razor page associated with this model.

8. What attribute is used in page models to activate model binding?

- a) [Required]
- b) [DisplayName("Price")]
- c) [ModelBinding]
- d) [BindProperty]

9. Fill in the below Razor markup so that the element's href attribute will render as `/Weather?forecast=tomorrow`.

@page

<a _____="/Weather" _____="tomorrow">Tomorrow

- asp-route-forecast
- asp-for
- asp-route
- asp-page
- asp-route-weather

Click or drag and drop to fill in the blank

10. Say that `FaqModel` is associated with the `/FAQ` page. What will happen when a GET response is handled by this page model?

```
public class FaqModel
{
    public void OnGet()
    {
        IActionResult redirect = RedirectToPage("/Index");
    }
}
```

- a) It will not do anything; this code will throw an error.
- b) It will return the /FAQ page.
- c) It will return the /Index page.
- d) It will return a 404 error.

11. Say that RetiredModel is associated with the page /Retired. Fill in the code so that it will send the /New page in response to every GET request.

```
public class RetiredModel
{
    public IActionResult OnGet()
    {
        return ____;
    }
}
```

- Page()
- NotFound()
- RedirectToPage("/New")

Click or drag and drop to fill in the blank

12. Say that a user is submitting a form on our delivery site, built with ASP.NET. The form generates a POST request and it is on the /Order page, which is associated with the OrderModel. What method will be invoked when the form is submitted?

- a) OrderModel.OnPostAsync()
- b) OrderModel.OnGetAsync()
- c) OrderModel.OnGet()
- d) DeliveryModel.OnPost()

13. Complete this method definition correctly.

```
public class WeatherModel
{
    public string Forecast { get; set; }

    public async _____ OnGetAsync()
    {
        Forecast = await GetForecastAsync();
    }
}
```

- IActionResult
- Task
- void

Click or drag and drop to fill in the blank

14. Say /Order is associated with OrderModel and a POST request is made to the below URL. What would Milkshakes be set to?

```
// POST request sent to:
// https://restaurant.net/Order?shakes=3&fries=1
```

```
public class OrderModel
{
    public int Fries { get; set; }
    public int Milkshakes { get; set; }

    public void OnPost(int fries, int shakes)
    {
        Fries = fries;
        Milkshakes = shakes;
    }
}
```

}

- a) 0
- b) null
- c) 3
- d) 1

15. Say that **MissingModel** is associated with the page **/Missing**. Fill in the code so that it will send a 404 status code in response to every **GET** request.

```
public class MissingModel
{
    public IActionResult OnGet()
    {
        return ____;
    }
}
```

- NotFound()
- Page()
- RedirectToPage("/NotFound")

Click or drag and drop to fill in the blank

16. Use Tag Helpers so that the Razor markup renders as the desired HTML.

<!-- Desired HTML -->

<input name="Fries" id="Fries" value="0" >

<!-- Your Razor markup -->

<input ____ = ____ value="0" >

- asp-for
- "Fries"
- asp-page
- value
- name

Click or drag and drop to fill in the blank

Project-9

Grocer.ly

Grocer.ly needs an ASP.NET developer with your skillset to help them build a new online shop.

You'll be building a Razor Pages app that displays each item in their store, complete with images, names, and prices. Each item will have its own "Details" page.

You'll be using Page Models, routing, and Razor syntax to make their vision a reality.

As you go through the tasks, make sure to Save your code to check that it compiles.

TASKS

Explore the App

1.

The Grocer.ly website has two "pages":

- The home page, which displays all of the grocery items and, at the bottom, a feedback form

- The details page, which displays a single item's details.

By the end of this project, you'll be able to click on each item in the home page to view its unique details page. The specific item will be listed as a URL segment, like Details/Aubergine and Details/Artichoke.

The app has two additional classes for grocery-specific data:

- GroceryItem - Represents a single grocery item, such as an apple or cupcake
- Inventory - A static class that lists all of the grocery stores items. Has one static method ToList(), which returns a List<GroceryItem> object.

Check out the page models

in **Pages/Index.cshtml.cs** and **Pages/Details.cshtml.cs**, then check out the two classes in **Models/GroceryItem.cs** and **Models/Inventory.cs**.

Home Page

2.

We'll need to access each grocery item in the page model for the home page.

In **Pages/Index.cshtml.cs**, define a Foods field that stores the result of Inventory.ToList().

(Inventory is a static class, and can be accessed by the IndexModel directly.)

3.

Set up the home page to generate some HTML for each item in Foods.

In **Pages/Index.cshtml**, within the <div> with id food-container, build an empty @foreach loop that iterates through the Foods property.

4.

We'll use Bootstrap styling to display each food. Use this template within the loop:

```
<div class="food-item pb-2 pt-2 row align-items-center">
  <div class="col-3 pl-1 pr-1 text-center" id="image-col">
    <img class="food-item-image" src=@food.ImageSrc alt=@food.Name>
  </div>
  <div class="col-6 pl-1 pr-1" id="info-col">
    <p class="food-item-info">@food.Name</p>
    <p class="food-item-info">${@String.Format("{0:0.00} / lb", food.Price)}</p>
  </div>
  <div class="col-3 pl-1 pr-1 text-center" id="button-col">
    <a asp-page="/Details" asp-route-
name=@System.Web.HttpUtility.UrlPathEncode(food.Name) class="btn text-
white">More info</a>
  </div>
</div>
```

- The src attribute should contain each food's ImageSrc property
- The alt attribute should contains each food's Name property
- The first <p> contents should be the food's Name property
- The second <p> contents should have the food's Price property, properly formatted
- The <a> should link to the Details page. It should have an additional URL segment that equals the food's Name property. For example, if the food is Artichoke, then the link should be /Details/Artichoke.

For the <a> link: Normally you don't need the HttpUtility — you would just use asp-route-name=@food.Name. We make an exception here because our environment on Codecademy is a bit different than working on your own computer.

5.

Run your code and make sure that each food in `Inventory.ToList()` is displayed properly.

Details Page

6.

In the home page, each grocery item has a link like `/Details/Artichoke`.

In **Pages/Details.cshtml**, make the Details page accept an additional route value by adding "{name}" after the `@page` directive.

7.

Before building out the Details page, make sure that users can get back to the home page.

Find the `<a>` element at the bottom of **Details.cshtml**. Add an `asp-page` attribute to the `<a>` element that points to the home page.

To check your work, click on view “More Info” for any item. The resulting page is the Details page. Click the button at bottom of the Details page. It should return you to the home page.

8.

Now that the URL route template is set up, let’s use that data in the page model.

In **Pages/Details.cshtml.cs**, define two properties:

- `Foods` — stores the result of `Inventory.ToList()`
- `CurrentFood` — a `GroceryItem` representing the food to be detailed

9.

Make sure that the {name} in the URL is captured in the page model.

Define an `OnGet()` method that accepts a name parameter of type string.

10.

Using the `Find()` method provided by LINQ, search the `Foods` property for an element whose `Name` property matches the `name` parameter. Assign the result of `Find()` to the `CurrentFood` property.

Here's an imaginary example that uses `Find()` to search for a car with the year 1985:

```
OldCar = Cars.Find(x => x.Year == 1985);
```

11.

Back in **Details.cshtml**, modify the existing HTML tags and add the `CurrentFood` properties in the following places:

- The `<h2>` should contain the current food's `Name`
- The image's `src` attribute should be the current food's `ImageSrc`
- The `<p>` should contain the current food's `Price`, formatted as a currency (see the hint for formatting tips)
- The next `<p>` should contain the current food's `Desc`
- The `<span>` below that should also contain the current food's `Price` (this is hidden from the user, but necessary to calculate total prices)

12.

Run your code and check that your `Details` page works!

You should be able to click on "More info" in the home page to see each item's details.

Async Details Page

13.

Grocer.ly has asked you to record a log of all visits to the `Details` page.

In **Pages/Details.cshtml.cs**:

- Change OnGet() to an asynchronous method
- Before the LINQ search, add this asynchronous code that writes the date and food name to a **log.txt** file:

```
using (StreamWriter writer = new StreamWriter("log.txt", append: true))
{
    await writer.WriteLineAsync($"{DateTime.Now} {name}");
}
```

14.

What happens when you look for a non-existent item, like localhost:8000/Details/UnicornMeat?

We'll need to use redirection methods to handle this case.

Start by changing the OnGetAsync() return type to Task<IActionResult> and returning Page() at the end of the method.

15.

When a user goes to a non-existent item, like Details/UnicornMeat, they should see a 404 - Not Found page.

After calling Find(), check if the result is null. If it is, show the users a "Not Found" page.

Extensions

16.

Well done! Grocer.ly is very pleased with your work. They're impressed with your knowledge of Razor Pages too!

If you'd like to keep practicing, try this optional challenge: Add a feedback form to the bottom of the home page:

- In **Index.cshtml**, write an HTML form.
- In **Index.cshtml.cs**, add some properties to capture the form values, then write an asynchronous `OnPostAsync()` method that writes the feedback to a text file.

Index.cshtml

```
@page
```

```
@model IndexModel
```

```
@{
```

```
    ViewData["Title"] = "Home";
```

```
}
```

```
<div class="text-center">
```

```
    <h1 id="store-name" >Grocer.ly</h1>
```

```
    <p>Browse our inventory online and shop in-store!</p>
```

```
</div>
```

```
<div id="food-container">
```

```
    <!-- Loop through Model.Foods -->
```

```
</div>
```

```
<hr class="mt-2"/>
```

Index.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.Extensions.Logging;

using GroceryStore.Models;

namespace GroceryStore.Pages
{
    public class IndexModel : PageModel
    {

        [BindProperty]
        public int Rating { get; set; }

        [BindProperty]
        public string Feedback { get; set; }

        public void OnGet()
        {
```

```
}
```

```
}
```

```
}
```

Details.cshtml

```
@page
```

```
@model DetailsModel
```

```
@{
```

```
    ViewData["Title"] = "Details";
```

```
}
```

```
<div id="top-container" class="text-center">
```

```
    <!-- Display CurrentFood's name -->
```

```
    <h2 class="font-weight-bold"> </h1>
```

```
    <!-- Display CurrentFood's image -->
```

```
    <img id="big-food-icon" >
```

```
    <!-- Display CurrentFood's price -->
```

```
    <p id="display-price" class="font-weight-bold text-left">
        $@String.Format("{0:0.00} / lb", 0)</p>
```

```
</div>
```

```
<hr />
```

```
<div id="description-container">
```

```

    <!-- Display CurrentFood's description -->
    <p> </p>
</div>

<hr />

<div id="pricing-container" class="d-flex font-weight-bold">
    <!-- Include CurrentFood's price -->
    <span id="unit-price" class="d-none">0</span>

    <div class="col text-left">
        <label for="quantity">Quantity:</label>
        <input type="number" id="quantity" name="quantity" min="1">
    </div>

    <div class="col text-right">
        Total: $<span id="total-price">0.00</span>
    </div>

</div>

<hr />

<div id="back-button-container" class="text-center">

```



```
<!-- Link back to home page-->
<a class="btn text-white">
    
    Back to home
</a>
</div>
```

Details.cshtml.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using GroceryStore.Models;

namespace GroceryStore.Pages
{
    public class DetailsModel : PageModel
    {

    }
}
```

OUTPUT

 **Grocer.ly** [Home](#) [Privacy](#)

 **Grocer.ly**

Browse our inventory online and shop in-store!

	Organic Honeycrisp Apple \$2.99 / lb	More info
	Artichoke \$3.49 / lb	More info
	Aubergine \$5.49 / lb	More info
	Hass Avocado \$3.99 / lb	More info



Broccoli
\$4.99 / lb

[More info](#)



Assorted Candy
\$0.99 / lb

[More info](#)



Chili Pepper
\$1.99 / lb

[More info](#)



Crab
\$22.49 / lb

[More info](#)



Chocolate Cupcake
\$5.99 / lb

[More info](#)



Organic Honeycrisp Apple



\$2.99 / lb

Organic, delicious Honeycrisp Apple, grown and harvested at a local farm.

Quantity:

Total: \$0.00

[← Back to home](#)

ASP.NET: Databases

Database Model

Entity Framework uses C# classes to define the database model. This is an in-memory representation of data stored in a database table. Several model classes combine to form the schema for the database.

Each property maps to a column in a database table. The bottom line in the example shows a type of Continent which implies a relationship to another table.

using System;

```
public class Country
{
    public string ID { get; set; }
    public string ContinentID { get; set; }
    public string Name { get; set; }
    public int? Population { get; set; }
    public int? Area { get; set; }
    public DateTime? UnitedNationsDate { get; set; }

    public Continent Continent { get; set; }
}
```

Database Context

The Entity Framework *database context* is a C# class that provides connectivity to an external database for an application. It relies on the Microsoft.EntityFrameworkCore library to define the DB context which maps model entities to database tables and columns.

The DbContextOptions are injected into the context class via the constructor. The options allow configuration changes per environment so the Development DB is used while coding and testing but the Production DB would be referenced for real work.

The DbSet is an in-memory representation of a table or view which has a number of member methods that can return a List<T> of records or a single record.

using Microsoft.EntityFrameworkCore;

```
public class CountryContext : DbContext
{
    public CountryContext(DbContextOptions<CountryContext> options)
        : base(options)
    {
    }

    public DbSet<Country> Countries { get; set; }
    public DbSet<Continent> Continents { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Country>().ToTable("Country");
        modelBuilder.Entity<Continent>().ToTable("Continent");
    }
}
```

```
}  
}
```

DbSet Type

The Entity Framework type `DbSet` represents a database table in memory. It is typically used with a `<T>` qualifier. The type, or `T`, is one of your database model classes. The `ModelBuilder` binds each database table entity to a corresponding `DbSet`.

`DbSet` has a number of member methods that can return a `List<T>` of records or a single record.

```
using Microsoft.EntityFrameworkCore;
```

```
public class CountryContext : DbContext  
{  
    public CountryContext(DbContextOptions<CountryContext> options)  
        : base(options)  
    {  
    }  
  
    public DbSet<Country> Countries { get; set; }  
    public DbSet<Continent> Continents { get; set; }  
  
    protected override void OnModelCreating(ModelBuilder modelBuilder)  
    {  
        modelBuilder.Entity<Country>().ToTable("Country");  
    }  
}
```

```
modelBuilder.Entity<Continent>().ToTable("Continent");  
}  
}
```

Entity Framework Configuration

In ASP.NET Core, a database may be connected to a web app using Entity Framework. There are four common steps for any setup:

1. Define one or more database model classes and annotate them
2. Define a database context class that uses DbSet to map entities to tables
3. Define a database connection string in **appsettings.json**
4. Add the Entity Framework service in Startup.ConfigureServices()

Database Connection String

The Entity Framework context depends on a database connection string that identifies a physical database connection. It is typically stored in **appsettings.json**. You can define multiple connection strings for different environments like Development, Test, or Production.

Each database product has specific requirements for the syntax of the connection string. This might contain the database name, user name, password, and other options.

```
{  
  "ConnectionStrings": {  
    "CountryContext": "Data Source=Country.db"  
  }  
}
```


Creating the Schema

Entity Framework provides command-line tools that help manage the connected database. Use these commands in the bash shell or Windows command prompt to create an initial database file and schema. This will read the context class and evaluate each database model represented by a DbSet. The SQL syntax necessary to create all schema objects is then generated and executed.

```
dotnet ef migrations add InitialCreate
```

```
dotnet ef database update
```

Model Binding

In ASP.NET Core, *model binding* is a feature that simplifies capturing and storing data in a web app. The process of model binding includes retrieving data from various sources, converting them to collections of .NET types, and passing them to controllers/page models. Helpers and attributes are used to render HTML with the contents of bound page models. Client- and server-side validation scripts are used to ensure integrity during data entry.

Adding Records

The Entity Framework context DbSet member provides the Add() and AddAsync() methods to insert a new record into the in-memory representation of the corresponding database table. A batch of multiple records can also be added in this fashion.

The record is passed from the browser in the <form> post back. In this case a Country member is declared with a [BindProperty] attribute so the entire record is passed back to the server.

Use the EF context SaveChanges() or SaveChangesAsync() methods to persist all new records to the database table.

```
// Assuming Country is of type Country
```

```
// Assuming _context is of a type inheriting DbSet
```

```
public async Task<IActionResult> OnPostAsync(string id)
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    await _context.Countries.AddAsync(Country);

    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}
```

Saving Changes

The Entity Framework context DbSet member provides the Attach() method to update an existing record, the Add() method to insert a new record, and the Remove() method to delete an existing record. Any combination of multiple records can be batched before saving.

Use the EF context SaveChanges() or SaveChangesAsync() methods to persist all inserted, updated, and deleted records to the database table.

```
// Assuming Country is of type Country
```

```
// Assuming _context is of a type inheriting DbSet
```

```
public async Task<IActionResult> OnPostAsync(string id)
{
    // update
    _context.Attach(Country).State = EntityState.Modified;

    // insert
    await _context.Countries.AddAsync(Country);

    // delete
    Country Country = await _context.Countries.FindAsync(id);

    if (Country != null)
    {
        _context.Countries.Remove(Country);
    }

    // all three methods must be followed by savechanges
}
```

```

        await _context.SaveChangesAsync();

        return RedirectToPage("./Index");
    }

```

Finding Records

The Entity Framework context DbSet member provides the Find() and FindAsync() methods to retrieve an existing record from the in-memory representation of the database table. Assign the result of this method to a local member in the page model.

This method generates the appropriate SQL syntax needed to access the record in the database table.

```
// Assuming Country is of type Country
```

```
// Assuming _context is of a type inheriting DbSet
```

```

public async Task<ActionResult> OnGetAsync(string id)
{
    if (id == null)
    {
        return NotFound();
    }

```

```

        Country Country = await _context.Countries.FindAsync(id);

```

```
    return Page();  
}
```

Deleting Records

The Entity Framework context DbSet member provides the Remove() method to delete an existing record from the in-memory representation of the database table. Any combination of multiple record deletions can be batched before saving.

Use the EF context SaveChanges() or SaveChangesAsync() methods to persist all deletions to the database table.

```
// Assuming Country is of type Country  
// Assuming _context is of a type inheriting DbSet  
  
public async Task<IActionResult> OnPostAsync(string id)  
{  
    if (id == null)  
    {  
        return NotFound();  
    }  
  
    Country Country = await _context.Countries.FindAsync(id);  
  
    if (Country != null)  
    {  
        _context.Countries.Remove(Country);  
    }  
}
```

```

    }

    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}

```

Updating Records

The Entity Framework context DbSet member provides the Attach() method to update an existing record in the in-memory representation of the corresponding database table. A batch of multiple records can also be updated in this fashion.

The record is passed from the browser in the <form> post back. In this case a Country member is declared with a [BindProperty] attribute so the entire record is passed back to the server.

Use the EF context SaveChanges() or SaveChangesAsync() methods to persist all updated records to the database table.

```

// Assuming Country is of type Country
// Assuming _context is of a type inheriting DbSet

```

```

public async Task<IActionResult> OnPostAsync(string id)
{
    if (!ModelState.IsValid)
    {
        return Page();
    }
}

```

```

    }

    _context.Attach(Country).State = EntityState.Modified;

    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}

```

Valid Model State

Entity Framework database models accept annotations that drive data validation at the property level. If you are using the `asp-validation-for` or `asp-validation-summary` HTML attributes, validation is handled client-side with JavaScript. The model is validated and the `<form>` post back won't occur until that model is valid.

Sometimes the client-side validation will not be available so it is considered best practice to also validate the model server-side, inside the `OnPostAsync()` method. This example checks for `ModelState.IsValid` and returns the same page if it is false. This effectively keeps the user on the same page until their entries are valid.

If the model is valid, the insert, update, or delete can proceed followed by `SaveChangesAsync()` to persist the changes.

```

public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }
}

```

```
}

_context.Continents.Add(Continent);

await _context.SaveChangesAsync();

return RedirectToPage("./Index");
}
```

Validation Attribute

The asp-for attribute in an <input> element will render HTML and JavaScript that handle the display and data entry for a field based on the model annotations. The JavaScript will set the valid flag on the field.

The asp-validation-for attribute in a element will display any error message generated when the property annotations are not valid.

In this example, the be rendered as this HTML:

```
<span class="field-validation-valid" data-valmsg-for="Continent.Name" data-  
valmsg-replace="true"></span>
```

```
<div>
```

```
<label asp-for="Continent.Name"></label>
```

```
<div>
```

```
<input asp-for="Continent.Name" />
```

```
<span asp-validation-for="Continent.Name"></span>
```

```
</div>
```


</div>

[Display] Attribute

The [Display] attribute specifies the caption for a label, textbox, or table heading.

Within a Razor Page, the @Html.DisplayName() helper defaults to the property name unless the [Display] attribute overrides it. In this case, Continent is displayed instead of the more technical ContinentID.

using System.ComponentModel.DataAnnotations;

```
public class Country
{
    [Display(Name = "Continent")]
    public string ContinentID { get; set; }
}
```

[DisplayFormat] Attribute

The [DisplayFormat] attribute can explicitly apply a C# format string. The optional ApplyFormatInEditMode means the format should also apply in edit mode.

[DisplayFormat] is often used in combination with the [DataType] attribute. Together they determine the rendered HTML when using the @Html.DisplayFor() helper.

using System.ComponentModel.DataAnnotations;

```

public class Country
{
    [DisplayFormat(DataFormatString = "{0:N0}", ApplyFormatInEditMode = true)]
    public int? Population { get; set; }
}

```

[DataType] Attribute

The [DataType] attribute specifies a more specific data type than the database column type. In this case, the database table will use a DateTime column but the render logic will only show the date.

The @Html.DisplayFor() helper knows about types and will render a default format to match the type. In this case, the HTML5 browser date picker will appear when editing the field.

```
using System.ComponentModel.DataAnnotations;
```

```

public class Country
{
    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:yyyy-MM-dd}", ApplyFormatInEditMode = true)]
    public DateTime? UnitedNationsDate { get; set; }
}

```

[Required] Attribute

The [Required] attribute can be applied to one or more properties in a database model class. EF will create a NOT NULL column in the database table for the property.

The client-side JavaScript validation scripts will ensure that a non-empty string or number is valid before posting the record from the browser on inserts and updates.

```
using System.ComponentModel.DataAnnotations;
```

```
public class Country
{
    [Required]
    public string Name { get; set; }
}
```

[RegularExpression] Attribute

The [RegularExpression] attribute can apply detailed restrictions for data input. The match expression is evaluated during data entry and the result returns true or false. If false, the model state will not be valid and the optional ErrorMessage will display.

In a Razor page, the @Html.DisplayFor() helper only shows the data in the field. The asp-validation-for attribute on a tag displays the ErrorMessage.

```
using System.ComponentModel.DataAnnotations;
```

```
public class Country
{
```

```
[RegularExpression(@"[A-Z]+", ErrorMessage = "Only upper case characters are allowed.")]
```

```
public string CountryCode { get; set; }  
}
```

[StringLength] Attribute

The [StringLength] attribute specifies the maximum length of characters that are allowed in a data field and optionally the minimum length. The model will not be flagged as valid if these restrictions are exceeded. In this case, the ContinentID must be exactly 2 characters in length.

In a Razor Page, the @Html.DisplayFor() helper only shows the data in the field. The client-side JavaScript validation scripts use the asp-validation-for attribute on a tag to display a default error message.

```
using System.ComponentModel.DataAnnotations;
```

```
public class Country  
{  
    [StringLength(2, MinimumLength = 2)]  
    public string ContinentCode { get; set; }  
}
```

[Range] Attribute

The [Range] attribute specifies the minimum and maximum values in a data field. The model will not be flagged as valid if these restrictions are exceeded. In this case, Population must be greater than 0 and less than the big number!

In a Razor page, the @Html.DisplayFor() helper only shows the data in the field. The client-side JavaScript validation scripts use the asp-validation-for attribute on a tag to display a default error message.

```
using System.ComponentModel.DataAnnotations;
```

```
public class Country
{
    [Range(1, 10000000000)]
    public int? Population { get; set; }
}
```

Select List Items Attribute

```
<select asp-for="Country.ContinentID" asp-items="Model.Continents"></select>
```

The asp-items attribute in a <select> element generates <option> tags according to the model property specified. It works in conjunction with the asp-for attribute to display the matching option and set the underlying value on a change.

```
using Microsoft.AspNetCore.Mvc.Rendering;
```

```
public SelectList Continents { get; set; }
```

```
public async Task<IActionResult> OnGetAsync(string id)
{
```

```
Continents = new SelectList(_context.Continents, nameof(Continent.ID),  
nameof(Continent.Name));  
}
```

The SelectList type is declared in the page model code and assigned to a new SelectList() where each record in Continents grabs the ID as the <option> value and Name as the <option> display text.

The included <select> in the example would render as this HTML:

```
<select class="valid" id="Country_ContinentID" name="Country.ContinentID"  
aria-invalid="false">  
  
  <option value="NA">North America</option>  
  
  <option value="SA">South America</option>  
  
  <option value="EU">Europe</option>  
  
  <!-- etc -->  
  
</select>
```